

THINKFREE FINANCE

Architecture & Engineering Methodology

A complete technical reference of the platform,
its pipeline, sub-agents, and data architecture

Prepared for engineering review · Generated from a direct reading of the source tree
AI Financial Intelligence Platform — research analyst, not a trading bot

CONFIDENTIAL — for prospective engineers and collaborators

Table of Contents

Part 1 — Executive Summary

- What ThinkFree Finance is
- The core translation mission
- The eight-question framework
- Political intelligence — transparency, not accusation
- What this document is
- System at a glance
- Scale
- How the numbers are produced (and what AI does)
- Recent updates and where to find them

Part 2 — Development Methodology & Engineering Pipeline

- 2.1 The three inputs that governed development
- 2.2 Working method (the "analyze before coding" loop)
- 2.3 The dual-mode architecture decision
- 2.4 Behavior-preserving verification (the readability pass)
- 2.5 Chronological engineering narrative
- 2.6 A note on the environment

Part 2B — Statistical Methodology and Corrections

- 2B.0 Why this section exists
- 2B.1 Expected return uses the true mean, not the median
- 2B.2 Probability of a move is an honest historical frequency
- 2B.3 Kelly sizing removed from user-facing outputs
- 2B.4 Backtest is walk-forward and point-in-time
- 2B.5 Recession trigger uses the Sahm rule
- 2B.6 Politician scoreboard ranks on excess return
- 2B.7 One clean signal definition
- 2B.8 Opportunity score separates outlook from suitability
- 2B.9 Every LLM number is validated against source
- 2B.10 Degradation flag on incomplete inputs
- 2B.11 Recent corrections, in one line each

Part 3 — System Architecture

- 3.1 High-level topology
- 3.2 The two delivery modes
- 3.3 Request lifecycle (server mode)
- 3.4 The 13-phase intelligence pipeline (logical view)
- 3.5 Verification-agent layer (cross-cutting)
- 3.6 The AI sub-agent layer (development-time)
- 3.7 Repository map

Part 4 — Offline Data Build Pipeline

- Summary Table
- Pipeline Execution Order
- `build\_data.py`
- `build\_states.py`
- `build\_census.py`
- `build\_bea.py`
- `build\_bls.py`
- `build\_fec.py`
- `build\_influence.py`
- `build\_legiscan.py`
- `build\_member\_bills.py`
- `build\_news\_intel.py`
- `build\_nonprofits.py`
- `build\_openstates.py`
- `build\_prices.py`

[`build\\_quant.py`](#)
[`build\\_relationships.py`](#)
[`build\\_sec.py`](#)
[`build\\_secbulk.py`](#)
[`build\\_sp500.py`](#)
[`build\\_usaspending.py`](#)
[`fetch\\_politician\\_photos.py`](#)
[`minify\\_data.py`](#)
[`scripts/extract\\_data\\_to\\_json.py`](#)
[Environment Variables Reference](#)
[Local File Prerequisites \(Non-API\)](#)

Part 5 — Backend Server Reference (FastAPI)

[Table of Contents](#)
[1. Dependency Map](#)
[2. File: `requirements.txt`](#)
[3. File: `.env.example`](#)
[4. File: `.gitignore`](#)
[5. File: `config.py`](#)
[6. File: `db.py`](#)
[7. File: `auth.py`](#)
[8. File: `data.py`](#)
[9. File: `main.py`](#)
[10. Endpoints Quick-Reference Table](#)
[11. Security Architecture Summary](#)

Part 6A — Frontend Reference: Core (boot.js, auth.js, app.js)

[Table of Contents](#)
[1. Script Load Order and HTML Structure](#)
[2. boot.js — Bootstrap and Dual-Mode Orchestrator](#)
[3. auth.js — Beta Gate, Session, and Tier Model](#)
[4. app.js — Main Controller, Router, and View Renderer](#)
[5. Window Globals Cross-Reference](#)

Part 6B — Frontend Reference: Feature Modules

[Table of Contents](#)
[influenceweb.js](#)
[congress.js](#)
[genimpact.js](#)
[predictions.js](#)
[scores.js](#)
[scoreinfo.js](#)
[glossary.js](#)

Part 7A — Core Intelligence Pipeline (Phases 3-13)

[Table of Contents](#)
[controller.py — Phase 12: Pipeline Orchestration](#)
[chef_gpt.py — Phase 11: Intelligence Synthesis](#)
[recession_signals.py — Phase 13: Recession Signals Engine](#)
[opportunity_score.py — Phase 8.9: Opportunity Scoring Engine](#)
[quantlib_metrics.py — Phase 8.7: QuantLib Risk Metrics](#)
[historical_correlation.py — Phase 7: Historical Correlation Engine](#)
[phase3_extraction.py — Phase 3: NLP Data Extraction](#)
[phase4_clustering.py — Phase 4: Article Scoring & Clustering](#)
[portfolio_tracker.py — Phase 10: Portfolio Tracker](#)
[congress_bills.py — Phase 6.5 Supplement: Congress Bills Intelligence](#)
[politician_performance.py — Phase 6.5 Supplement: Politician Trading Performance](#)
[Cross-Module Data Flow Summary](#)

Part 7B — Supporting Engines: Economic Reasoning, Clustering, Technical Analysis & Backtesting

[Table of Contents](#)
[1. `GPT\\_Economy/economic\\_engine.py`](#)

2. `GPT\_Economy/Reasoning\_Report.py`
3. `Profile/article\_clustering.py`
4. `Profile/GPT\_article\_clustering\_summary.py`
5. `Module\_2\_Technical\_Analysis/ta\_analysis.py`
6. `Module\_2\_Technical\_Analysis/phase\_2\_q\_lib.py`
7. `Module\_2\_Technical\_Analysis/phase\_3\_signal.py`
8. `Module\_2\_Technical\_Analysis/pipeline\_alpha.py`
9. `Module\_2\_Technical\_Analysis/prepare\_qlib\_data.py`
10. `Module\_2\_Technical\_Analysis/phase\_4\_backtrader.py`
11. `validate\_env.py`

Appendix: Data Flow Summary

The Money Trail Detective Engine

- The 6-link model
- Scoring formula and weights
- Data sources per link
- File and function map
- The Case File UI
- Phase 4: gpt-4o materiality (enabled, titles + summaries path)
- Known limitations (evidenced vs inferred)

Part 8 — Python Verification Agent Layer

- Summary Table
- Package Root — `agents/\_\_init\_\_.py`
- DataQualityAgent — `agents/data\_quality.py`
- SecurityAuditAgent — `agents/security\_audit.py`
- StrategyAuditAgent — `agents/strategy\_audit.py`
- VerificationRunner — `agents/verification\_runner.py`
- Cross-Agent Architecture Notes

Part 9 — The Cybersecurity Sub-Agent and Its Training

- 9.1 Agent definition
- 9.2 Operating rules (baked into the prompt)
- 9.3 Stack translation (why generic advice was not enough)
- 9.4 The training corpus (source folder)
- 9.5 Knowledge file 1 — `vibe-guard-rules.md` (the 28-rule checklist)
- 9.6 Knowledge file 2 — `pentest-playbook.md` (active-test methodology)
- 9.7 Knowledge file 3 — `payloads-cheatsheet.md` (payloads + wordlists)
- 9.8 How the agent uses its training
- 9.9 Relationship to the runtime verification agents
- 9.10 Distribution note

Part 10 — Library & Dependency Inventory

- 10.1 Backend dependencies (server/requirements.txt)
- 10.2 Pipeline dependencies (requirements.txt)
- 10.3 Standard-library usage (notable)
- 10.4 Version-pin rationale
- 10.5 Frontend dependencies
- 10.6 Security tooling (referenced by CI and the audit agent)

Part 11 — Security Model & Deployment

- 11.1 The security model in one line
- 11.2 What the model achieves
- 11.3 Authentication & session security
- 11.4 Signup integrity controls
- 11.5 Transport & HTTP hardening (server middleware)
- 11.6 Secret hygiene
- 11.7 CI security gate
- 11.8 Local run
- 11.9 Production hosting (recommended path)
- 11.10 Pre-launch security checklist
- 11.11 Known not-yet-done (by design)

Part 12 — Dataset & Data-Source Catalog

- Master Catalog Table
- Dataset Groups
- Server-Mode Endpoint Map
- Build Script Reference

Part 12B — Current State of the Live Application

- 12B.1 Deployment on Render
- 12B.2 Tiered refresh and pricing
- 12B.3 The Money Trail engine
- 12B.4 Congressional trading, QuiverQuant, and corrected statistics
- 12B.5 Scheduler and autonomous refresh
- 12B.6 OFR Hedge Fund Monitor
- 12B.7 Data Centers tracker
- 12B.8 Economy and cost-of-living tracker
- 12B.9 Presidential feed
- 12B.10 Security hardening
- 12B.11 Public landing and app theme

Part 13 — Appendices

- Appendix A — Project file index (non-vendored)
- Appendix B — Environment variables
- Appendix C — Glossary
- Appendix D — Document provenance

Part 1 — Executive Summary

What ThinkFree Finance is

ThinkFree Finance is an **AI-powered financial-intelligence platform** that helps investors — and everyday people — understand what is happening in financial markets, the economy, and government activity, and **why it matters to their lives**. It sits between a Bloomberg Terminal and plain English.

It is deliberately **not** a trading bot, brokerage, robo-advisor, portfolio manager, or automated execution system. It is an **AI research analyst and economic translator**. The final artifact a user receives is an **intelligence report**, never a trade order.

Target audience: the average person trying to understand the economy — the blue-collar worker, the renter, the small-business owner, the college student — not Wall Street professionals.

The core translation mission

Most financial news assumes the reader already understands economics. ThinkFree bridges that gap. When the Fed raises rates 0.25%, ThinkFree explains what that means for rent and mortgages, credit cards and car loans, jobs and hiring, grocery prices, small-business borrowing, and savings accounts.

The eight-question framework

- Every major analysis answers eight questions:
- 1. What happened?
 - 2. Why did it happen?
 - 3. Who benefits?
 - 4. Who is negatively affected?
 - 5. How could this affect consumers (rent, groceries, gas, credit cards, mortgages, jobs)?
 - 6. How could this affect businesses?
 - 7. How could this affect financial markets?
 - 8. What happened historically in similar situations?

Political intelligence — transparency, not accusation

ThinkFree tracks congressional trading, lobbying, government contracts, and legislative activity. The framing rule is non-negotiable: **all political- intelligence output describes timing relationships between publicly available government disclosures and market events. It never characterizes intent or implies wrongdoing.** Every such output carries the disclaimer:

"This analysis identifies timing relationships between publicly available government disclosures and market events. It does not imply or allege wrongdoing of any kind."

Data sources include STOCK Act disclosures, USASpending.gov, OpenSecrets, and QuiverQuant.

What this document is

This is a complete architecture-and-methodology reference for engineers. It documents every subsystem, every Python file and function, every build script, every backend route, every frontend module, the offline data pipeline, the 13-phase intelligence pipeline, the Python verification-agent layer, the AI sub-agent (and exactly what it was trained on), every third-party library, the security model, and the deployment story. It also records the **development methodology** — how the platform was built from the founder's instructions, the canonical blueprint (the project's CLAUDE.md), and the prior code base.

System at a glance

Layer	Technology	Purpose
Delivery (web)	Static HTML/CSS/JS in <code>webapp/</code>	The user-facing intelligence UI
Delivery (legacy)	Streamlit <code>dashboard/app.py</code>	Original prototype dashboard (kept, untouched)
Backend	FastAPI in <code>server/</code>	Auth, gated data, tier enforcement, rate limiting

Layer	Technology	Purpose
Identity	bcrypt + PyJWT httpOnly cookie + SQLite	Real authentication for cloud hosting
Offline pipeline	~22 Python build scripts in <code>webapp/</code>	Fetch external data → <code>*_data.js</code> / <code>private_data/*.json</code>
Intelligence pipeline	13 phases across the repo root	Scrape → extract → cluster → reason → quantify → brief
Verification	4 Python agents in <code>agents/</code>	Data-quality, security, strategy, orchestration gates
AI sub-agent	<code>.claude/agents/cybersecurity-agent.md</code> + KB	Attacker-perspective security review
Native app	SwiftUI in <code>iphone-app/</code>	iOS client reusing the web data (not build-verified)

Scale

- ~16,700 lines of project Python across ~60 files (excluding vendored venvs).
- ~5,700 lines of frontend JavaScript across ~10 logic modules.
- 30+ pinned Python libraries for the pipeline; 5 for the backend.
- 17 generated datasets sourced from 11+ external APIs / government disclosures.

How the numbers are produced (and what AI does)

Every quantitative result is **computed in code** by the Python quant engine, not by a language model. The AI layer is a **translator**: it turns the deterministic numbers into plain English and is constrained so it can phrase a figure but never invent one. This division of labor is central to the product's honesty claims.

Recent updates and where to find them

- **Part 2B, Statistical Methodology and Corrections**, documents the current,

audited quant methodology (true-mean expected return, historical-frequency probabilities, walk-forward point-in-time backtesting, the Sahm-rule recession trigger, the excess-return politician scoreboard, and related corrections). It is written as the updated approach for review before the implementing code merges.

- **Part 12B, Current State of the Live Application**, documents the production

deployment on Render, the tiered refresh and pricing, the Money Trail engine, the QuiverQuant congressional-trade integration, the scheduler and autonomous refresh, the Hedge Fund Monitor, the Data Centers and Economy trackers, the Presidential feed, the security hardening, and the public landing plus the app theme upgrade.

Part 2 — Development Methodology & Engineering Pipeline

This part records *how* ThinkFree was built: the inputs that governed every decision, the working method, and the chronological engineering narrative. It is written so an external engineer can understand not just *what* the code does but *why* it is shaped the way it is.

2.1 The three inputs that governed development

Every change was made against three sources of truth, in priority order:

1. **The founder's direct instructions** — the feature requests and corrections

issued conversationally (e.g. "make login always land on the dashboard", "no emojis in the UI", "make the backend completely safe for beta testing").

1. **The canonical blueprint (`CLAUDE.md`)** — a committed project charter that

defines the mission statement, the eight-question framework, the political- intelligence framing rule, the 13-phase roadmap with per-phase status, the security requirements, and the planned verification-agent layer. This file is treated as non-negotiable: the mission ("research analyst, not trading bot") and the framing rule ("transparency, not accusation") override any other consideration.

1. **The prior code base** — the existing Python pipeline, the Streamlit

prototype, and the data already built (e.g. the QuiverQuant integration and the government-contract files). New work extends this rather than replacing it.

2.2 Working method (the "analyze before coding" loop)

The blueprint mandates a senior-architect discipline. Before any code is written, five questions are answered:

1. What phase are we currently in?
2. What dependencies already exist?
3. What modules should be created next?
4. Does the requested feature fit the roadmap?
5. How does the new code integrate into the existing ecosystem?

Three principles are enforced throughout:

- **Respect phase dependencies** — never build Phase 8 logic on broken Phase 3 data.
- **Preserve financial facts** — summaries and analyses must never lose numerical precision.
- **Behavior-preserving change** — when refactoring or hardening, function must not change; only readability or safety improves. This was verified mechanically (see 2.4).

2.3 The dual-mode architecture decision

A central architectural decision was to make the front end **dual-mode** so it runs both as a plain static site (for development and offline use) and as a server-backed app (for secure cloud hosting), with no code fork:

- **Static mode** — the page loads `*_data.js` bundles directly and gates access

with a soft client-side overlay (localStorage + a non-cryptographic `cyrb53` hash). Suitable for local `file://` use.

- **Server mode** — the same shell detects a FastAPI backend via ``GET`

`/api/healthz`, then performs real login against `/api/auth/` and fetches data from `/api/data/` (keys, Python, and datasets never reach the browser).

`webapp/js/boot.js` is the switch: it probes health, sets the mode, and injects the data and logic scripts **in the correct order** only after authentication. This was necessary because every logic module captures its data at evaluation

time (e.g. `const D = window.TF_DATA || {}`), so data globals must exist before the logic scripts run.

2.4 Behavior-preserving verification (the readability pass)

The founder requested the codebase be made readable to an external engineer — normal human comments instead of decorative `===== banners =====`, with textual and visual structure — **without changing what the code does**. To guarantee the "no behavior change" constraint, a **stack-based JavaScript normalizer** was used: it strips comments and whitespace while correctly handling string literals,

nested template literals (`${...}` containing backticks), and regex literals, then compares the normalized bytes before and after. After the pass, all nine reformatted modules were proven byte-identical in behavior. This is the standard applied to any future cleanup work.

2.5 Chronological engineering narrative

The following is the actual sequence of engineering work, with the problem, the diagnosis, and the resolution for each item. It doubles as a record of the non-obvious bugs that were found and fixed.

2.5.1 Behavioral onboarding quiz + settings styling

The signup flow was completed with a five-question behavioral profile quiz (age, experience, goal, time horizon, loss reaction) and the CSS for quiz and settings inputs. The quiz scores the user into a risk tolerance and a two-part `profile_id` used to tailor the briefing (see Part 6A, `computeProfile`).

2.5.2 Logout hang ("page unresponsive")

Symptom: logging out buffered back to the loading screen and Chrome reported "page unresponsive." **Diagnosis:** logout called `location.reload()`, which re-parsed the ~6 MB single-page app synchronously. **Fix:** logout was rewritten to re-lock the gate *in place* (clear session, re-add the lock overlay) without reloading the document.

2.5.3 The three-minute full hang on load

Symptom: the site "just reloaded for three minutes" — a complete freeze. **Diagnosis (via headless-Chrome bisection):** the `guard()` function installed a `MutationObserver` on `document.body` and its callback wrote to the body's class, which re-triggered the observer — an infinite microtask loop that starved the `load` event. **Fix:** the observer now watches only `childList`, disconnects itself while it performs its own writes, and reconnects afterward; a low-frequency `setInterval` provides a safety re-assert. The page loads normally.

2.5.4 Broken accessibility-statement link

The "accessibility statement" link pointed at `#` and led nowhere. It was wired to open a modal containing a full WCAG-style accessibility statement (`#allyStatementLink` → `openModal(...)` in `app.js`).

2.5.5 Market-hours API economy

Request: stop generating API calls after the market closes, *unless* the user is on a higher tier (the 30-minute and 15-minute refresh subscriptions), which should keep refreshing after hours. **Implementation:** `marketOpen()` computes US Eastern market minutes (Mon–Fri 09:30–16:00 via `toLocaleString` with `America/New_York`); `pausedForClose(tier)` pauses refresh only for slower tiers when the market is closed; the live-data refresh returns early and stamps "Markets closed" when paused. The same pause is enforced server-side in `data.py`.

2.5.6 The neon goodbye animation (no emojis)

Request evolution: change the logout farewell to "Goodbye, friend" with a smiley + wave; then explicitly *not* an Apple emoji wave but a custom hand-drawn SVG with a neon line outline, hollow and transparent inside; and a hard rule of **no emojis anywhere in the UI**.

Implementation: two inline SVGs (a smiley face and a waving hand) drawn as stroked paths with `currentColor`, given a neon glow via layered `drop-shadow` filters and gentle bob/wave keyframe animations.

2.5.7 De-vibe-coding the codebase

Per the founder's "developer's hell" feedback, all decorative banner comments were replaced with normal human comments and file headers, and the code was given textual/visual structure for an external reader — verified behavior- identical per 2.4. The `app.js` reformat had to be done by hand because its size exceeded a single agent's output budget.

2.5.8 Cloud-hosting security hardening

Request: prepare for cloud hosting with maximum security — no API keys exposed, the Python unreadable from the page, data files not downloadable, and the whole thing served from a server. This produced the FastAPI backend (Part 5), the data extractor that moves datasets into gitignored `private_data/`, the per-tier server-side rate limiting and market-hours pause, the security- headers/HTTPS/CORS middleware, brute-force throttling, JWT revocation, and the CI security workflow. The honest caveat was communicated and recorded: front-end JavaScript is always readable by the browser; only keys, Python, and bulk data can truly be hidden — and those are.

2.5.9 Login lands on the Dashboard; test account; beta key

Login was changed to always open the Dashboard (not the last-visited page) via a `window.TF.home` entry point. A `test1` account was seeded locally (in the gitignored `auth_seed.js`) and a beta invite key was provided for trials.

2.5.10 GitHub upload with secret-safety gate

Before pushing, every sensitive path (`.env`, `auth_seed.js`, `users.db`, `private_data/*`, `.claude/`) was confirmed gitignored *and* never previously tracked, and a key-pattern scan was run over everything committable. Only then was the commit pushed.

2.5.11 Signup age-gating + real email validation

Request: restrict under-age signups (accept 18+, or 13+ with parental permission) to limit liability, and reject nonsense emails like `sfdjk@fdsjkhdsdkhj.com` while still accepting legitimate **custom domains**. **Implementation:** a required consent checkbox on signup; a hard age floor of 13 enforced both client- and server-side; a stricter email-format regex; and a DNS-over-HTTPS domain-existence check (MX, falling back to A record) that rejects unregistered domains but accepts any real custom domain, failing **open** if the DNS service is unreachable so real users are never blocked.

2.5.12 The cybersecurity sub-agent and its training

A Claude Code sub-agent was created for attacker-perspective security review, then **trained** on a curated knowledge base extracted from the founder's security-research folder (see Part 9).

2.6 A note on the environment

Development occurred under real-world constraints worth recording for reproducibility: the working tree was at one point offloaded by iCloud ("Optimize Mac Storage") into zero-byte placeholder files, which had to be re-materialized before git could operate; the legacy `tf_env` virtualenv was broken by a stale conda path; and the sandbox had no outbound network. Where the pipeline could not be executed end-to-end in this environment, that is stated honestly in the relevant section rather than asserted as verified.

Part 2B — Statistical Methodology and Corrections

This part documents the **current statistical methodology** for the quantitative engine, incorporating the corrections from an in-progress statistical audit. It is written as the **updated, intended approach** so it can be reviewed before the implementing code merges. Where a correction replaces a prior method, the prior method is described only to make the change legible. Nothing here quotes a live point figure, because those change with the data and are computed at run time.

2B.0 Why this section exists

The engine was audited for statistical honesty: places where a formula returned a subtly wrong quantity, where a model number was presented as if it were a fact, or where an output read like advice. The corrections below make every number mean exactly what it says, state its lookback and assumptions, and stop short of anything that resembles a recommendation. The theme is **honesty over impressiveness**: fewer numbers, each one defensible.

2B.1 Expected return uses the true mean, not the median

- **Correction.** Expected return is computed as the true expected price,

$E[S_T] = S_0 * \exp(r * T)$, where S_0 is spot, r the drift, and T the horizon. Expected return is $E[S_T] / S_0 - 1$.

- **What was wrong.** The prior lognormal expression returned the **median**

terminal price, $S_0 * \exp((r - \sigma^2 / 2) * T)$, and labeled it the expected (mean) return. For a lognormal, the mean exceeds the median, so the old figure understated the expected value by the volatility-drag term. The label now matches the quantity.

2B.2 Probability of a move is an honest historical frequency

- **Correction.** "Probability of a +5% move" (and similar thresholds) is reported

as the **historical frequency**: over the past five years of **rolling 21-day windows**, the fraction of windows in which the asset gained 5% or more. The lookback is stated alongside the number. No confidence interval is attached, because a single realized-path frequency does not carry one.

- **What was wrong.** The prior figure was the risk-neutral $N(d_2)$ from the

Black-Scholes world. That is a pricing-measure probability, not a real-world chance of the event, and presenting it as "the probability you make 5%" is misleading. The historical frequency answers the question a person actually asks ("how often has this happened before?") and is transparent about its window.

2B.3 Kelly sizing removed from user-facing outputs

- **Correction.** Kelly "percent of portfolio" is **removed** from every user-facing

output, briefing, and LLM prompt. Kelly may remain an internal research quantity, but it is never shown to a user and never fed to the language model.

- **What was wrong.** Kelly fractions estimated from roughly one year of samples

are unstable, and a "put X percent of your portfolio here" number reads as individualized financial advice, which the platform must not give. Removing it keeps the product on the research-analyst side of the line.

2B.4 Backtest is walk-forward and point-in-time

- **Correction.** The backtest is **walk-forward** and free of look-ahead:

- The universe is **frozen at the backtest start** from **point-in-time S&P 500 constituents**, so membership is what it was then, not today's survivors.

- Data is split by **time** into train, validation, and test windows.
- Only **test-window** metrics are ever reported; train and validation results

are diagnostic and are not surfaced.

- **What was wrong.** Earlier backtests were contaminated by survivorship bias

(today's index members applied to the past) and by fitting and reporting on the same period. Those **old backtest numbers are archived and suppressed**; they no longer appear in any output.

2B.5 Recession trigger uses the Sahm rule

- **Correction.** The recession trigger uses the **Sahm rule**: it fires when the three-month average unemployment rate rises **0.5 percentage points** above its **lowest three-month average of the prior twelve months**. When an input series is short or partially available, the engine **discloses the reduced-data reweighting** it applied rather than silently substituting.
- **Why.** The Sahm rule is a published, well-understood real-time recession indicator with a clear, reproducible definition, which suits a transparency-first product better than an ad hoc composite.

2B.6 Politician scoreboard ranks on excess return

- **Correction.** The congressional-trading scoreboard ranks members on ****excess return versus SPY over the identical holding window**** (the member's return minus the market's return over the same dates), not on raw dollars.
- **Sells are framed honestly.** A sale is described as **avoided or foregone** exposure ("avoided a subsequent decline"), and is **never summed together with buy profit-and-loss** into one number, because a sale's counterfactual is not the same quantity as a realized gain.
- **Totals are ranges.** Dollar totals are shown as **ranges**, reflecting that disclosures report brackets rather than exact amounts.
- The political-intelligence framing rule (timing relationships, not intent) still governs every line of this output.

2B.7 One clean signal definition

- **Correction.** The trading signal is a ****single, clearly stated momentum definition****. Any "phantom" indicators (terms that were named in explanations but not actually computed, or double-counted) are removed. What the signal says it uses is exactly what it computes.

2B.8 Opportunity score separates outlook from suitability

- **Correction.** The opportunity score presents **two separate axes**:
 - **Outlook** — the model's directional read and its strength.
 - **Suitability** — how the idea fits the user's stated risk profile.
- These are shown side by side and are ****not multiplied into a single inflated composite****. The prior "times ten" composite conflated a market view with a personal-fit judgment and exaggerated small differences.

2B.9 Every LLM number is validated against source

- **Correction.** Any number that appears in language-model output is ****validated** by set membership against the source values^{**}: the figure must match a value that the deterministic engine actually produced, or it is rejected and regenerated. The language model may phrase numbers, never invent them.
- **Prompt change.** The prompt line that asked the model **"what stocks to buy"** is **removed**. The model translates and explains; it does not recommend purchases.

2B.10 Degradation flag on incomplete inputs

- **Correction.** When a briefing is assembled from **incomplete inputs** (a feed was stale, an API returned partial data, a metric could not be computed), a **degradation flag surfaces in the briefing itself**, so the reader knows the output was built on reduced data rather than assuming full coverage.

2B.11 Recent corrections, in one line each

- Expected return now uses the mean $S_0 * \exp(r T)$, not the lognormal median.
- Probability of a move is a stated-lookback historical frequency, not $N(d2)$.
- Kelly percent removed from all user-facing outputs and prompts.
- Backtest is walk-forward, point-in-time universe, test-window metrics only; old

numbers archived.

- Recession trigger is the Sahm rule with disclosed reduced-data reweighting.
- Scoreboard ranks on excess-vs-SPY; sells framed as avoided, never summed with

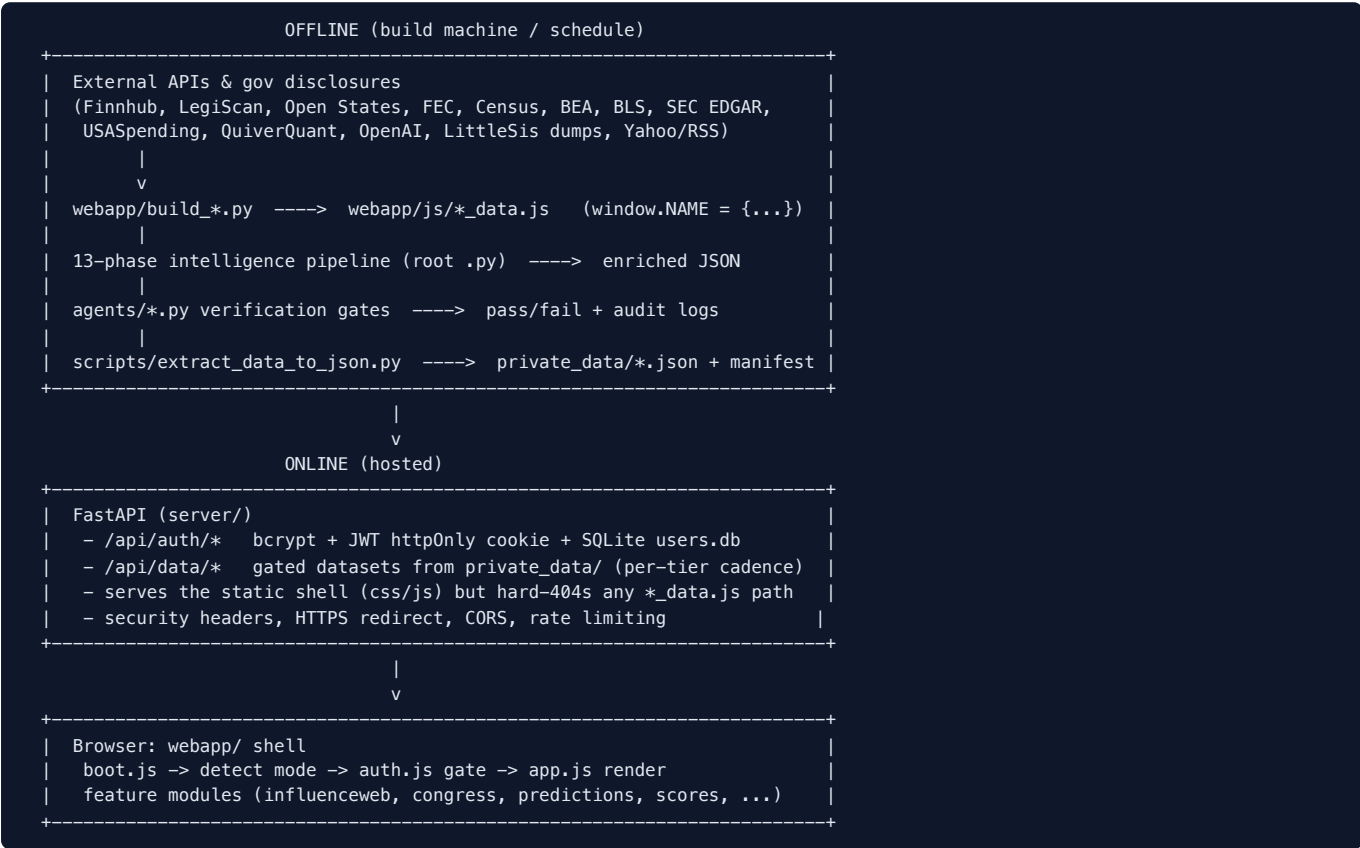
buys; totals as ranges.

- One clean momentum signal, no phantom indicators.
- Opportunity score splits outlook from suitability, no times-ten composite.
- LLM numbers validated by set membership; the "what to buy" prompt line removed.
- A degradation flag appears on any briefing built from incomplete inputs.

Part 3 — System Architecture

3.1 High-level topology

ThinkFree separates a heavy **offline data/intelligence pipeline** (runs on a build machine or schedule, in Python) from a **lightweight delivery layer** (what the user touches). The browser receives only a UI shell; everything sensitive stays server-side in the hosted configuration.



3.2 The two delivery modes

The same `webapp/` shell runs in two modes with no code fork. `boot.js` decides at startup by probing `GET /api/healthz`.

Concern	Static mode	Server mode		
Trigger	No backend responds to <code>/api/healthz</code>	Backend responds with <code>{ready:true}</code>		
Auth	localStorage + <code>cyrb53</code> soft gate	<code>/api/auth/*</code> , bcrypt + JWT cookie		
Data	<code>js/*_data.js</code> injected directly	<code>/api/data/bundle</code>	state	live <code>fetch</code> d, then <code>Object.assign(window, json)`</code>
Secrets	none present (data is public)	keys/Python/data never sent to browser		
Use case	local dev / offline demo	secure beta + production		

The hard ordering constraint: logic modules read their data at eval time, so `boot.js` injects data globals *before* the logic scripts in both modes.

3.3 Request lifecycle (server mode)

1. Browser GET /
2. boot.js GET /api/healthz
3. boot.js GET /api/auth/me
4. User submits login
- > FastAPI serves index.html (the shell)
-> {ready, mode} => mode = "server"
-> 401 (not logged in) => show gate
-> POST /api/auth/login

```

5. boot.ensureAppLoaded()      bcrypt verify -> set httpOnly JWT cookie
                                -> GET /api/data/bundle (+ /api/data/live)
                                Object.assign(window, json); inject logic JS
6. app.js init()              -> render Dashboard (window.TF.home)
7. State Legislature view      -> GET /api/data/state (lazy)
8. Periodic refresh (per tier) -> GET /api/data/live (429 / {paused} handled)

```

3.4 The 13-phase intelligence pipeline (logical view)

The blueprint defines a 13-phase roadmap. The pipeline is orchestrated by `controller.py` (Phase 12) and culminates in `chef_gpt.py` (Phase 11), which only runs after the upstream phases complete for a cycle.

Phase	Name	Primary file(s)	Status per blueprint
1	User personalization	<code>Profile/</code> , quiz in <code>auth.js</code>	Complete
2	Financial data scraping	<code>Z_0ld_versions/News/</code> , RSS sources	Complete
3	Data extraction & structuring	<code>phase3_extraction.py</code>	Partial
4	Article scoring & clustering	<code>phase4_clustering.py</code> , <code>Profile/article_clustering.py</code>	Mostly complete
5	GPT summarization & validation	<code>Profile/GPT_article_clustering_summary.py</code>	Partial
6	Economic reasoning engine	<code>GPT_Economy/economic_engine.py</code> , <code>Reasoning_Report.py</code>	Partial (FAISS)
6.5	Political intelligence	<code>congress_bills.py</code> , <code>politician_performance.py</code> , <code>webapp/build_*</code>	Built (web)
7	Historical correlation	<code>historical_correlation.py</code>	Implemented
8	Opportunity quantification	<code>opportunity_score.py</code> , <code>quantlib_metrics.py</code>	Implemented
9	Backtesting engine	<code>Module_2_Technical_Analysis/phase_4_backtrader.py</code>	Active dev
10	Portfolio tracker & dashboard	<code>portfolio_tracker.py</code> , <code>dashboard/app.py</code>	Prototype
11	Chef GPT (final layer)	<code>chef_gpt.py</code>	Implemented
12	Control logic / human-in-the-loop	<code>controller.py</code>	Implemented
13	Recession indicator engine	<code>recession_signals.py</code>	Implemented

Each phase is documented function-by-function in Parts 7A and 7B.

3.5 Verification-agent layer (cross-cutting)

A set of independent Python agents validate concerns at pipeline checkpoints and return a pass/fail verdict; a failed verdict halts the step and is logged. These are documented in Part 8. The blueprint also defines additional planned agents (financial-fact, economic-reasoning, recession-signal) not yet implemented; the gap is noted in Part 8.

3.6 The AI sub-agent layer (development-time)

Separate from the runtime Python agents, a **Claude Code sub-agent** assists development: an attacker-perspective cybersecurity reviewer defined in `.claude/agents/cybersecurity-agent.md` and trained on an extracted knowledge base in `.claude/agents/cybersecurity-kb/`. This is documented in full in Part 9.

3.7 Repository map

```

ThinkFree-main/
  CLAUDE.md           # the canonical blueprint
  DEPLOY.md           # hosting + security guide
  requirements.txt     # pipeline Python deps
  controller.py        # Phase 12 orchestrator
  chef_gpt.py          # Phase 11 final briefing
  recession_signals.py # Phase 13
  opportunity_score.py / quantlib_metrics.py # Phase 8
  historical_correlation.py # Phase 7
  phase3_extraction.py / phase4_clustering.py # Phases 3-4
  portfolio_tracker.py # Phase 10
  congress_bills.py / politician_performance.py # Phase 6.5
  validate_env.py      # env/config validation
  agents/              # runtime verification agents (Part 8)

```

```
GPT_Economy/          # economic reasoning engine (Part 7B)
Profile/              # personalization + clustering (Part 7B)
Module_2_Technical_Analysis/ # TA + backtesting (Part 7B)
server/              # FastAPI backend (Part 5)
scripts/extract_data_to_json.py # data extractor (Part 4)
webapp/
  index.html          # the shell
  css/styles.css
  js/
  build_*.py          # boot/auth/app + feature modules + *_data.js (Parts 6A/6B/12)
                        # offline data build scripts (Part 4)
private_data/         # gitignored gated datasets (server mode)
dashboard/app.py       # legacy Streamlit prototype (kept)
iphone-app/           # SwiftUI client (not build-verified)
.claude/agents/        # the cybersecurity sub-agent + KB (Part 9)
```

Part 4 — Offline Data Build Pipeline

This document is an exhaustive technical reference for every Python script that constitutes the ThinkFree offline data build pipeline. These scripts fetch external data and emit the `webapp/js/*_data.js` bundles (each assigned to a `window.NAME` global) consumed by the static front-end, as well as `private_data/` JSON files served by the authenticated FastAPI backend.

All scripts live under `/Users/allanaziz/Desktop/ThinkFree/ThinkFree-main/webapp/` unless noted otherwise.

Summary Table

Script	Data Source	Output File	Env Var / API Key
<code>build_data.py</code>	Local JSON outputs (ThinkFree pipeline)	<code>webapp/js/data.js</code> (<code>window.TF_DATA</code>)	None (reads project JSON)
<code>build_states.py</code>	FRED (St. Louis Fed)	<code>webapp/js/states_data.js</code> (<code>window.STATES_DATA</code>)	<code>FRED_API_KEY</code>
<code>build_census.py</code>	U.S. Census Bureau ACS 1-Year API	<code>webapp/js/states_data.js</code> (enriches)	<code>CENSUS_API_KEY</code>
<code>build_bea.py</code>	BEA Regional Price Parities API	<code>webapp/js/states_data.js</code> (enriches)	<code>BEA_API_KEY</code>
<code>build_bls.py</code>	BLS Regional CPI API v2	<code>webapp/js/states_data.js</code> (enriches)	<code>BLS_API_KEY</code>
<code>build_fec.py</code>	OpenFEC API (api.open.fec.gov)	<code>webapp/js/fec_data.js</code> (<code>window.FEC_DATA</code>)	<code>FEC_API_KEY</code>
<code>build_influence.py</code>	LittleSis bulk dump (.gz)	<code>webapp/js/influence_data.js</code> (<code>window.IW_DATA</code>)	None (local files)
<code>build_legiscan.py</code>	LegiScan Public API	<code>webapp/js/legiscan_data.js</code> (<code>window.LEGISCAN_DATA</code>)	<code>LEGISCAN_API_KEY</code>
<code>build_member_bills.py</code>	Congress.gov API v3	<code>webapp/js/member_bills.js</code> (<code>window.MEMBER_BILLS</code>)	<code>CONGRESS_API_KEY</code>
<code>build_news_intel.py</code>	OpenAI Chat API (via local pre-processing)	<code>webapp/js/news_intel.js</code> (<code>window.NEWS_INTEL</code>)	<code>OPENAI_API_KEY</code>
<code>build_nonprofits.py</code>	ProPublica Nonprofit Explorer API	<code>webapp/js/nonprofit_data.js</code> (<code>window.NP_DATA</code>)	None (public API)
<code>build_openstates.py</code>	Open States / Plural API v3	<code>webapp/js/openstates_data.js</code> (<code>window.OPENSTATES_DATA</code>)	<code>OPENSTATES_API_KEY</code>
<code>build_prices.py</code>	Finnhub REST API	<code>webapp/js/prices_data.js</code> (<code>window.PRICES_DATA</code>)	<code>FINNHUB_API_KEY</code>
<code>build_quant.py</code>	yfinance (via <code>quantlib_metrics.py</code>) + <code>ta</code> library	<code>webapp/js/quant_data.js</code> (<code>window.QUANT_DATA</code>)	None (yfinance)
<code>build_relationships.py</code>	U.S. Senate LDA API (anonymous)	<code>webapp/js/relationships_data.js</code> (<code>window.RELATIONSHIPS</code>)	None (anonymous)
<code>build_sec.py</code>	sec-api.io REST API	<code>webapp/js/sec_data.js</code> (<code>window.SEC_DATA</code>)	<code>SECAPI_IO_KEY</code>
<code>build_secbulk.py</code>	SEC EDGAR bulk ZIPs (local)	<code>webapp/js/secbulk_data.js</code> (<code>window.SECBULK_DATA</code>)	None (local files)
<code>build_sp500.py</code>	GitHub datasets/s-and-p-500-companies CSV	<code>webapp/js/sp500_data.js</code> (<code>window.SP500</code>)	None
<code>build_usaspending.py</code>	USASpending.gov API v2	<code>webapp/js/usaspending_data.js</code> (<code>window.USA_DATA</code>)	None (public API)
<code>fetch_politician_photos.py</code>	unitedstates/images GitHub Pages	<code>webapp/assets/politicians/{bioguide}.jpg</code>	None
<code>minify_data.py</code>	Local <code>webapp/js/*.js</code> files	Overwrites all data <code>.js</code> files in-place	None

Script	Data Source	Output File	Env Var / API Key
<code>scripts/extract_data_to_json.py</code>	Local <code>webapp/js/*_data.js</code> files	<code>private_data/*.json</code> + <code>private_data/manifest.json</code>	None

Pipeline Execution Order

The scripts form a dependency chain. The correct build sequence is:

```

build_sp500.py
build_states.py          # requires FRED
build_census.py          # enriches states_data.js; requires Census
build_bea.py             # enriches states_data.js; requires BEA
build_bls.py             # enriches states_data.js; requires BLS
build_influence.py       # requires LittleSis .gz files in ~/Downloads
build_secbulk.py         # requires SEC EDGAR ZIPs in ~/Downloads + influence_data.js
build_usaspending.py     # requires sp500_data.js, prices_data.js, influence_data.js
build_sec.py            # requires influence_data.js; SEC_API_IO_KEY
build_fec.py            # requires politician_performance.json; FEC_API_KEY
build_data.py           # requires all project JSON outputs (pipeline phases 1-13)
build_legiscan.py       # LEGISCAN_API_KEY
build_openstates.py     # OPENSTATES_API_KEY; enriches LegiScan roster
build_member_bills.py   # requires data.js (for bioguide IDs); CONGRESS_API_KEY
build_nonprofits.py     # no key
build_relationships.py  # requires influence_data.js, prices_data.js, sp500_data.js
build_prices.py         # requires sp500_data.js, secbulk_data.js, influence_data.js,
                        # usaspending_data.js, data.js; FINNHUB_API_KEY
build_quant.py         # requires data.js, prices_data.js; yfinance + ta
build_news_intel.py     # requires data.js, quant_data.js, prices_data.js,
                        # signal_output_phase3.json, sector_summaries.json,
                        # economic_reasoning_summary.json; OPENAI_API_KEY
fetch_politician_photos.py # requires politician_parties.json
minify_data.py          # final step; run before deploy
scripts/extract_data_to_json.py # run after all JS files written; creates private_data/

```

build_data.py

File: `webapp/build_data.py` **Purpose:** The central aggregator. Reads all upstream ThinkFree pipeline JSON outputs (recession signals, politician trades, congress bills, news summaries, historical events, portfolio snapshot, opportunity scores) and emits `webapp/js/data.js` as `window.TF_DATA = {...}`. This is the primary data bundle consumed by the static front-end and is a prerequisite for most other build scripts.

Data source: Local project JSON files only — no external API calls.

Env vars / API keys: None.

Output: `webapp/js/data.js` — `window.TF_DATA`

Output shape (top-level keys)

```

{
  "generated_at": "<ISO-8601 UTC>",
  "user": {"name": "Allan", "plan": "Free Plan"},
  "disclaimer": "<STOCK Act disclaimer string>",
  "market_ticker": [...],
  "recession": {...},
  "portfolio": {...},
  "politicians": [...],
  "recent_trades": [...],
  "sectors": [...],
  "tickers_opp": [...],
  "parallels": [...],
  "events": [...],
  "bills": [...],
  "correlation": {...},
  "news": [...],
  "means": [...],

```

```
"everyday": [...]  
}
```

Input files read

File (relative to repo root)	Used for
politician_performance.json	Politician trade summary, trade detail, disclaimer
politician_parties.json	Bioguide ID map (for photo lookups)
portfolio_snapshot.json	Portfolio metrics and positions
recession_signals_output.json	Recession score, factors, raw indicators
opportunity_scores.json	Sector and ticker opportunity scoring
historical_parallelsl.json	Historical analog periods
news_output/summaries.json	Summarized news articles
news_output/clustered_summaries.json	Clustered article summaries ("what this means for you")
news_output/enriched_news_sentiment.json	Source, sentiment, credibility for each article
congress_bills.json	Bill correlations and correlated trades
historical_events_db.json	20-event historical library (Tulip mania → AI boom)

Functions

load(name, default=None)

Loads a JSON file by path relative to the repo root. Returns `default` (or `{}`) on any error, so missing upstream files degrade gracefully instead of crashing the build.

money(n: float) -> str

Formats a raw number into a human-readable currency string (`$1.23B` , `$4.5M` , `$1.2K` , `$123`). Used throughout the data to pre-format all monetary values for display.

source_credibility(source: str) -> float

Looks up a numeric credibility score (0–1) for a news source string against a hardcoded `CREDIBILITY` dict. Scores range from 0.97 (Reuters/AP) down to 0.38 (AccessWire). Sources scoring below `MIN_CREDIBILITY = 0.45` are dropped entirely from the news feed.

credibility_tier(score: float) -> str

Maps a credibility score to a display label: `"Trusted"` (≥ 0.9), `"Reliable"` (≥ 0.75), `"Mixed"` (≥ 0.6), `"Low / Opinion"` (≥ 0.45), `"Unreliable"` (< 0.45).

_clean_str(s: str) -> str

Strips em dashes and en dashes from a string (replacing with commas or hyphens), collapses double spaces, and removes orphaned punctuation. The front-end prohibits the `–` character.

strip_dashes(obj)

Recursively walks any nested JSON structure (dict, list, str) and applies `_clean_str` to all string values. Called on the entire `data` dict just before writing.

ordinal(n) -> str

Converts an integer to its ordinal string (`1st` , `2nd` , `3rd` , `119th` , etc.) for use in Congress.gov URLs.

bill_url(bill_id: str) -> str

Converts a ThinkFree bill ID of the form `"HR1234-119"` or `"S2393-119"` into a congress.gov canonical URL. Supports HR, S, HRES, SRES, HJRES, SJRES, HCONRES, SCONRES bill types.

quiver_url(name: str, bg: str) -> str

Builds a QuiverQuant politician trading URL by URL-encoding the politician's name and appending their bioguide ID.

`make_donors(name: str) -> list`

Generates a deterministic (seeded by the character sum of the politician's name) list of 6 sample donor PAC records. **These are sample data, clearly labeled as such in the UI**, intended to be replaced with real OpenSecrets/FEC API data when a key is available.

`make_supporters(name: str) -> list`

Generates a deterministic list of 4 sample outside-spending group records (Support/Oppose stance). Same seeding approach as

`make_donors`.

`lobby_for(sectors: list) -> list`

Maps a list of sector names to associated industry lobbying groups (from the hardcoded `_SECTOR_LOBBY` dict) and returns up to 6 `{"org": ..., "sector": ...}` records. Falls back to `"Multi-industry coalition"` if no sector match is found.

`plain_bill(title: str, sectors, tickers) -> str`

Generates a best-effort plain-English explanation of what a bill does, based on keyword matching in the title (e.g., "chips" → "boosts U.S. semiconductor manufacturing"). Returns a fallback explanation referencing the matched sectors and tickers if no keyword matches.

`build_everyday(raw: dict, oil_price: float, rec: dict) -> list`

Translates market and economic indicators into a list of plain-English "What This Means for You" items across eight categories: Rent & Mortgages, Credit Cards & Auto Loans, Gas Prices, Groceries, Utilities & Electricity, Jobs & Hiring, Consumer Goods & Tariffs, Savings & CDs. Each item has `category`, `dir` ("up" / "down" / "neutral"), `impact` (one-line headline), and `detail` (2–3 sentence plain-English explanation). This is the core ThinkFree translation-for-everyday-people feature.

`enrich(nm: str, p_meta: dict) -> dict`

Builds a rich per-politician record including: a plain-English narrative summary, top traded tickers by count, estimated portfolio value and net-worth range, donor PAC list, outside-spending group list, related correlated bills, a full chronological trade timeline (with associated bill, days-delta, and P&L), and a count of trades that preceded a bill action ("influenced count").

`build() -> None`

The main orchestration function. Calls all of the above in sequence, assembles the final `data` dict, applies `strip_dashes`, and writes `webapp/js/data.js`.

Credibility-filtered news assembly: For each article in `summaries.json`, the function joins it with `enriched_news_sentiment.json` by URL, computes a credibility-weighted impact score ($cred \times (0.5 + |compound|) \times (1 + \min(richness, 1))$), drops sources below `MIN_CREDIBILITY`, and round-robins across tickers for diversity before sorting by `(credibility, impact)` descending. Up to 160 articles are kept.

Conflict index: Computes a 0–100 "Conflict Index" from three components: breadth (45% — share of related bills that had trades opened beforehand), intensity (35% — share of correlated trades placed before the action), and lead (20% — advance positioning capped at 90 days).

`build_states.py`

File: `webapp/build_states.py` **Purpose:** Fetches per-state economic indicators from the FRED API and computes two ThinkFree-specific scores — Constituent Prosperity Score and Constituent Pressure Score — for all 50 states plus DC. Writes the initial `webapp/js/states_data.js`. Must be run first in the four-part states chain (`build_states` → `build_census` → `build_bea` → `build_bls`).

Data source: FRED (Federal Reserve Bank of St. Louis) — api.stlouisfed.org/fred/series/observations

Env vars / API keys: `FRED_API_KEY` (read from `.env`)

Output: `webapp/js/states_data.js` — `window.STATES_DATA`

Rate limiting: `time.sleep(0.15)` between states (≈340 calls for 51 jurisdictions × 4 series each). No explicit rate limit documented by FRED; 0.15s provides conservative headroom.

FRED Series Used

Series pattern	Metric
{ST}UR	Unemployment rate (4 observations)
{ST}STHPI	All-transactions house price index (8 observations)
{ST}PCPI	Per-capita personal income (6 observations)
MEH0INUS{ST}A672N	Real median household income (3 observations)

Output shape (`byState` entry)

```
{
  "name": "California",
  "unemployment": 4.8,
  "house_price_index": 542.1,
  "house_price_growth": 3.2,
  "per_capita_income": 74200,
  "income_growth": 1.8,
  "median_household_income": 85300,
  "prosperity_score": 62,
  "pressure_score": 41
}
```

Functions

fred(series: str, limit: int = 8) -> list

Fetches observations from the FRED API for a given series ID, returning up to `limit` observations in descending order. Returns an empty list on any error.

latest(obs: list) -> tuple[float | None, str | None]

Returns the first valid `(float(value), date)` pair from a FRED observations list, skipping non-numeric entries.

yoy(obs: list) -> float | None

Computes the year-over-year percentage change using the most recent value vs. approximately 4 quarters back (index position 4 in descending order). Returns `None` if fewer than 5 observations are available.

clamp(v, lo=0, hi=100) -> float

Clamps a value to `[lo, hi]`.

main() -> None

Iterates over all 51 jurisdictions, fetches 4 FRED series per state, computes prosperity and pressure scores (relative scoring across the full 51-jurisdiction set), and writes `states_data.js`.

Prosperity score formula: `(income_rank × 45) + (clamp((7 - unemp) / 7 × 35, 0, 35)) + (clamp(incg × 3, -10, 20))`

Pressure score formula: `clamp((hpi_growth - income_growth) × 6, 0, 60) + clamp((unemp - 3.5) × 6, 0, 25) + clamp(hpi_growth × 1.2, 0, 15)`

build_census.py

File: `webapp/build_census.py` **Purpose:** Enriches `webapp/js/states_data.js` with housing and affordability data from the U.S. Census Bureau ACS 1-Year API. Adds median home value, median rent, median household income, poverty rate, population, price-to-income ratio, affordability score, and rent burden percentage. Then recomputes prosperity and pressure scores incorporating affordability. Must run after `build_states.py`.

Data source: Census ACS 1-Year API — `api.census.gov/data/{YEAR}/acs/acs1` (year hardcoded to 2023)

Env vars / API keys: `CENSUS_API_KEY` (read from `.env`)

Output: `webapp/js/states_data.js` — `window.STATES_DATA` (in-place enrichment)

Rate limiting: One API call for all states simultaneously (single batch request using `for=state:*`). No `sleep` needed.

ACS Variables Fetched

ACS variable	Field added
B19013_001E	median_household_income
B25077_001E	median_home_value
B25064_001E	median_rent
B17001_002E	poverty_count (used to derive poverty_rate)
B01003_001E	population

Derived fields

Field	Formula
poverty_rate	$\text{poverty_count} / \text{population} \times 100$
price_to_income	$\text{median_home_value} / \text{median_household_income}$
affordability	$\text{clamp}((9 - (\text{home_value} / \text{income})) / 7 \times 100, 0, 100)$ — 100 = most affordable ($\leq 2 \times$ income), 0 = least ($\geq 9 \times$)
rent_burden_pct	$\text{median_rent} \times 12 / \text{median_household_income} \times 100$

Functions

main() -> None

Reads the current `states_data.js`, parses the JSON payload, makes a single Census API call for all states using FIPS-to-postal mapping (FIPS dict), derives the computed fields above, recomputes prosperity and pressure scores with affordability included, and rewrites `states_data.js`.

Updated prosperity score formula: $(\text{affordability} \times 0.30) + (\text{clamp}((7 - \text{unemp}) / 7 \times 30, 0, 30)) + (\text{clamp}((18 - \text{poverty_rate}) / 18 \times 25, 0, 25)) + \text{clamp}(\text{incg} \times 2, -8, 15)$

Updated pressure score formula: $((100 - \text{affordability}) \times 0.45) + \text{clamp}((\text{hpg} - \text{incg}) \times 4, 0, 30) + \text{clamp}((\text{rent_burden_pct} - 25) \times 1.5, 0, 25)$

build_bea.py

File: `webapp/build_bea.py` **Purpose:** Adds BEA Regional Price Parities (RPP, US = 100) to each state in `states_data.js`. RPP measures cost of living relative to the national average. Also computes `real_purchasing_power` (nominal income adjusted for local price level) and folds cost of living into the prosperity and pressure scores. Must run after `build_census.py`.

Data source: BEA Regional API — `apps.bea.gov/api/data?datasetname=Regional&TableName=SARPP&LineCode=1&GeoFips=STATE` (year hardcoded to 2023)

Env vars / API keys: `BEA_API_KEY` (read from `.env`)

Output: `webapp/js/states_data.js` — `window.STATES_DATA` (in-place enrichment)

Rate limiting: One API call covers all states. No `sleep` needed.

Fields added per state

Field	Description
cost_of_living	RPP value (US = 100; e.g., Hawaii ≈ 118 , Mississippi ≈ 84)
real_purchasing_power	$\text{median_household_income} / (\text{rpp} / 100)$ — inflation-adjusted income

Score adjustment logic

- `pressure_score` += $\text{clamp}((\text{rpp} - 100) \times 1.2, 0, 18)$ — above-average cost of living adds up to 18 pressure points.
- `prosperity_score` += $\text{clamp}((\text{real_purchasing_power} - 70000) / 70000 \times 10, -8, 8)$ — real income vs. \$70k baseline.

Functions

main() -> None

Reads `states_data.js`, fetches BEA SARPP data for all states, maps state names to postal abbreviations using `NAME_TO_AB` dict, adds RPP and derived fields, adjusts scores, and rewrites the file.

`build_bls.py`

File: `webapp/build_bls.py` **Purpose:** Adds year-over-year CPI inflation by U.S. Census region (Northeast, Midwest, South, West) to each state in `states_data.js` and folds regional inflation into the Pressure score. Must run after `build_bea.py`.

Data source: BLS Public API v2 — `api.bls.gov/publicAPI/v2/timeseries/data/` (POST, JSON body). Series IDs 2023–2025, 4 regional series.

Env vars / API keys: `BLS_API_KEY` (read from `.env`)

Output: `webapp/js/states_data.js` — `window.STATES_DATA` (in-place enrichment, final states enrichment step)

Rate limiting: One API call fetches all 4 regional series in a single POST. No `sleep` needed.

Regional CPI Series

Region	BLS Series ID
Northeast	<code>CUUR0100SA0</code>
Midwest	<code>CUUR0200SA0</code>
South	<code>CUUR0300SA0</code>
West	<code>CUUR0400SA0</code>

Fields added per state

Field	Description
<code>region</code>	Census region name (<code>"Northeast"</code> , <code>"Midwest"</code> , <code>"South"</code> , <code>"West"</code>)
<code>inflation</code>	YoY CPI % change for the state's region

Score adjustment

`pressure_score += clamp((inflation - 2.0) × 6, 0, 16)` — inflation above the 2% target adds up to 16 pressure points.

Functions

`main()` -> `None`

Reads `states_data.js`, fetches BLS regional CPI (YoY computed from index position 0 vs. 12), maps each state to its region via `STATE_REGION` dict, adds `region` and `inflation` fields, adjusts pressure scores, and rewrites the file with the final combined source label.

`build_fec.py`

File: `webapp/build_fec.py` **Purpose:** Fetches real campaign-finance data for every politician tracked in ThinkFree (from `politician_performance.json`) via the OpenFEC API. Retrieves total receipts, individual contributions, PAC contributions, disbursements, cash on hand, and (for the top 30 by estimated P&L) a top-contributor breakdown by employer. Writes `webapp/js/fec_data.js`.

Data source: OpenFEC API — `api.open.fec.gov/v1/` (endpoints: `candidates/search/`, `candidate/{cid}/totals/`, `schedules/schedule_a/by_employer/`)

Env vars / API keys: `FEC_API_KEY` (read from `.env`)

Output: `webapp/js/fec_data.js` — `window.FEC_DATA`

Rate limiting: `time.sleep(0.25)` between each politician (≈ 240 calls/min on 2 calls per politician). The FEC API has rate limits not publicly documented; 0.25s is conservative.

Output shape (`byName` entry)

```
{
  "CandidateName": {
```

```

    "candidate_id": "H8CA01234",
    "fec_name": "PELOSI, NANCY",
    "party": "DEM",
    "office": "U.S. House",
    "url": "https://www.fec.gov/data/candidate/H8CA01234/",
    "cycle": 2024,
    "receipts": 4500000,
    "receipts_fmt": "$4.5M",
    "from_individuals": 3200000,
    "from_individuals_fmt": "$3.2M",
    "from_pacs": 800000,
    "from_pacs_fmt": "$800K",
    "disbursements": 3800000,
    "disbursements_fmt": "$3.8M",
    "cash": 1200000,
    "cash_fmt": "$1.2M",
    "top_employers": [
      {"employer": "University of California", "total_fmt": "$42K"}
    ]
  }
}

```

Functions

get(path: str, **params) -> dict

Appends `api_key` to params and makes a GET request to the OpenFEC base URL. Returns parsed JSON.

money(n) -> str | None

Formats a number to a display currency string (B/M/K suffixes). Returns `None` for falsy input.

politicians() -> list

Reads `politician_performance.json` from the repo root and returns the `politician_summary` list.

find_candidate(name: str, chamber: str, state: str) -> dict | None

Searches the FEC `candidates/search/` endpoint for a politician by name and optionally by office (`H` for House, `S` for Senate). Sorts results by state match, presence of principal committees, and recency of last filing. Returns the best candidate match or `None`.

main() -> None

Loads all politicians, sorts by estimated P&L (descending) to identify the top 30 for employer breakdown, then for each politician: finds the FEC candidate record, fetches cycle totals, and optionally fetches top-employer breakdown. Writes `fec_data.js`.

build_influence.py

File: `webapp/build_influence.py` **Purpose:** Extracts the slice of the LittleSis relationship database that connects to companies already referenced in ThinkFree (tickers found in `data.js` bills, trades, and correlation data). Streams two large compressed files (`entities.json.gz` ~73 MB, `relationships.json.gz` ~105 MB) via `ijson` to avoid loading them entirely into memory, and writes only the matching board members, executives, owners, and donors.

Data source: LittleSis bulk data dump — `entities.json.gz` and `relationships.json.gz` (expected in `~/Downloads/`). No network call at runtime.

Env vars / API keys: None. Files must be manually downloaded.

Local file requirements:

- `~/Downloads/entities.json.gz`
- `~/Downloads/relationships.json.gz`

Output: `webapp/js/influence_data.js` — `window.IW_DATA`

Rate limiting: N/A (local file reads). Scans all entities (~millions of records) and relationships sequentially.

LittleSis Category IDs Used

Category	ID
CAT_POSITION	1 (board member / executive)
CAT_DONATION	5
CAT_OWNERSHIP	10
CAT_LOBBYING	7 (defined but not currently extracted)

Output shape (`companies` entry)

```
{
  "AAPL": {
    "name": "Apple Inc.",
    "entity_id": 12345,
    "blurb": "...",
    "website": "https://apple.com",
    "domain": "apple.com",
    "employees": 165000,
    "revenue": 383285000000,
    "fedspending_id": "...",
    "lda_registrant_id": "...",
    "board": [{"name": "Tim Cook", "title": "CEO", "current": true}],
    "executives": [...],
    "owners": [...],
    "donors_in": [...]
  }
}
```

Caps: board ≤ 14 members, executives ≤ 10 , owners ≤ 8 , donors_in ≤ 8 .

Functions

`norm(t: str) -> str`

Normalizes a ticker by stripping all non-alphanumeric characters and uppercasing — used for fuzzy ticker matching against LittleSis `PublicCompany.ticker` field.

`our_tickers() -> set`

Reads `webapp/js/data.js` and extracts all ticker symbols referenced in bills, recent trades, and correlation top bills. Returns a set of ticker strings.

`domain_of(url: str) -> str`

Extracts the bare domain name from a URL (strips `http(s)://www.`).

`main() -> None`

Two-pass algorithm:

- Pass 1 (entities):** Streams `entities.json.gz` with `ijson`, identifies entities with a `PublicCompany.ticker` that matches one of the ThinkFree tickers, records entity ID $\rightarrow$ ticker mapping and company metadata.
- Pass 2 (relationships):** Streams `relationships.json.gz` with `ijson`, for each relationship involving a matched company entity extracts position (board/executive), ownership, or donation data and appends to the company's record.

Writes `influence_data.js` with all matched companies.

`build_legiscan.py`

File: `webapp/build_legiscan.py` **Purpose:** Fetches real state-legislative data for all 50 states plus DC from the LegiScan Public API. For each jurisdiction: retrieves the current session, the full bill master list with session-wide status breakdown, the most recently-active economically-relevant bills (tagged with ThinkFree topics), deep bill details (sponsors + roll-call votes) for the top 6 most recent relevant bills, and the full legislator roster with party/chamber composition. Writes `webapp/js/legiscan_data.js`.

Data source: LegiScan API — `api.legiscan.com/` (operations: `getSessionList`, `getMasterList`, `getBill`, `getSessionPeople`)

Env vars / API keys: `LEGISCAN_API_KEY` (read from `.env`)

Output: `webapp/js/legiscan_data.js` — `window.LEGISCAN_DATA`

Rate limiting: `SLEEP = 0.3` seconds between API calls. Budget: the build spends approximately 51×3 calls (session list + master list + people) plus up to 51×6 deep-dive calls $\approx \sim 460$ total, well within the 30,000-query monthly free-tier limit.

Economic Topic Tags

Bills are matched against 10 topic categories by keyword scanning the bill title:

Topic	Sample Keywords
Taxes	tax, taxation, revenue, levy
Budget	appropriat, budget, spending, fund transfer
Housing	housing, rent, tenant, landlord, eviction, mortgage
Wages & Labor	minimum wage, wage, labor, employ, overtime, union
Healthcare	health, medicaid, medicare, insurance, prescription
Energy & Utilities	energy, utility, electric, natural gas, fuel
Education	education, school, tuition, student, university
Consumer & Credit	consumer, credit, loan, lending, debt, payday
Benefits	unemployment, snap, benefit, pension, retirement
Business	business, small business, commerce, license, regulation

Output shape (`byState` entry)

```
{
  "CA": {
    "state": "CA",
    "name": "California",
    "session": {
      "id": 1789,
      "name": "2023-2024 Regular Session",
      "year_start": 2023,
      "year_end": 2024,
      "special": false
    },
    "bill_total": 4182,
    "status_breakdown": {"Introduced": 2100, "Engrossed": 800, ...},
    "relevant_count": 40,
    "bills": [
      {
        "bill_id": 1234567,
        "number": "SB 567",
        "url": "...",
        "title": "Tenant Protections Act",
        "status": "Engrossed",
        "status_code": 2,
        "last_action_date": "2024-03-15",
        "last_action": "Read second time",
        "topics": ["Housing", "Wages & Labor"],
        "sponsors": [...],
        "votes": [...]
      }
    ],
    "legislator_total": 118,
    "composition": {"Senate": {"D": 31, "R": 8, "I": 1, "Other": 0}, "House": {...}},
    "legislators": [...]
  }
}
```

Functions

`get(**params) -> dict`

Makes a GET request to the LegiScan API with the given parameters plus `key`. Raises `RuntimeError` if the response status is not `"OK"`.

`topics_for(text: str) -> list`

Lowercases the input and checks for keyword matches against `TOPICS`. Returns a list of matching topic strings.

`pick_session(sessions: list) -> dict`

Selects the most recent regular (non-special) session from the LegiScan session list. Falls back to most recent overall session if no regular session exists.

`clean_people(raw: list) -> list`

Filters out committee sponsors (`committee_sponsor == 1`) and roles other than "Sen" / "Rep", then trims each legislator record to: `people_id`, `name`, `party`, `chamber`, `district`, `ballotpedia`, `votesmart_id`.

`composition(legislators: list) -> dict`

Counts D/R/I/Other legislators per chamber (Senate/House) and returns a nested breakdown dict.

`deep_dive(bill_id: int) -> dict | None`

Fetches full bill detail via `getBill`, extracts sponsors (up to 8, with party/role/district) and the 5 most recent roll-call vote summaries (date, description, yea/nay counts, passed flag, chamber, URL). Returns `None` on error.

`build_state(abbr: str) -> dict | None`

Orchestrates the full data fetch for one state: session list → master list → status breakdown → topic tagging → sort by recency → cap at `RELEVANT_CAP = 40` bills → deep-dive top `DEEP_DIVE = 6` → legislators → composition. Returns the assembled state dict.

`main() -> None`

Iterates over all 51 jurisdictions, calls `build_state`, logs progress, and writes `legiscan_data.js`.

`build_member_bills.py`

File: `webapp/build_member_bills.py` **Purpose:** Fixes an accuracy problem in the politician hover cards: the original `data.js` showed 0–1 bills per member (only trade-correlated bills). This script fetches each tracked member's real sponsored and co-sponsored legislation counts from the Congress.gov API v3, keyed by bioguide ID, so the UI shows true legislative activity.

Data source: Congress.gov API v3 — `api.congress.gov/v3/member/{bioguide}/sponsored-legislation` and `.../cosponsored-legislation` (pagination count only, `limit=1`)

Env vars / API keys: `CONGRESS_API_KEY` (read from `.env`)

Output: `webapp/js/member_bills.js` — `window.MEMBER_BILLS`

Rate limiting: `SLEEP = 0.25` seconds between each API call (2 calls per member — sponsored + cosponsored).

Output shape

```
{
  "source": "Congress.gov API",
  "byBioguide": {
    "P000197": {"sponsored": 423, "cosponsored": 1892, "total": 2315}
  },
  "count": 47
}
```

Functions

`politicians() -> list`

Reads the `webapp/js/data.js` file (already built by `build_data.py`), parses `window.TF_DATA`, and returns the `politicians` list.

`count(bioguide: str, kind: str) -> int`

Fetches the Congress.gov endpoint for `kind` (`"sponsored-legislation"` or `"cosponsored-legislation"`) with `limit=1` and returns the `pagination.count` value — an efficient count-only query that avoids pulling all bill records.

`main() -> None`

Filters politicians to those with a non-empty `bioguide` field, fetches sponsored and cosponsored counts for each, and writes `member_bills.js`.

`build_news_intel.py`

File: `webapp/build_news_intel.py` **Purpose:** The "Chef GPT" news intelligence pipeline for the webapp. Uses a cost-conscious two-phase design: heavy analysis is done locally (no API cost) and only a thin translation step calls the OpenAI API.

Phase 1 (local, no API cost): Groups already-fetched news by sector, attaches each ticker's technical signal (from `signal_output_phase3.json`), QuantLib metrics (from `quant_data.js`), and sector economic-reasoning summaries (from `news_output/sector_summaries.json`). Selects the top 8 most important stories per sector using a `credibility × |impact|` importance score.

Phase 2 (one batched Chef GPT call per sector, ~8 total): Sends the condensed brief (news + signals + context) to `gpt-4o-mini` (or the model in `THINKFREE_NEWS_MODEL` env var). The prompt asks for a 3–4 sentence plain-English sector summary plus a per-ticker `summary` + `what_it_means` for each mentioned ticker.

Data source: OpenAI Chat Completions API — `api.openai.com/v1/chat/completions`

Env vars / API keys:

- `OPENAI_API_KEY` (read from `.env`)
- `THINKFREE_NEWS_MODEL` (optional, default `gpt-4o-mini`)

Local file dependencies:

- `webapp/js/data.js` (news articles with credibility/sentiment)
- `webapp/js/quant_data.js` (QuantLib + TA signals per ticker)
- `webapp/js/prices_data.js` (company names for relevance filtering)
- `Module_2_Technical_Analysis/signal_output_phase3.json` (technical signals)
- `news_output/sector_summaries.json` (sector economic context)
- `news_output/economic_reasoning_summary.json` (overall economic context)

Output: `webapp/js/news_intel.js` — `window.NEWS_INTEL`

CLI flag: `--limit-sectors N` limits to the top N sectors by article count (useful for testing).

Output shape

```
{
  "source": "ThinkFree pipeline: local TA/QuantLib/economic analysis -> Chef GPT translation",
  "model": "gpt-4o-mini",
  "api_calls": 8,
  "bySector": {
    "Information Technology": {"summary": "...", "stories": 23}
  },
  "byTicker": {
    "NVDA": {"summary": "...", "what_it_means": "...", "sector": "Information Technology"}
  },
  "disclaimer": "Plain-English summary generated from public data and our models. Not financial advice."
}
```

Functions

`chat_json(system: str, user: str, model: str, timeout: int = 60) -> dict`

Makes a POST to the OpenAI Chat Completions API with `response_format: {"type": "json_object"}` and `temperature: 0.3`. Returns the parsed JSON content of the first choice message.

`load_window(fname: str, marker: str) -> dict`

Parses a `window.NAME = {...}`; data file from the `webapp/js/` directory by splitting on the `marker` string.

`load_json(p: Path, default) -> dict | list`

Loads any JSON file with a fallback default on error.

```
norm_sector(s: str) -> str
```

Normalizes sector names using `SECTOR_ALIASES` so news sector labels (e.g., `"Technology"`, `"Healthcare"`) map to GICS standard names (`"Information Technology"`, `"Health Care"`).

```
news_relevant(item: dict, tk: str, names: dict) -> bool
```

Filters out common-word ticker mismatches (e.g., `ON`, `IT`, `DD`). Returns `True` only if the article text actually mentions the company name or the ticker symbol in a recognizable format (`$TICK` or `(TICK)`).

```
importance(n: dict) -> float
```

Scores a news item for sector ranking: `impact × 2 + credibility`.

```
main() -> None
```

Groups news by normalized sector, selects top 8 by importance, builds a condensed brief with TA signals and QuantLib metrics attached, calls OpenAI once per sector, and writes `news_intel.js`.

```
build_nonprofits.py
```

File: `webapp/build_nonprofits.py` **Purpose:** Fetches IRS Form 990 financial data for the lobbying and advocacy organizations shown in InfluenceWeb (the `_SECTOR_LOBBY` and `_OUTSIDE_GROUPS` constants in `build_data.py`). Uses ProPublica's Nonprofit Explorer API — completely free, no authentication. Writes `webapp/js/nonprofit_data.js`.

Data source: ProPublica Nonprofit Explorer API — `projects.propublica.org/nonprofits/api/v2` (endpoints: `search.json`, `organizations/{ein}.json`)

Env vars / API keys: None. The API is public and unauthenticated.

Output: `webapp/js/nonprofit_data.js` — `window.NP_DATA`

Rate limiting: `time.sleep(0.3)` between each organization (2 calls per org: search + detail).

Hardcoded Organizations (24 total)

Industry trade groups (e.g., PhRMA, American Petroleum Institute, National Association of Realtors) and political 501(c)(4) advocacy groups (e.g., League of Conservation Voters, Americans for Prosperity, Club for Growth).

Output shape (`byName` entry)

```
{
  "American Petroleum Institute": {
    "ein": "13-5467760",
    "name": "American Petroleum Institute",
    "city": "Washington",
    "state": "DC",
    "ntee": "S41",
    "ntee_label": "Community/Economic Dev.",
    "revenue": 284000000,
    "revenue_fmt": "$284.0M",
    "expenses": 278000000,
    "expenses_fmt": "$278.0M",
    "assets": 320000000,
    "assets_fmt": "$320.0M",
    "year": 2022,
    "url": "https://projects.propublica.org/nonprofits/organizations/..."
  }
}
```

Functions

```
get(url: str) -> dict
```

Simple GET request with `ThinkFree/1.0` user-agent. Returns parsed JSON.

```
best_match(query: str) -> dict | None
```

Searches ProPublica by organization name and returns the first result, or `None` if no match.

```
latest_financials(ein: str) -> tuple[dict, dict]
```

Fetches the full organization record for an EIN. Returns `(org_record, best_filing)` where `best_filing` is the most recent filing with non-zero `totrevenue`, or the most recent filing overall.

```
money(n) -> str | None
```

Formats a number to a currency display string.

```
main() -> None
```

For each of the 24 hardcoded organizations: searches by display query, fetches financials for the matched EIN, extracts NTEE classification, and writes `nonprofit_data.js`.

build_openstates.py

File: `webapp/build_openstates.py` **Purpose:** Secondary source complementing LegiScan. Open States v3 provides clean, normalized legislator records with photos, email addresses, and Open States profile links. The front-end cross-references these against the LegiScan roster by name to show legislator headshots and contact info for accountability. The script degrades gracefully — states that fail to fetch (common due to Open States gateway 502s) are skipped and listed in `skipped`.

Data source: Open States / Plural API v3 — `v3.openstates.org/people` (paginated, 50 per page)

Env vars / API keys: `OPENSTATES_API_KEY` (read from `.env`)

Output: `webapp/js/openstates_data.js` — `window.OPENSTATES_DATA`

Rate limiting: `RATE = 1.15` seconds between each API call (≥ 1 request/sec as required by the free tier). Up to `MAX_PAGES = 12` pages per state, `PER_PAGE = 50` records. Each failed call retries up to `RETRIES = 4` times with linear backoff.

Output shape

```
{
  "source": "Open States / Plural (openstates.org) API v3",
  "byState": {
    "CA": {
      "name": "California",
      "legislators": [
        {
          "name": "Toni G. Atkins",
          "party": "D",
          "chamber": "Senate",
          "district": "39",
          "image": "https://...",
          "email": "senator.atkins@senate.ca.gov",
          "openstates_url": "https://openstates.org/person/..."
        }
      ]
    }
  },
  "count": 49,
  "skipped": ["WY"]
}
```

Functions

```
fetch_page(jurisdiction: str, page: int) -> dict
```

Fetches one page of legislators for a jurisdiction name (full state name, e.g., `"California"`). Retries up to 4 times with linear backoff on any error (502, 429, timeout).

```
build_state(abbr: str) -> list
```

Paginates through all pages for a state, extracting `name`, `party` (normalized to D/R/I), `chamber` (normalized to Senate/House/Legislature), `district`, `image`, `email`, and `openstates_url` for each legislator. Stops at `MAX_PAGES`.

```
main() -> None
```

Iterates all 51 jurisdictions, calls `build_state`, counts legislators with photos, and writes `openstates_data.js`.

build_prices.py

File: `webapp/build_prices.py` **Purpose:** Fetches live price, daily change, market cap, company name, industry, exchange, logo, and website URL from Finnhub for every unique ticker referenced anywhere in the ThinkFree webapp. This creates the lookup table used by the ticker/company popup throughout the front-end. Two Finnhub calls per ticker (quote + profile2).

Data source: Finnhub REST API — `finnhub.io/api/v1/` (endpoints: `quote`, `stock/profile2`)

Env vars / API keys: `FINNHUB_API_KEY` (read from `.env`)

Output: `webapp/js/prices_data.js` — `window.PRICES_DATA`

Rate limiting: `SLEEP = 1.1` seconds between tickers (2 calls per ticker → approximately 55 tickers/minute on the free tier's 60 calls/minute cap). Tickers with no quote (price = 0 or None) are skipped after one call.

Ticker Universe Assembly

`ticker_universe()` collects tickers from all previously-built JS bundles:

1. `sp500_data.js` — full S&P 500 (≈ 503 tickers)
2. `secbulk_data.js` — LittleSis-matched companies
3. `influence_data.js` — InfluenceWeb companies
4. `usaspending_data.js` — government contract recipients
5. `data.js` — congressional trade tickers and bill tickers and news symbols

Only tickers matching `[A-Z]{1,5}` (1–5 uppercase letters, no digits, no dots) are included — index labels and non-equity symbols are discarded.

Output shape (`byTicker` entry)

```
{
  "NVDA": {
    "price": 875.23,
    "change": 12.45,
    "change_pct": 1.44,
    "market_cap": 2160000000000,
    "market_cap_fmt": "$2.16T",
    "name": "NVIDIA Corporation",
    "industry": "Semiconductors",
    "exchange": "NASDAQ NMS - GLOBAL MARKET",
    "logo": "https://...",
    "weburl": "https://www.nvidia.com"
  }
}
```

Functions

`_load_window(fname: str, var: str) -> dict`

Parses a `window.NAME = {...};` file by finding `var` in the text. Returns `{}` if the file does not exist.

`ticker_universe() -> list`

Collects all ticker strings from all existing JS data files and returns a sorted, deduplicated, filtered list of valid equity tickers.

`get(path: str, **params) -> dict`

Makes a GET request to the Finnhub API endpoint, appending the API token.

`money(n) -> str | None`

Formats a number to a display string with T/B/M suffixes.

`main() -> None`

Fetches quote and profile2 for every ticker in the universe, skips tickers with no price data, and writes `prices_data.js`.

build_quant.py

File: `webapp/build_quant.py` **Purpose:** The quantitative heart of the Predictive Market Signals engine. Computes real QuantLib-style metrics (Black-Scholes N(d2) probability, annualized volatility, 95% VaR, lognormal expected return, Kelly fraction) from `quantlib_metrics.py`, AND computes technical indicators (RSI, MACD, SMA-50/SMA-200 crossovers) via the `ta` library on live yfinance price history. Writes `webapp/js/quant_data.js` which feeds `predictions.js` and `build_news_intel.py`.

Data source: yfinance (Yahoo Finance) price history via `quantlib_metrics.fetch_recent_prices()`. No additional API key required.

Env vars / API keys: None. yfinance is unauthenticated.

Output: `webapp/js/quant_data.js` — `window.QUANT_DATA`

Rate limiting: Inherits yfinance's rate limiting. No explicit `sleep` in the script.

CLI: `./tf_env/bin/python webapp/build_quant.py [TICKER ...]` — explicit tickers override the automatic candidate selection.

Output shape (byTicker entry)

```
{
  "NVDA": {
    "price": 875.23,
    "volatility": 42.1,
    "var95": -3.21,
    "prob_up": 38.5,
    "prob_down": 18.2,
    "exp_return_1mo": 1.43,
    "kelly": 12.3,
    "sharpe": 1.84,
    "rsi": 68.4,
    "rsi_signal": "neutral",
    "macd": 2.341,
    "macd_signal_line": 1.982,
    "macd_cross": "bullish",
    "sma50": 820.11,
    "sma200": 750.33,
    "trend": "golden",
    "ta_score": 78,
    "ta_signal": "BUY",
    "ql_signal": "BUY"
  }
}
```

Functions

`candidates(n: int = 22) -> list`

Selects up to 22 candidate tickers for analysis. Scores tickers by frequency: each news article mention = 1 point, each congressional trade = 0.5 points. Filters to only tickers present in `prices_data.js` and matching `[A-Z]+` (alphabetic only). Returns the top-n most-mentioned.

`sharpe(prices: list) -> float`

Computes the annualized Sharpe ratio from log returns: $(\text{mean_daily_log_return} \times 252 - \text{RISK_FREE_RATE}) / (\text{daily_std} \times \sqrt{252})$.

`ta_block(prices: list) -> dict`

Computes technical indicators using the `ta` library on a pandas Series:

- **RSI** (14-period): signals `"oversold"` (< 30), `"overbought"` (> 70), `"neutral"`
- **MACD** (12/26/9): signals `"bullish"` (MACD > signal line) or `"bearish"`
- **SMA-50 / SMA-200 crossover**: `"golden"` (SMA-50 > SMA-200) or `"death"` cross

Composite `ta_score` (0–100): starts at 50, adds/subtracts 12 for RSI, ±14 for MACD, ±14 for trend. Maps score to `ta_signal`: `"BUY"` if ≥ 60, `"SELL"` if ≤ 40, `"HOLD"` otherwise.

`main() -> None`

Gets the ticker list (CLI args or `candidates()`), for each ticker fetches ≥ 30 days of price history, runs QuantLib metrics, runs `ta_block`, derives `ql_signal` from expected return and probability balance, and writes `quant_data.js`.

QuantLib metrics reused (imported from `quantlib_metrics.py`):

- `compute_annualized_vol(prices)` — annualized historical volatility
- `compute_var_95(prices)` — 95% Value-at-Risk (as % move)
- `bs_prob_above_target(spot, target, vol)` — Black-Scholes N(d2) probability
- `expected_return_lognormal(spot, vol)` — lognormal price expectation
- `kelly_fraction(prices)` — Kelly criterion fraction
- `RISK_FREE_RATE` — risk-free rate constant

build_relationships.py

File: `webapp/build_relationships.py` **Purpose:** Fetches federal lobbying relationships for every tracked company from the U.S. Senate Lobbying Disclosure Act (LDA) database. For each company: identifies the lobbying firms hired, individual lobbyists, issue areas lobbied, specific bills mentioned in lobbying descriptions, and total lobbying spend. This is purely a relationship graph — who hires whom and for what. The LDA API is anonymous (no key required).

Data source: U.S. Senate LDA API — `lda.senate.gov/api/v1/filings/` (GET, anonymous, ~15 requests/minute)

Env vars / API keys: None. The LDA API is fully public.

Output: `webapp/js/relationships_data.js` — `window.RELATIONSHIPS`

Rate limiting: `SLEEP = 4.2` seconds between each company (anonymous LDA limit $\approx$ 15 requests/minute). Large runs benefit from `--limit N`.

CLI flags:

- `--year 2024` — fiscal year for lobbying data (default 2024)
- `--limit N` — process only the first N companies (for testing)

Input file dependencies: `webapp/js/influence_data.js`, `webapp/js/prices_data.js` (for names), `webapp/js/sp500_data.js` (for names)

Output shape (`byTicker` entry)

```
{
  "MSFT": {
    "client": "Microsoft Corporation",
    "filings": 48,
    "spend_fmt": "$8.5M",
    "firms": ["Covington & Burling LLP", "Squire Patton Bogs"],
    "lobbyists": ["John Smith", "Jane Doe"],
    "issues": ["Technology", "Tax", "Trade (International)"],
    "bills": ["H.R. 1234", "S. 567"]
  }
}
```

Functions

`load_window(fname: str, marker: str) -> dict`

Parses a `window.NAME = {...};` file from `webapp/js/`.

`search_name(name: str) -> str`

Strips common generic corporate suffixes (Inc, Corp, Ltd, Holdings, etc.) from a company name to produce a cleaner LDA search term. Falls back to the original name if stripping leaves nothing.

`get(url: str) -> dict`

Simple GET request with JSON Accept header.

`fetch_company(name: str, year: int) -> tuple[dict | None, str | None]`

Queries the LDA API for filings by `client_name` and `filing_year` (up to 25 results). Aggregates: lobbying firms (Counter), individual lobbyists (set), issue codes (Counter), bill numbers (regex-extracted from lobbying descriptions using `BILL_RE`), and total spend (income + expenses summed across filings). Returns `(record, None)` on success or `(None, error_message)` on failure.

`_money(n: float) -> str | None`

Formats a spend amount.

`main() -> None`

Merges company names from S&P 500, Finnhub, and LittleSis datasets (LittleSis legal names take precedence), fetches LDA data for each ticker, and writes `relationships_data.js`.

`build_sec.py`

File: `webapp/build_sec.py` **Purpose:** Fetches current board/director records and the latest 10-K filing metadata for each company in InfluenceWeb via the sec-api.io commercial API. Implements resume logic: if `sec_data.js` already exists, previously-fetched companies are preserved and only new ones are fetched. Stops immediately and prints a warning on HTTP 429 rate-limit responses.

Data source: sec-api.io — `api.sec-api.io/directors-and-board-members?token=KEY` (POST) and `api.sec-api.io?token=KEY` (POST for 10-K query)

Env vars / API keys: `SECAPI_IO_KEY` (read from `.env`)

Output: `webapp/js/sec_data.js` — `window.SEC_DATA`

Rate limiting: `time.sleep(0.15)` between companies. The sec-api.io free tier has strict rate limits; the resume logic allows incremental runs across multiple sessions.

Output shape (byTicker entry)

```
{
  "AAPL": {
    "entityName": "APPLE INC",
    "filedAt": "2024-11-01",
    "board": [
      {
        "name": "Tim Cook",
        "position": "CEO and Director",
        "age": 63,
        "since": "2011",
        "committees": ["Audit"],
        "independent": false
      }
    ],
    "latest_10k": {
      "filedAt": "2024-11-01",
      "url": "https://..."
    }
  }
}
```

Functions

`post(url: str, payload: dict) -> dict`

Makes a POST request with JSON body. Handles gzip-compressed responses by checking for the `\x1f\x8b` magic bytes.

`tickers() -> list`

Reads `webapp/js/influence_data.js` and returns the list of matched company tickers.

`main() -> None`

Reads any existing `sec_data.js` to resume (preserving already-fetched companies), fetches board members and 10-K metadata for each new company, stops on 429, and writes the accumulated result.

`build_secbulk.py`

File: `webapp/build_secbulk.py` **Purpose:** Extracts XBRL financial data and filing metadata from the SEC EDGAR bulk datasets (`companyfacts.zip` and `submissions.zip`) for companies matched in InfluenceWeb. Completely offline — no network calls, no rate limits. These are large official SEC downloads (~multi-GB compressed) that must be placed in `~/Downloads/` manually.

Data source: SEC EDGAR bulk data — `companyfacts.zip` and `submissions.zip` (from <https://www.sec.gov/dera/data/>)

Env vars / API keys: None.

Local file requirements:

- `~/Downloads/companyfacts.zip`
- `~/Downloads/submissions.zip`

Output: `webapp/js/secbulk_data.js` — `window.SECBULK_DATA`

Two-Pass Algorithm

Pass 1 (submissions.zip): Walks every `CIK*.json` file in the submissions ZIP, looking for CIK files whose `tickers` list intersects with the ThinkFree ticker set. Extracts company name, SIC description, exchange, EIN, filing activity summary (total filings, last filing date, form type counts), and builds the ticker-to-CIK map.

Pass 2 (companyfacts.zip): For each matched CIK, opens `CIK{padded_10}.json` in the companyfacts ZIP and extracts the latest annual XBRL values for:

XBRL Concepts	Field
<code>RevenueFromContractWithCustomerExcludingAssessedTax</code> , <code>Revenues</code> , <code>SalesRevenueNet</code>	<code>revenue</code>
<code>Assets</code>	<code>assets</code>
<code>NetIncomeLoss</code> , <code>ProfitLoss</code>	<code>net_income</code>
<code>Liabilities</code>	<code>liabilities</code>

Output shape (`byTicker` entry)

```
{
  "MSFT": {
    "cik": "789019",
    "name": "MICROSOFT CORP",
    "sic": "Prepackaged Software",
    "exchange": "Nasdaq",
    "ein": "91-1144442",
    "filings_total": 1247,
    "last_filing": "2024-10-25",
    "form_counts": {"10-Q": 142, "8-K": 380, "10-K": 38, ...},
    "revenue": 211915000000,
    "revenue_fmt": "$211.9B",
    "assets": 512163000000,
    "assets_fmt": "$512.2B",
    "net_income": 72361000000,
    "net_income_fmt": "$72.4B",
    "liabilities": 243686000000,
    "liabilities_fmt": "$243.7B",
    "fy": 2024,
    "xbrl_concepts": 612,
    "url": "https://www.sec.gov/cgi-bin/browse-edgar?action=getcompany&CIK=789019&type=10-K"
  }
}
```

Functions

`load_ciks() -> dict`

Currently returns an empty dict — placeholder. CIK resolution is done via the submissions ZIP scan instead.

`fmt(n) -> str | None`

Formats a number to T/B/M/K currency string.

`latest_fact(us: dict, concepts: list) -> tuple[float | None, int | None, str | None]`

Searches the `us-gaap` XBRL facts dict for the most recent annual value among the given concept list. Prefers 10-K/20-F forms; falls back to any form. Returns `(value, fiscal_year, unit_key)`.

`our_tickers() -> set`

Reads `webapp/js/influence_data.js` and returns the set of company ticker keys.

`main() -> None`

Opens both ZIPs, runs the two-pass algorithm, and writes `secbulk_data.js`.

`build_sp500.py`

File: `webapp/build_sp500.py` **Purpose:** Downloads the S&P 500 constituent list (ticker, company name, GICS sector, GICS sub-industry) from the public GitHub `datasets/s-and-p-500-companies` repository and maps each company to one of ThinkFree's InfluenceWeb sector categories. Requires no API key.

Data source: GitHub raw CSV — `raw.githubusercontent.com/datasets/s-and-p-500-companies/main/data/constituents.csv` (public domain)

Env vars / API keys: None.

Output: `webapp/js/sp500_data.js` — `window.SP500`

Rate limiting: One HTTP request for the CSV. No delays needed.

GICS to ThinkFree Sector Mapping

GICS Sector	ThinkFree Sector (selected rules)
Industrials (Aerospace & Defense sub)	Defense
Industrials (Transport sub-industries)	Transportation
Industrials (other)	Industrials
Information Technology	Technology
Health Care (pharma/biotech/life sci.)	Pharmaceuticals
Health Care (other)	Healthcare
Financials	Financial Services
Energy	Oil & Gas
Utilities	Energy
Communication Services	Telecommunications
Real Estate	Real Estate
Consumer Discretionary / Staples	Consumer Goods
Materials	Materials

Output shape (`byTicker` entry)

```
{
  "NVDA": {
    "name": "NVIDIA Corporation",
    "sector": "Technology",
    "gics": "Information Technology",
    "sub": "Semiconductors & Semiconductor Equipment"
  }
}
```

Functions

`tf_sector(gics: str, sub: str) -> str`

Maps a GICS sector + sub-industry string to one of ThinkFree's 12 custom sector labels using a chain of `if / elif` checks with `in` substring matching on the lowercased sub-industry.

`main() -> None`

Fetches the CSV, parses it with `csv.DictReader`, maps each row, and writes `sp500_data.js`. Prints a per-sector count after writing.

build_usaspending.py

File: `webapp/build_usaspending.py` **Purpose:** Fetches total federal contract award dollars (2020–2025) and the top awarding agencies for each company tracked across all ThinkFree datasets (full S&P 500 + Finnhub names + LittleSis companies). Uses the public USASpending.gov API — no authentication required. Implements resume logic: companies already in `usaspending_data.js` are preserved across runs.

Data source: USASpending.gov API v2 — `api.usaspending.gov/api/v2/search/spending_by_category/awarding_agency/` (POST)

Env vars / API keys: None.

Output: `webapp/js/usaspending_data.js` — `window.USA_DATA`

Rate limiting: `time.sleep(0.5)` between companies. Uses SSL via `certifi` where available, falls back to no-verify SSL context.

Award types: `["A", "B", "C", "D"]` — federal contracts only (excludes grants and loans).

Date range: 2020-01-01 to 2025-12-31.

Output shape (`byTicker` entry)

```
{
  "LMT": {
    "recipient": "Lockheed Martin",
    "total_contracts": 47200000000,
    "total_contracts_fmt": "$47.2B",
    "top_agencies": [
      {"agency": "Department of Defense", "amount": 43100000000, "amount_fmt": "$43.1B"}
    ],
    "url": "https://www.usaspending.gov/search?keyword=Lockheed+Martin"
  }
}
```

Functions

`company_names() -> dict`

Builds a `ticker -> company_name` map from three sources in priority order (each overwrites earlier): S&P 500 names → Finnhub names → LittleSis legal names (highest priority). Strips common suffixes (" Inc.", " Inc.", ",") from all names before searching.

`post(url: str, payload: dict) -> dict`

Makes a POST request with JSON body using the SSL context.

`fmt(n) -> str | None`

Formats a number to a T/B/M/K/\$ currency string.

`fetch(name: str) -> tuple[float, list]`

Posts the spending-by-awarding-agency query for a given recipient name. Returns `(total_dollars, agencies_list)` where `agencies_list` contains up to 6 agencies with name, amount, and formatted amount.

`main() -> None`

Assembles the company name map, loads any existing `usaspending_data.js` for resume, fetches data for new companies, and writes the result.

fetch_politician_photos.py

File: `webapp/fetch_politician_photos.py` **Purpose:** Downloads official public-domain congressional portrait photos for every politician tracked in ThinkFree. Uses the `@unitedstates/images` GitHub Pages collection, keyed by bioguide ID — the same IDs already stored in `politician_parties.json`. Photos are saved as `webapp/assets/politicians/{bioguide}.jpg`. Skips already-downloaded files (idempotent).

Data source: `unitedstates.github.io/images/congress/450x550/{bg}.jpg` (primary) with fallback to `raw.githubusercontent.com/unitedstates/images/gh-pages/congress/450x550/{bg}.jpg`. Public domain / no key required.

Env vars / API keys: None.

Output: `webapp/assets/politicians/{bioguide}.jpg` (one file per politician)

Rate limiting: `time.sleep(0.15)` between each download. The GitHub CDN has no documented rate limit at this cadence.

Functions

`bioguide_ids() -> dict`

Reads `politician_parties.json` and extracts a `{bioguide_id: politician_name}` dict for all politicians that have a `bioguide` field.

`fetch(bg: str) -> bytes | None`

Tries each URL template in `SOURCES`, returns raw image bytes if the response is HTTP 200 and `len(data) > 1000` bytes. Returns `None` if all sources fail.

`main() -> None`

Creates `webapp/assets/politicians/` if needed, iterates all bioguide IDs, skips files already on disk with size > 1000 bytes, and downloads missing photos. Reports totals: downloaded, cached (skipped), missing.

`minify_data.py`

File: `webapp/minify_data.py` **Purpose:** Production optimization step. The build scripts emit pretty-printed (indented) JSON for readability during development. This script recompacts every `window.*_DATA` assignment in `webapp/js/*.js` to strip all whitespace, achieving approximately 30–40% size reduction. It is safe and idempotent: it only changes whitespace, never data. Code files (`app.js`, `scores.js`, etc.) are explicitly excluded. Run this once as the final step before deploying.

Data source: Local `webapp/js/*.js` files.

Env vars / API keys: None.

Output: Overwrites data `.js` files in `webapp/js/` in place.

Excluded Code Files

`app.js`, `scores.js`, `genimpact.js`, `scoreinfo.js`, `predictions.js`, `glossary.js`, `congress.js`, `influenceweb.js`

Functions

`compact(txt: str) -> str`

Uses Python's `json.JSONDecoder.raw_decode()` to find each `window.NAME = {...}` assignment in the file text, re-serializes the JSON value with `json.dumps(..., separators=(",", ":"))` (no whitespace), and reconstructs the file text. Handles files with multiple window assignments.

`main() -> None`

Iterates all `*.js` files in `webapp/js/` that contain `"window."` and are not in the `CODE` exclusion set, applies `compact`, overwrites the file if the result is smaller, and reports the size savings.

`scripts/extract_data_to_json.py`

File: `scripts/extract_data_to_json.py` **Purpose:** One-time (repeatable) migration step that converts the public `webapp/js/*_data.js` bundles into plain JSON files under `private_data/`, which the FastAPI backend server serves only to authenticated users. Also writes a `manifest.json` classifying each dataset as `core` (needed immediately after login), `lazy` (large, loaded on demand), or `live` (refreshed on a cadence). Does **not** delete the originals from `webapp/js/` so the current static app keeps working during the transition.

Data source: Local `webapp/js/*.js` files.

Env vars / API keys: None (stdlib only).

Output:

- `private_data/{GLOBAL_NAME}.json` — one file per dataset (minified JSON, no `window.` prefix)
- `private_data/manifest.json` — classification of all datasets

Category Classification

Category	Files	Rationale
core	<code>data.js</code> , <code>influence_data.js</code> , <code>relationships_data.js</code> , <code>nonprofit_data.js</code> , <code>fec_data.js</code> , <code>sec_data.js</code> , <code>secbulk_data.js</code> , <code>usaspending_data.js</code> , <code>states_data.js</code> , <code>sp500_data.js</code> , <code>quant_data.js</code> , <code>member_bills.js</code>	Must render app right after login
lazy	<code>legiscan_data.js</code> , <code>openstates_data.js</code>	Large; fetched only when State Legislature view is opened
live	<code>prices_data.js</code> , <code>news_intel.js</code>	Refreshed on user's subscription cadence

Manifest shape

```
{
  "core": ["TF_DATA", "IW_DATA", "RELATIONSHIPS", "NP_DATA", "FEC_DATA", "SEC_DATA", "SECBULK_DATA", "USA_DATA",
    "STATES_DATA", "SP500", "QUANT_DATA", "MEMBER_BILLS"],
  "lazy": ["LEGISCAN_DATA", "OPENSTATES_DATA"],
  "live": ["PRICES_DATA", "NEWS_INTEL"]
}
```

Functions

`extract_one(js_path: Path) -> tuple[str, object]`

Reads one `window.NAME = <json>` file, applies a regex (`ASSIGN_RE`) to extract the global name and JSON value string, and calls `json.loads()` to validate and parse the value. Raises `ValueError` if the file is not in the expected format.

`main() -> None`

Creates `private_data/` directory, iterates all categories and files, calls `extract_one`, writes the compact JSON output (using `separators=(",", ":)` – no whitespace), accumulates the manifest, and writes `manifest.json`.

Environment Variables Reference

The following environment variables must be present in the `.env` file at the repository root. No API keys are ever written to any output file or log.

Env Var	Required By	External Service
<code>FRED_API_KEY</code>	<code>build_states.py</code>	Federal Reserve Bank of St. Louis FRED API
<code>CENSUS_API_KEY</code>	<code>build_census.py</code>	U.S. Census Bureau ACS API
<code>BEA_API_KEY</code>	<code>build_bea.py</code>	Bureau of Economic Analysis Regional API
<code>BLS_API_KEY</code>	<code>build_bls.py</code>	Bureau of Labor Statistics Public API v2
<code>FEC_API_KEY</code>	<code>build_fec.py</code>	OpenFEC (Federal Election Commission)
<code>LEGISCAN_API_KEY</code>	<code>build_legiscan.py</code>	LegiScan State Legislature API
<code>CONGRESS_API_KEY</code>	<code>build_member_bills.py</code>	Congress.gov API v3
<code>OPENAI_API_KEY</code>	<code>build_news_intel.py</code>	OpenAI Chat Completions API
<code>OPENSTATES_API_KEY</code>	<code>build_openstates.py</code>	Open States / Plural API v3
<code>FINNHUB_API_KEY</code>	<code>build_prices.py</code>	Finnhub REST API
<code>SECAPI_IO_KEY</code>	<code>build_sec.py</code>	sec-api.io (SEC EDGAR commercial API)
<code>THINKFREE_NEWS_MODEL</code>	<code>build_news_intel.py</code> (optional)	OpenAI model override (default <code>gpt-4o-mini</code>)

Scripts with no env var requirement: `build_data.py`, `build_influence.py`, `build_nonprofits.py`, `build_relationships.py` (LDA is anonymous), `build_secbulk.py`, `build_sp500.py`, `build_usaspending.py`, `fetch_politician_photos.py`, `minify_data.py`, `scripts/extract_data_to_json.py`.

Local File Prerequisites (Non-API)

Some scripts require large files that must be downloaded manually and are not committed to the repository.

File	Location	Required By	Source
entities.json.gz	~/Downloads/	build_influence.py	LittleSis bulk data exports
relationships.json.gz	~/Downloads/	build_influence.py	LittleSis bulk data exports
companyfacts.zip	~/Downloads/	build_secbulk.py	SEC EDGAR (https://www.sec.gov/dera/data/)
submissions.zip	~/Downloads/	build_secbulk.py	SEC EDGAR (https://www.sec.gov/dera/data/)

Part 5 — Backend Server Reference (FastAPI)

This document is an exhaustive technical reference for every file in `server/`. It covers every function, class, constant, route, middleware, and security mechanism exactly as written in the source code. File-and-line citations appear where precision matters.

Table of Contents

- 1. [Dependency Map](#)
- 2. [File: requirements.txt](#)
- 3. [File: .env.example](#)
- 4. [File: .gitignore](#)
- 5. [File: config.py](#)
- 6. [File: db.py](#)
- 7. [File: auth.py](#)
- 8. [File: data.py](#)
- 9. [File: main.py](#)
- 10. [Endpoints Quick-Reference Table](#)
- 11. [Security Architecture Summary](#)

1. Dependency Map

```
main.py
├── config.py      (settings, resolved paths, tier definitions)
├── db.py          (SQLite user store)
├── auth.py        (router: /api/auth/*, JWT, bcrypt, rate limiting)
│   ├── config.py
│   └── db.py
├── data.py        (router: /api/data/*, gated datasets)
│   ├── config.py
│   └── auth.py    (imports current_user dependency)
```

Import order matters: `db.init_db()` is called inside `main.py` at module load time, before any request is handled.

2. File: requirements.txt

Responsibility: Declares the exact pinned Python dependencies for the isolated server virtualenv. The comment at the top of the file explicitly warns that this virtualenv must be kept separate from the data-pipeline virtualenv (`tf_env`) because the version pins are different.

Package	Pinned Version	Purpose
fastapi	0.111.0	ASGI web framework; routes, middleware, dependency injection
uvicorn[standard]	0.30.1	ASGI server (includes <code>httptools</code> , <code>uvloop</code> , <code>websockets</code>)
bcrypt	4.1.3	Password hashing; slow by design to resist offline brute-force
PyJWT	2.8.0	Signs and verifies JWT session tokens with HS256
python-dotenv	1.0.1	Loads <code>server/.env</code> into <code>os.environ</code> at startup

No ORM, no async DB driver, no Redis. The design is intentionally minimal for a single-process closed beta: SQLite + in-memory rate-limit state is sufficient and avoids operational complexity.

3. File: .env.example

Responsibility: Template for `server/.env` , which is gitignored. Documents every environment variable the server reads. Operators copy this file to `server/.env` and fill in real values, or set the variables directly in the hosting platform dashboard (Render / Railway / Fly).

Variables

Variable	Default in example	Purpose
TF_SECRET_KEY	change-me-to-a-long-random-string	HMAC-SHA256 signing key for JWT session tokens. Must be replaced before production.
TF_BETA_KEY	<redacted — configured via TF_BETA_KEY>	Closed-beta invite key. Users must present this at signup.
TF_PRODUCTION	0	Set to "1" to enable Secure cookies, HSTS header, and HTTPS redirect.
TF_ALLOWED_ORIGINS	http://localhost:8000	Comma-separated CORS allow-list. In production: exact HTTPS domain.
LEGISCAN_API_KEY	_(empty)_	Used by offline build scripts only; never sent to the browser.
OPENSTATES_API_KEY	_(empty)_	Used by offline build scripts only; never sent to the browser.
FINNHUB_API_KEY	_(empty)_	Used by offline build scripts only; never sent to the browser.
OPENAI_API_KEY	_(empty)_	Used by offline build scripts only; never sent to the browser.

The comment in the file notes a method for generating a strong signing key:

```
python3 -c "import secrets; print(secrets.token_urlsafe(48))"
```

4. File: .gitignore

Responsibility: Prevents secrets and local artifacts from entering version control. The server .gitignore file is intentionally narrow — it only covers server-specific artifacts.

Entries:

- .env — the live secrets file
- .venv/ — the server's isolated virtualenv
- __pycache__/ — compiled bytecode
- *.pyc — compiled bytecode
- users.db — the SQLite database containing hashed passwords and user records

5. File: config.py

Responsibility: Central configuration for the entire backend. Loads all settings from the environment (via python-dotenv) and exposes them as module-level constants. All other modules import from here; nothing is hardcoded anywhere else.

Module-Level Constants

Path Constants

Resolved at import time using Path(__file__).resolve().parent so the server runs correctly regardless of the working directory when uvicorn is started.

Constant	Value / Resolution	Type
SERVER_DIR	Directory containing config.py itself (../ThinkFree-main/server)	Path
ROOT_DIR	Parent of SERVER_DIR (../ThinkFree-main)	Path
WEBAPP_DIR	ROOT_DIR / "webapp" — the static front-end shell	Path
PRIVATE_DATA_DIR	ROOT_DIR / "private_data" — built JSON datasets, never web-served	Path
DB_PATH	SERVER_DIR / "users.db" — the SQLite database file	Path

Runtime Settings (from environment)

Constant	Environment Variable	Default	Purpose
SECRET_KEY	TF_SECRET_KEY	"change-me-to-a-long-random-string"	JWT signing key (HS256)

Constant	Environment Variable	Default	Purpose
BETA_KEY	TF_BETA_KEY	"<redacted – configured via TF_BETA_KEY>"	Closed-beta invite key
PRODUCTION	TF_PRODUCTION	False (when env var != "1")	Enables Secure cookies, HSTS, HTTPS redirect
ALLOWED_ORIGINS	TF_ALLOWED_ORIGINS	["http://localhost:8000"]	CORS allow-list (split on , , stripped)
SESSION_TTL_SECONDS	_(hardcoded)_	604800 (7 days)	JWT and cookie expiry

Tier Definitions

TIERS: `dict[str, dict]` — maps tier slug to display name and refresh cadence. The `refresh_min` value is the minimum number of minutes between `/api/data/live` calls for users on that tier. The server enforces this value; it cannot be bypassed by the browser.

```
TIERS = {
    "free": {"name": "Free", "refresh_min": 1440}, # once/day
    "hourly": {"name": "Pro – Hourly", "refresh_min": 60},
    "half": {"name": "Pro – 30-min", "refresh_min": 30},
    "live": {"name": "Pro – 15-min", "refresh_min": 15},
    "beta": {"name": "Beta Access", "refresh_min": 15}, # full access during beta
}
```

DEFAULT_TIER: `str = "beta"` — all new accounts created during the closed beta receive the `"beta"` tier, giving them 15-minute refresh access.

Functions

`_get`

```
def _get(name: str, default: str = "") -> str
```

Parameters:

- `name: str` — environment variable name
- `default: str` — value to use if the variable is absent

Returns: `str` — the environment variable value, or `default`.

What it does: A thin wrapper around `os.environ.get`. Used internally by every config setting to read from the environment.

Security relevance: All sensitive values (`SECRET_KEY` , `BETA_KEY`) pass through this function. By routing all env reads through one function, the module makes it impossible to accidentally hardcode a value inline and forget to read from the environment.

`is_placeholder_secret`

```
def is_placeholder_secret() -> bool
```

Parameters: None.

Returns: `bool` — `True` if `SECRET_KEY` is either the empty string or the default placeholder `"change-me-to-a-long-random-string"`.

What it does: Checks whether the JWT signing key is still the insecure default. Called in `main.py` at startup; if it returns `True` while `PRODUCTION` is also `True`, the application raises `RuntimeError` and refuses to start.

Security relevance: Fail-loud guard against deploying with a guessable signing key. Without this check, a misconfigured production deploy would silently accept any token signed with the known placeholder.

6. File: `db.py`

Responsibility: A minimal SQLite-backed user store. Uses only Python's standard library `sqlite3` module (no ORM). The design explicitly notes that this module is the single swap point if the project grows to need Postgres: callers use the public functions, never the SQLite connection directly.

Every function opens a fresh short-lived connection and commits or reads within a single `with` block. This avoids shared-connection threading issues while remaining safe across FastAPI's thread pool.

Database Schema

Table: `users`

Column	Type	Constraints	Description
<code>email</code>	TEXT	PRIMARY KEY	Lowercase email address; the account identifier
<code>name</code>	TEXT	NOT NULL	Display name
<code>pass_hash</code>	TEXT	NOT NULL	bcrypt hash of the password (never plaintext)
<code>tier</code>	TEXT	NOT NULL	Tier slug (<code>"free"</code> , <code>"beta"</code> , etc.)
<code>ally</code>	INTEGER	NOT NULL DEFAULT 0	Accessibility mode flag; <code>0</code> =off, <code>1</code> =on
<code>profile</code>	TEXT	nullable	JSON-encoded behavioral quiz answers
<code>created</code>	TEXT	NOT NULL	ISO date string (<code>YYYY-MM-DD</code>)
<code>token_valid_after</code>	INTEGER	NOT NULL DEFAULT 0	Unix timestamp; tokens with <code>iat</code> before this are rejected

The `token_valid_after` column was added after initial deployment. `init_db()` handles live migration using `PRAGMA table_info`.

Functions

`_connect`

```
def _connect() -> sqlite3.Connection
```

Parameters: None.

Returns: `sqlite3.Connection` — a new connection to the database at `config.DB_PATH`, with `row_factory = sqlite3.Row` (so rows behave like dicts).

What it does: Opens a fresh SQLite connection on every call. `check_same_thread=False` is set because FastAPI runs handlers across multiple threads; opening per-call avoids sharing a single connection across threads. `row_factory = sqlite3.Row` allows columns to be accessed by name.

Side effects: Creates `users.db` on disk if it does not exist.

`init_db`

```
def init_db() -> None
```

Parameters: None.

Returns: `None`.

What it does (step by step):

1. Opens a connection via `_connect()`.
2. Executes `CREATE TABLE IF NOT EXISTS users (...)` with all columns.
3. Runs `PRAGMA table_info(users)` to get the set of existing column names.
4. If `token_valid_after` is not present (pre-migration database), issues `ALTER TABLE users ADD COLUMN token_valid_after INTEGER NOT NULL DEFAULT 0` to add it non-destructively.
5. Commits.

Side effects: Modifies the database schema if the migration column is missing.

Called by: `main.py` at module load time, before any request is handled (`db.init_db()` at line 36 of `main.py`).

`get_user`

```
def get_user(email: str) -> Optional[dict]
```

Parameters:

- `email: str` — the email address to look up (lowercased internally)

Returns: `dict | None` — the full account row as a Python dict (with `profile` JSON decoded and `ally` converted to `bool`), or `None` if no matching row exists.

What it does (step by step):

1. Executes `SELECT * FROM users WHERE email = ?` with `email.lower()`.
2. If no row, returns `None`.
3. Converts the `sqlite3.Row` to a plain `dict`.
4. Casts `ally` from integer to `bool`.
5. Parses `profile` from JSON string to dict/None.
6. Returns the augmented dict.

Security relevance: Always lowercases the lookup key, matching the insert behavior in `create_user`. Prevents account duplication from case differences. The returned dict contains `pass_hash`; callers in `auth.py` are responsible for stripping it before returning anything to the browser (see `_public_user`).

`create_user`

```
def create_user(email: str, name: str, pass_hash: str, tier: str, created: str) -> None
```

Parameters:

- `email: str` — lowercased email (primary key)
- `name: str` — display name (already stripped by caller)
- `pass_hash: str` — bcrypt-hashed password (plaintext never passed here)
- `tier: str` — tier slug
- `created: str` — ISO date string

Returns: `None`.

What it does: Executes a parameterized `INSERT INTO users` with `ally=0` and `profile=NULL`. Commits on success.

Side effects: Raises `sqlite3.IntegrityError` if the email already exists (primary key violation). The caller (`auth.signup`) checks for an existing account before calling this, so this is a safety net.

Security relevance: Uses parameterized queries throughout — no string interpolation, no SQL injection vector. The `pass_hash` parameter is always a bcrypt digest; the plaintext password is never passed to this function.

`update_profile`

```
def update_profile(email: str, profile: Optional[dict]) -> None
```

Parameters:

- `email: str` — account to update
- `profile: Optional[dict]` — behavioral quiz answers, or `None` to clear

Returns: `None`.

What it does: Serializes `profile` to JSON (or stores `NULL`) and executes a parameterized `UPDATE users SET profile = ? WHERE email = ?`. Commits on success.

Side effects: Overwrites the existing profile.

`update_ally`

```
def update_ally(email: str, ally: bool) -> None
```

Parameters:

- `email: str` — account to update
- `ally: bool` — `True` to enable accessibility mode

Returns: `None`.

What it does: Converts `ally` to `1` or `0` and executes a parameterized `UPDATE users SET ally = ? WHERE email = ?`. Commits on success.

update_token_valid_after

```
def update_token_valid_after(email: str, ts: int) -> None
```

Parameters:

- **email: str** — account whose tokens should be invalidated
- **ts: int** — Unix timestamp (seconds); any token with `iat < ts` is rejected

Returns: `None`.**What it does:** Executes a parameterized `UPDATE users SET token_valid_after = ? WHERE email = ?` and commits.

Security relevance: This is the JWT revocation mechanism. On logout, `auth.logout` calls this with `int(time.time())` as `ts`. From that point forward, any JWT that was issued before the logout — including a token the attacker may have captured — is rejected by `auth.current_user` even if the token has not yet expired. This closes the window between logout and natural token expiry.

7. File: `auth.py`

Responsibility: All authentication and session management for the closed beta. Implements signup (with invite-key gating), login, logout, and the `current_user` FastAPI dependency. Security properties enforced here: bcrypt password hashing, httpOnly/Secure/SameSite JWT cookie, constant-time invite-key comparison, in-memory brute-force rate limiting, and JWT revocation via `token_valid_after`.

All routes are mounted under the prefix `/api/auth`.**Module-Level Constants**

Constant	Value	Purpose
<code>COOKIE_NAME</code>	<code>"tf_session"</code>	Name of the httpOnly session cookie
<code>_EMAIL_RE</code>	compiled regex	Validates email format (requires real-looking TLD, 2+ letters); case-insensitive
<code>_LOGIN_MAX_FAILS</code>	<code>5</code>	Max failed logins per (IP + email) before 429
<code>_LOGIN_WINDOW</code>	<code>900</code> (15 min)	Rolling window for per-account login failures
<code>_LOGIN_IP_MAX_FAILS</code>	<code>20</code>	Max failed logins per IP across all accounts before 429
<code>_LOGIN_IP_WINDOW</code>	<code>900</code> (15 min)	Rolling window for per-IP login failures
<code>_SIGNUP_MAX_FAILS</code>	<code>10</code>	Max failed signup attempts per IP before 429
<code>_SIGNUP_WINDOW</code>	<code>3600</code> (1 hour)	Rolling window for per-IP signup failures
<code>_fails</code>	<code>dict[str, deque]</code>	In-memory store of failure timestamps, keyed by rate-limit key string

Brute-Force Rate Limiting

The limiter uses a **rolling window** strategy: each failure is recorded as a timestamp in a `collections.deque`. On each check, entries older than the window are dropped from the front. If the remaining count meets or exceeds the threshold, HTTP 429 is raised with a `Retry-After` header indicating seconds until the oldest failure ages out.

Three separate rate-limit dimensions are tracked:

1. **Per (IP + email)** for login — `"login:{ip}:{email}"` — stops targeted password guessing against one account.
2. **Per IP** for login — `"login_ip:{ip}"` — stops credential-stuffing sprays across many accounts from one source.
3. **Per IP** for signup — `"signup:{ip}"` — stops brute-forcing the closed-beta invite key via repeated signup attempts.

A successful login clears both the per-account and per-IP login keys for that IP, so a legitimate user is never penalized by their own past failures.

Important operational note: The limiter is in-memory and per-process. It resets on server restart. A multi-worker deployment would need Redis. This is documented in the source comments.

Pydantic Models**SignupBody**

```
class SignupBody(BaseModel):
    beta_key: str = Field(min_length=1)
```

```
name: str = Field(min_length=1, max_length=80)
email: str
password: str = Field(min_length=8, max_length=200)
consent: bool = False
```

Fields:

- `beta_key` — the closed-beta invite key; must be non-empty (Pydantic rejects empty string)
- `name` — display name; 1–80 characters
- `email` — email address string; format is validated server-side by `_EMAIL_RE` after Pydantic accepts it
- `password` — minimum 8 characters, maximum 200 characters (prevents bcrypt input length attacks)
- `consent` — age-requirement attestation (18+, or 13+ with parental permission); defaults to `False`; must be `True` to proceed

LoginBody

```
class LoginBody(BaseModel):
    email: str
    password: str
```

No length constraints enforced by Pydantic at this level; the handler normalizes email and delegates all further validation to `verify_password`.

ProfileBody

```
class ProfileBody(BaseModel):
    age: str | None = None
    experience: str | None = None
    goal: str | None = None
    timeline: str | None = None
    emotional: str | None = None
```

All fields optional. The age field is a string (to match the front-end's form value) and is cast to `int` in the handler for age-floor validation.

Private Functions

`_client_ip`

```
def _client_ip(request: Request) -> str
```

Parameters:

- `request: Request` — the FastAPI/Starlette request object

Returns: `str` — the best-effort client IP address.

What it does: Reads the `X-Forwarded-For` header; if present, returns the first (leftmost) IP in the comma-separated list, which is the original client as set by the hosting proxy (Render/Railway/Cloudflare). Falls back to `request.client.host` (the raw socket peer), then to `"unknown"` if the client is absent.

Security relevance: Used as part of rate-limit keys. Taking only the first XFF hop prevents a client from injecting a fake IP into the header to bypass a ban — the proxy always prepends the real client IP before any attacker-controlled values.

`_check_locked`

```
def _check_locked(key: str, max_fails: int, window: int) -> None
```

Parameters:

- `key: str` — rate-limit bucket key (e.g. `"login:1.2.3.4:user@example.com"`)
- `max_fails: int` — threshold before 429 is raised
- `window: int` — rolling window in seconds

Returns: `None` (raises on lockout).

What it does (step by step):

1. Gets or creates a `deque` for `key` in `_fails`.
2. Pops entries from the front of the deque that are older than `window` seconds.
3. If `len(dq) >= max_fails`: computes `retry = int(window - (now - dq[0])) + 1` (seconds until oldest failure ages out), raises `HTTPException(429)` with `Retry-After: {retry}` header.

Security relevance: Called before any password or invite-key comparison, so the attacker is blocked before consuming any bcrypt compute cycles.

`_record_fail`

```
def _record_fail(key: str) -> None
```

Parameters:

- `key: str` — rate-limit bucket key

Returns: `None`.

What it does: Appends `time.time()` to the deque for `key`. Called after a failed login or signup attempt.

`_clear_fails`

```
def _clear_fails(*keys: str) -> None
```

Parameters:

- `*keys: str` — one or more rate-limit bucket keys to clear

Returns: `None`.

What it does: Removes all failure history for each given key from `_fails`. Called on successful login to reset both the per-account and per-IP counters for that user/IP combination, so a legitimate user is not penalized.

`hash_password`

```
def hash_password(plain: str) -> str
```

Parameters:

- `plain: str` — the plaintext password

Returns: `str` — the bcrypt hash (includes salt; each call produces a unique hash).

What it does: Encodes `plain` to UTF-8, calls `bcrypt.hashpw` with a fresh `bcrypt.gensalt()`, and decodes the result to a UTF-8 string for storage.

Security relevance: bcrypt is intentionally slow (cost factor controlled by `gensalt()`). Each call generates a new random salt, so identical passwords produce different hashes. The plaintext is never stored or logged.

`verify_password`

```
def verify_password(plain: str, hashed: str) -> bool
```

Parameters:

- `plain: str` — plaintext password from the login attempt
- `hashed: str` — bcrypt hash stored in the database

Returns: `bool` — `True` if the password matches.

What it does: Encodes both strings to UTF-8 and calls `bcrypt.checkpw`. Catches `ValueError` and `TypeError` (malformed hash) and returns `False` instead of raising.

Security relevance: `bcrypt.checkpw` is constant-time relative to the hash computation, preventing timing-based password extraction. The `try/except` prevents malformed hash values in the database from causing unhandled exceptions that could leak information.

`_make_token`

```
def _make_token(email: str, tier: str) -> str
```

Parameters:

- `email: str` — the account email (becomes the JWT `sub` claim)
- `tier: str` — the account tier slug (stored in the JWT as `tier` claim)

Returns: `str` — a signed JWT string.

What it does: Builds a payload dict with claims `sub`, `tier`, `iat` (issued-at, UTC), and `exp` (expiry, UTC now + `config.SESSION_TTL_SECONDS`). Encodes with `jwt.encode(..., config.SECRET_KEY, algorithm="HS256")`.

Security relevance: The `iat` claim is used by `current_user` to implement JWT revocation by comparing against `token_valid_after`. The `tier` claim is embedded in the token so the server does not need a DB lookup to know the tier on data requests — but `current_user` re-reads from the DB to confirm the account still exists, so a deleted account is always rejected.

`_set_session_cookie`

```
def _set_session_cookie(response: Response, token: str) -> None
```

Parameters:

- `response: Response` — the outgoing FastAPI response
- `token: str` — the JWT string to store in the cookie

Returns: `None`.

What it does: Calls `response.set_cookie` with:

- `key="tf_session"`
- `value=token`
- `max_age=config.SESSION_TTL_SECONDS` (604800; 7 days)
- `httponly=True` — cookie is invisible to JavaScript
- `secure=config.PRODUCTION` — only sent over HTTPS in production; `False` locally so `http://localhost` works
- `samesite="lax"` — sent on top-level navigations but not on cross-site POST requests (CSRF defense)
- `path="/"` — cookie is sent on all paths

Security relevance: The three cookie flags (`httponly`, `secure`, `samesite`) together prevent the three main session theft vectors: XSS (cannot read the cookie), network eavesdropping (HTTPS-only in production), and cross-site request forgery (SameSite=Lax blocks cross-origin POST).

`_public_user`

```
def _public_user(user: dict) -> dict
```

Parameters:

- `user: dict` — full account dict as returned by `db.get_user` (includes `pass_hash`)

Returns: `dict` — safe subset of the account for sending to the browser.

What it does: Constructs and returns a new dict with only: `email`, `name`, `tier`, `tier_name` (looked up from `config.TIERS`), `ally`, and `profile`. The `pass_hash`, `created`, and `token_valid_after` fields are deliberately excluded.

Security relevance: Central stripping point. Every endpoint that returns user data calls this function, ensuring the bcrypt hash is never accidentally leaked to the browser.

`current_user`

```
def current_user(tf_session: str | None = Cookie(default=None)) -> dict
```

Parameters:

- `tf_session: str | None` — the value of the `tf_session` cookie, injected by FastAPI's dependency injection from the request's cookie jar

Returns: `dict` — the full account row (including `pass_hash`) for use by the calling handler. Handlers that return data to the browser must pass the result through `_public_user`.

What it does (step by step):

1. If `tf_session` is `None` (cookie absent): raises `HTTPException(401, "Not authenticated")`.
2. Decodes and verifies the JWT using `config.SECRET_KEY` and `HS256`. On any `jwt.PyJWTError` (invalid signature, expired, malformed): raises `HTTPException(401, "Invalid or expired session")`.
3. Looks up the account via `db.get_user(payload["sub"])`. If not found: raises `HTTPException(401, "Account not found")`.
4. Compares `payload["iat"]` (issued-at) against `user["token_valid_after"]`. If `iat < token_valid_after`: raises `HTTPException(401, "Session revoked")`.
5. Returns the full user dict.

Security relevance: This function is the single enforcement point for authentication. It is declared as a FastAPI `Depends(current_user)` dependency on every protected endpoint. It enforces: cookie presence, JWT signature validity, JWT expiry, account existence, and JWT revocation. An attacker who steals a valid JWT is blocked the moment the legitimate user logs out (because logout bumps `token_valid_after`).

Auth Endpoints

POST /api/auth/signup

```
@router.post("/signup")
def signup(body: SignupBody, request: Request, response: Response)
```

Auth required: No (public).

What it does (step by step):

1. Constructs the signup rate-limit key `"signup:{client_ip}"` and calls `_check_locked` (10 fails / 1 hour per IP). Raises 429 if locked.
2. Compares `body.beta_key.strip()` against `config.BETA_KEY` using `hmac.compare_digest` (constant-time). If mismatch: calls `_record_fail`, raises 403.
3. If `body.consent` is `False`: raises 400.
4. Normalizes `email = body.email.strip().lower()`. If it does not match `_EMAIL_RE`: raises 400.
5. Calls `db.get_user(email)`. If an account exists: raises 409.
6. Calls `db.create_user(email, name, hash_password(body.password), config.DEFAULT_TIER, today)`.
7. Calls `_make_token` and `_set_session_cookie` to issue a session immediately.
8. Returns `{"user": _public_user(...)}`.

Returns: `{"user": {...}}` — the safe public user dict.

Security relevance: `hmac.compare_digest` prevents timing attacks on the invite key. Signup rate limit prevents brute-forcing the invite key. Consent check is server-side (cannot be skipped from browser). Generic conflict message on duplicate email does not reveal which emails exist.

POST /api/auth/login

```
@router.post("/login")
def login(body: LoginBody, request: Request, response: Response)
```

Auth required: No (public).

What it does (step by step):

1. Normalizes `email = body.email.strip().lower()`.
2. Computes `ip = _client_ip(request)`.
3. Constructs rate-limit keys `k_user = "login:{ip}:{email}"` and `k_ip = "login_ip:{ip}"`.
4. Calls `_check_locked(k_user, 5, 900)` — per-account limit. Raises 429 if locked.
5. Calls `_check_locked(k_ip, 20, 900)` — per-IP limit. Raises 429 if locked.
6. Calls `db.get_user(email)`. If not found, or if `verify_password` returns `False`: records failure on both keys, raises 401 with a generic `"Incorrect email or password"` message (same message for both cases to avoid user enumeration).
7. On success: calls `_clear_fails(k_user, k_ip)`.
8. Calls `_make_token` and `_set_session_cookie`.
9. Returns `{"user": _public_user(user)}`.

Returns: `{"user": {...}}` — the safe public user dict.

Security relevance: Two-level rate limiting (per-account and per-IP). Unified error message prevents account enumeration. Failure counters are cleared on success so legitimate users never get locked out. bcrypt comparison is constant-time.

POST /api/auth/logout

```
@router.post("/logout")
def logout(response: Response, tf_session: str | None = Cookie(default=None))
```

Auth required: No (intentionally — a logged-out user must be able to call this; invalid cookies are handled gracefully).

What it does (step by step):

1. If `tf_session` is present:
 - a. Decodes the JWT (without raising — `jwt.PyJWTError` is silently passed). b. If `sub` (email) is valid and the account exists: calls `db.update_token_valid_after(email, int(time.time()))` to bump the validity epoch, immediately invalidating all outstanding tokens for that account.
 1. Calls `response.delete_cookie(COOKIE_NAME, path="/")` to clear the session cookie in the browser.
 2. Returns `{"ok": True}`.

Returns: `{"ok": True}`.

Security relevance: Two-layer logout: cookie deletion (browser) and JWT revocation (server-side `token_valid_after`). The second layer ensures a stolen token is invalidated on logout, not just at expiry. The try/except silently handles invalid or already-expired cookies so logout always succeeds from the browser's perspective.

GET /api/auth/me

```
@router.get("/me")
def me(user: dict = Depends(current_user))
```

Auth required: Yes (`current_user` dependency).

What it does: Returns the currently authenticated account.

Returns: `{"user": _public_user(user)}`.

Usage: Called by the front-end on page load to check whether a session cookie is still valid. If the user is not logged in, `current_user` raises 401 and the front-end redirects to login.

POST /api/auth/profile

```
@router.post("/profile")
def save_profile(body: ProfileBody, user: dict = Depends(current_user))
```

Auth required: Yes (`current_user` dependency).

What it does (step by step):

1. Casts `body.age` to `int` (catches `TypeError / ValueError`, defaults to `0`).
2. If `age < 13`: raises 400 "You must be at least 13 years old to use ThinkFree".
3. If `age > 120`: raises 400 "Enter a valid age".
4. Calls `db.update_profile(user["email"], body.model_dump())`.
5. Re-fetches the updated account and returns `{"user": _public_user(...)}`.

Returns: `{"user": {...}}` — updated public user dict.

Security relevance: Age floor is enforced server-side (COPPA compliance baseline). Cannot be bypassed by sending a manipulated request from the browser.

POST /api/auth/allly

```
@router.post("/allly")
def save_allly(on: bool, user: dict = Depends(current_user))
```

Auth required: Yes (`current_user` dependency).

Parameters (query/body):

- `on: bool` — enable (`True`) or disable (`False`) accessibility mode

What it does: Calls `db.update_ally(user["email"], on)`.

Returns: `{"ok": True}`.

8. File: `data.py`

Responsibility: Gated data endpoints. The large datasets that previously shipped as publicly-accessible `webapp/js/*_data.js` files have been moved to `private_data/` (never web-served). This module serves them only to authenticated users. Anonymous requests receive 401. Per-tier refresh cadence and a market-hours gate are enforced server-side.

All routes are mounted under the prefix `/api/data`.

Dataset Organization

Datasets are organized in `private_data/manifest.json` (read by `_manifest()`) into three named lists:

Manifest key	Used by endpoint	Description
<code>core</code>	<code>GET /api/data/bundle</code>	Datasets the dashboard needs immediately after login
<code>lazy</code>	<code>GET /api/data/state</code>	Large state-legislature datasets, loaded on demand
<code>live</code>	<code>GET /api/data/live</code>	Price and news data, refreshed per tier cadence

Module-Level State

Constant / Variable	Type	Purpose
<code>_last_live</code>	<code>dict[str, float]</code>	In-memory map of <code>email -> unix timestamp</code> of last successful <code>/live</code> pull; enforces tier cadence

Like the auth rate-limiter, `_last_live` is in-memory and per-process. It resets on restart and is noted in the source as a Redis candidate for multi-worker deployments.

Private Functions

`_manifest`

```
def _manifest() -> dict
```

Parameters: None.

Returns: `dict` — the parsed contents of `private_data/manifest.json`.

What it does: Constructs the path `config.PRIVATE_DATA_DIR / "manifest.json"`. If the file does not exist, raises `HTTPException(503, "Data not built yet. Run scripts/extract_data_to_json.py")`. Otherwise reads and JSON-parses the file.

Side effects: File I/O on every call (not cached). The manifest is small so this is not a bottleneck.

`_load`

```
@lru_cache(maxsize=64)
def _load(global_name: str) -> str
```

Parameters:

- `global_name: str` — the dataset name (matches a JSON file in `private_data/` without extension)

Returns: `str` — the raw JSON text of the dataset.

Decorator: `@lru_cache(maxsize=64)` — the dataset text is cached in memory after the first disk read. Since the data files are large and static (rebuilt by offline scripts, not by the server), caching eliminates repeated disk I/O for the most common datasets.

What it does: Constructs the path `config.PRIVATE_DATA_DIR / f"{global_name}.json"`. If the file does not exist, raises `HTTPException(404, f"Unknown dataset: {global_name}")`. Otherwise reads and returns the raw text (not parsed).

Design choice: Returns raw JSON string, not a parsed object, to avoid a parse + re-serialize round trip when assembling the bundle response.

`_bundle_json`

```
def _bundle_json(names: list[str]) -> str
```

Parameters:

- `names: list[str]` — list of dataset names to bundle

Returns: `str` — a single JSON object string with each dataset name as a key mapped to its dataset.

What it does: For each name, calls `_load(name)` to get the raw JSON text, then constructs `"name": rawJSON` pairs and joins them into `{pair1,pair2,...}`. This produces a valid JSON object without ever deserializing and re-serializing the large dataset payloads.

`_market_open`

```
def _market_open(now: dt.datetime | None = None) -> bool
```

Parameters:

- `now: dt.datetime | None` — if provided, uses this datetime instead of the real current time (for testing)

Returns: `bool` — `True` if US equity markets are currently open.

What it does:

1. Imports `zoneinfo.ZoneInfo` and creates an `America/New_York` timezone-aware datetime (current time if `now` is not provided).
2. If today is Saturday or Sunday (`weekday() >= 5`): returns `False`.
3. Computes `minutes = hour * 60 + minute`. Market hours are 09:30–16:00 ET, which is `570 <= minutes < 960`.
4. Returns `True` if within those bounds.

Error handling: If `zoneinfo` is unavailable (missing system timezone data), the `except Exception` clause returns `True` (does not block refreshes) to fail open rather than erroneously blocking legitimate requests.

Used by: The `/live` endpoint to gate lower-tier users during off-hours.

Data Endpoints

GET `/api/data/bundle`

```
@router.get("/bundle")
def bundle(user: dict = Depends(current_user))
```

Auth required: Yes (`current_user` dependency).

What it does:

1. Calls `_manifest()` to get the dataset lists.
2. Calls `_bundle_json(m["core"])` to assemble all core datasets into a single JSON object string.
3. Returns a `Response` with `media_type="application/json"` and the raw JSON string as the body.

Returns: JSON object containing all core datasets needed to render the app shell.

Security: Requires a valid session cookie. Returns 401 if unauthenticated, 503 if the data has not been built yet.

GET `/api/data/state`

```
@router.get("/state")
def state(user: dict = Depends(current_user))
```

Auth required: Yes (`current_user` dependency).

What it does:

1. Calls `_manifest()`.
2. Calls `_bundle_json(m["lazy"])` to assemble the state-legislature datasets.

3. Returns a `Response` with `media_type="application/json"`.

Returns: JSON object containing LegiScan + Open States data.

Design: These datasets are large and loaded lazily — the front-end calls this endpoint only when the user navigates to the State Legislature view, rather than on initial load.

GET /api/data/live

```
@router.get("/live")
def live(user: dict = Depends(current_user))
```

Auth required: Yes (`current_user` dependency).

What it does (step by step):

1. Looks up the user's tier in `config.TIERS` (falls back to `DEFAULT_TIER` if tier slug is unknown).
2. Gets `refresh_min = tier["refresh_min"]`.
3. **Market-hours gate:** If `refresh_min > 30` (i.e., tier is `"hourly"` or `"free"`) AND `_market_open()` is `False`: returns `{"paused": True, "reason": "market_closed"}` immediately without consuming a refresh slot.
4. **Cadence enforcement:** Computes `wait = refresh_min * 60 - (now - _last_live[user["email"]])`. If `wait > 0`: raises `HTTPException(429, f"Refresh available in {int(wait)}s")` with `Retry-After: {wait}` header.
5. Records `_last_live[user["email"]] = now`.
6. Calls `_manifest()` and `_bundle_json(m["live"])`.
7. Returns a `Response` with `media_type="application/json"`.

Returns: JSON object with current prices and news data, or `{"paused": True, "reason": "market_closed"}`.

Rate-limit behavior by tier:

Tier	refresh_min	Market-hours gate applies?
free	1440 (24 hrs)	Yes (> 30 min)
hourly	60	Yes (> 30 min)
half	30	No (== 30 min)
live	15	No (< 30 min)
beta	15	No (< 30 min)

Security: Both the cadence and the market-hours gate are enforced server-side. The browser cannot bypass them by modifying JavaScript or sending extra requests — the server tracks `_last_live` per user email.

9. File: `main.py`

Responsibility: The application entry point. Creates the FastAPI app instance, wires up routers and middleware, mounts static file directories, and defines two simple top-level routes (`/healthz` and `/`). Also enforces the startup guard (refuse to run in production with a placeholder secret).

Startup Guard

```
# main.py:31-32
if config.PRODUCTION and config.is_placeholder_secret():
    raise RuntimeError("TF_SECRET_KEY is unset/placeholder. ...")
```

Called at module load time (before the ASGI server accepts any connections). If `TF_PRODUCTION=1` is set but `TF_SECRET_KEY` is still the default placeholder, the process exits with `RuntimeError`. This prevents a misconfigured production deploy from silently running with a guessable signing key.

FastAPI Instance

```
app = FastAPI(title="ThinkFree Finance", docs_url=None, redoc_url=None, openapi_url=None)
```

`docs_url=None`, `redoc_url=None`, `openapi_url=None` — all auto-generated API documentation endpoints are disabled. The interactive Swagger UI and OpenAPI schema are not accessible in any environment.

Initialization Sequence

```
db.init_db()          # Create/migrate the SQLite users table
app.include_router(auth.router)  # Mount /api/auth/*
app.include_router(data.router)  # Mount /api/data/*
```

These three lines run at import time, before the ASGI server starts accepting requests.

CORS Middleware

```
app.add_middleware(
    CORSMiddleware,
    allow_origins=config.ALLOWED_ORIGINS,
    allow_credentials=True,
    allow_methods=["GET", "POST", "OPTIONS"],
    allow_headers=["Content-Type"],
)
```

Configuration details:

- `allow_origins` — from `config.ALLOWED_ORIGINS` (env-configured, not `"*"`)
- `allow_credentials=True` — required because the session travels in a cookie; this means the wildcard `"*"` origin is forbidden by the CORS spec and would cause an error
- `allow_methods` — only `GET`, `POST`, and `OPTIONS` (no `PUT`, `DELETE`, `PATCH`)
- `allow_headers` — only `Content-Type`

The source comment notes: in the recommended single-server setup, the front-end and API share one origin, so the browser never makes a cross-origin request and this middleware is effectively inert. It is present for deployments where the front-end is hosted on a different domain.

Blocked Path Constants

```
_BLOCKED_SUFFIXES = ("_data.js", "news_intel.js", "member_bills.js")
```

A tuple of filename suffixes that must never be served directly. Even if copies of these files still exist under `webapp/js/` (during the migration from the old public-data architecture), the middleware returns 404 before the static file handler can serve them.

Middleware

security_and_blocklist

```
@app.middleware("http")
async def security_and_blocklist(request: Request, call_next)
```

Decorator: `@app.middleware("http")` — applied to every HTTP request and response.

Parameters:

- `request: Request` — the incoming ASGI request
- `call_next` — callable; passes the request to the next handler in the stack

Returns: A `Response` object.

What it does (step by step):

Step 0 — Production HTTPS redirect (applied before path checks):

- If `config.PRODUCTION` is `True` AND the path is not `"/healthz"` (health check is exempted to avoid redirecting the load balancer probe):
 - Reads the `X-Forwarded-Proto` header (set by the TLS-terminating proxy). Takes the first value (before any comma), strips whitespace. Defaults to `"https"` when the header is absent (so a direct HTTPS hit is never redirected).
 - If the scheme is `"http"`: returns `RedirectResponse` to the same URL with `scheme="https"` and HTTP **308** (Permanent Redirect, preserving the request method).

Step 1 — Blocked path enforcement:

- If the path starts with `"/js/"` AND ends with any of `_BLOCKED_SUFFIXES`: returns `JSONResponse({"detail": "Not found"}, status_code=404)`.

- This blocks direct browser access to `*_data.js`, `news_intel.js`, and `member_bills.js` regardless of where the files sit on disk.

Step 2 — Pass to next handler:

- Calls `await call_next(request)` to execute the actual route handler or static file handler.

Step 3 — Security headers (applied to all responses):

Header	Value	Purpose
X-Content-Type-Options	<code>nosniff</code>	Prevents browsers from MIME-sniffing responses away from the declared content type
X-Frame-Options	<code>DENY</code>	Prevents the app from being embedded in any iframe (clickjacking defense)
Referrer-Policy	<code>no-referrer</code>	Suppresses the <code>Referer</code> header on all outgoing requests, preventing URL leakage
Permissions-Policy	<code>geolocation=(), microphone=(), camera=()</code>	Disables access to sensitive browser APIs
Content-Security-Policy	See below	Restricts resource loading to trusted sources

CSP value:

```
default-src 'self';
script-src 'self';
style-src 'self' 'unsafe-inline';
img-src 'self' data: https:;
connect-src 'self';
frame-ancestors 'none';
base-uri 'self'
```

- Scripts: only from the same origin (no CDN, no inline scripts)
- Styles: same origin plus `'unsafe-inline'` (because the UI uses `style=""` attributes)
- Images: same origin, `data:` URIs, and any `https:` URL
- `frame-ancestors 'none'`: reinforces X-Frame-Options at the CSP level
- `base-uri 'self'`: prevents base-tag injection attacks

Step 4 — HSTS (production only):

```
Strict-Transport-Security: max-age=31536000; includeSubDomains
```

Applied only when `config.PRODUCTION` is `True`. Instructs browsers to only contact the server over HTTPS for the next 365 days, including all subdomains.

Static Mounts

```
app.mount("/css", StaticFiles(directory=config.WEBAPP_DIR / "css"), name="css")
app.mount("/js", StaticFiles(directory=config.WEBAPP_DIR / "js"), name="js")
```

Serves `webapp/css/*` at `/css/*` and `webapp/js/*` at `/js/*`. The `_BLOCKED_SUFFIXES` check in the middleware intercepts requests for blocked filenames before they reach these handlers.

Top-Level Routes

GET /healthz

```
@app.get("/healthz")
def healthz()
```

Auth required: No.

What it does: Returns `{"ok": True}`.

Returns: JSON `{"ok": True}`.

Purpose: Simple liveness probe for the hosting platform's load balancer or health-check system. Deliberately exempted from the HTTPS redirect (step 0 of the middleware) so the host's HTTP probe is never redirected.

GET /

```
@app.get("/")
def index()
```

Auth required: No (the HTML shell itself is public; data is gated behind `/api/data/*`).

What it does: Returns `FileResponse(config.WEBAPP_DIR / "index.html")`.

Returns: The `webapp/index.html` app shell.

Design: The shell HTML contains no data and no secrets. It loads JS and CSS, checks for a session cookie via `GET /api/auth/me`, and redirects to login if unauthenticated. All actual data is fetched from the authenticated `/api/data/*` endpoints after login.

10. Endpoints Quick-Reference Table

Method	Path	Handler	Auth Required	Returns
GET	/healthz	main.healthz	No	{"ok": True}
GET	/	main.index	No	webapp/index.html (FileResponse)
POST	/api/auth/signup	auth.signup	No	{"user": {...}} + sets session cookie
POST	/api/auth/login	auth.login	No	{"user": {...}} + sets session cookie
POST	/api/auth/logout	auth.logout	No (graceful)	{"ok": True} + clears cookie, revokes JWT
GET	/api/auth/me	auth.me	Yes	{"user": {...}}
POST	/api/auth/profile	auth.save_profile	Yes	{"user": {...}} (updated)
POST	/api/auth/ally	auth.save_ally	Yes	{"ok": True}
GET	/api/data/bundle	data.bundle	Yes	JSON bundle of core datasets
GET	/api/data/state	data.state	Yes	JSON bundle of state-legislature datasets
GET	/api/data/live	data.live	Yes	JSON bundle of live data, or {"paused": True, "reason": "market_closed"}
GET	/css/*	StaticFiles	No	CSS files from webapp/css/
GET	/js/*_data.js	Middleware blocklist	—	404 (blocked)
GET	/js/news_intel.js	Middleware blocklist	—	404 (blocked)
GET	/js/member_bills.js	Middleware blocklist	—	404 (blocked)
GET	/js/* (other)	StaticFiles	No	JS files from webapp/js/

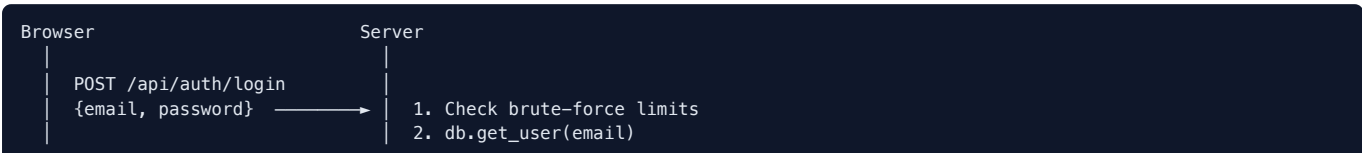
Notes:

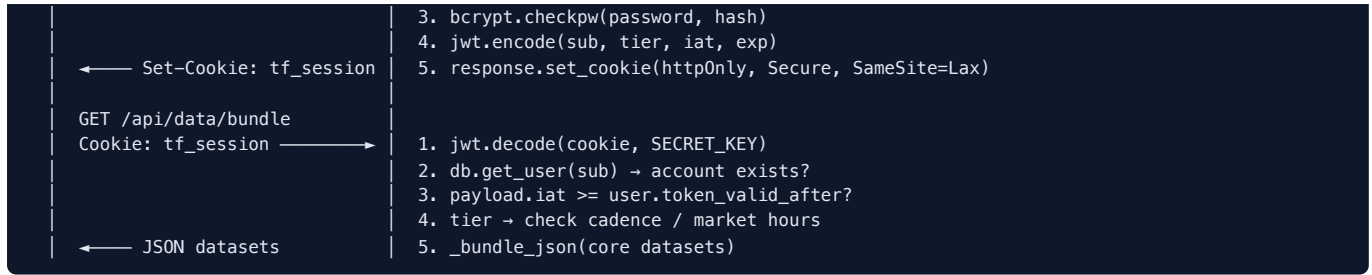
- `docs_url`, `redoc_url`, and `openapi_url` are all set to `None` — no auto-generated documentation is accessible.
- All CORS is scoped to `config.ALLOWED_ORIGINS` (never `"*"`).
- All endpoints except `/healthz` are subject to the security headers middleware.

11. Security Architecture Summary

This section synthesizes the security mechanisms across all files into a single reference.

Authentication Flow





Security Controls Index

Control	Location	Mechanism
Password hashing	<code>auth.py:hash_password</code>	bcrypt with random salt; <code>bcrypt.gensalt()</code> per call
Password verification	<code>auth.py:verify_password</code>	<code>bcrypt.checkpw</code> (constant-time); exception-safe
Session token	<code>auth.py:_make_token</code>	JWT signed HS256 with <code>TF_SECRET_KEY</code> ; 7-day expiry
Cookie security	<code>auth.py:_set_session_cookie</code>	<code>httpOnly=True</code> , <code>Secure=PRODUCTION</code> , <code>SameSite=lax</code>
JWT revocation	<code>db.py:update_token_valid_after</code> + <code>auth.py:current_user</code>	<code>token_valid_after</code> epoch; bumped on logout
Invite-key comparison	<code>auth.py:signup</code>	<code>hmac.compare_digest</code> (constant-time; prevents timing attacks)
Login brute-force	<code>auth.py:_check_locked</code>	Rolling-window deque; per (IP+account) and per-IP
Signup brute-force	<code>auth.py:_check_locked</code>	Rolling-window deque; per IP
User enumeration	<code>auth.py:login</code>	Single generic error for wrong email OR wrong password
Age floor	<code>auth.py:save_profile</code>	Server-side: <code>age < 13</code> → 400
Consent	<code>auth.py:signup</code>	Server-side: <code>consent == False</code> → 400
Password stripping	<code>auth.py:_public_user</code>	Removes <code>pass_hash</code> before any data leaves the server
Startup secret guard	<code>main.py:31-32</code>	Refuses to start in production with placeholder key
Data file blocklist	<code>main.py:security_and_blocklist</code>	404 for <code>*_data.js</code> , <code>news_intel.js</code> , <code>member_bills.js</code>
HTTPS redirect	<code>main.py:security_and_blocklist</code>	308 redirect for <code>X-Forwarded-Proto: http</code> in production
Security headers	<code>main.py:security_and_blocklist</code>	X-Content-Type-Options, X-Frame-Options, Referrer-Policy, Permissions-Policy, CSP, HSTS
CORS	<code>main.py</code>	Explicit allow-list; never <code>"*"</code> ; credentials allowed
OpenAPI docs	<code>main.py</code>	All disabled (<code>docs_url=None</code> , etc.)
SQL injection	<code>db.py</code> (all functions)	Parameterized queries throughout; no string interpolation
Per-tier cadence	<code>data.py:live</code>	In-memory <code>_last_live</code> enforced server-side
Market-hours gate	<code>data.py:_market_open</code>	DST-aware America/New_York check; blocks lower tiers after hours
Secrets in env	<code>config.py:_get</code>	All sensitive values read from environment variables only
Database gitignored	<code>server/.gitignore</code>	<code>users.db</code> never committed
<code>.env</code> gitignored	<code>server/.gitignore</code>	<code>.env</code> never committed

Part 6A — Frontend Reference: Core (boot.js, auth.js, app.js)

This document is a complete, function-by-function reference for the three JavaScript files that form the entire operational spine of the ThinkFree Finance static webapp. Every function is documented with its signature, parameter types, return value, step-by-step behavior, side effects, and design rationale. Line citations use the format `file:line`.

Table of Contents

1. [Script Load Order and HTML Structure](#)
2. [boot.js — Bootstrap and Dual-Mode Orchestrator](#)
3. [auth.js — Beta Gate, Session, and Tier Model](#)
4. [app.js — Main Controller, Router, and View Renderer](#)
5. [Window Globals Cross-Reference](#)

1. Script Load Order and HTML Structure

1.1 The HTML Shell (`index.html`)

`index.html` (`webapp/index.html`) is a single-page shell. Its `<body>` starts with `class="tf-locked"` applied statically — this hides all app content before any JavaScript runs (`index.html:9`). It contains:

- **#tf-splash** (`index.html:10–14`): A static, CSS-animated loading logo rendered by the browser immediately, before any JS. Removed by `killSplash()` in `auth.js`.
- **.app** (sidebar + **.main**): The full application chrome — sidebar nav, user chip, dark-mode toggle, top bar with greeting and search input, the viewport with eleven `<section class="page">` elements (one per route), and the shared modal overlay.
- **#modal** / **#modalBox** / **#modalContent** (`index.html:112–117`): The shared detail overlay used for politician profiles, bill details, stock details, trades views, event details, and the accessibility statement.
- **#ticker** (`index.html:75`): An empty `<div>` where `initTicker()` in `app.js` injects the scrolling market ticker.
- **#tf-updated-slot** (`index.html:68`): A `<div>` in the sidebar footer where `auth.js`'s `stampEl()` injects the live-data status line.

1.2 Script Load Order

Three `<script defer>` tags at the bottom of `<body>` (`index.html:125–129`):

```
1. js/boot.js      - IIFE; runs immediately at eval time; sets window.TFBoot
2. js/auth_seed.js - local-only / gitignored dev helper; absent on server (404, harmless)
3. js/auth.js      - IIFE; runs immediately; sets window.Auth; calls boot() on DOMContentLoaded
```

Because all three use `defer`, they evaluate in document order after the DOM is parsed but before `DOMContentLoaded` fires. The data scripts (`js/*_data.js`) and all logic scripts (`js/scores.js`, `js/app.js`, etc.) are **not** present as static `<script>` tags; they are injected dynamically by `boot.js`'s `ensureAppLoaded()` only after authentication, which is why the old list of `<script>` tags was removed from `index.html` (see the comment at `index.html:119–124`).

1.3 Evaluation-Time Data Capture

Every logic script captures its data globals into a module-level `const` at script-eval time. For example, `app.js` line 15:

```
const D = window.TF_DATA || {};
```

This is the central design constraint that drives `boot.js`'s entire architecture: if `app.js` evaluates before `window.TF_DATA` exists, `D` is permanently `{}` for the lifetime of the page. `Boot.js` guarantees that all `window.*_DATA` globals are populated **before** any logic script is injected.

2. boot.js — Bootstrap and Dual-Mode Orchestrator

File: `webapp/js/boot.js` **Structure:** A single IIFE (`"use strict"; (function(){ ... })()`), with `window.TFBoot` as its only export.

Purpose: Detect whether the app is running under the FastAPI backend ("server" mode) or as a plain static file ("static" mode), then orchestrate loading all data globals and logic scripts in the correct order, after authentication.

2.1 Module-Level Constants

DATA_SCRIPTS (boot.js:34–39)

```
const DATA_SCRIPTS = [
  "js/data.js", "js/influence_data.js", "js/relationships_data.js", "js/nonprofit_data.js",
  "js/fec_data.js", "js/sec_data.js", "js/secbulk_data.js", "js/usaspending_data.js",
  "js/states_data.js", "js/sp500_data.js", "js/quant_data.js", "js/member_bills.js",
  "js/prices_data.js", "js/news_intel.js",
];
```

An ordered list of data script paths. Each file, when evaluated, assigns a `window.*_DATA` global (e.g., `window.TF_DATA`, `window.PRICES_DATA`, `window.NEWS_INTEL`). In **static mode** these are injected as `<script>` tags (in this exact order) so that each global exists before the next file executes. In **server mode** these files are 404-blocked by `server/main.py`; the same globals are populated from `/api/data/bundle` and `/api/data/live` API responses instead.

Not in this list: `js/legiscan_data.js` and `js/openstates_data.js`. Those are ~4.5 MB combined and are lazy-loaded by `app.js`'s `ensureStateData()` only on the first visit to the State Rankings page.

LOGIC_SCRIPTS (boot.js:45–49)

```
const LOGIC_SCRIPTS = [
  "js/scores.js", "js/genimpact.js", "js/app.js", "js/influencweb.js",
  "js/usmap_paths.js", "js/congress.js", "js/glossary.js", "js/scoreinfo.js",
  "js/predictions.js",
];
```

Scripts that read the window globals populated by `DATA_SCRIPTS`. Loaded in both modes, always after data globals exist, in this order to preserve the relative eval ordering from the old `index.html`. `js/usmap_paths.js` contains static SVG path geometry (not sensitive, not blocked by the server), so it loads normally in both modes.

2.2 Functions

injectScript(src) (boot.js:52–59)

Signature: `function injectScript(src: string): Promise<string>` **Returns:** A `Promise` that resolves with `src` when the script loads, or rejects with an `Error` if it fails.

Behavior:

1. Creates a new `<script>` element.
2. Sets `s.src = src`.
3. Attaches `onload` → `resolve(src)` and `onerror` → `reject(new Error(...))`.
4. Appends the element to `document.body`.

Design note: Appending to `body` (not `head`) ensures the script loads after the existing DOM. There is no `async` or `defer` attribute on the injected tag, so the browser loads and evaluates each script as it is appended. The sequential guarantee is enforced by `loadInOrder`, not by the tag attributes.

loadInOrder(list) (boot.js:63–65)

Signature: `async function loadInOrder(list: string[]): Promise<void>` **Returns:** `Promise<void>` — resolves when every script in the list has been injected and evaluated, in order.

Behavior:

- Iterates `list` sequentially with `for...await`. Each `await injectScript(list[i])` pauses the loop until the current script's `onload` fires before the next script tag is created.

Design note: This is the mechanism that makes the data-before-logic guarantee work. A `Promise.all` would inject all tags in parallel and break the eval order.

api(path, opts) (boot.js:70–81)

Signature: `async function api(path: string, opts?: {method?: string, body?: any}): Promise<{ok: boolean, status: number, data: any}>` **Returns:** An object `{ok, status, data}`. Never rejects (only network failures would propagate, but the surrounding callers

wrap in try/catch).

Parameters:

- `path`: A same-origin URL path, e.g., `"/api/healthz"` or `"/api/data/bundle"`.
- `opts.method`: HTTP method; defaults to `"GET"`.
- `opts.body`: If present, serialized to JSON; `Content-Type: application/json` is added.

Behavior:

1. Builds a `fetch` init object with `credentials: "same-origin"` so the browser sends the session cookie on every request.
2. If `opts.body` is defined, stringifies it and adds the JSON Content-Type header.
3. Calls `fetch(path, init)`.
4. Attempts `res.json()` in a try/catch; sets `data = null` on parse failure (e.g., a 204 No Content or an HTML error page).
5. Returns `{ok: res.ok, status: res.status, data}`.

Design note: Callers branch on `res.status` (401/403/409/429) directly without try/catch noise. Returning `{ok, status, data}` instead of throwing keeps all error logic in the caller as readable `if` branches.

Exported as: `window.TFBoot.api` — consumed by `auth.js` for all `/api/auth/*` calls and by `app.js`'s `ensureStateData()`.

`detectMode()` (boot.js:86–93)

Signature: `async function detectMode(): Promise<"server" | "static">` **Returns:** `"server"` if `/api/healthz` responds with HTTP 200 OK; `"static"` otherwise.

Behavior:

1. `fetch("/api/healthz", { credentials: "same-origin" })`.
2. If the fetch succeeds and `res.ok` is true → `"server"`.
3. If the response is not OK (404, 500, etc.) → `"static"`.
4. If `fetch` itself throws (network error, `file://` origin blocked, CORS) → catches and returns `"static"`.

Design note: Three cases all collapse to "static": (a) `file://` protocol (fetch throws a `TypeError`), (b) a dumb static host that 404s the `healthz` path, (c) any network failure. Only the FastAPI backend, which explicitly handles `GET /api/healthz` with a 200, triggers server mode.

`populateData(mode)` (boot.js:112–122)

Signature: `async function populateData(mode: "server" | "static"): Promise<void>`

Behavior — server mode:

1. Calls `api("/api/data/bundle")`. If `ok` and `data` exist, `Object.assign(window, bundle.data)` — the server returns a JSON object whose keys are exactly the `window.*_DATA` names the logic scripts expect (e.g., `{TF_DATA: {...}, FEC_DATA: {...}, ...}`).
2. Calls `api("/api/data/live")`. If `ok` and `data` and `!data.paused`, `Object.assign(window, live.data)` — adds or overwrites `window.PRICES_DATA` and `window.NEWS_INTEL` with the freshest live data.
3. If `live.data.paused` is true (server-side market-hours pause), the live globals are not set for this initial load (the old data is stale, but the UI shows a "markets closed" stamp via `stamp()` in `auth.js`).

Behavior — static mode:

1. Calls `loadInOrder(DATA_SCRIPTS)` — injects each `js/*_data.js` as a `<script>` tag in order. Each script assigns its global (`window.TF_DATA = {...}`, etc.) as a side effect of evaluation.

`ensureAppLoaded()` (boot.js:127–137)

Signature: `function ensureAppLoaded(): Promise<void>` **Returns:** A `Promise<void>` that resolves when (a) the data globals are set, (b) all logic scripts have evaluated, and (c) `window.TF.init` is confirmed to exist.

Behavior:

1. **Idempotency check:** If `appLoadedPromise` is already set (a previous call was made), returns the same promise. Subsequent calls await the same single chain — no double-loading.
2. Awaits `ready` (the mode detection promise) to get the confirmed mode string.
3. Awaits `populateData(mode)`.
4. Awaits `loadInOrder(LOGIC_SCRIPTS)` — injects `scores.js`, `genimpact.js`, `app.js`, etc.

5. Guards: if `window.TF && window.TF.init` is falsy after all scripts load, throws `Error("ThinkFree app failed to initialize")`. This surfaces as the "Could not load your dashboard" error in `auth.js:unlock`.

Side effects:

- Appends 9+ `<script>` elements to `document.body`.
- Sets all `window.TF_DATA`, `window.PRICES_DATA`, etc. globals.
- `app.js` assigns `window.TF.init` synchronously at its eval time, making `window.TF` available immediately after `loadInOrder(LOGIC_SCRIPTS)` resolves.

Called by: `auth.js:unlock()` — awaited before calling `window.TF.init()`.

2.3 window.TFBoot Public API

Assigned at `boot.js:139–147`:

```

window.TFBoot = {
  ready,           // Promise<"server"|"static"> – resolves once mode is known
  mode,           // "server"|"static"|null – set synchronously after ready resolves
  ensureAppLoaded, // () => Promise<void> – idempotent boot + script loader
  api,            // (path, opts?) => Promise<{ok, status, data}>
  DATA_SCRIPTS,  // string[] – the ordered data script list (exposed for debugging)
  LOGIC_SCRIPTS,  // string[] – the ordered logic script list (exposed for debugging)
};

```

`ready` is the pending promise from `detectMode().then(...)`. The `.then` callback sets `TFBoot.mode` before resolving the promise, so any awaiter of `ready` can immediately read `TFBoot.mode` synchronously.

Static mode warning: The `.then` callback also calls `console.warn` (`boot.js:102`) with a reminder that the localStorage gate is a dev fallback, not real security.

3. auth.js — Beta Gate, Session, and Tier Model

File: `webapp/js/auth.js` **Structure:** A single IIFE (`"use strict"; (function(){ ... })()`), with `window.Auth` as its only export.

Purpose: Manage the entire pre-authentication flow: login, beta key verification, account signup, behavioral-quiz profile creation, accessibility onboarding, session management, live-data refresh scheduling, and logout. The gate overlay is rendered and controlled entirely from within this IIFE; no auth HTML exists in `index.html`.

3.1 Module-Level Constants and Variables

BETA_HASH (auth.js:28)

```
const BETA_HASH = "5sw0ilypx6";
```

The `cyrb53` hash of the plaintext beta invite key (`"<redacted – configured via TF_BETA_KEY>"`). In **static mode**, `cyrb53(enteredKey) === BETA_HASH` is the pre-check before the signup form is shown. In **server mode**, this constant is present but unused for the actual gate check — the plaintext key is never in the bundle; instead, the entered key is sent to `/api/auth/signup` where the server validates it in constant time.

TIERS (auth.js:44–50)

```

const TIERS = {
  free: { id: "free", name: "Free", refreshMin: 1440, price: "$0", blurb: "Daily news briefing" },
  hourly: { id: "hourly", name: "Pro · Hourly", refreshMin: 60, price: "$9/mo", blurb: "Refreshes every hour" },
  half: { id: "half", name: "Pro · 30-min", refreshMin: 30, price: "$19/mo", blurb: "Refreshes every 30 minutes" },
  live: { id: "live", name: "Pro · 15-min", refreshMin: 15, price: "$39/mo", blurb: "Refreshes every 15 minutes – fastest feed" },
  beta: { id: "beta", name: "Beta Access", refreshMin: 15, price: "Free (beta)", blurb: "Full 15-minute access during beta" },
};

```

Each tier controls how frequently the live-data interval fires (`refreshMin` × 60 × 1000 ms). `beta` tier grants 15-minute cadence for free during the closed beta period. Prices are placeholder values, noted in source as needing to be set before launch.

Exported as: `window.Auth.TIERS`.

LS — localStorage Helper (auth.js:55-60)

```
const LS = {
  get accounts() { try { return JSON.parse(localStorage.getItem("tf_accounts")) || "{}"; } catch (e) { return {}; } },
  set accounts(v) { localStorage.setItem("tf_accounts", JSON.stringify(v)); },
  get session() { return localStorage.getItem("tf_session") || ""; },
  set session(v) { v ? localStorage.setItem("tf_session", v) : localStorage.removeItem("tf_session"); },
};
```

A thin abstraction over `localStorage`. The accounts object is a dictionary keyed by email address; each value holds `{ name, pass (cyrb53 hash), tier, ally, profile, created }`. The session is a single email string pointing to the active account. Setting `session` to a falsy value removes the key entirely (used on logout).

Design note: `LS` is only read and written in **static mode**. In server mode, the session lives in an httpOnly cookie managed by the FastAPI backend; `LS` is never touched.

E — HTML Escaper (auth.js:63)

```
const E = (s) => String(s ?? "").replace(/&<>"/g, (c) => ({ "&": "&amp;", "<": "&lt;", ">": "&gt;", "'": "&quot;" }[c]));
```

A minimal HTML-escaper used everywhere user-supplied text is interpolated into `innerHTML`. Guards against XSS in the auth gate views.

validId (auth.js:66)

```
const validId = (s) => /^[^\s@]+@[^\s@]+\.[^\s@]+$/i.test(s) || /^[a-z0-9_-]{3,}$/i.test(s);
```

Accepts either a valid email address or a simple alphanumeric username (3+ characters). Used on the login form to allow logging in by email or a username alias (e.g., `"admin"`).

validEmail (auth.js:71)

```
const validEmail = (s) => /^[^\s@]+@[a-z0-9](?:[a-z0-9-]*[a-z0-9])?(?:\.[a-z0-9](?:[a-z0-9-]*[a-z0-9])?)*\.[a-z]{2,}$/i.test(s);
```

A stricter check used on the signup form. Requires a TLD of 2+ letters. Rejects obviously malformed emails like `"a@b"` or `"a@b.1"`. A real-looking email that passes this check is still subject to the async `emailDomainReal()` domain-existence check.

SERVER() (auth.js:103)

```
const SERVER = () => !(window.TFBoot && window.TFBoot.mode === "server");
```

A zero-argument function (not a stored value) that reads `window.TFBoot.mode` at call time. Every auth/data branch that differs between modes calls `SERVER()` to decide which path to take. Using a function rather than a constant means it correctly reflects the mode even if called before `TFBoot.ready` has settled (in which case `mode` is still `null`, and `SERVER()` returns `false`, correctly defaulting to static behavior).

betaKeyEntered (auth.js:108)

```
let betaKeyEntered = "";
```

Carries the invite key entered in the `"beta"` view forward to the signup POST. In server mode the local cyrb53 check is skipped; the raw key string is sent to `/api/auth/signup` where the server validates it. If the server returns 403, the UI re-renders the `"beta"` view and shows the error message.

currentUser / **refreshTimer** / **unlocked** (auth.js:139)

```
let currentUser = null, refreshTimer = null, unlocked = false;
```

- currentUser**: The active account object. In static mode it is `accountFor()`'s return value (email + LS entry). In server mode it is the enriched server response from `adoptServerUser()`. Set to `null` on logout.
- refreshTimer**: The `setInterval` handle for the live-data refresh loop. Stored so it can be `clearInterval`'d when the tier changes or the user logs out.

- `unlocked`: Boolean; `true` after successful login. The `guard()` tamper-protection checks this — once true it stops re-locking. Set to `false` on logout.

Immediate Lock (auth.js:144)

```
if (document.body) document.body.classList.add("tf-locked");
```

Applied at IIFE eval time (before `DOMContentLoaded`). If `document.body` is already available (e.g., the script is in a deferred position), the class is applied immediately, hiding the app before any content renders. This is belt-and-suspenders alongside the `class="tf-locked"` already on `<body>` in the HTML.

3.2 Functions

`cyrb53(str, seed)` (auth.js:33–39)

Signature: `function cyrb53(str: string, seed?: number): string` **Returns:** A base-36 string hash of `str`.

Behavior:

- Implements the cyrb53 non-cryptographic hash algorithm. Uses two 32-bit integers, applies the MurmurHash3-style mixing of `Math.imul`, and combines them into a 53-bit integer.
- The `seed` parameter (default `0`) allows salting; it is always called without a seed in this codebase.
- Returns `(4294967296 * (2097151 & h2) + (h1 >>> 0)).toString(36)` — a compact base-36 string.

Usage:

- `cyrb53(betaKey) === BETA_HASH` — beta key verification in static mode (`bind()`, data-act `"verify"`).
- `cyrb53(pass)` — password hashing for static-mode account storage. Passwords are stored and compared as their cyrb53 hash, never in plaintext.

Design note: cyrb53 is explicitly not suitable for real authentication (no salt per account, not cryptographically resistant to preimage attacks). The code comment says "Good enough for a soft gate; not suitable for real auth." Real authentication uses the FastAPI backend with bcrypt.

`emailDomainReal(email)` (auth.js:79–97)

Signature: `async function emailDomainReal(email: string): Promise<boolean>` **Returns:** `true` if the email's domain appears to exist and be able to receive mail; `false` if the domain is definitively nonexistent.

Behavior:

1. Extracts the domain from the email (`email.split("@")[1]`). If absent or has no `.`, returns `false`.
2. Defines a helper `ask(type)` that fetches `https://dns.google/resolve?name={domain}&type={type}` (Google DNS-over-HTTPS), returning parsed JSON or `null` on failure.
3. Calls `ask("MX")`:
 - If the DNS response is `null` (network unreachable), returns `true` — **fail open** to avoid locking out real users on flaky networks.
 - If `mx.Status === 3` (NXDOMAIN — domain does not exist in DNS), returns `false`.
 - If `mx.Answer` is non-empty, returns `true` (domain has a real mail server).
4. If MX lookup returns no answer records, falls back to an `ask("A")` lookup:
 - If `null` → `true` (fail open).
 - If `a.Answer` is non-empty → `true` (domain resolves at all, even without explicit MX).
 - Otherwise → `false`.
5. Any uncaught exception → `true` (fail open).

Side effects: Disables and re-enables the signup button during the async DNS lookup (`bind()`, data-act `"signup"`) to prevent double-submission.

Design note: The fail-open policy means a real user with a legitimate but temporarily unreachable domain is never locked out. The cost is that some nonsense domains with DNS outages might pass, but those users still require a real password and beta key, so the gate is still effective.

`adoptServerUser(u)` (auth.js:114–119)

Signature: `function adoptServerUser(u: object): object` **Returns:** The same user object, mutated with an enriched `profile` if the raw answers are present.

Behavior:

1. Checks if `u && u.profile && u.profile.experience`. If so, calls `computeProfile(u.profile)` and overwrites `u.profile` with the enriched result.
2. If `computeProfile` throws (e.g., malformed answers), silently keeps the raw profile.
3. Returns `u`.

Purpose: The server persists only the five raw profile answers (`{age, experience, goal, timeline, emotional}`). The UI expects the enriched shape produced by `computeProfile`: `{age, experience, goal, timeline, emotional, risk_tolerance, risk_score, profile_id}`. This function bridges the gap so that server-mode and static-mode sessions look identical to `app.js` — which reads `p.risk_tolerance` and `p.goal` when tailoring news.

Called by: All server-mode authentication responses: `boot()` (`/api/auth/me`), `login` handler (`/api/auth/login`), `signup` handler (`/api/auth/signup`), `quiz-submit` handler (`/api/auth/profile`).

`computeProfile(a)` (auth.js:125–137)

Signature: `function computeProfile(a: {age: string|number, experience: string, goal: string, timeline: string, emotional: string}): object` **Returns:** An enriched profile object: `{age, experience, goal, timeline, emotional, risk_tolerance, risk_score, profile_id}`.

Behavior — point accumulation:

Condition	Points
<code>age < 30</code>	+2
<code>30 ≤ age < 50</code>	+1
<code>experience === "advanced"</code>	+2
<code>experience === "intermediate"</code>	+1
<code>goal === "growth"</code>	+2
<code>goal === "income"</code>	+1
<code>emotional === "buy more"</code>	+2
<code>emotional === "hold"</code>	+1

Risk classification:

- `pts ≥ 7` → `"high"`
- `pts ≥ 4` → `"moderate"`
- `pts < 4` → `"low"`

Profile ID construction:

- `ageGroup`: `"A1_<25"` | `"A2_25–34"` | `"A3_35–50"` | `"A4_50+"`.
- `riskGroup`: `"R1_Cons"` | `"R2_Mod"` | `"R3_High"`.
- `profile_id` = `ageGroup` + `"_"` + `riskGroup` (e.g., `"A2_25–34_R2_Mod"`).

Design note: This is a client-side port of `phase1_user_personalization.py` so that the web front-end and the Python pipeline share identical scoring logic. The `profile_id` is a compact string suitable as a cache/personalization key.

Exported as: `window.Auth.computeProfile`.

`accountFor()` (auth.js:148–151)

Signature: `function accountFor(): object | null` **Returns:** The current session's account object with `email` merged in, or `null` if no session exists.

Behavior:

1. Reads `LS.accounts[LS.session]` — the stored account for the current session email.
2. If found, returns `{ email: LS.session, ...a }`.
3. If `LS.session` is empty or no matching account exists, returns `null`.

Used by: `enter()`, `boot()` (static path), and the `ally-on/off` and `quiz-submit` handlers after LS mutations.

killSplash() (auth.js:158)**Signature:** `function killSplash(): void`**Behavior:** Finds `#tf-splash` by ID and removes it from the DOM. No-op if already removed. Called by `showGate()` and `unlock()`.**Design note:** The splash screen is a pure HTML/CSS animation that the browser renders immediately during the initial parse, giving the user visual feedback before any JavaScript runs. It must be explicitly removed once auth/app takes over.**showGate(view)** (auth.js:162–167)**Signature:** `function showGate(view?: string): void`**Behavior:**

1. Calls `killSplash()`.
2. If `gate` (the module-level overlay element) does not yet exist, creates a `<div class="auth-gate">` and appends it to `document.body`.
3. Sets `gate.style.display = "flex"` to make it visible.
4. Calls `render(view || "login", {})` to populate the gate with the requested view's HTML.

Called by: `boot()` when no valid session is found, and `logout()` after the goodbye animation finishes (to show the login view again without reloading the page).**hideGate()** (auth.js:169)**Signature:** `function hideGate(): void`**Behavior:** Sets `gate.style.display = "none"` if gate exists. Not currently called in the main flow (the gate is removed from the DOM entirely on unlock); retained as a utility.**shell(inner)** (auth.js:173–179)**Signature:** `function shell(inner: string): string` **Returns:** An HTML string wrapping `inner` in the branded auth card frame.**Behavior:** Returns the outer card structure including the "TF" logo mark, the "ThinkFree Finance" brand name, the `inner` HTML, and the legal disclaimer footer. Keeps individual view templates lean by centralizing the frame.**render(view, ctx)** (auth.js:183–226)**Signature:** `function render(view: "login"|"beta"|"signup"|"quiz"|"ally", ctx: object): void`**Behavior:** Builds the inner HTML for the requested view, wraps it in `shell()`, and assigns to `gate.innerHTML`. The five views are:

- **"login"**: Email + password inputs, a Log In button (`data-act="login"`), and a link to the beta key entry view (`data-act="to-beta"`). Uses `autocomplete="username"` and `autocomplete="current-password"` for browser autofill support.
- **"beta"**: A single text input for the invite key, a Verify button (`data-act="verify"`), and a back link (`data-act="to-login"`).
- **"signup"**: Name, email, and password inputs; an age/consent checkbox (`#au-consent`); a Create Account button (`data-act="signup"`); a note that beta accounts get the 15-minute tier free.
- **"quiz"**: Five form controls — a number input for age, and four `<select>` elements for experience, goal, timeline, and emotional response. Pre-fills from `currentUser.profile` if the user is revisiting from Settings. A Continue button (`data-act="quiz-submit"`).
- **"ally"**: A plain question with two buttons — `data-act="ally-on"` and `data-act="ally-off"`. No inputs.

Design note: All event wiring is done via the single delegated listener in `bind()` using `data-act` attributes, not inline `onclick` handlers. This keeps `render()` free of event registration.**err(msg)** (auth.js:229)**Signature:** `function err(msg: string): void`**Behavior:** Finds `#auth-err` inside the gate. If `msg` is truthy, sets `textContent` and `style.display = "block"`. If empty, hides the element. Safe to call even if `#auth-err` doesn't exist in the current view.**val(id)** (auth.js:232)**Signature:** `const val = (id: string) => string`

Returns: The `.value` of the element with the given ID, or `""` if not found.

`enter(email)` (auth.js:237–240)

Signature: `function enter(email: string): void`

Behavior:

1. In **static mode**: Persists `email` to `LS.session` and sets `currentUser = accountFor()`.
2. In **server mode**: No localStorage write; `currentUser` was already set from the API response before `enter()` was called.
3. Calls `unlock(true)` (animated unlock).

Called by: The `"login"` data-act handler after credentials are verified.

`applyA11y(on)` (auth.js:244)

Signature: `function applyA11y(on: boolean): void`

Behavior: Calls `document.body.classList.toggle("a11y-mode", !!on)`. The CSS file uses `.a11y-mode` to activate larger text, higher contrast, stronger focus outlines, and `prefers-reduced-motion`-style animation removal.

`unlock(animated)` (auth.js:254–287)

Signature: `async function unlock(animated: boolean): Promise<void>`

Behavior:

1. Sets `unlocked = true` — stops `guard()` from re-locking.
2. Calls `killSplash()`.
3. If `animated && gate`: Replaces the gate's innerHTML with a "Welcome / Unlocking your dashboard" card. This gives the user visual feedback during the potentially multi-second data fetch.
4. Awaits `window.TFBoot.ensureAppLoaded()`. On failure, shows the "Could not load your dashboard" error and returns early.
5. Calls `applyA11y(currentUser && currentUser.a11y)` — honors the user's stored accessibility preference immediately.
6. Calls `window.TF.init()` if it exists — triggers the first full app render. Guarded with `if (window.TF && window.TF.init)` because `init()` is idempotent (`_inited` flag prevents double-initialization).
7. Calls `window.TF.onRefresh()` — updates the greeting and re-renders the active page for this specific account's data.
8. Calls `applyUser()` — writes the user's name, plan, and avatar initial into the sidebar.
9. Calls `startRefresh()` — starts the tier-appropriate live-data interval.
10. **Animated path:** Wraps the gate-removal in a `setTimeout(700ms)` to let the welcome card settle, then removes `"tf-locked"` from `<body>` (revealing the rendered app behind the gate), adds class `"unlocking"` to the gate (a CSS slide-up animation), and removes the gate element when `animationend` fires (or after 1000ms as a timeout fallback).
11. **Non-animated path:** Removes `"tf-locked"` immediately and drops the gate synchronously.

Design note: The 700ms welcome card delay is intentional — it gives `ensureAppLoaded()` time to fetch data and inject scripts before the app is revealed. Without it, the page could flash an empty state. The `animationend` + 1000ms fallback pattern prevents a memory leak if the animation event never fires (e.g., `prefers-reduced-motion` OS setting skips animations).

`bind()` (auth.js:292–383)

Signature: `function bind(): void`

Behavior: Attaches a single delegated `click` listener on `document`. The listener:

1. Finds `e.target.closest("[data-act]")`. If not found, or if `gate` is absent or hidden, returns early.
2. Reads `t.dataset.act` and dispatches to the appropriate handler.

Handlers (by `data-act` value):

- `"to-beta"`: Calls `render("beta")`.
- `"to-login"`: Calls `render("login")`.
- `"verify"`:
 - Static mode: Checks `cyrb53(key) === BETA_HASH`. On match → `render("signup")`; on mismatch → `err(...)`.
 - Server mode: Stores key in `betaKeyEntered` and calls `render("signup")` unconditionally (server validates the key at signup time).
- `"signup"`:

1. Reads name, email (lowercased), password, and consent checkbox.
 2. Validates: name non-empty, `validEmail(email)`, consent checked.
 3. Disables the button, awaits `emailDomainReal(email)`, re-enables.
 4. If domain fails → `err(...)`.
 5. **Server mode:** Requires password ≥ 8 chars; POSTs to `/api/auth/signup` with `{beta_key, name, email, password, consent: true}`; on 403 re-renders "beta" with the error; on other error shows the detail message; on success sets `currentUser = adoptServerUser(res.data.user)` and renders "quiz".
 6. **Static mode:** Requires password ≥ 6 chars; checks for duplicate email in `LS.accounts`; creates the account with `cyrb53(pass)` as the stored password; sets `LS.session` and `currentUser`; renders "quiz".
- **"quiz-submit"**:
 1. Reads the five quiz field values.
 2. Hard floor: if `ageNum < 13` → `err(...)`. Max: if `ageNum > 120` → `err(...)`.
 3. **Server mode:** POSTs to `/api/auth/profile`; sets `currentUser = adoptServerUser(res.data.user)` on success; renders "a1ly".
 4. **Static mode:** Calls `computeProfile(answers)`; writes enriched profile to `LS.accounts[LS.session].profile`; refreshes `currentUser`; renders "a1ly".
 - **"a1ly-on" / "a1ly-off"**:
 1. Determines desired state (`on = a === "a1ly-on"`).
 2. **Server mode:** POSTs to `/api/auth/a1ly?on={on}` (fire-and-forget with await for ordering); sets `currentUser.a1ly = on`; calls `applyA1ly(on)`.
 3. **Static mode:** Mutates `LS.accounts`; refreshes `currentUser`; calls `applyA1ly(on)`.
 4. Both modes: calls `unlock(true)` — initiates the full animated unlock sequence.
 - **"login"**:
 1. Reads email (lowercased) and password.
 2. **Server mode:** POSTs to `/api/auth/login`; on failure → `err(...)`; on success → `currentUser = adoptServerUser(res.data.user)` and `enter(email)`.
 3. **Static mode:** Reads `LS.accounts[email]`; checks `acc.pass === cyrb53(pass)`; on match → `enter(email)`; on mismatch → `err(...)`.

Called at IIFE scope (auth.js:505). Set up before DOMContentLoaded, so the listener is active for the lifetime of the page.

`applyUser()` (auth.js:387–393)

Signature: `function applyUser(): void`

Behavior: Reads `currentUser`. If null, no-op. Otherwise:

- `#userName.textContent = u.name || u.email`
- `#userPlan.textContent = TIERS[u.tier].name || TIERS.beta.name`
- `#userAvatar.textContent = (u.name || u.email)[0].toUpperCase()`

Called by: `unlock()`.

`startRefresh()` (auth.js:398–404)

Signature: `function startRefresh(): void`

Behavior:

1. Clears any existing `refreshTimer` with `clearInterval`.
2. Looks up the tier from `TIERS[(currentUser && currentUser.tier) || "beta"]`, falling back to `TIERS.beta`.
3. Computes `ms = tier.refreshMin * 60 * 1000`.
4. Sets `refreshTimer = setInterval(() => refreshLiveData(tier), ms)`.
5. Calls `stamp(tier)` immediately to render the current status into the sidebar.

Design note: Clearing before setting means tier upgrades take effect immediately — no doubled intervals.

`stampEl()` (auth.js:408–415)

Signature: `function stampEl(): HTMLElement | null`

Behavior: Tries to find `#tf-updated` by ID. If not found, tries `#tf-updated-slot` then `.sidebar-foot` as the parent; creates a `<div id="tf-updated" class="tf-updated" role="status">` and appends it. Returns the element, or `null` if no parent slot is found.

Design note: Lazily creating the element means the stamp works even if it's called before the sidebar is in the DOM (though in practice it's called after `unlock()` which runs after `window.TF.init()` which renders the sidebar).

`stamp(tier)` (auth.js:419–425)

Signature: `function stamp(tier: object): void`

Behavior:

1. Gets or creates the stamp element via `stampEl()`. Returns if null.
2. If `pausedForClose(tier)` → sets innerHTML to: `<span class="tf-dot closed"></span>Markets closed · updates resume 9:30 AM ET`.
3. Otherwise: computes whether we're in "extended hours" (`!marketOpen()`), formats the tier name and cadence, and sets innerHTML to: `<span class="tf-dot"></span>Live · {tier.name} · every {N min or day}{extended}`.

`marketOpen()` (auth.js:430–438)

Signature: `function marketOpen(): boolean` **Returns:** `true` during US equity core market hours; `false` otherwise.

Behavior:

1. Constructs the current Eastern time using `new Date(new Date().toLocaleString("en-US", { timeZone: "America/New_York" })))`. This approach is DST-correct without any timezone library.
2. Returns `false` on weekends (day `0` = Sunday, `6` = Saturday).
3. Computes `mins = hours * 60 + minutes`. Returns `true` if `mins >= 570 && mins < 960` (9:30 AM to 4:00 PM ET).
4. On any `toLocaleString` failure (rare; some environments don't support the IANA database) → returns `true` (fail open: don't block refresh).

Design note: Market holidays are not modeled. The refresh simply skips during a holiday and resumes the next session when `marketOpen()` returns `true` again.

`pausedForClose(tier)` (auth.js:443)

Signature: `function pausedForClose(tier: object): boolean` **Returns:** `true` if the market is closed AND the tier has a `refreshMin > 30` (i.e., is the daily or hourly tier).

Logic: `return !marketOpen() && (tier.refreshMin || 0) > 30`.

Rationale: The 15-minute and 30-minute tiers (`refreshMin ≤ 30`) are paid for coverage that includes extended hours. The daily and hourly tiers do not have that entitlement, so they pause when the core market is closed to save API budget.

`refreshLiveData(tier)` (auth.js:448–478)

Signature: `async function refreshLiveData(tier: object): Promise<void>`

Behavior — server mode (auth.js:452–464):

1. Calls `api("/api/data/live")`.
2. If `res.status === 429` (rate-limited — too soon for this tier's cadence): calls `stamp(tier)` and returns without updating data.
3. If `ok && data && !data.paused`: `Object.assign(window, res.data)` (refreshes `PRICES_DATA`, `NEWS_INTEL`); calls `window.TF.onRefresh()`; updates `#tf-updated` with "Updated HH:MM · {tier.name}".
4. If `paused` (market closed server-side): calls `stamp(tier)` only.
5. Network error: silently swallowed — keeps current data.

Behavior — static mode (auth.js:466–477):

1. Checks `pausedForClose(tier)`. If paused: calls `stamp(tier)` and returns.
2. For each of `["js/prices_data.js", "js/news_intel.js"]`:
 - Fetches the file with `?t={Date.now()}` cache-bust.`
 - Parses the JSON out of the file text by slicing from the `window.NAME =` marker, stripping the trailing `;`, and calling `JSON.parse`. This avoids injecting a new `<script>` tag (which would evaluate the whole file again and might conflict with the existing global).
 - Assigns `window[key] = JSON.parse(json)`.

3. Calls `window.TF.onRefresh()`.
4. Updates the `#tf-updated` timestamp line.

Design note: The raw-text parsing approach (slice + JSON.parse) in static mode is deliberate: injecting a second `<script>` tag for the same file would re-run the global assignment but the old constant captures in logic scripts would not update. By directly mutating `window.PRICES_DATA`, all code that reads `window.PRICES_DATA` at refresh time (rather than at eval time) gets the fresh data.

`guard()` (auth.js:484–502)

Signature: `function guard(): void`

Behavior:

1. Creates an inner `reassert` function:
 - If `unlocked` is true → returns immediately (the app is legitimately unlocked).
 - Disconnects the `MutationObserver` before touching the DOM. **Critical:** Without this disconnect, the observer's own DOM write (`appendChild(gate)`, `classList.add`) would fire the callback again as a microtask, creating an infinite loop that starved the page event loop and froze the tab.
 - Re-adds `"tf-locked"` to `<body>` if it was removed.
 - Re-appends `gate` to `<body>` if it was removed.
 - Sets `gate.style.display = "flex"` if it was hidden.
 - Re-observes `document.body` for `childList` changes after the DOM writes are done.
2. Tries to create a `MutationObserver(reassert)` observing `document.body` with `{ childList: true }`. This fires when someone removes the gate via devtools `Remove element`.
3. Regardless of observer support, sets `setInterval(reassert, 700)` — a 700ms periodic fallback that catches style/class edits (e.g., removing `"tf-locked"` via devtools `Edit attribute`) that the `childList` observer doesn't see.

Called at IIFE scope (auth.js:506) — active from page load.

Design note: The observer/interval combination makes it impractical to bypass the gate via devtools. However, the code comments explicitly acknowledge this is a soft gate: "It is NOT real security." The comment in `boot.js` reiterates the same — real data protection happens server-side, where the data files are 404-blocked.

`boot()` (auth.js:512–526)

Signature: `async function boot(): Promise<void>`

Behavior:

1. Adds `"tf-locked"` to `document.body` (in case JS was slow to run and the class was temporarily absent).
2. Awaits `window.TFBoot.ready` to get the confirmed `mode` string. Falls back to `"static"` if `TFBoot` is absent.
3. **Server mode:**
 - Calls `api("/api/auth/me")` to check for an existing session cookie.
 - If the response is `ok` and has a `user`: `currentUser = adoptServerUser(res.data.user)`; calls `unlock(false)` (silent, no animation — returning user).
 - Otherwise: calls `showGate("login")`.
4. **Static mode:**
 - If `LS.session && LS.accounts[LS.session]`: `currentUser = accountFor()`; calls `unlock(false)`.
 - Otherwise: calls `showGate("login")`.

Entry point: Called via:

```
if (document.readyState === "loading") document.addEventListener("DOMContentLoaded", boot);
else boot();
```

This handles both the common case (script `defer` 'd, DOM already parsed) and edge cases where the script might evaluate after `DOMContentLoaded`.

`logout()` (auth.js:532–555)

Signature: `function logout(): void`

Behavior:

1. **Server mode:** Fires `api("/api/auth/logout", { method: "POST" })` as fire-and-forget (no await, inside try/catch) — clears the httpOnly session cookie server-side. The UI proceeds regardless.
2. **Both modes:** `LS.session = ""`; `currentUser = null`; `unlocked = false`.
3. Clears `refreshTimer`.
4. Adds `"tf-locked"` to `<body>` — hides the app.
5. Removes any existing gate element.
6. Creates a new `<div class="auth-gate goodnight" role="status">` and appends it to `<body>`. Its content is a goodbye card with:
 - A smiling face SVG (`gn-face`).
 - A waving hand SVG (`gn-wave`). Both are inline SVGs with neon-style CSS stroke animations.
 - "Goodbye, friend" and "See you next session" text.
7. After 1800ms, removes the goodbye element and calls `showGate("login")` — back to the login view.

Design note: `location.reload()` was deliberately avoided. A full reload would re-parse ~6 MB of data scripts (the `DATA_SCRIPTS` list) and re-run all the logic scripts, which was observed to trigger Chrome's "Page Unresponsive" dialog. The no-reload path re-uses the already-loaded app; the next login simply calls `window.TF.onRefresh()` to re-render with the new account's data.

Event wiring (auth.js:559): A separate `click` listener on `document` checks `e.target.closest("#logoutBtn")` and calls `logout()`. Bound on `document` so it survives sidebar re-renders.

`window.Auth` Public API (auth.js:562–593)

Assigned at the end of the IIFE:

Property	Type	Description		
<code>TIERS</code>	object	The tier definitions (see §3.1).		
<code>user()</code>	<code>() => object\</code>	<code>null</code>	Returns <code>currentUser</code> .	
<code>tier()</code>	<code>() => object</code>	Returns <code>TIERS[currentUser.tier] \</code>	<code>\</code>	<code>TIERS.free</code> .
<code>ally()</code>	<code>() => boolean</code>	Returns <code>!!(currentUser && currentUser.ally)</code> .		
<code>profile()</code>	<code>() => object\</code>	<code>null</code>	Returns <code>currentUser.profile</code> or <code>null</code> .	
<code>computeProfile</code>	function	The scoring function (exported for use in Settings and other modules).		
<code>setProfile(answers)</code>	function	Updates the investor profile (Settings editor).		
<code>setAlly(on)</code>	function	Updates accessibility preference (Settings toggle).		
<code>logout</code>	function	The logout function.		

`window.Auth.setProfile(answers)` (auth.js:569–580)

Behavior:

1. Calls `computeProfile(answers)` to get the enriched profile.
2. **Server mode:** POSTs raw answers to `/api/auth/profile` (fire-and-forget); sets `currentUser.profile = prof`.
3. **Static mode:** Writes enriched profile to `LS.accounts` and `currentUser.profile`.
4. Returns the enriched profile object.

Called by `app.js` when the user clicks "Save profile" in the Settings page.

`window.Auth.setAlly(on)` (auth.js:581–591)

Behavior:

1. **Server mode:** POSTs to `/api/auth/ally?on={on}` (fire-and-forget); updates `currentUser.ally`.
2. **Static mode:** Writes to `LS.accounts` and `currentUser.ally`.
3. Both modes: calls `applyAlly(on)`.

Called by `app.js` 's delegated `#allyToggle` click/keydown handler.

4. app.js — Main Controller, Router, and View Renderer

File: `webapp/js/app.js` **Structure:** Non-IIFE module-level code; all identifiers are in the global scope of the script file (not `window.*` — they are function-scoped to the module by virtue of being top-level `const/let/function` in "use strict" but not explicitly attached to `window`). Exports only `window.TF.init` and `window.TF.onRefresh`, plus `window.tfShowLegState`. **Injected by:** `boot.js:loadInOrder(LOGIC_SCRIPTS)` — evaluated after all data globals exist.

4.1 Evaluation-Time Data Capture

`const D = window.TF_DATA || {}` (app.js:15)

The primary data object. Captured at script-eval time. Contains: `politicians`, `recent_trades`, `bills`, `correlation`, `sectors`, `tickersOpp`, `news`, `everyday`, `means`, `portfolio`, `recession`, `parallels`, `events`, `market_ticker`, `disclaimer`, `generated_at`, `user`.

All `render*()` functions read from `D`. Because `D` is a `const` reference to the object, if `window.TF_DATA` is mutated in place, `D` reflects the mutation. But `auth.js:refreshLiveData()` uses `Object.assign(window, data)` which replaces `window.TF_DATA` (a new reference), so `D` in `app.js` would not update for `TF_DATA`. Only `window.PRICES_DATA` and `window.NEWS_INTEL` are refreshed at runtime; `D` is treated as static.

`const STATES` (app.js:78)

```
const STATES = (window.STATES_DATA || {}).byState || {};
```

State economic data keyed by abbreviation (e.g., `STATES["CA"]`). Captured at eval time from `window.STATES_DATA` (set by `js/states_data.js`).

`let LEGIS / OSTATES` (app.js:80–81)

```
let LEGIS = (window.LEGISCAN_DATA || {}).byState || {};
let OSTATES = (window.OPENSTATES_DATA || {}).byState || {};
```

State legislative data. Declared with `let` (not `const`) because `ensureStateData()` can update them after lazy-loading the `legiscan_data.js` / `openstates_data.js` scripts.

`const NEWS_INTEL` (app.js:105)

```
const NEWS_INTEL = window.NEWS_INTEL || { bySector: {}, byTicker: {} };
```

Chef GPT news intelligence index. Captured at eval time. `auth.js:refreshLiveData()` updates `window.NEWS_INTEL` with fresh data, but because `NEWS_INTEL` is a `const` reference to the object-at-eval-time, it **does not auto-update**. `tfNewsSummary()` reads directly from `NEWS_INTEL` (the eval-time snapshot). This is a known trade-off: the summary data is re-fetched on the next page render triggered by `window.TF.onRefresh()`, which calls `go(currentPage)`, which re-renders the active view's HTML including fresh `tfNewsSummary()` output — so the content updates, just not the constant reference itself.

`const SECBULK / USASTOCK / IWCOS / FEC` (app.js:359–167)

Read-only at eval time:

- `SECBULK`: SEC bulk financial data keyed by ticker (from `window.SECBULK_DATA.byTicker`).
- `USASTOCK`: USASpending government contracts keyed by ticker (from `window.USA_DATA.byTicker`).
- `IWCOS`: InfluenceWeb company names and metadata (from `window.IW_DATA.companies`).
- `FEC`: OpenFEC campaign finance data keyed by politician name (from `window.FEC_DATA.byName`).

4.2 DOM and Formatting Utilities

`$` (app.js:18)

```
const $ = (sel, root = document) => root.querySelector(sel);
```

A brief alias for `querySelector`. Used throughout as `$("#greet")`, `$(".sidebar-foot")`, etc.

`esc(s)` (app.js:19)

```
const esc = (s) => String(s ?? "").replace(/&<"/g, (c) => ({ ... }[c]));
```

HTML escaper, identical in behavior to `auth.js`'s `E`. Used for all user-visible string interpolation in `app.js`'s templates. Handles `null`/`undefined` gracefully via `?? ""`.

initials(name) (app.js:20)

```
const initials = (name) => String(name || "?").split(" ").map(w => w[0]).slice(0, 2).join("").toUpperCase();
```

Returns up to 2 uppercase initials from a display name. Used for avatar placeholders.

colorFor(v) (app.js:21)

```
const colorFor = (v) => (v > 0 ? "up" : v < 0 ? "down" : "");
```

Maps a signed numeric value to a CSS class string: `"up"` (green), `"down"` (red), or `""` (neutral). Used in financial data cells.

pct(v) (app.js:22)

```
const pct = (v) => `${v > 0 ? "+" : ""}${Number(v).toFixed(2)}%`;
```

Formats a number as a percentage string with sign and 2 decimal places (e.g., `"+3.45%"` or `"-1.20%"`).

Color Constants C (app.js:75)

```
const C = { green: "#00C46A", red: "#EF4444", blue: "#38BDF8", amber: "#F59E0B", purple: "#A855F7" };
```

Shared color tokens used in `gauge()`, `sparkline()`, and `chamberBar()`.

money0(n) (app.js:121)

```
const money0 = (n) => n == null ? "n/a" : "$" + Number(n).toLocaleString(undefined, { maximumFractionDigits: 0 });
```

Formats a number as a dollar amount with no decimal places (e.g., `"$85,000"`). Returns `"n/a"` for null/undefined.

4.3 Visual Component Functions

photoEl(name, bioguide, cls) (app.js:26–32)

Signature: `function photoEl(name: string, bioguide: string|null, cls?: string): string` **Returns:** An `<img>` HTML string pointing to `assets/politicians/{bioguide}.jpg`, with an `onerror` handler that replaces the image with an initials placeholder `<div>`. If `bioguide` is absent, returns the placeholder immediately.

Design note: The `onerror` attribute's inner HTML uses escaped single quotes (`\'`) because it is embedded in a double-quoted HTML attribute string. This avoids the XSS-via-onerror pattern since `esc(initials(name))` sanitizes the injected content.

gauge(value, max, color, label) (app.js:35–47)

Signature: `function gauge(value: number, max: number, color: string, label: string): string` **Returns:** An HTML string with a 116×116 SVG radial gauge and a centered label.

Behavior:

- Circle radius `r = 50`. Circumference `c = 2π × 50 ≈ 314.16`.
- `frac = clamp(value/max, 0, 1)`. `strokeDashoffset = c * (1 - frac)` — so `0` fills the full circle, `c` is empty.
- Background circle: `rgba(148,163,184,0.14)` — a subtle track.
- Foreground arc: `stroke={color}`, `stroke-linecap="round"` for rounded endpoints.

sparkline(seed, color, dir) (app.js:51–73)

Signature: `function sparkline(seed: number, color: string, dir?: "up"|"down"): string` **Returns:** An HTML string with an SVG sparkline chart (viewBox 240×38).

Behavior:

- Generates 24 pseudo-random data points using a linear congruential generator (LCG): `s = (s * 9301 + 49297) % 233280`. The LCG is seeded by `seed`, making charts deterministic — the same seed always produces the same shape.
- `dir = "up"` biases the random step: `v += (rnd - 0.45) * 10` (upward drift); `dir = "down"` uses `0.55` (downward drift).
- Renders a `<path>` for the line and a gradient-filled `<path>` for the area underneath.
- The gradient ID is `"g" + Math.abs(seed % 99999)` — unique-enough per sparkline to avoid SVG gradient ID collisions.

Design note: Deterministic pseudo-random sparklines are used for KPI cards and portfolio summaries where actual time-series data is not available. They convey visual "direction" without pretending to be real price data.

4.4 Data Enrichment and Intelligence Functions

`ensureStateData(cb)` (app.js:83–103)

Signature: `function ensureStateData(cb?: () => void): void`

Behavior:

- If `window.LEGISCAN_DATA` already exists (pre-loaded or cached from a prior call): updates the module-level `LEGIS` and `OSTATES` references and calls `cb()` synchronously.
- If `_stateDataLoading` is true (a load is already in progress): returns immediately to avoid double-loading.
- Sets `_stateDataLoading = true`.
- Defines `apply()`: updates `LEGIS` / `OSTATES` references and calls `cb()`.
- Server mode:** Calls `window.TFBoot.api("/api/data/state")`. On resolution (success or error), calls `Object.assign(window, res.data)` if ok, then `apply()`.
- Static mode:** Injects `js/legiscan_data.js` and `js/openstates_data.js` as `<script>` tags in parallel (not via `loadInOrder` — they are independent). Uses a countdown counter (`left = 2`) — `apply()` is called when both scripts fire `onload` or `onerror`.

Called by: `go("states")` in the router, and re-called if the user revisits the States page after `LEGISCAN_DATA` is populated.

Design note: This function avoids loading ~4.5 MB of state legislative data on initial page load. The data is only fetched once per session. The server mode fetches it as a gated authenticated bundle (`/api/data/state`) rather than exposing the raw files.

`tfNewsSummary(kind, key)` (app.js:107–119)

Signature: `function tfNewsSummary(kind: "ticker"|"sector", key: string): string` **Returns:** An HTML string with the "ThinkFree Summary" styled card, or `""` if no summary exists.

Behavior — ticker:

- Looks up `NEWS_INTEL.byTicker[key]`. If absent, returns `""`.
- Renders: badge, `v.summary` body, optional `v.what_it_means` section, and the "Chef GPT, not financial advice" footer.

Behavior — sector:

- Looks up `NEWS_INTEL.bySector[key]`, with fallback via `NI_SECTOR_ALIAS` (e.g., `"Technology"` → `"Information Technology"`).
- Renders: badge, sector label, `sec.summary` body, and the footer.

`accountabilityBlock(p)` (app.js:124–164)

Signature: `function accountabilityBlock(p: politician): string` **Returns:** An HTML section with the "Constituent Accountability" block, or `""` if no state data exists for `p.state`.

Behavior:

- Maps `p.state` (full name, e.g., `"California"`) to a 2-letter abbreviation via `STATE_ABBR`.
- Reads `STATES[ab]` for the state's economic data.
- Computes `alignment: 50 + (prosperity - 50) * 0.6 - max(0, repGain) * 0.8 - (pressure - 50) * 0.4`, clamped to 0–100.
 - High constituent prosperity + high pressure improvement + low representative trading gains → high alignment.
 - High trading gains while constituents struggle → low alignment.
- Renders: an alignment score pill, meter bars for Prosperity / Affordability / Cost Pressure, economic stats summary line, and a Representative Outcomes section (trading return, P&L, PAC funding if FEC data exists).

Framing note: The `<div class="sample-note">` footer reads: "Alignment is not a corruption score. It contrasts how the district is doing economically with the representative's disclosed financial outcomes." This is the legally required non-accusatory framing mandated by

CLAUDE.md .

polScore3(p) (app.js:174–181)

Signature: `function polScore3(p: politician): {influence: number, publicImpact: number, transparency: number}`

Computes three scores for a politician:

- **Influence:** `min(100, 30 + trades*0.3 + pac*0.6 + |ret|*0.5)` — measures market access and funding leverage.
- **Public Impact:** `max(10, 70 - pac*0.5 + individual*0.2)` — measures grassroots vs PAC funding mix.
- **Transparency:** `max(5, 100 - pac*1.4 - trades*0.15)` — inversely weighted by PAC dependence and trade frequency.

All three are integers between 0 and 100. The `polBadge()` function wraps them in interactive badge elements that trigger `ScoreInfo` explainers on click.

fecBlock(name) (app.js:183–200)

Signature: `function fecBlock(name: string): string` **Returns:** An HTML section with FEC campaign finance data for the politician, or `""` if no FEC data exists.

Renders: Total raised, from individuals, from PACs, cash on hand, top employer-contributor table, and a source link to the FEC profile.

stockName(tk) (app.js:362)

```
function stockName(tk) { return (IWCOS[tk] && IWCOS[tk].name) || (SECBULK[tk] && SECBULK[tk].name) || tk; }
```

Returns the company name for a ticker. Prefers InfluenceWeb data (more curated), falls back to SEC bulk data, falls back to the ticker symbol itself.

tickerCompanyName(tk) (app.js:368)

```
function tickerCompanyName(tk) { return ((window.PRICES_DATA || {}).byTicker || {})[tk] || {}).name || stockName(tk) || tk; }
```

Like `stockName()` but also checks `window.PRICES_DATA` (which is refreshed by the live-data timer), so it reflects the freshest company name.

newsRelevant(a, tk) (app.js:369–377)

Signature: `function newsRelevant(a: article, tk: string): boolean` **Returns:** `true` if the article is genuinely about the company represented by `tk`.

Problem it solves: Tickers like `"ON"`, `"IT"`, `"ALL"`, `"NOW"`, `"T"`, `"D"` are common English words. A naive `includes(ticker)` check would match irrelevant articles (e.g., any article containing the word "on" would match `ON Semiconductor`). `COMMON_TK` is a `Set` of such ambiguous tickers; `GENERIC_CO` is a `Set` of generic company-name words excluded from the company-name match.

Logic (three tests, first match wins):

1. **Company name match:** Tokenizes the company name (split on whitespace, length ≥ 4 , not a generic word). If any token appears in the article blob, returns `true`.
2. **Explicit ticker mention:** Checks for `$TK` or `(TK)` in the blob. Returns `true`.
3. **Distinctive ticker token:** If `tk` is not in `COMMON_TK` and has ≥ 3 characters, checks for `\bTK\b` as a standalone word. Returns `true` if found.

newsForTicker(tk) (app.js:378)

```
function newsForTicker(tk) { return (D.news || []).filter((n) => n.symbol === tk && newsRelevant(n, tk)); }
```

Filters all news to items whose `symbol` field matches `tk` AND passes the `newsRelevant()` check.

preferredSectors() (app.js:381–387)

Signature: `function preferredSectors(): string[]` **Returns:** An array of lowercase sector name strings matching the user's investment profile.

Logic:

- Growth/high-risk profile → technology, IT, communication services, consumer discretionary.
- Income/low-risk profile → utilities, consumer staples, real estate, financials, health care.
- Otherwise → a balanced default set.

Called by: `tailorNews()` and `renderNews()` (for the "tailored to your profile" label).

tailorNews(list) (app.js:388–396)

Signature: `function tailorNews(list: article[]): article[]` **Returns:** A re-ordered copy of `list` where preferred-sector articles sort first, preserving relative order within each group.

Behavior: Uses a stable sort — items get a priority of `0` (preferred) or `1` (other), and ties preserve the original index ordering. If `preferredSectors()` returns an empty set, the list is returned as-is.

tidySummary(s) (app.js:399–407)

Signature: `function tidySummary(s: string): string` **Returns:** A cleaned summary string that ends on a complete sentence boundary.

Behavior:

1. If `s` does not end in `"..."` or `"..."`, returns `s` unchanged.
2. Strips the trailing ellipsis.
3. Finds the last sentence-ending punctuation character (`.`, `!`, or `?`) with `re.exec` in a loop (to get the last match, not the first).
4. If a boundary is found at index ≥ 40 chars in, slices there.
5. Fallback (boundary too early or absent): strips the last dangling word and appends `"..."`.

Purpose: News article previews from Finnhub and other sources are often truncated mid-sentence with `"..."`. Showing a mid-sentence cut looks broken; this function makes the preview end cleanly.

detectThemes(items, max) (app.js:421–424)

Signature: `function detectThemes(items: article[], max?: number): string[]` **Returns:** Up to `max` (default 3) plain-English theme strings matched against the combined headline+summary text of all articles.

Behavior: Tests each `NEWS_THEMES` regex against the concatenated article blob. Returns the friendly string label for each matching theme.

`NEWS_THEMES` (app.js:410–420): 9 regex patterns covering earnings, product launches, M&A, dividends, leadership changes, analyst ratings, litigation/regulation, daily stock moves, and partnerships.

joinList(arr) (app.js:425–429)

Signature: `function joinList(arr: string[]): string` **Returns:** A comma-and-"and" English list (e.g., `"earnings, dividends and leadership changes"`).

genImpact(sectors) (app.js:451–457)

Signature: `function genImpact(sectors: string[]): {gen: string, score: number}[]` **Returns:** Array of 5 generational impact scores.

Behavior: Looks up each sector in `GEN_SECTOR_W`. The weight for each generation is averaged across all input sectors. Multiplied by 70 and rounded. If no sectors match, uses `GEN_SECTOR_W.default`.

genImpactBlock(sectors, title) (app.js:459–475)

Signature: `function genImpactBlock(sectors: string[], title?: string): string` **Returns:** HTML for the generation impact panel.

Behavior:

- If `window.GenImpact` (the richer `genimpact.js` module) is available, delegates to `window.GenImpact.forSectors()` and `window.GenImpact.panel()`.
- Otherwise, falls back to the local `genImpact()` computation and renders bar rows inline.

sourceName(url) (app.js:477–485)

```
function sourceName(url): string
```

Extracts the publisher name from a URL. Strips `www.` and the TLD; maps known hostnames to friendly names (e.g., `"finnhub"` → `"Finnhub"`).

sourceLink(url, source) / **newsHeadline(a)** / **readMore()** (app.js:487–495)

- **sourceLink(url, source)** : Returns an `<a>` tag linking to the original article with the publisher name.
- **newsHeadline(a)** : Returns `esc(a.headline)` — deliberately kept in-app (no link-out) so readers consume the ThinkFree Summary rather than leaving the app.
- **readMore()** : Returns `""` — placeholder; no "read more" link is emitted.

credBadge(n) (app.js:497–502)

Maps `n.credibility_tier` to a CSS class and renders a badge: `"Trusted"` → `.up`, `"Reliable"` → `.info`, `"Mixed"` → `.warn`, anything else → `.down`.

newsSentiment(n) (app.js:503–509)

Returns `[text, pillClass]` tuple: `"positive"` → `["Positive", "up"]`, `"negative"` → `["Negative", "down"]`, otherwise `["Neutral", "info"]`.

4.5 Modal Functions

openModal(html) (app.js:207–216)

Signature: `function openModal(html: string): void`

Behavior:

1. Sets `#modalContent.innerHTML = html`.
2. Adds class `"open"` to `#modal` (the overlay). CSS transitions the overlay to full opacity/visibility.
3. Resets scroll: `overlay.scrollTop = 0` and `#modalBox.scrollTop = 0`. Repeated inside `requestAnimationFrame()` to handle browser scroll restoration that might override the synchronous reset.

closeModal() (app.js:217–219)

Signature: `function closeModal(): void`

Behavior: Removes class `"open"` from `#modal`. The CSS transition hides the overlay.

Called by: The `#modalClose` button click, clicking the `#modal` overlay directly (not the `#modalBox` content area), and `Escape` keydown.

4.6 Detail View Functions

politicianProfile(name) (app.js:222–319)

Signature: `function politicianProfile(name: string): void`

Behavior:

1. Looks up the politician from `D.politicians` by name.
2. Computes `polScore3(p)` for the three accountability scores.
3. Builds the full HTML including:
 - `photoEl()` headshot.
 - Portfolio stats (net worth, portfolio value, 6-month return, P&L).
 - Three accountability score badges via `polBadge()`.
 - Summary text.
 - `fecBlock(p.name)` — FEC campaign finance section.
 - `accountabilityBlock(p)` — district vs. representative outcomes.
 - Most-traded tickers chip row.
 - Top donors and outside spending tables.

- Trade timeline (up to 40 entries) with bill correlation data, timing indicators (before/after), and per-trade P&L.
- Related legislation list.
- Disclaimer per the framing rule.

4. Calls `openModal(html)`.

`allTradesView()` (app.js:322–339)

Signature: `function allTradesView(): void`

Behavior: Renders a complete table of all `D.recent_trades` (not capped at 14 like the Political Watch page renders). Opens in the modal.

`stockDetail(tk)` (app.js:511–599)

Signature: `function stockDetail(tk: string): void`

Behavior:

1. Gathers data from `SECBULK[tk]`, `USASTOCK[tk]`, `IWCOS[tk]`, `window.PRICES_DATA.byTicker[tk]`.
 2. Computes `TFScores.influenceScore(tk)` and `TFScores.dependencyScore(tk)`.
 3. Filters congressional trades and related bills for the ticker.
 4. Calls `newsForTicker(tk)` with the relevance filter.
 5. **Holistic summary construction:** Builds a 2–4 sentence plain-English narrative using `detectThemes(news)`, revenue/income from SEC data, government contracts from USASpending, and congressional trading activity.
 6. Renders a full company detail panel including:
 - Company logo (with initials fallback on `onerror`).
 - Price and change percentage.
 - `tfNewsSummary("ticker", tk)` — Chef GPT summary.
 - Key financial figures (market cap, revenue, net income, assets, federal contracts, exchange).
 - Generation Impact panel (via `genImpactBlock()` or `window.GenImpact`).
 - Congressional trades table.
 - Related legislation chip row.
 - Recent news feed (up to 6 items with relevance filter applied).
 7. Calls `openModal(html)`.
-

`billDetail(id)` (app.js:602–653)

Signature: `function billDetail(id: string): void`

Behavior:

1. Calls `billById(id)` to find the bill (preferring `correlation.top_bills` rich data over `bills` basic data).
 2. Renders:
 - Header: sector tags, bill title, congress.gov link, action date.
 - Plain-English summary (`b.plain_summary` || `b.action_text`).
 - `genImpactBlock(b.sectors)` — generation impact with status note.
 - "Who traded it" table with before/after timing highlight.
 - PAC/lobbying chip row.
 - Disclaimer.
 3. Calls `openModal(html)`.
-

`billById(id)` (app.js:349–356)

Signature: `function billById(id: string): object | undefined`

Behavior: Lazily builds `_billMap` (a merged dictionary from `D.bills` and `D.correlation.top_bills`). Rich `top_bills` entries overwrite basic `bills` entries for the same `bill_id`. Returns the entry for `id`.

`eventDetail(id)` (app.js:1218–1235)

Signature: `function eventDetail(id: string): void`

Behavior: Finds the event in `D.events` by `id`. Renders: category + date range badge, event name, six `row()` sections (What Happened, Why Similar, What Happened After, History's Warning, Personal Impact), modern parallel themes chips. Opens in the modal.

4.7 Page Render Functions

All `render*()` functions return an HTML string. They are registered in the `PAGES` map and called lazily by `go()` on first navigation to each page.

`renderDashboard()` (app.js:656–822)

Returns: Four grid sections:

1. **KPIs row:** Market Mood (72%, bullish, sparkline), Recession Risk (computed from `D.recession`), Market Direction, Top Story.
2. **Political block:** Feature politician (top `D.politicians[0]`) with photo, stats, and "Top 6 by Trading Gains" rank list; Investigative Overview gauge (Bill-Trade Correlation Index from `D.correlation`).
3. **Insight row:** What Matters Today (`D.means`), How This Affects You (`D.everyday`), Market Signals (`D.tickers_opp`), Sector Scorecard (bar chart from `D.sectors`).
4. **Bottom row:** Opportunity Scores, My Portfolio KPI, Historical Parallels preview, Quick Actions grid.

`marketSummary()` (app.js:826–873)

Signature: `function marketSummary(): string` **Returns:** A plain-English paragraph summarizing the day's news for the `renderNews()` Market Overview card.

Behavior:

1. Groups news by sector and company; counts positive/neutral/cautious sentiment.
2. Identifies top sectors (mapped to plain English, e.g., `"Information Technology"` → `"tech companies"`).
3. Detects themes via the `NEWS_THEMES` regex set.
4. Assembles a 3–4 sentence narrative: what sectors, which companies, what themes, what the overall mood is.

`sectorSummaries()` (app.js:876–913)

Signature: `function sectorSummaries(): {name, sec, count, text}[]` **Returns:** Array of sector summary objects, sorted by article count descending.

Behavior: For each sector group, generates a 4-sentence plain-English paragraph: coverage share, detected themes, company names, overall tone. The leading sentence varies: the top sector says "busiest part of the market today"; others say "has N stories in today's feed."

`renderNews()` (app.js:933–997)

Returns: The News page HTML, with:

- Page header + story count.
- Market Overview card (using `marketSummary()`).
- Market ticker row (6 categories from `D.market_ticker`).
- "What's Happening by Sector" card (from `sectorSummaries()`; each row has a `data-secnews` chip that opens `sectorNewsModal()`).
- Two-column grid: left = impact-sorted (via `tailorNews()`) full news feed; right = "What This Means For You" (`D.everyday` items).

`renderPolitical()` (app.js:1000–1105)

Returns: The Political Watch page HTML, with:

- Top politician feature card (photo, stats).
- Top 10 politicians table (with three accountability score columns via `polBadge()`).
- Recent trades table (14 rows; "View all" triggers `allTradesView()`).
- Investigative Overview card: Correlation Index gauge, four summary stats, "How Bills Were Influenced" timeline (clickable via `data-bill` attributes).
- Disclaimer at page bottom.

`renderPortfolio()` (app.js:1107–1136)

Returns: The Portfolio page HTML: four KPI cards (value, exposure, cash, win rate) from `D.portfolio`, and the open positions table.

`renderMarkets()` (app.js:1139–1159)

Returns: The Markets page HTML: sector opportunity bar chart, ticker signals table.

`renderReasoning()` (app.js:1162–1179)

Returns: The Economic Reasoning page HTML: recession risk gauge (score/max from `D.recession`), "What The Indicators Say" summary and factor list.

`renderHistory()` (app.js:1182–1215)

Returns: The Historical Parallels page HTML:

- Parallel analog cards (from `D.parallels`): each with a timeline of historical periods.
- Historical Events Library grid (from `D.events`): card-per-event with category badge, summary preview, and modern parallel theme chips. Clicking a card triggers `eventDetail()`.

`renderSettings()` (app.js:1237–1281)

Returns: The Settings page HTML:

- Profile stat card (name, plan, data generation date from `D.generated_at`).
- Investor Profile editor card: pre-fills form inputs from `window.Auth.profile()`. Shows current risk tolerance as a pill. "Save profile" button (`#saveProfileBtn`).
- Accessibility toggle card (`#a11yToggle`): the current on/off state read from `window.Auth.a11y()`.
- About ThinkFree card with inline accessibility statement link (`#a11yStatementLink`).

`renderIntelligence()` (app.js:1302–1381)

Returns: The Influence Intelligence page HTML:

- Influence Score top companies table (computed via `TFScores.influenceScore()`).
- Government Dependency top companies table (computed via `TFScores.dependencyScore()`).
- Industry Intelligence Dashboards table (aggregated via `TFScores.industryRollup()`).
- PAC Money top recipients table (from `FEC_DATA`).
- Lobbying industry groups table (from `NP_DATA` via ProPublica).

`renderStates()` (app.js:1456–1487)

Returns: The State Rankings page HTML:

- Four ranking tables: Constituent Prosperity, Constituent Pressure, Housing Affordability, Real Purchasing Power.
- `#leg-watch-mount` div: if `LEGISCAN_DATA` is already loaded, renders the Legislature Watch card; otherwise shows a "Loading..." placeholder. The `go()` router's `ensureStateData()` call will update this mount point asynchronously.

`renderLegWatch()` (app.js:1489–1508)

Returns: The State Legislature Watch card HTML:

- Header with totals (bills, legislators, statehouses).
- `<select id="leg-select">` with all available states, defaulting to `"CA"` if present.
- `#leg-detail` div pre-rendered with `renderLegDetail(defaultState)`.

`window.tfShowLegState(ab)` (app.js:1451–1454): The `onchange` handler of `#leg-select`. Replaces `#leg-detail.innerHTML` with `renderLegDetail(ab)`. Exposed on `window` because it is called from an inline `onchange` attribute in the HTML string returned by `renderLegWatch()`.

`renderLegDetail(ab)` (app.js:1412–1450)

Signature: `function renderLegDetail(ab: string): string` **Returns:** The HTML for one state's legislature detail: session metadata, bill status breakdown pills, chamber composition bars (`chamberBar()`), economically-relevant bills table, legislator roster with Open States photos.

Helpers:

- `osPhotoIndex(ab)` : Builds and caches a lookup map from `lastname|firstInitial` → `{image, url, email}` using Open States data.
- `osKey(name)` : Normalizes a name to the `lastname|firstInitial` key.
- `partyColor(p)` : Returns a hex color for D/R/I/Other.
- `chamberBar(name, c)` : Renders a proportional party-composition bar with legend.

`renderInfluence()` (app.js:1511–1556)

Returns: The InfluenceWeb page HTML: the graph stage (canvas, zoom controls, breadcrumb, legend, tooltip, HUD, drill panel, side panel). The actual graph engine is initialized separately by `influenceweb.js` when `window.IW.activate()` is called.

Notable: Optionally prepends `window.Predictions.render()` at the top if the Predictions module is loaded.

4.8 Search System

`searchIndex()` (app.js:1560–1595)

Signature: `function searchIndex(): object[]` **Returns:** A lazily-built flat array of search items. Memoized after first call via `_searchIndex`.

Item types:

- `{type: "pol", key: name, group: "Politicians", label, sub, ico}` — one per politician.
 - `{type: "bill", key: bill_id, group: "Bills", label, sub: title, ico: "BILL"}` — deduplicated across `D.bills` and `D.correlation.top_bills`.
 - `{type: "event", key: id, group: "Historical Events", label: name, sub: date_range, ico: "HIST"}`.
 - `{type: "page", key: "political", group: "Tickers", label: ticker, sub: "N congressional trades", ico}` — from unique tickers in `D.recent_trades`.
 - `{type: "page", key: "news", group: "News", label: headline, sub: sector, ico: symbol}` — one per news article.
 - `{type: "page", key: "news", group: "Your Costs", label: category, sub: impact, ico: "$"}` — from `D.everyday`.
-

`runSearch(q)` (app.js:1598–1625)

Signature: `function runSearch(q: string): void`

Behavior:

1. If `q.trim().length < 2`: clears and hides `#searchResults`; returns.
 2. Iterates `searchIndex()`. For each item where ``${label} ${sub}.toLowerCase().includes(q)``, pushes to the item's group bucket.
 3. Renders results by group (`["Politicians", "Bills", "Tickers", "Historical Events", "News", "Your Costs"]`) with per-group caps (`{Politicians: 6, Bills: 6, "Historical Events": 4, Tickers: 6, News: 4, "Your Costs": 4}`).
 4. Sets `#searchResults.innerHTML` and adds class `"open"` if any results exist; otherwise shows a "No matches found" message.
-

`searchDispatch(type, key)` (app.js:1627–1635)

Signature: `function searchDispatch(type: string, key: string): void`

Behavior:

1. Clears and hides `#searchResults`; clears the search input.
 2. Routes by `type`:
 - `"pol"` → `politicianProfile(key)`.
 - `"bill"` → `billDetail(key)`.
 - `"event"` → `go("history")` then `eventDetail(key)`.
 - `"page"` → `go(key)` (navigates to the target page).
-

4.9 Router

PAGES (app.js:1638–1650)

A plain object mapping page name strings to their render functions:

```
const PAGES = {
  dashboard: renderDashboard, influence: renderInfluence, intelligence: renderIntelligence,
  states: renderStates, news: renderNews, political: renderPolitical,
  portfolio: renderPortfolio, markets: renderMarkets, reasoning: renderReasoning,
  history: renderHistory, settings: renderSettings,
};
```

rendered (app.js:1651)

```
const rendered = {};
```

A dictionary recording which pages have been rendered at least once. `rendered[page] = true` after first render; `rendered[page] = false` to force a re-render on next `go()` call.

go(page) (app.js:1653–1682)

Signature: `function go(page: string): void`

Behavior:

1. If `PAGES[page]` is not found, defaults to `"dashboard"`.
2. **Lazy render:** If the host element exists and `rendered[page]` is falsy, calls `PAGES[page]()` and sets `host.innerHTML` to the result. Marks `rendered[page] = true`.
3. **Page activation:** Removes `"active"` from all `.page` elements; adds it to `#page-{page}`. Updates `.nav-item` active state.
4. **Scroll reset:** `document.querySelector(".viewport").scrollTop = 0`.
5. **Hash update:** `location.hash = page`.
6. **State Legislature lazy load** (if `page === "states"`): Calls `ensureStateData()` with a callback that replaces `#leg-watch-mount.innerHTML` with `renderLegWatch()`.
7. **InfluenceWeb activation/deactivation:** If `page === "influence"` → `requestAnimationFrame(() => { window.IW.activate(); window.IW.resetView(); })`. Otherwise → `window.IW.deactivate()`. The `requestAnimationFrame` ensures the page element is visible (display not none) before the graph engine tries to measure canvas dimensions.

Design note: Pages are rendered once and cached. Exceptions: `rendered.settings` and `rendered.news` are explicitly reset to `false` after a profile save (in the `#saveProfileBtn` handler in `init()`) so the next navigation re-renders them with fresh tailoring data.

initTicker() (app.js:1684–1688)

Signature: `function initTicker(): void`

Behavior: Sets `#ticker.innerHTML` to a horizontal strip of market data spans from `D.market_ticker`. Each item has a label, value, and direction-colored change percentage.

Called by: `init()` and `window.TF.onRefresh()`.

4.10 Initialization and Lifecycle

setGreeting() (app.js:1692–1702)

Signature: `function setGreeting(): void`

Behavior:

1. Reads the display name: prefers `window.Auth.user().name`, falls back to `D.user.name`, then to the hardcoded default `"Allan"`.
2. Computes the time-of-day greeting: `< 12` → "Good morning"; `< 17` → "Good afternoon"; else "Good evening".
3. Sets text content of `#userName`, `#userPlan`, `#userAvatar`, `#greet`.

Called by: `init()` (once) and `window.TF.onRefresh()` (on every re-login or data refresh). Keeping it separate from `init()` means it runs after a second-user login without reinitializing the whole app.

init() (app.js:1703–1833)**Signature:** `function init(): void`**Behavior (guarded by `_inited` flag — runs only once):**

1. `setGreeting()` .
2. `initTicker()` .
3. **Nav click** delegation on `#nav` : `go(item.dataset.page)` on click.
4. **Nav keyboard** delegation on `#nav` : `Enter` / `Space` on focused nav items → `go()` (WCAG 2.1.1 compliance).
5. **Global body click** delegation — handles all `data-*` actions in rendered page content:
 - `[data-secnews]` → `sectorNewsModal(sec)` .
 - `[data-stock]` → `stockDetail(ticker)` .
 - `[data-iwco]` → `go("influence") + window.IW.openCompany(ticker)` .
 - `[data-pol]` → `politicianProfile(name)` .
 - `[data-action="alltrades"]` → `allTradesView()` .
 - `[data-bill]` → `billDetail(id)` .
 - `[data-event]` → `eventDetail(id)` .
 - `[data-goto]` → `go(page)` .
 - Does not hijack real `<a>` link clicks.
6. **Modal close:** `#modalClose` click → `closeModal()` ; overlay `#modal` click (not the box) → `closeModal()` ; `Escape` keydown → `closeModal()` + close search dropdown.
7. **Search:** `#searchInput` `input` event → `runSearch()` ; `focus` event → re-run if non-empty; `#searchResults` click → `searchDispatch()` ; outside-click handler → close dropdown.
8. **Dark/light theme toggle:**
 - Reads `localStorage.getItem("tf-theme")` on init.
 - `applyTheme(light)` : toggles `class="light"` on `<body>` and the toggle's `"off"` class; sets `aria-checked` .
 - `toggleTheme()` : flips `lightMode` , calls `applyTheme` , persists to `localStorage` .
 - `#darkToggle` click and `Enter` / `Space` keydown both trigger `toggleTheme()` .
9. **Accessibility mode toggle** (delegated, because `#a11yToggle` is inside the Settings page which renders lazily):
 - `toggleA11y(t)` : reads current state from toggle's `"off"` class; calls `window.Auth.setA11y(turningOn)` .
 - Wired to both click and `Enter` / `Space` keydown on `#a11yToggle` .
10. **Profile save** (`#saveProfileBtn` click):
 - Reads all five Settings form inputs.
 - Calls `window.Auth.setProfile(answers)` .
 - Sets `rendered.settings = false; rendered.news = false; go("settings")` — forces re-render of both Settings (to show new risk badge) and News (to re-apply tailoring).
11. **Accessibility statement link** (`#a11yStatementLink` click):
 - Prevents default navigation.
 - Calls `openModal()` with the full WCAG 2.1 AA statement, including keyboard navigation claims, accessibility mode description, known limitations (InfluenceWeb graph is visual-only), and feedback email.
12. **Initial route:** `go(location.hash.replace("#", "") || "dashboard")` — restores the last visited page from the URL hash, defaulting to dashboard.

Fallback: `document.addEventListener("DOMContentLoaded", function() { if (!window.Auth) init(); })` at app.js:1847 — if the auth gate is somehow absent, `init()` runs directly on DOM ready.

window.TF.init (app.js:1837)

```

window.TF = window.TF || {};
window.TF.init = init;

```

The public entry point called by `auth.js:unlock()` . Delegates directly to `init()` . The `_inited` guard inside `init()` makes this idempotent — calling it a second time (e.g., if `onRefresh()` triggers a re-auth) is a no-op.

window.TF.onRefresh (app.js:1838–1843)

```

window.TF.onRefresh = function () {
  setGreeting();
  const p = (location.hash || "#dashboard").slice(1);
  if (rendered[p]) { rendered[p] = false; go(p); }
  if (typeof initTicker === "function") initTicker();
};

```

Called by: `auth.js:unlock()` (immediately after `TF.init()` on first login) and `auth.js:refreshLiveData()` (on every data refresh interval tick).

Behavior:

1. `setGreeting()` — updates name/greeting/avatar for the current user.
2. Gets the current page from `location.hash`. If it has been rendered (`rendered[p]` is true), resets the flag and calls `go(p)` to re-render it with fresh data.
3. `initTicker()` — refreshes the market ticker strip.

Design note: Only the currently-visible page is re-rendered on a data refresh. Other pages' `rendered[page]` flags remain `true`; when the user navigates to them, they will serve the cached version until the next call to `onRefresh()`. This is acceptable because data refreshes happen on the same cadence as a user is likely to browse between pages.

5. Window Globals Cross-Reference

Global	Set by	Read by
<code>window.TFBoot</code>	<code>boot.js</code>	<code>auth.js</code> (all auth calls), <code>app.js</code> (<code>ensureStateData</code>), <code>influenceweb.js</code>
<code>window.Auth</code>	<code>auth.js</code>	<code>app.js</code> (<code>preferredSectors</code> , <code>renderSettings</code> , <code>init</code> handlers), other modules
<code>window.TF</code>	<code>app.js</code>	<code>auth.js</code> (<code>unlock</code> : <code>TF.init</code> , <code>TF.onRefresh</code>)
<code>window.TF_DATA</code>	<code>js/data.js</code> (static) or <code>/api/data/bundle</code> (server)	<code>app.js</code> (captured as <code>const D</code> at eval time)
<code>window.PRICES_DATA</code>	<code>js/prices_data.js</code> or <code>/api/data/live</code>	<code>app.js</code> (<code>tickerCompanyName</code> , <code>stockDetail</code>); refreshed by <code>auth.js:refreshLiveData</code>
<code>window.NEWS_INTEL</code>	<code>js/news_intel.js</code> or <code>/api/data/live</code>	<code>app.js</code> (captured as <code>const NEWS_INTEL</code> at eval time); refreshed globally by <code>refreshLiveData</code>
<code>window.STATES_DATA</code>	<code>js/states_data.js</code> or <code>/api/data/bundle</code>	<code>app.js</code> (captured as <code>const STATES</code>)
<code>window.LEGISCAN_DATA</code>	<code>js/legiscan_data.js</code> or <code>/api/data/state</code>	<code>app.js:ensureStateData</code> (lazy)
<code>window.OPENSTATES_DATA</code>	<code>js/openstates_data.js</code> or <code>/api/data/state</code>	<code>app.js:ensureStateData</code> (lazy)
<code>window.SECBULK_DATA</code>	<code>js/secbulk_data.js</code> or bundle	<code>app.js</code> (captured as <code>const SECBULK</code>)
<code>window.FEC_DATA</code>	<code>js/fec_data.js</code> or bundle	<code>app.js</code> (captured as <code>const FEC</code>)
<code>window.IW_DATA</code>	<code>js/influence_data.js</code> or bundle	<code>app.js</code> (captured as <code>const IWCOS</code> , <code>const IW_NAMES</code>)
<code>window.USA_DATA</code>	<code>js/usaspending_data.js</code> or bundle	<code>app.js</code> (captured as <code>const USASTOCK</code> , <code>const USADATA</code>)
<code>window.NP_DATA</code>	<code>js/nonprofit_data.js</code> or bundle	<code>app.js:renderIntelligence</code>
<code>window.TFScores</code>	<code>js/scores.js</code>	<code>app.js</code> (<code>stockDetail</code> , <code>renderIntelligence</code> , <code>polScore3</code>)
<code>window.GenImpact</code>	<code>js/genimpact.js</code>	<code>app.js:genImpactBlock</code> , <code>app.js:stockDetail</code>
<code>window.ScoreInfo</code>	<code>js/scoreinfo.js</code>	<code>app.js:polBadge</code>
<code>window.Predictions</code>	<code>js/predictions.js</code>	<code>app.js:renderInfluence</code>
<code>window.IW</code>	<code>js/influenceweb.js</code>	<code>app.js:go</code> (activate/deactivate/openCompany)
<code>window.tfShowLegState</code>	<code>app.js</code>	Inline <code>onchange</code> in <code>renderLegWatch()</code> HTML string

Part 6B — Frontend Reference: Feature Modules

This document is an exhaustive technical reference for the seven feature-module JavaScript files that power the ThinkFree Finance static webapp. Each module is self-contained in an IIFE, reads only from pre-loaded `window.*` globals, and writes its public API back onto `window.*`.

Table of Contents

1. [influenceweb.js — Interactive Relationship Graph](#)
2. [congress.js — Political-Intelligence Drill-Down](#)
3. [genimpact.js — Generation Impact Engine](#)
4. [predictions.js — Predictive Market Signals](#)
5. [scores.js — ThinkFree Scores Engine](#)
6. [scoreinfo.js — Progressive Score Explanation System](#)
7. [glossary.js — Finance-Jargon Tooltips](#)

influenceweb.js

File: `webapp/js/influenceweb.js` **Public API:** `window.IW` **Lines:** 1–1201

Purpose

Renders and controls the interactive "InfluenceWeb" relationship graph — a force-directed bubble map rooted at a U.S. Congress hub node. Clicking any node expands it outward into children: Congress → Sectors → Companies → Category nodes (Board, Shareholders, Political Connections, Legislation, Lobbying, Government Contracts). A side panel opens alongside each focused node showing detailed accountability data. Cross-company relationship edges (shared lobby firm, shared institutional owner, shared board member) are drawn as dashed xlink lines across sectors.

window.* Data Consumed

Global	Shape	Purpose
<code>window.TF_DATA</code>	<code>{bills, recent_trades, politicians, news, correlation, market_ticker}</code>	Bills, trades, politician list, news sentiment
<code>window.IW_DATA</code>	<code>{companies: { [ticker]: {name, domain, blurb, board, executives, owners, lda_registrant_id, fedspending_id} }}</code>	LittleSis company/board/owner data
<code>window.NP_DATA</code>	<code>{byName: { [orgName]: {name, ein, revenue_fmt, year, url} }}</code>	ProPublica nonprofit financials for lobbying groups
<code>window.SEC_DATA</code>	<code>{byTicker: { [ticker]: {entityName, board[], filedAt, latest_10k, filings} }}</code>	SEC EDGAR current board members
<code>window.SECBULK_DATA</code>	<code>{byTicker: { [ticker]: {revenue, net_income_fmt, assets, liabilities, filings_total, fy, sic} }}</code>	SEC EDGAR bulk financials
<code>window.FEC_DATA</code>	<code>{byName: {...}}</code>	OpenFEC campaign finance (passed through to other modules)
<code>window.USA_DATA</code>	<code>{byTicker: { [ticker]: {total_contracts, total_contracts_fmt, top_agencies[], url} }}</code>	USASpending.gov federal contracts
<code>window.RELATIONSHIPS</code>	<code>{byTicker: { [ticker]: {firms[], lobbyists[], issues[], bills[], spend_fmt, filings} }}</code>	Senate LDA lobbying relationships
<code>window.SP500</code>	<code>{byTicker: { [ticker]: {sector, name} }}</code>	S&P 500 membership for sector merging
<code>window.PRICES_DATA</code>	<code>{byTicker: { [ticker]: {name, market_cap, change_pct} }}</code>	Market price / cap data for sorting
<code>window.TFScores</code>	computed by <code>scores.js</code>	Influence / dependency scores displayed in panels
<code>window.CongressDrill</code>	computed by <code>congress.js</code>	Drill-down overlay opened on Congress click

Graph Data Model

Nodes are plain objects held in the module-scoped `nodes` array:

```
{
  id: "n<seq>",          // unique string assigned by addNode()
  type: "congress" | "sector" | "company" | "more" | "category",
  colorType: "company" | "government" | "political" | "financial" | "other",
  label: string,
  ticker?: string,        // company and category nodes
  icon: string,           // key into the I{} SVG path map
  parent?: string,        // id of parent node (null for root)
  size: number,           // bubble diameter in px
  x, y: number,           // current world-space position
  vx, vy: number,         // velocity (physics integration)
  tx?, ty?: number,       // target slot position
  angle?: number,         // radial angle from parent (layout hint)
  targetDist?: number,    // target radial distance from parent
  expanded?: boolean,
  collapsing?: boolean,
  fresh?: boolean,        // true = newly added, triggers fade-in
  sensitive?: boolean,    // red ring: trades + bills + lobby all overlap
  _all?, _shown?: any,    // sector pagination state
  moreOf?: string,        // id of parent sector for "more" nodes
  cat?: string,           // category key for category nodes
  sub?: string,           // subtitle string for category nodes
  el?: HTMLElement,       // live DOM element (set by rebuild())
}
```

Edges are objects in the module-scoped `edges` array:

```
{
  a: string,              // source node id
  b: string,              // target node id
  kind: "main" | "xlink",
  label?: string,        // xlink only: human-readable connection reason
  el?: SVGLineElement,
}
```

`nodeById` is a plain object mapping `id` → `node` for O(1) lookup.

Layout and Rendering

Coordinate system: World space is centered at (0,0). The Congress root is pinned at the origin. The CSS `transform: translate(cam.x, cam.y) scale(cam.zoom)` is applied to `#iw-scene`, converting world→screen. Zoom bounds: `ZOOM_MIN = 0.3`, `ZOOM_MAX = 3.2`. Auto-fit never zooms closer than `FIT_MAX = 1.5`.

Node sizes (bubble diameter in px):

Node type	Size
congress	118
sector	94
company, more	72
category	56
leaf / other	50

Sector ring: 11 curated sectors + any additional sectors from SP500 merge. Each sector is placed at angle $-\pi/2 + (i/n) \times 2\pi$ on a dynamic radius computed by `dynRadius()`.

Company fan layout (staggered zig-zag): When a sector expands, company nodes fan outward in an arc away from the grandparent. When more than 5 children share a slot, alternate children are placed on an inner and outer radius row (stagger = `childFoot × 0.6`) to prevent overlap. Computed by `layoutChildren()` (influenceweb.js:340–374).

Physics: A two-pass force model in `physics()`:

1. Collision-only repulsion: pairs whose centres are closer than $(ra + rb + 34)$ are pushed apart.
2. Spring toward slot: each node is pulled toward its `(tx, ty)` target with strength 0.14.
3. Integration: $vx = (vx + fx) \times 0.55$. The Congress node is pinned at (0,0).

Orbit dots: Each bubble's animated ring has coloured dot `<span>` elements. Dot count encodes relationship richness: up to 10 for Congress/sector nodes, up to 8 for company nodes. Colours cycle through `["#38BDF8", "#E9C46A", "#EF4444", "#14B8A6"]`.

Company logos: Clearbit logo CDN is used when `IWD[ticker].domain` is present. On image error, falls back to a 2–3 character initials `<span>` (influenceweb.js:516–518).

Sensitivity ring: A company node's `--ring` CSS variable is set to `#EF4444` (red) when `sensitive === true` (meaning trades, bills, and lobbying all overlap for that ticker), otherwise the type color (influenceweb.js:524).

Interactions

Zoom/pan (mouse):

- Wheel on `#iw-canvas` zooms anchored at cursor world position (influenceweb.js:982–989).
- Mousedown + mousemove on canvas pans. A move of >4px sets `drag.moved = true`, suppressing the click handler (influenceweb.js:992–1003).
- `+` / `-` toolbar buttons call `zoomBtn(1.25)` / `zoomBtn(0.8)` centered on viewport midpoint.
- Reset/fit button calls `reset()`.

Click on nodes:

- `more` node: calls `revealMore()` to load the next batch of `SHOW_LIMIT=12` companies.
- `congress` node: calls `IW.zoomToCongress()` then `CongressDrill.open()`.
- Expanded node (with children): calls `collapse()` to retract the branch, then re-fits camera.
- All other nodes: calls `focus()`.

Hover:

- `mouseover` on a node element calls `highlight(id, true)` (dims unrelated nodes/edges) and `showTip(n, e)` (floating brief tooltip).
- `mouseout` clears highlight and hides tooltip.

Side panel delegated click handler (influenceweb.js:1046–1056):

- `[data-conn]` → `connectionDetail()` (person-company connection panel)
- `[data-boardback]` → `boardListPanel()` (back to board list)
- `[data-copanel]` → `openPanel()` for the company node
- `[data-exec]` → `personPanel()` (person profile)
- `[data-co]` → `focus()` on matching company node, or expand sector first
- `[data-person]` → `window.politicianProfile()`
- `[data-bill]` → `window.billDetail()`
- `[data-up]` → `focus(nodeById[id])` (navigate up in breadcrumb)
- `[data-board]` → `boardListPanel()` (View All members button)

Breadcrumb: Click on a `.iw-crumb[data-cr]` calls `focus(nodeById[id])` to jump directly to that ancestor (influenceweb.js:1059).

Cross-Company Relationship Links

xlink edges are drawn as dashed SVG lines styled with class `iw-xlink`. They appear when `showXlinks === true` (toggled via the Filters panel checkbox).

`linkSharedOwners()` (influenceweb.js:378–387) scans all visible company nodes pairwise ($O(n^2)$) and calls `sharedLink(a, b)`. `sharedLink` checks in priority order:

1. Shared lobbying firm (from `REL[tk].firms`) → label "Both lobby via {firm}"
2. Shared institutional owner (from `IWD[tk].owners`) → label "Shared owner: {name}"
3. Shared board member (from `boardOf()`) → label "Shared board member: {name}"

The label string is stored as a native SVG `<title>` child so it appears in browser-native tooltips.

Functions

`boardOf(tk)` — influenceweb.js:25–38

Signature: `boardOf(tk: string): BoardMember[]` Returns the board member list for ticker `tk`. Prefers live SEC EDGAR data from `SECD[tk].board` (normalised to `{name, title, age, committees, independent, since, source:"SEC"}`). Falls back to LittleSis `IWD[tk].board` and `.executives` (source:"LittleSis"). Normalises `independent` flag by checking the `position` string for the word "independent".

`buildPersonIndex()` — influenceweb.js:140–148

Signature: `buildPersonIndex(): void` Walks every ticker in the union of `IWD` and `SECD`, calls `boardOf()` for each, and populates `personIndex[name]` with `{ticker, company, title, current, role}` entries. Called once from `indexData()`. Side effect: populates the module-scoped `personIndex` object used by revolving-door detection.

`pushPerson(m, tk, company, role)` — `influenceweb.js:151–153`

Signature: `pushPerson(m: {name, title, current}, tk: string, company: string, role: string): void` Appends a single person-company entry to `personIndex[m.name]`. Called exclusively by `buildPersonIndex()`.

`indexData()` — `influenceweb.js:156–166`

Signature: `indexData(): void` Called once at first activate. Builds all secondary indexes:

- Calls `buildPersonIndex()`
- Deduplicates all bills (merging `D.bills` with `D.correlation.top_bills`) into `billsByTicker` and `lobbyBySector`
- Counts `tradeCount[ticker]` and populates `polsByTicker[ticker]` from `D.recent_trades`

Side effects: mutates `tradeCount`, `billsByTicker`, `polsByTicker`, `lobbyBySector`, `personIndex`.

`compName(tk)` — `influenceweb.js:169`

Signature: `compName(tk: string): string` Returns the best available display name for a ticker. Priority: `IWD` → `SECD` → `PX` → `SP` → `tk`.

`companyHasData(tk)` — `influenceweb.js:172`

Signature: `companyHasData(tk: string): boolean` Returns true when any of `IWD`, `SECD`, `tradeCount`, or `billsByTicker` has data for the ticker.

`sectorCompanies(sec)` — `influenceweb.js:176–178`

Signature: `sectorCompanies(sec: SectorDef): string[]` Returns all tickers in the sector sorted by market cap descending, then by `companyInfluence()` descending, then alphabetically.

`companyInfluence(tk)` — `influenceweb.js:182–189`

Signature: `companyInfluence(tk: string): number` Composite heuristic influence score (not capped). Formula: $18 + \text{board.length} \times 1.4 + \text{owners.length} \times 2.2 + \text{bills} \times 2.2 + \min(\text{trades}, 60) \times 0.5$ Used for graph sorting and orbit-dot count. Not the same as the formal `TFScores.influenceScore()`.

`sectorInfluence(sec)` — `influenceweb.js:193–199`

Signature: `sectorInfluence(sec: SectorDef): number` Aggregates company influence scores for a sector with a per-company cap ($\times 0.18$) plus `lobbyBySector` group count ($\times 4$). Used to rank sectors.

`influenceLabel(tk)` — `influenceweb.js:203–205`

Signature: `influenceLabel(tk: string): "High" | "Medium" | "Moderate"` Maps `companyInfluence()` to a tier: $> 55 \rightarrow \text{High}$, $> 35 \rightarrow \text{Medium}$, else $\rightarrow \text{Moderate}$.

`sharedOwner(a, b)` — `influenceweb.js:210–213`

Signature: `sharedOwner(a: string, b: string): boolean` Returns true when `IWD[a].owners` and `IWD[b].owners` share any owner name (Set intersection).

`sharedLink(a, b)` — `influenceweb.js:217–233`

Signature: `sharedLink(a: string, b: string): string` Returns a human-readable connection description (or `""` if none). Priority: shared lobby firm > shared institutional owner > shared board member. Used for xlink edge labels.

`addNode(o)` — `influenceweb.js:245–248`

Signature: `addNode(o: Partial<Node>): Node` Assigns `o.id = "n" + seq++`, initialises `vx/vy = 0`, `fresh = true`, pushes to `nodes`, indexes in `nodeById`. Returns the node.

`addEdge(a, b, kind)` — `influenceweb.js:252`

Signature: `addEdge(a: Node|null, b: Node|null, kind: "main"|"xlink"): void` Pushes `{a:a.id, b:b.id, kind}` to `edges`. Guards against null inputs.

sizeForType(o) — influenceweb.js:256–264

Signature: `sizeForType(o: Node): number` Returns the bubble diameter (px) for a node's type. See the size table above.

dynRadius(parent, count, childSize) — influenceweb.js:269–274

Signature: `dynRadius(parent: Node, count: number, childSize?: number): number` Computes a ring radius large enough so `count` children of diameter `childSize` fit without overlapping. Returns $\max(\text{parentRadius} + \text{childRadius} + 120, (\text{count} \times \text{footprint}) / (2\pi), 260)$.

expandCongress(cn) — influenceweb.js:281–292

Signature: `expandCongress(cn: Node): void` Guards against double-expansion. Creates one sector node per entry in `SECTORS`, placed at evenly spaced angles on a `dynRadius()` circle. Each node starts at the Congress position and springs outward. Adds main edges Congress → sector.

makeCompanyNode(sn, tk) — influenceweb.js:295–300

Signature: `makeCompanyNode(sn: Node, tk: string): Node` Creates a company node, computes `sensitive` flag (true when trades + bills + sector lobby all present), adds a main edge from the sector node. Returns the new node.

makeMoreNode(sn) — influenceweb.js:303–308

Signature: `makeMoreNode(sn: Node): Node` Creates a "+N More" node showing how many companies remain hidden. Sets `moreOf = sn.id`. Adds a main edge from the sector node.

expandSector(sn) — influenceweb.js:311–319

Signature: `expandSector(sn: Node): void` Guards against double-expansion. Calls `sectorCompanies()`, creates up to `SHOW_LIMIT=12` company nodes plus a More node if needed, calls `layoutChildren()` and `linkSharedOwners()`.

revealMore(moreNode) — influenceweb.js:322–335

Signature: `revealMore(moreNode: Node): void` Removes the More node from `nodes / edges / nodeById`, creates the next batch of up to `SHOW_LIMIT` company nodes, increments `sn._shown`, creates a new More node if still needed, then calls `layoutChildren()`, `linkSharedOwners()`, `rebuild()`, and `fitSubtree(sn, 0.74)`.

layoutChildren(parent) — influenceweb.js:340–374

Signature: `layoutChildren(parent: Node): number` Distributes all children of `parent` around it. For root (no grandparent): full circle $\text{span} = 2\pi$. For non-root: outward fan with $\text{span} = \min(\pi \times 0.9, 0.6 + (n-1) \times 0.2)$. Applies staggered zig-zag (2 rows) when `hasGrandparent` && `n > 5`. Sets `(tx, ty)` and `angle` on each child. Returns the outer radius.

linkSharedOwners() — influenceweb.js:378–387

Signature: `linkSharedOwners(): void` $O(n^2)$ scan of all visible company nodes. For each pair not already connected by an xlink, calls `sharedLink()` and pushes an xlink edge if a connection exists.

politicalTies(tk) — influenceweb.js:392–401

Signature: `politicalTies(tk: string): PoliticalTies` Returns `{traders, industry, firms, lobbyists, issues, bills, contracts, registered, spend, filings, sec, count}` for the Political Connections panel. Combines `polByTicker`, `SECTOR_LOBBY`, `CROSS_LOBBY`, and `REL[tk]` lobbying data.

expandCompany(cn) — influenceweb.js:417–426

Signature: `expandCompany(cn: Node): void` Guards against double-expansion. Filters `CATEGORIES` (omitting `contracts` when no USASpending record exists), creates one category node per entry, calls `layoutChildren()`.

expandNode(n) — influenceweb.js:429–433

Signature: `expandNode(n: Node): void` Dispatcher: routes to `expandCongress`, `expandSector`, or `expandCompany` based on `n.type`.

isDescendant(d, p) — influenceweb.js:436–439

Signature: `isDescendant(d: Node, p: Node): boolean` Walks the parent chain of `d`; returns true if `p.id` appears at any level.

collapse(node) — influenceweb.js:443–451

Signature: `collapse(node: Node): void` Marks all non-collapsing descendants with `collapsing = true` and sets their `(tx, ty)` to the node's current position (glide inward). Pushes them to `leaving`. Calls `scheduleSweep()`.

scheduleSweep() — influenceweb.js:456–467

Signature: `scheduleSweep(): void` Defers DOM cleanup by 300ms (CSS animation duration). On timeout: removes leaving nodes/edges from the live arrays, deletes from `nodeById`, clears `leaving`, calls `rebuild()`. Guards against duplicate timers.

collapseSiblings(n) — influenceweb.js:471–472

Signature: `collapseSiblings(n: Node): void` Finds all nodes that share `n.parent`, are not `n`, and are expanded. Calls `collapse()` on each.

s2w(sx, sy) — influenceweb.js:477

Signature: `s2w(sx: number, sy: number): {x, y}` Converts screen (viewport-relative) coordinates to world (unzoomed) coordinates using the current camera.

applyCam() — influenceweb.js:480

Signature: `applyCam(): void` Writes `translate(cam.x px, cam.y px) scale(cam.zoom)` to `scene.style.transform`.

rebuild() — influenceweb.js:484–543

Signature: `rebuild(): void` Full DOM reconstruction. Removes all `.iw-node` divs and clears SVG. Re-creates one SVG `<line>` per edge and one `<div class="iw-node">` per node (with bubble, orbit dots, icon/logo, and label). Applies `iw-faded` to non-active sector nodes. Sets initial opacity for fresh/collapsing nodes and triggers fade transitions via `requestAnimationFrame`. Calls `syncDOM()` at the end.

physics() — influenceweb.js:547–582

Signature: `physics(): number`

Single physics tick. Two passes: (1) collision-only repulsion for overlapping pairs; (2) spring-toward-slot for each non-congress node. Integrates with damping 0.55. Returns total `|vx| + |vy|` across all nodes (used to detect settlement).

syncDOM() — influenceweb.js:585–592

Signature: `syncDOM(): void` Cheap position sync: updates `.el.style.left/top` for every node and `x1/y1/x2/y2` for every edge SVG line. Called every animation frame.

fitSubtree(parent, pad) — influenceweb.js:596–612

Signature: `fitSubtree(parent: Node, pad?: number): void` Computes the axis-aligned bounding box of `parent` and its direct children (using `tx/ty` target positions). Sets `camT` to a zoom/pan that fits the bbox into the viewport with margin `pad` (default 0.85), bounded by `ZOOM_MIN` and `FIT_MAX`.

orbitDots(n) — influenceweb.js:616–627

Signature: `orbitDots(n: Node): string` Returns HTML `<span class="iw-odot">` elements arranged around the bubble ring. Count: up to 10 for Congress/sector, up to 8 for company, 0 for others. Colour cycles through 4 palette entries.

setBreadcrumb(n) — influenceweb.js:630–635

Signature: `setBreadcrumb(n: Node): void` Walks the parent chain of `n` to build a path array, then injects `<span class="iw-crumb">` elements separated by `>` into `#iw-breadcrumb`.

focus(n) — influenceweb.js:639–659

Signature: `focus(n: Node): void` The primary interaction handler. Sets `selectedId`, calls `collapseSiblings()`, updates `focusedSector`, calls `expandNode()`, `rebuild()`, `setBreadcrumb()`, `openPanel()`. Adds `iw-focus` CSS class to the node element. Calls `fitSubtree()` or snaps the camera to the node. Sets `settle = 90`.

influenceBadge(tk) — influenceweb.js:664

Signature: `influenceBadge(tk: string): string` Returns a coloured `<span>` for the influence tier label ("High"=red, "Medium"=yellow, "Moderate"=teal). Used inside panel HTML.

scoreBlock(tk) — `influenceweb.js:668–680`

Signature: `scoreBlock(tk: string): string` Returns HTML for two `iw-meter` bar rows (Influence Score + Government Dependency) if `window.TFScores` is loaded. Each bar includes a `ScoreInfo` `si` anchor with `data-si-kind`, `data-si-key`, and `data-si-val` attributes for the progressive-disclosure tooltip system.

secFinancials(tk) — `influenceweb.js:683–695`

Signature: `secFinancials(tk: string): string` Returns a 6-cell grid of SEC EDGAR financials (Revenue, Net Income, Assets, Liabilities, Filings, SIC Industry) from `SECBULK_DATA`. Returns `""` when no data is available.

openPanel(n) — `influenceweb.js:698–748`

Signature: `openPanel(n: Node): void` Builds and injects the panel `innerHTML` based on `n.type`:

- **congress:** stats grid (sectors, companies, bills count) + hint text.
- **sector:** company count + chip list of first 12 tickers.
- **company:** back button, score block, stats grid, SEC financials, "Connected To Congress Through" section, board member rows, peers chip list, latest 10-K link.
- **category:** delegates to `categoryPanel(n)`.

Adds `open` class to the panel element.

currentSectorPeers(tk) — `influenceweb.js:752–755`

Signature: `currentSectorPeers(tk: string): string[]` Returns all other tickers in the same sector as `tk`, sorted by `sectorCompanies()`.

memberRow(m, fromTk) — `influenceweb.js:758–771`

Signature: `memberRow(m: BoardMember, fromTk: string): string` Returns an HTML `<div class="iw-member iw-clickable">` with initials avatar, name, title, and optional badges (Independent, age, committees, connection count). Includes `data-exec` and `data-from` attributes for the click delegation handler.

boardListPanel(tk) — `influenceweb.js:775–784`

Signature: `boardListPanel(tk: string): void` Injects the full board member list for a company into the panel body. Shows source attribution (SEC EDGAR vs. LittleSis). Back button navigates to the company panel.

personPanel(name, fromTk) — `influenceweb.js:788–824`

Signature: `personPanel(name: string, fromTk?: string): void` Renders the person profile panel. Splits `personIndex[name]` into current and previous positions. Current items show normal avatars; previous items show amber avatars with "Revolving Door" label. Each item is a clickable row that opens `connectionDetail()`.

connectionDetail(name, tk, title, isCurrent, fromTk) — `influenceweb.js:828–841`

Signature: `connectionDetail(name: string, tk: string, title: string, isCurrent: boolean, fromTk: string): void` Renders an explanation of a single person–company relationship. Active connections describe a current board role. Previous connections use the revolving-door framing: "career transition; does not imply wrongdoing."

howtoPanel() — `influenceweb.js:844–866`

Signature: `howtoPanel(): void` Renders a static guide panel explaining navigation controls, graph levels, and node colours. Triggered by the `?` toolbar button.

filtersPanel() — `influenceweb.js:869–884`

Signature: `filtersPanel(): void` Renders the graph filters panel including a checkbox to toggle cross-links (xlinks). The checkbox `change` event mutates `showXlinks` and calls `rebuild()`. Also shows the relationship type legend.

categoryPanel(n) — `influenceweb.js:887–965`

Signature: `categoryPanel(n: Node): string` Returns the HTML body string for a category node's panel. Branches by `n.cat`:

- **board**: board member list (same as `boardListPanel` body).
- **shareholders**: owner list from `IWD[tk].owners`.
- **political**: full political connections breakdown (lobby firms, bills lobbied, issues, lobbyists, congressional traders, industry associations, contracts). Ends with transparency disclaimer.
- **bills**: chip list of `billsByTicker[tk]` bill IDs.
- **lobbying**: ProPublica 990 rows for industry/cross-sector lobby groups with revenue, EIN, link.
- **contracts** (default fallback): USASpending total + top agency breakdown with link.

logo(tk) — `influenceweb.js:968–971`

Signature: `logo(tk: string): string` Returns a Clearbit `<img>` with fallback to a 3-character initials `<span>` for use in panel chip lists.

closePanel() — `influenceweb.js:974`

Signature: `closePanel(): void` Removes the `open` CSS class from the panel element.

bind() — `influenceweb.js:979–1065`

Signature: `bind(): void` Attaches all interaction event listeners:

- Wheel zoom on canvas (passive:false, calls `s2w` for anchor-at-cursor).
- Mousedown/mousemove/mouseup for drag-to-pan.
- Click delegation on `#iw-scene` for node clicks.
- Mouseover/mouseout on scene for highlight + tooltip.
- Click delegation on panel for back navigation and sub-panels.
- Panel close button.
- Breadcrumb click.
- Zoom +/- and fit/reset buttons.
- How-to and filters toolbar buttons.

zoomBtn(f) — `influenceweb.js:1068–1072`

Signature: `zoomBtn(f: number): void` Zooms by factor `f` centered on the viewport midpoint. Does not use `camT`; applies immediately.

highlight(id) — `influenceweb.js:1075–1084`

Signature: `highlight(id?: string): void` When `id` is given: dims all nodes/edges that are not connected to `id`. When `id` is null/undefined: clears all dim/hot classes. Uses edge adjacency set for O(edges) lookup.

showTip(n, e) — `influenceweb.js:1087–1097`

Signature: `showTip(n: Node, e: MouseEvent): void` Positions and shows `#iw-tip` with the node's label and a brief subtitle. Tooltip is positioned at cursor + 16px offset within the canvas.

reset() — `influenceweb.js:1100–1114`

Signature: `reset(): void` Clears all node/edge arrays and counters. Creates the Congress root node, calls `expandCongress()`, calls `rebuild()`. Sets `cam` to `{x: W/2, y: H/2, zoom: 0.45}`, calls `fitSubtree()`. Sets `settle = 140`. Shows HUD. Closes panel.

loop() — `influenceweb.js:1117–1132`

Signature: `loop(): void` Main `requestAnimationFrame` loop. When `active`: runs `physics()` and `syncDOM()` while `settle > 0` or `moving >= 0.6`. Interpolates `camT` toward `cam` at rate 0.1 per frame. Calls `applyCam()` every frame.

size() — `influenceweb.js:1135`

Signature: `size(): void` Reads `canvas.getBoundingClientRect()` into `W, H`. Resets SVG dimensions to 1x1 (SVG overflow handles the actual size).

window.IW Public API

IW.activate() — `influenceweb.js:1140–1150`

Binds scene/canvas/panel/tip/hud DOM references. On first call: runs `indexData()`, `size()`, `bind()`, `reset()`, attaches resize handler, starts the animation loop. On subsequent calls: sets `active = true` and re-fits camera.

IW.deactivate() — **influenceweb.js:1152**

Sets `active = false` and hides the tooltip.

IW.zoomToCongress(cb) — **influenceweb.js:1155–1165**

Animates the camera to zoom in (scale 2.4) centered on the Congress node. All non-Congress nodes fade to 6% opacity. Fires callback after 540ms delay. Saves pre-zoom camera to `preZoomCam`.

IW.zoomOut() — **influenceweb.js:1167–1172**

Reverses the Congress zoom: restores all node opacities, restores `preZoomCam` to `camT` for smooth interpolation.

IW.resetView() — **influenceweb.js:1175–1179**

Closes the drill overlay, clears `preZoomCam`, calls `reset()`.

IW.openCompany(tk) — **influenceweb.js:1181–1199**

Deep-links to a specific company. Calls `this.activate()`, finds the sector node, calls `focus(sn)`, then after 120ms focuses the company node (or reveals more companies first if the ticker is beyond the initial 12).

congress.js

File: `webapp/js/congress.js` **Public API:** `window.CongressDrill` **Lines:** 1–582

Purpose

Renders the U.S. Congress drill-down overlay (`#iw-drill`). Provides four sub-views accessible from the Congress hub node:

1. **House constellation** — scatter plot of tracked House members
2. **Senate constellation** — scatter plot of tracked Senate/Congress members
3. **Governors** — SVG choropleth / CSS tile cartogram of all 50 state governors
4. **Generation Impact** — per-bill generation impact scores

Per-member hover cards surface FEC funding influence breakdowns (individual vs. PAC vs. other) with optional pie charts. State hover cards show Census economic indicators. The module enforces the transparency-not-accusation framing throughout: all political intelligence output describes timing/funding relationships from public data and never implies wrongdoing.

window.* Data Consumed

Global	Shape	Purpose
<code>window.TF_DATA</code>	<code>{politicians[], bills[], correlation.top_bills[], disclaimer}</code>	Politician list, bill list, disclaimer text
<code>window.FEC_DATA</code>	<code>{byName: { [name]: {receipts, from_individuals, from_pacs, receipts_fmt, url} }}</code>	OpenFEC campaign finance receipts
<code>window.STATES_DATA</code>	<code>{byState: { [stAbbr]: {name, cost_of_living, median_household_income, poverty_rate, unemployment, inflation, affordability, pressure_score, prosperity_score, real_purchasing_power, income_growth} }}</code>	Census state economics
<code>window.MEMBER_BILLS</code>	<code>{byBioguide: { [id]: {total, sponsored, cosponsored} }}</code>	Congress.gov sponsored/cosponsored bill counts
<code>window.GenImpact</code>	computed by <code>genimpact.js</code>	Generation impact engine (optional, falls back to local calculation)
<code>window.ScoreInfo</code>	computed by <code>scoreinfo.js</code>	Badge renderer for score spans (optional)
<code>window.US_MAP_PATHS</code>	<code>{[stAbbr]: svgPathData}</code>	SVG state outline paths (optional, falls back to tile grid)
<code>window.US_MAP_VIEWBOX</code>	string	SVG viewBox for the US map (default: "174 100 959 593")

Global	Shape	Purpose
<code>window.IW</code>	computed by <code>influenceweb.js</code>	Graph reset on drill close
<code>window.politicianProfile</code>	function	External handler for full politician profile
<code>window.stockDetail</code>	function	External handler for stock detail drawer
<code>window.billDetail</code>	function	External handler for bill detail

Transparency-Not-Accusation Framing

Every political intelligence sub-view includes disclaimer text anchored in the HTML. The footer of `modulesView()` renders `D.disclaimer` (falling back to a hardcoded string). The hover card expanded view always ends with:

"Funding influence breakdown from FEC and OpenSecrets-style disclosures. Describes outside-funding exposure; does not imply wrongdoing."

The `fundNote` logic (`congress.js:249–251`) derives a plain-English funding observation from PAC share thresholds:

- $\geq 40\%$: "Potential influence concentration: high outside-funding exposure."
- $\geq 20\%$: "Moderate industry-linked support."
- $< 20\%$: "Largely individually funded."

DOM Target

`#iw-drill` — a full-height slide-in panel. All views are injected via `show(html)` (`congress.js:134`). The element is made visible with class `open`.

Functions

`funding(name)` — `congress.js:61–68`

Signature: `funding(name: string): FundingBreakdown | null` Reads `FEC[name]` and computes percentage breakdowns. Returns `{individual, pac, other, total_fmt, raised}` or `null` when no FEC record exists.

`scores(p)` — `congress.js:73–80`

Signature: `scores(p: PoliticianRecord): {influence, transparency, publicImpact, exposure}` Derives 0–100 scores from live signals:

- `influence = min(100, 30 + trades × 0.3 + pacShare × 0.6 + |ret| × 0.5)`
- `transparency = max(5, 100 - pacShare × 1.4 - trades × 0.15)`
- `publicImpact = max(10, 70 - pacShare × 0.5 + individual × 0.2)`

These mirror the formulas in `scoreinfo.js` to ensure consistency between the hover card and the full modal.

`billGenerationImpact(bill)` — `congress.js:111–124`

Signature: `billGenerationImpact(bill: BillRecord): GenImpactRow[]` Local fallback (used when `window.GenImpact` is unavailable). Averages sector weights from `SECTOR_GEN` across all bill sectors, scales to `-70..+70`. Returns `[{gen, score, reason}]` for each of the five cohorts.

`ensureHost()` — `congress.js:129–132`

Signature: `ensureHost(): HTMLElement | null` Gets or assigns the `host` reference to `#iw-drill`.

`show(html)` — `congress.js:134`

Signature: `show(html: string): void` Injects `html` into `host.innerHTML`, adds class `open`, resets scroll to top.

`close()` — `congress.js:135`

Signature: `close(): void` Removes class `open` from the host. Calls `window.IW.resetView()` if available.

`modulesView()` — `congress.js:138–172`

Signature: `modulesView(): void` Renders the Level 1 Congress landing view: four module cards (House, Senate, Governors, Generation Impact) with member counts from `D.politicians`. Includes the disclaimer footer. Uses `show()`.

constellation(kind) — `congress.js:177–207`

Signature: `constellation(kind: "house" | "senate"): void` Renders the Level 2 member constellation view. Filters `D.politicians` by chamber. Lays dots using the golden-angle spiral: $\text{angle} = i \times 137.5^\circ \times \pi/180$, $r = 6 + 40 \times \sqrt{i / n}$. Each dot is a `<span class="cd-dot">` with party-color background. Shows a party count legend. Includes `#cd-hover` placeholder for the hover card.

fundingPie(f, size) — `congress.js:212–229`

Signature: `fundingPie(f: FundingBreakdown, size?: number): string` Returns an SVG pie chart string. Segments: individual (green `#22C55E`), PAC (red `#EF4444`), other (gray `#64748B`). When only one non-zero segment exists, renders a filled circle instead. Default size 40px.

hoverCard(name, dotEl, big) — `congress.js:236–310`

Signature: `hoverCard(name: string, dotEl: HTMLElement|null, big: boolean): void` Renders the politician hover card into `#cd-hover`. Compact view (`big=false`): small pie, activity summary (trades, bills, return), ticker chips. Expanded view (`big=true`): large pie with totals, legislative career totals, PAC/outside spending list, top donors/industries, disclaimer footer. Calls `placeCard()` to position it. Stores `curPolName` and `curDotEl` for toggle continuity.

placeCard(dotEl) — `congress.js:314–328`

Signature: `placeCard(dotEl: HTMLElement): void` Positions `#cd-hover` relative to the dot element, flipping left when right-side overflow would occur. Clamps to drill panel bounds.

hideHover() — `congress.js:331`

Signature: `hideHover(): void` Removes `open` and `big` classes from `#cd-hover`. Clears `curPolName`.

applyMapTf() — `congress.js:338–341`

Signature: `applyMapTf(): void` Applies `translate(x y) scale(s)` to the `#cd-map-g` SVG group. Used for governors map pan/zoom.

partyCount(ab) — `congress.js:344`

Signature: `partyCount(ab: string): number` Counts how many governors in `GOVERNORS` have the given party abbreviation.

governorsView() — `congress.js:348–375`

Signature: `governorsView(): void` Renders the governors panel. Prefers SVG choropleth from `window.US_MAP_PATHS`: renders `<path class="cd-state-path">` elements with fill `${partyColor}33` and matching stroke. Falls back to a CSS grid tile cartogram using `TILE` row/col positions. Includes zoom (+/-/reset) buttons and the hover card placeholder.

stateHover(st, ev) — `congress.js:379–403`

Signature: `stateHover(st: string, ev: MouseEvent): void` Shows a transient hover card near the mouse with Census economics for state `st`: cost of living, median income, poverty rate, unemployment, inflation, affordability, cost pressure tier, and prosperity score. Positioned relative to cursor, clamped to drill panel.

generationView(billId) — `congress.js:408–438`

Signature: `generationView(billId?: string): void` Renders the Generation Impact overlay. Finds the bill in `D.bills + D.correlation.top_bills`. If `window.GenImpact` is loaded, calls `GI.forBill(bill)` and `GI.panel(res, {reasons:true, dims:true})`. Otherwise calls local `billGenerationImpact()` and renders coloured horizontal bars. Shows a bill-picker chip row (first 8 bills). Shows affected sectors, companies, and politicians chips below the bars.

genImpactFromState(s) — `congress.js:443–448`

Signature: `genImpactFromState(s: StateRecord): GenImpactRow[]` Derives generation impact scores from state cost statistics: pressure score, affordability index, and inflation rate. Applies generation-specific tilt multipliers (`Gen Z: 1.2, Millennials: 1.4, Gen X: 0.9, Boomers: 0.5, Retirees: 0.6`). Returns `[{gen, score}]` clamped to `[-90, +40]`.

genBars(impacts) — `congress.js:452–459`

Signature: `genBars(impacts: GenImpactRow[]): string` Renders compact horizontal bar rows for a state detail panel (used when `window.GenImpact` is unavailable).

stateDetail(st) — `congress.js:463–502`

Signature: `stateDetail(st: string): void` Renders the full state detail panel. Shows governor name and party, two card columns (State Economy + Pressure Snapshot), a generation impact section (via `window.GenImpact.panel()` or `genBars()`), and a Federal Bills In Play chip list.

bind() — `congress.js:515–573`

Signature: `bind(): void` Attaches all event listeners to `#iw-drill` using a sentinel `_bound` flag to prevent double-binding. Single delegated `click` handler routes to all sub-view functions via `data-dc` attributes. `mousemove` handler activates politician dot hover cards and state hover cards. Separate `mousedown` / `mousemove` / `mouseup` listeners on `window` manage SVG map dragging. Wheel zoom on `#cd-map-wrap` (passive:false).

window.CongressDrill Public API

CongressDrill.open() — `congress.js:579`

Signature: `open(): void` Calls `bind()` then `modulesView()`. Entry point called by `influenceweb.js` when the Congress node is clicked.

CongressDrill.generation(billId) — `congress.js:580`

Signature: `generation(billId: string): void` Calls `bind()` then `generationView(billId)`. Allows other modules to jump directly to the generation-impact overlay for a specific bill.

genimpact.js

File: `webapp/js/genimpact.js` **Public API:** `window.GenImpact` **Lines:** 1–337

Purpose

The Generation Impact engine. A reusable heuristic translation layer that estimates how a bill, company, sector, or state's economic activity may land on each of the five U.S. generational cohorts across six life dimensions. Outputs per-generation signed scores (–100..+100), a confidence percentage, and a plain-English reason. Used by `app.js`, `congress.js`, and `influenceweb.js`.

Framing Rule (Non-Negotiable)

Every rendered panel ends with:

"Estimate of likely exposure by generation. Plain-English, non-partisan; does not imply wrongdoing."

window.* Data Consumed

`window.ScoreInfo` — optional; when loaded, score values become clickable info chips.

Data Tables

GENS (`genimpact.js:27`): `["Gen Z", "Millennials", "Gen X", "Baby Boomers", "Retirees"]`

DIMS (`genimpact.js:31–38`): six dimension pairs `[key, displayLabel]: costOfLiving, housing, income, healthcare, jobs, retirement`

GEN_WEIGHT (`genimpact.js:42–48`): How exposed each generation is to each dimension (0..1). Gen Z weights jobs highest (1.0). Retirees weight healthcare (1.0) and retirement (1.0) highest.

AREA (`genimpact.js:54–80`): Signed per-dimension effect vectors for each policy area (–1..+1). 23 entries including Real Estate, Healthcare, Technology, Wages & Labor, Taxes, Education, Benefits, and a "default" fallback.

REASON_DIM (`genimpact.js:84–97`): Plain-English reason fragments keyed by dimension and direction (`harm` / `help`).

Functions

clamp(v, lo, hi) — `genimpact.js:100`

Signature: `clamp(v: number, lo: number, hi: number): number` Standard clamp. Used throughout all score calculations.

`matchArea(s)` — `genimpact.js:103–129`

Signature: `matchArea(s: string): string | null` Maps a free-form sector/industry/topic string to an `AREA` key. Tries exact match first (avoids substring collisions), then falls back to substring keyword matching (e.g. "pharma" → "Pharmaceuticals", "health" → "Health Care"). Returns `null` when no match found.

`fromAreas(areas, opts)` — `genimpact.js:134–176`

Signature: `fromAreas(areas: string[], opts?: {orientation?: number, confidenceBoost?: number}): GenerationResult` **Core computation function.** Maps each string in `areas` through `matchArea()`, aggregates signed dimension effects (averaged across matched areas), then for each generation computes:

- Per-dimension score: `clamp(round(eff[d] × w[d] × 110 × orient), -100, 100)`
- Overall score: weighted average across dimensions
- Reason: uses the `REASON_DIM` template for the strongest-magnitude dimension

Confidence formula: `clamp(50 + matched × 12 + confidenceBoost, 35, 95)`

Returns `{gens, confidence, areas, matched}`.

`forSectors(sectors, opts)` — `genimpact.js:181`

Signature: `forSectors(sectors: string[], opts?: object): GenerationResult` Thin wrapper: calls `fromAreas(sectors, opts)` directly.

`forBill(bill, opts)` — `genimpact.js:185–196`

Signature: `forBill(bill: BillRecord, opts?: object): GenerationResult` Concatenates `bill.sectors` and `bill.topics`, calls `fromAreas()`. Augments result with `{status, title, id, politicians, companies, sectors}` metadata. Status is derived by `billStatus()`.

`forCompany(ticker, industry, sectors, opts)` — `genimpact.js:200–205`

Signature: `forCompany(ticker: string, industry: string, sectors: string[], opts?: object): GenerationResult` Combines `sectors` and `industry` into an `areas` array, falls back to `["default"]`. Augments result with `{ticker}`.

`forState(s, opts)` — `genimpact.js:210–234`

Signature: `forState(s: StateRecord, opts?: object): GenerationResult` Builds a synthetic dimension effect vector directly from real state statistics:

- `costOfLiving = clamp(-(pressure - 40) / 60, -1, 0.4)`
- `housing = clamp(-(60 - afford) / 60, -1, 0.4)`
- `income = clamp(income_growth / 6, -1, 1)`
- `healthcare = clamp(-(infl - 2) / 6, -1, 0.3)`
- `jobs = clamp((4.5 - unemp) / 4, -1, 1)`
- `retirement = clamp(-(infl - 2) / 8, -1, 0.3)`

Runs the same generation-weighting pass as `fromAreas`. Returns `confidence=70` (fixed, since state data is directly measured).

`billStatus(bill)` — `genimpact.js:237–243`

Signature: `billStatus(bill: BillRecord): "Passed" | "Failed" | "Proposed"` Checks `bill.status` or `bill.action_text` for keywords. Returns "Passed" on enacted/signed, "Failed" on vetoed/dead, "Proposed" otherwise.

`siGen(g, res)` — `genimpact.js:257–271`

Signature: `siGen(g: GenResult, res: GenerationResult): ScoreInfoPayload` Builds the structured `ScoreInfo` explainer payload for one generation's impact score. Sorts dimensions by absolute magnitude, takes the top two for contributor rows, and includes the full weighted-score formula.

`bars(res, opts)` — `genimpact.js:275–289`

Signature: `bars(res: GenerationResult, opts?: {reasons?: boolean}): string`

Renders horizontal bar chart HTML (`<div class="gimp-bars">`). Each row shows the generation label, coloured score (with `ScoreInfo` chip if `window.ScoreInfo` loaded), a proportional bar, and optional reason text. Green for positive scores, red for negative, gray for neutral ($|\text{score}| \leq 8$).

confBadge(res) — `genimpact.js:293–303`

Signature: `confBadge(res: GenerationResult): string` Returns the confidence percentage as either a plain string or a `ScoreInfo` chip explaining how it was calculated.

dimGrid(res) — `genimpact.js:306–315`

Signature: `dimGrid(res: GenerationResult): string`

Returns a 6-cell grid showing each dimension's score for the most-affected generation (highest |overall|). Used in the detailed view.

panel(res, opts) — `genimpact.js:319–328`

Signature: `panel(res: GenerationResult, opts?: {title?: string, reasons?: boolean, dims?: boolean}): string` Assembles the complete Generation Impact panel HTML: header (with status badge and/or confidence badge), `bars()`, optional `dimGrid()`, optional confidence line, and the non-partisan disclaimer footer. Returns an HTML string.

window.GenImpact Public API

```

window.GenImpact = {
  GENS,      // ["Gen Z", "Millennials", "Gen X", "Baby Boomers", "Retirees"]
  DIMS,      // [[key, label], ...]
  forSectors, forBill, forCompany, forState, billStatus,
  bars, dimGrid, panel,
}

```

predictions.js

File: `webapp/js/predictions.js` **Public API:** `window.Predictions` **Lines:** 1–323

Purpose

Renders the "Predictive Market Signals" section. Reads precomputed technical-analysis signals (RSI, MACD, moving-average crossovers, QuantLib Black-Scholes metrics), market-wide VIX/index data, congressional trading flow, and policy exposure to produce a bullish-probability score, directional label, forecast cone SVG, confidence rating, and time horizon for each candidate ticker. Registers itself with `window.ScoreInfo` so the progressive-disclosure system can open full breakdowns on click.

window.* Data Consumed

Global	Shape	Purpose
<code>window.TF_DATA</code>	<code>{news[], recent_trades[], market_ticker[]}</code>	News sentiment, congressional trades, VIX/indices
<code>window.PRICES_DATA</code>	<code>{byTicker: {name, change_pct}}</code>	Current price and daily change
<code>window.USA_DATA</code>	<code>{byTicker: {total_contracts}}</code>	USASpending presence (used in candidate ranking)
<code>window.SP500</code>	<code>{byTicker: {sector, name}}</code>	Sector membership for peer momentum
<code>window.QUANT_DATA</code>	<code>{byTicker: {exp_return_1mo, prob_up, prob_down, volatility, ta_score, rsi, rsi_signal, macd_cross, trend, var95, sharpe}}</code>	QuantLib Black-Scholes + TA metrics
<code>window.TFScores</code>	computed by <code>scores.js</code>	<code>politicalExposure()</code> for policy factor
<code>window.ScoreInfo</code>	computed by <code>scoreinfo.js</code>	Explainer registration and badge rendering

Factor Weights

Eight factors summed with weights totalling 1.0:

Factor	Weight	Signal Source
News Sentiment	0.20	Ratio pos/neg headlines from <code>D.news</code>
Social Sentiment	0.15	Congressional buy vs. sell ratio from <code>D.recent_trades</code>
QuantLib Forecast / Historical Pattern	0.15	<code>Q.exp_return_1mo</code> , <code>Q.prob_up/down</code> ; falls back to price momentum
Technical (RSI/MACD) / Technical Momentum	0.15	<code>Q.ta_score</code> ; falls back to <code>chg × 4</code>
VIX / Fear	0.10	<code>VIX</code> + <code>Q.volatility</code> penalty
Sector Strength	0.10	Average sector peer daily change
Macro Environment	0.10	S&P + NASDAQ change minus rate pressure
Policy Impact	0.05	Congressional flow + contracts + bill count

All factors return a 0–100 score where 50 = neutral.

Functions

`factors(tk)` — `predictions.js:59–114`

Signature: `factors(tk: string): Factor[]` Returns an array of 8 factor objects `{key, label, w, v, ...extra}`. Each factor computes its score from the relevant signals as described in the weight table. Uses `Q[tk]` (QuantLib data) when available and falls back to simpler proxies.

- **newsScore:** `clamp(50 + (pos - neg) / news.length × 45, 5, 95)`
- **socialScore:** `clamp(50 + (buys - sells) / (buys + sells) × 40, 8, 92)`
- **patternScore** (with QuantLib): `clamp(50 + Q.exp_return_1mo × 9 + (Q.prob_up - Q.prob_down) × 1.2, 8, 92)`
- **techScore** (with QuantLib): `Q.ta_score` directly (already 0–100)
- **vixScore:** `clamp(100 - (VIX - 12) × 3.5 - ownVolPen, 8, 95)` where `ownVolPen = clamp((Q.volatility - 25) × 0.6, -8, 24)`
- **sectorScore:** `clamp(50 + sectorChg[sec] × 6, 12, 92)`
- **macroScore:** `clamp(50 + (SPCHG + NDCHG) × 6 - YLDCHG × 30, 15, 88)`
- **policyScore:** base 50 + trade flow ± 22 + 6 if contracts + min(bills, 10)

`compute(tk)` — `predictions.js:127–149`

Signature: `compute(tk: string): Prediction` Calls `factors(tk)`, computes the weighted sum score, maps to `dirLabel/dirClass` via `DIR()`. Computes confidence from data depth (news count, trade count, price data, contracts, QuantLib) and factor agreement. Derives time horizon from VIX level. Returns `{ticker, name, score, dirLabel, dirClass, confidence, confScore, horizon, factors, bull}`.

Confidence levels: depth/7 × 50 + agreement/8 × 50 → High ≥ 66, Medium ≥ 45, Low < 45. **Horizon:** VIX ≥ 25 → "1–7 Days", ≥ 16 → "7 Days", < 16 → "30 Days".

`drivers(p)` — `predictions.js:152–183`

Signature: `drivers(p: Prediction): {up: string[], down: string[]}` Converts factor scores to plain-English upside/downside driver lists. Factors ≥ 56 → upside phrase, ≤ 44 → downside phrase. Supplements with specific QuantLib signal names when `Q[tk]` is available (QuantLib positive/negative drift, RSI oversold/overbought, golden/death cross, high volatility). Adds market-level phrases for rising rates and elevated VIX.

`explain(tk)` — `predictions.js:187–248`

Signature: `explain(tk: string): ScoreInfoPayload` Builds the full ScoreInfo explainer payload for a ticker. Calls `compute()` and `drivers()`. Assembles:

- QuantLib-derived input rows (volatility, Black-Scholes prob, VaR, expected return, Sharpe, RSI, MACD, trend)
- Sentiment/macro/policy rows
- Weighted-score calculation table for all 8 factors
- Eight source attributions (QuantLib, TA library, Finnhub, news, STOCK Act, USASpending, Congress.gov, SEC)
- Non-financial-advice disclaimer footer

`cone(p)` — `predictions.js:254–272`

Signature: `cone(p: Prediction): string` Returns an SVG forecast cone. The triangle's slope encodes directional bias (`score - 50` / `50`). The band half-width encodes uncertainty: `Q.volatility / 100 × H × 0.55` when QuantLib is available, else `(1 - confScore/100) ×`

$H \times 0.42$. Colour: green ≥ 55 , red ≤ 45 , gray otherwise. Uses a `linearGradient` that fades from 32% to 6% opacity toward the right.

`card(p)` — predictions.js:277–292

Signature: `card(p: Prediction): string` Returns HTML for one prediction card: ticker + name, directional label (with `ScoreInfo` chip), bullish probability score (with `ScoreInfo` chip), forecast cone SVG, horizon and confidence metadata (with `ScoreInfo` chip), and a "Why?" button (also a `ScoreInfo` chip). All `.si` anchors carry `data-si-kind="prediction"` and `data-si-key=ticker`.

`candidates()` — predictions.js:296–301

Signature: `candidates(): string[]` Selects up to 8 tickers from the union of `newsByTk` and `tradeByTk` keys. Filters to valid uppercase ticker symbols with price data. Ranks by activity score: `news.length + trades.length × 0.5 + (has contracts ? 1 : 0)`.

`render()` — predictions.js:305–317

Signature: `render(): string` Calls `candidates()` and `compute()` for each, wraps cards in a `<section class="pm-wrap">` with a header, grid, and disclaimer. Returns `""` when no candidates exist.

window.Predictions Public API

```
window.Predictions = { compute, explain, render }
```

Registers the "prediction" kind with `window.ScoreInfo` at load time (predictions.js:320): `ScoreInfo.register("prediction", (tk) => explain(tk))`

scores.js

File: `webapp/js/scores.js` **Public API:** `window.TFScores` **Lines:** 1–211

Purpose

The ThinkFree accountability scores engine. Calculates Influence Score, Government Dependency Score, and Corporate Political Exposure from browser-loaded public data with no extra network requests. Also provides sector-level rollup aggregation, a money formatter, and the `formula()` HTML expander that makes the scoring transparent to users. This is the calculation layer only; it never renders UI directly.

window.* Data Consumed

Global	Shape	Purpose
<code>window.TF_DATA</code>	<code>{bills[], correlation.top_bills[], recent_trades[]}</code>	Bills and congressional trade disclosures
<code>window.IW_DATA</code>	<code>{companies: {...}}</code>	Board, executives, owners, LDA registrant id
<code>window.NP_DATA</code>	<code>{byName: {...}}</code>	Nonprofit financials (passed to influencweb)
<code>window.SECBULK_DATA</code>	<code>{byTicker: {revenue}}</code>	Revenue for dependency score denominator
<code>window.USA_DATA</code>	<code>{byTicker: {total_contracts, total_contracts_fmt}}</code>	Federal contract dollars
<code>window.FEC_DATA</code>	<code>{byName: {...}}</code>	Campaign finance (referenced in formula text)
<code>window.RELATIONSHIPS</code>	<code>{byTicker: {firms[], spend_fmt, bills[], filings}}</code>	Senate LDA lobbying signals
<code>window.SEC_DATA</code>	<code>{byTicker: {board[]}}</code>	SEC board count

Index Structures (built once at load)

- `tradeCount[ticker]`: total number of STOCK Act trade disclosures
- `polByTicker[ticker]: Set<string>` of member names who traded the ticker
- `billsByTicker[ticker]: Set<string>` of bill IDs that mention the ticker

Functions

`moneyToNum(fmt)` — scores.js:31–37

Signature: `moneyToNum(fmt: string): number` Parses formatted money strings such as `"$9.8M"` → 9,800,000. Handles B/M/K suffixes. Returns 0 on parse failure.

influenceScore(tk) — scores.js:75–100

Signature: `influenceScore(tk: string): number` **Returns 0–100.** Capped-component sum formula:

Component	Formula	Cap
Federal contracts	$(\log_{10}(\text{contracts}) - 5) \times 8$	38
Lobby spend	$(\log_{10}(\text{lobbySpend}) - 4) \times 4$	16
Lobby footprint	$\text{firms} \times 1.3 + \text{lobbyBills} \times 0.6$	16
Registered	4 if registered else 0	4
Bill mentions	$\text{bills} \times 2$	12
Congressional trades	$\text{trades} \times 0.3 + \text{pols} \times 1.0$	14
Board size	$\text{boardN} \times 0.3$	4

Sum is clamped to [0, 100].

dependencyScore(tk) — scores.js:106–113

Signature: `dependencyScore(tk: string): number` **Returns 0–100.** $\text{contracts} / (5\text{yr_revenue}) \times 140$, clamped at 100. When no revenue data: falls back to $\log_{10}(\text{contracts}) \times 8$. Returns 0 when no contracts exist.

label(v) — scores.js:116–118

Signature: `label(v: number): "High" | "Medium" | "Low" | "Minimal"` Bucket: $\geq 70 \rightarrow \text{High}$, $\geq 40 \rightarrow \text{Medium}$, $\geq 15 \rightarrow \text{Low}$, $< 15 \rightarrow \text{Minimal}$.

politicalExposure(tk) — scores.js:122–138

Signature: `politicalExposure(tk: string): PoliticalExposure` Returns the raw itemised signals: `{contracts, contracts_raw, bills, congressional_traders, trades, lobby_spend, lobby_firms, lobby_bills, lobbying, influence, dependency}`. Used by `scoreinfo.js`, `predictions.js`, and `influenceweb.js`.

industryRollup(tickers) — scores.js:143–167

Signature: `industryRollup(tickers: string[]): RollupResult` Aggregates across a list of tickers: total contracts (summed), unique bill set (union), total trades, average influence score, average dependency score. Returns `{companies, contracts, contracts_fmt, bills, trades, influence, dependency}`.

money(n) — scores.js:170–178

Signature: `money(n: number): string` Compact money formatter: T for trillions, B for billions, M for millions, K for thousands.

formula(kind) — scores.js:197–204

Signature: `formula(kind: "company" | "pol"): string` Returns an HTML `<details>` expander with the plain-English formula for the given score family. `FORMULAS.company` explains Influence Score + Government Dependency. `FORMULAS.pol` explains Influence, Public Impact, and Transparency scores. Ends with a note that all inputs come from public data and scores are transparency estimates, not accusations of wrongdoing.

window.TFScores Public API

```

window.TFScores = {
  influenceScore, dependencyScore, politicalExposure,
  industryRollup, label, money, formula,
  raw: { tradeCount, polsByTicker, billsByTicker },
}
```

scoreinfo.js

File: `webapp/js/scoreinfo.js` **Public API:** `window.ScoreInfo` **Lines:** 1–302

Purpose

Implements the progressive score-explanation system. Any HTML element with class `si` and `data-si-kind` / `data-si-key` (or `data-si-payload`) attributes becomes interactive:

- **Desktop:** hover shows a brief tooltip; click opens the full breakdown modal.
- **Mobile:** tap opens a bottom sheet with a "View Full Breakdown" button that promotes to the modal.

Built-in explainers for three political score kinds (`pol-influence`, `pol-public`, `pol-transparency`) and two company score kinds (`company-influence`, `company-dependency`) are registered at load time. Other modules (e.g. `predictions.js`) register additional kinds via `ScoreInfo.register()`. The modal calls `Glossary.annotate()` to wrap finance jargon in hover-definition spans when available.

window.* Data Consumed

Global	Purpose
<code>window.TF_DATA</code>	<code>politicians[]</code> — politician lookups
<code>window.FEC_DATA</code>	<code>{byName}</code> — funding breakdowns
<code>window.MEMBER_BILLS</code>	<code>{byBioguide}</code> — bill sponsorship counts
<code>window.TFScores</code>	Company score calculations (via <code>companyExp()</code>)
<code>window.PRICES_DATA</code>	Company name lookup for company explainers
<code>window.Glossary</code>	Optional jargon annotation in modal body

DOM Elements (lazily created)

- `tipEl` — `<div class="si-tip">` appended to `<body>`. Desktop hover tooltip.
- `sheetEl` — `<div class="si-sheet-wrap">` with background overlay and inner `<div class="si-sheet">`. Mobile bottom sheet.
- `modalEl` — `<div class="si-modal-wrap">` with `<div class="si-modal">`. Full breakdown modal.

Explainer Object Shape

```
{
  title: string,           // "Political Influence Score"
  value: string,           // "72/100"
  whyLabel?: string,       // Section heading (default "Why this score was assigned")
  bullets: string[],       // 3-4 key points for the tooltip/sheet
  why: string,             // Full plain-English explanation paragraph
  upside?: string[],       // Optional upside drivers list
  downside?: string[],     // Optional downside risks list
  inputs?: {label, value}[], // Key model inputs table
  contributors?: {label, detail}[], // Numbered contributor rows
  calc?: {formula, rows: {factor, value}[], total}, // Collapsible calculation
  sources?: {label, url?}[], // Collapsible sources
  foot?: string,           // Disclaimer footer
  calcLabel?: string,      // "How was this calculated?" override
}
```

Functions

`register(kind, fn)` — `scoreinfo.js:25-26`

Signature: `register(kind: string, fn: (key: string, val?: string) => ExplainerObj): void` Adds `fn` to the `REG` registry under `kind`. When a `.si` element with `data-si-kind=kind` is hovered/clicked, `fn(el.dataset.siKey, el.dataset.siVal)` is called to build the explainer.

`explainEl(el)` — `scoreinfo.js:30-34`

Signature: `explainEl(el: HTMLElement): ExplainerObj | null` Resolves an explainer from an `.si` element. Prefers inline `data-si-payload` JSON; falls back to the registered builder for `data-si-kind`.

`polFunding(name)` — `scoreinfo.js:41-46`

Signature: `polFunding(name: string): FundingBreakdown | null` Reads `FEC[name]` and computes individual/PAC/other percentages. Returns null when no data.

pol(name) — scoreinfo.js:50

Signature: `pol(name: string): PoliticianRecord | undefined` Finds a politician by exact name in `D.politicians`.

polCommon(p) — scoreinfo.js:53–56

Signature: `polCommon(p: PoliticianRecord): {f, pacShare, mb, trades, ret}` Extracts commonly needed fields for all three political score explainers in one call.

Built-in: "pol-influence" explainer — scoreinfo.js:61–96

Registered for `data-si-kind="pol-influence"`. Returns an explainer with:

- Formula: `30 base + trades × 0.3 + PAC share % × 0.6 + |return %| × 0.5 (capped 100)`
- Contributor rows: legislative activity, disclosed trades, outside funding exposure, trading performance
- Sources: Congress.gov (linked with bioguide), OpenFEC (linked), STOCK Act

Built-in: "pol-public" explainer — scoreinfo.js:100–120

Registered for `data-si-kind="pol-public"`. Returns an explainer with:

- Formula: `70 - PAC share % × 0.5 + individual share % × 0.2 (floor 10)`
- "Public impact rises when a member's funding leans toward individual donors rather than PACs."

Built-in: "pol-transparency" explainer — scoreinfo.js:124–143

Registered for `data-si-kind="pol-transparency"`. Returns an explainer with:

- Formula: `100 - PAC share % × 1.4 - trades × 0.15 (floor 5)`
- "It is a measure of exposure, not an accusation."

companyExp(tk, which) — scoreinfo.js:148–186

Signature: `companyExp(tk: string, which: "influence" | "dependency"): ExplainerObj | null` Delegates to `window.TFScores.politicalExposure(tk)` for raw signals. Builds the explainer for either:

- **influence:** aggregates contracts, lobbying spend/footprint, legislation, congressional trading, board size. Formula: the full capped-component sum.
- **dependency:** federal contracts ÷ ~5yr revenue × 140 (capped 100).

Registered as `"company-influence"` and `"company-dependency"` kinds (scoreinfo.js:189–190).

build() — scoreinfo.js:197–205

Signature: `build(): void` Lazily creates the three overlay DOM elements (tooltip, bottom sheet, modal) and attaches their close handlers. The sheet background click closes the sheet. Modal click on wrapper or × button closes the modal. Escape key closes both.

bullets(b) — scoreinfo.js:208

Signature: `bullets(b: string[]): string` Returns `<ul class="si-bullets">` with HTML-escaped `<li>` items.

showTip(exp, x, y) — scoreinfo.js:212–221

Signature: `showTip(exp: ExplainerObj, x: number, y: number): void` Calls `build()`, injects "Why this score?" + bullet list + "Click for full breakdown →" into `tipEl`. Positions it at `(x+16, y+16)` flipping horizontally/vertically when it would overflow the viewport. Adds class `open`.

hideTip() — scoreinfo.js:222

Signature: `hideTip(): void` Removes class `open` from `tipEl`.

openSheet(exp) — scoreinfo.js:226–236

Signature: `openSheet(exp: ExplainerObj): void` Mobile path. Injects the sheet content: drag grip, title, value, "Why this score?" + bullets, "View Full Breakdown" button. The button calls `closeSheet()` then `openModal(exp)`. Adds class `open`.

closeSheet() — scoreinfo.js:237

Signature: `closeSheet(): void` Removes class `open` from `sheetEl`.

openModal(exp) — scoreinfo.js:242–266

Signature: `openModal(exp: ExplainerObj): void` Renders the full breakdown modal. Calls `build()`, `hideTip()`, `closeSheet()`. Calls `window.Glossary.annotate()` on the `why` text and all driver/input strings when Glossary is loaded. Modal body sections (rendered in order):

1. Title + value
2. "Why this score?" section + `why` paragraph
3. Optional upside drivers `<ul>` (green)
4. Optional downside risks `<ul>` (red)
5. Optional key model inputs table
6. Numbered contributor rows
7. Collapsible `<details>` with formula + factor table
8. Collapsible `<details>` with source links
9. Disclaimer footer

closeModal() — scoreinfo.js:267

Signature: `closeModal(): void` Removes class `open` from `modalEl`.

bindOnce() — scoreinfo.js:271–288

Signature: `bindOnce(): void` Attaches delegated document-level listeners once (via `document._siBound` flag):

- Desktop: `mouseover`, `mousemove` (position tracking), `mouseout` → show/hide tooltip.
- All devices: `click` (capture phase) on `.si` → `openSheet()` (touch) or `openModal()` (mouse).

badge(kind, key, value, opts) — scoreinfo.js:294–298

Signature: `badge(kind: string, key: string, value: string|number, opts?: {payload?: object}): string` Returns a `<span class="si">` HTML string with the appropriate `data-si-*` attributes. When `opts.payload` is provided, the explanation is embedded inline as JSON in `data-si-payload`. Otherwise resolution happens lazily via the registered builder. The span includes an `ⓘ` info icon.

window.ScoreInfo Public API

```
window.ScoreInfo = { register, badge, openModal, openSheet, explainEl }
```

glossary.js

File: `webapp/js/glossary.js` **Public API:** `window.Glossary` **Lines:** 1–95

Purpose

Provides jargon-glossary hover tooltips throughout the ThinkFree UI. Detects known finance/quant terms in rendered HTML strings, wraps the first occurrence of each term in a `<span class="term">`, and shows a floating plain-English definition on hover or tap. Designed for non-expert users: definitions are written in everyday language, not formal finance terminology.

Glossary Terms

All 16 terms (glossary.js:13–30):

Term	Plain-English Definition
golden cross	"A sign the stock's price trend has recently turned upward..."
death cross	"A sign the stock's price trend has recently turned downward..."
rsi	"A 0-to-100 speed gauge. Near 70+ the price shot up fast; near 30- it dropped fast..."
oversold	"The price dropped fast and may be due for a bounce back up."
overbought	"The price ran up fast and may be due to cool off."
macd	"A momentum gauge. 'Bullish' means upward push is building..."
moving average	"The average price over recent weeks or months..."

Term	Plain-English Definition
volatility	"How much the price jumps around. Higher means bigger, less predictable swings..."
var	"A rough estimate of the worst loss to expect on a normal day (about 19 out of 20)."
sharpe ratio	"Whether the gains were worth the risk taken. Higher is better; above 1 is good."
kelly	"A math suggestion for how big a position could be... Shown for context only, not advice."
black-scholes	"A well-known finance formula, used here to estimate the chance of a price move."
bullish	"Leaning positive: the signs point toward the price possibly going up."
bearish	"Leaning negative: the signs point toward the price possibly going down."
drift	"The model's best guess at which way the price tends to lean over time."
beta	"How much a stock usually moves compared with the overall market."

Functions

`annotate(s)` — `glossary.js:41–50`

Signature: `annotate(s: string): string` Processes an already-HTML-escaped text string. Sorts all term keys longest-first to avoid partial matches (e.g. "moving average" before "average"). For each term, applies a word-boundary `RegExp` (case-insensitive) and replaces the **first** occurrence with `<span class="term" data-term="{key}">{match}</span>`. Subsequent occurrences are left unwrapped (the `used` map prevents double-wrapping). Returns the annotated string.

Side effect: none. Pure string transformation.

`ensure()` — `glossary.js:57`

Signature: `ensure(): void` Lazily creates the singleton `<div class="term-tip">` tooltip element and appends it to `<body>`.

`show(key, x, y)` — `glossary.js:61–69`

Signature: `show(key: string, x: number, y: number): void` Looks up `TERMS[key.toLowerCase()]`. Sets `tip.textContent` to the definition, adds class `open`, positions the tooltip at `(x+14, y+14)`, flipping sides if it would overflow the viewport.

`hide()` — `glossary.js:72`

Signature: `hide(): void` Removes class `open` from `tip`.

`bind()` — `glossary.js:76–91`

Signature: `bind(): void` Attaches delegated event listeners to `document` once (via `document._termBound` flag):

- `mouseover` → `show()` when cursor enters a `.term` span
- `mousemove` → follow cursor while inside `.term`; `hide()` otherwise
- `mouseout` → `hide()` when leaving a `.term` span (checks `relatedTarget` to avoid hiding on child elements)
- `click` → `show()` briefly for 3800ms then `hide()` (mobile tap support)

window.Glossary Public API

```
window.Glossary = { annotate, TERMS }
```

`annotate()` is called by `scoreinfo.js:openModal()` to annotate the `why` paragraph and driver lists with term spans before injecting them into the modal body.

End of Part 6B — Frontend Reference: Feature Modules

Part 7A — Core Intelligence Pipeline (Phases 3-13)

This document is an exhaustive technical reference for every Python module in the ThinkFree Finance core intelligence pipeline. Each file section documents the module's role, its place in the 13-phase roadmap, and every function it contains — including exact signatures, parameters, return values, step-by-step behavior, file I/O, external libraries used, GPT/LLM prompts, and scoring formulas reproduced verbatim from the source code.

Table of Contents

- 1. [controller.py — Phase 12: Pipeline Orchestration](#)
- 2. [chef_gpt.py — Phase 11: Intelligence Synthesis](#)
- 3. [recession_signals.py — Phase 13: Recession Signals Engine](#)
- 4. [opportunity_score.py — Phase 8.9: Opportunity Scoring Engine](#)
- 5. [quantlib_metrics.py — Phase 8.7: QuantLib Risk Metrics](#)
- 6. [historical_correlation.py — Phase 7: Historical Correlation Engine](#)
- 7. [phase3_extraction.py — Phase 3: NLP Data Extraction](#)
- 8. [phase4_clustering.py — Phase 4: Article Scoring & Clustering](#)
- 9. [portfolio_tracker.py — Phase 10: Portfolio Tracker](#)
- 10. [congress_bills.py — Phase 6.5 Supplement: Congress Bills Intelligence](#)
- 11. [politician_performance.py — Phase 6.5 Supplement: Politician Trading Performance](#)

controller.py — Phase 12: Pipeline Orchestration

File: `controller.py` **Phase:** 12 — Controller / Human-in-the-Loop **Role:** The master entry point for the entire ThinkFree pipeline. Defines the ordered sequence of all pipeline phases as a list of step descriptors, then runs them in sequence (or selectively) via subprocess calls. Tracks pass/fail state per phase in a JSON state file. Runs security and data-quality audit checkpoints between phases. Provides a full CLI with flags for single-phase execution, resume-from-phase, chef-only mode, and audit-only mode.

Key constants:

Name	Value	Purpose
<code>BASE_DIR</code>	<code>Path(__file__).parent</code>	Root of the project
<code>STATE_FILE</code>	<code>BASE_DIR / "pipeline_state.json"</code>	Persisted run state

Pipeline definition (`PIPELINE`): A module-level list of dicts (`controller.py:43`). Each dict has:

Key	Type	Meaning
<code>id</code>	<code>int</code> or <code>str</code>	Phase identifier (e.g. <code>1</code> , <code>"6.5"</code> , <code>"8.7"</code>)
<code>name</code>	<code>str</code>	Human-readable phase name
<code>description</code>	<code>str</code>	One-line description shown during run
<code>script</code>	<code>Path</code>	Absolute path to the phase script
<code>output_check</code>	<code>Path</code>	Path whose existence signals "phase already ran"
<code>skip_if_output_exists</code>	<code>bool</code>	If True and output exists, skip silently
<code>required</code>	<code>bool</code>	If True and the phase fails, the pipeline halts
<code>audit_checkpoint</code>	<code>str</code> or <code>None</code>	Which verification agent to run after this phase

Phases in pipeline order (controller.py:43–203):

Phase ID	Name	Required	Audit Checkpoint
1	User Personalization	False	None
2	News Scraping	True	<code>"data"</code>
3	Data Extraction & Enrichment	True	<code>"data"</code>

Phase ID	Name	Required	Audit Checkpoint
4	Article Clustering	True	None
5	GPT Summarization	True	None
6	Economic Reasoning Engine	True	None
6.5	Political Intelligence	False	None
7	Historical Correlation Engine	False	None
8	Technical Analysis	False	None
8.5	Signal Generation	False	None
8.7	QuantLib Risk Metrics	False	None
8.9	Opportunity Scoring	False	None
9	Backtesting Engine	False	None
13	Recession Signals	False	None
10	Portfolio Tracker	False	None
11	Chef GPT — Intelligence Synthesis	True	None

log(msg: str) -> None

Location: controller.py:211

Parameters:

- **msg (str):** The message to print.

Returns: None

Behavior: Prepends an HH:MM:SS timestamp to msg and prints to stdout with flush=True.

External libs: datetime

output_exists(step: dict) -> bool

Location: controller.py:216

Parameters:

- **step (dict):** A pipeline step descriptor dict from PIPELINE.

Returns: bool — True if the step's output_check path exists on disk.

Behavior:

1. Reads step.get("output_check"). If None, returns False.
2. Returns Path(check).exists().

External libs: pathlib.Path

load_state() -> dict

Location: controller.py:223

Parameters: None

Returns: dict with keys completed (list), failed (list), last_run (str or None).

Behavior:

1. Checks if STATE_FILE (pipeline_state.json) exists.
2. If yes, opens and parses it as JSON.
3. If file missing or parse error, returns {"completed": [], "failed": [], "last_run": None}.

Files read: pipeline_state.json **External libs:** json, pathlib

save_state(state: dict) -> None

Location: controller.py:233

Parameters:

- **state** (dict): Current pipeline state dict.

Returns: None

Behavior:

1. Sets `state["last_run"]` to current UTC ISO timestamp with "Z" suffix.
2. Writes state dict to `pipeline_state.json` as pretty-printed JSON (indent=2).

Files written: pipeline_state.json **External libs:** json, datetime

run_phase(step: dict, state: dict) -> bool

Location: controller.py:239

Parameters:

- **step** (dict): A pipeline step descriptor dict from PIPELINE.
- **state** (dict): Current pipeline state dict (mutated in-place).

Returns: bool — True if the phase succeeded or was skipped, False if it failed.

Behavior (step by step):

1. Extracts `phase_id = str(step["id"])` and `script = Path(step["script"])`.
2. Prints a formatted header banner showing phase id, name, and description.
3. If the script file does not exist on disk, logs [SKIP] Script not found and returns True (non-blocking).
4. If `skip_if_output_exists` is True and `output_exists(step)` is True, logs skip and returns True.
5. Runs the script using `subprocess.run([sys.executable, str(script)], cwd=str(BASE_DIR), capture_output=False)`. Output streams to the terminal directly.
6. Checks `result.returncode == 0` for success.
7. If success: appends `phase_id` to `state["completed"]`, removes from `state["failed"]` if present.
8. If failure: appends `phase_id` to `state["failed"]`.
9. Calls `save_state(state)`.
10. Returns success boolean.

Files written: pipeline_state.json (via save_state) **External libs:** subprocess, sys, pathlib

run_audit_checkpoint(agent: str) -> bool

Location: controller.py:280

Parameters:

- **agent** (str): The audit agent name (e.g. "security", "data").

Returns: bool — True if passed or agent unavailable, False if flagged issues.

Behavior:

1. Checks for `agents/verification_runner.py`. If not found, returns True (non-blocking).
2. Runs `verification_runner.py --phase <agent> --quiet` via `subprocess.run`.
3. If `returncode != 0`, logs a warning and points to `pipeline_audit.json`. Returns False.
4. If `returncode == 0`, logs pass. Returns True.

External libs: subprocess, sys, pathlib

check_env() -> bool

Location: controller.py:299

Parameters: None

Returns: bool — True if all critical API keys are present, False if any are missing.

Behavior:

1. Calls `load_dotenv()` to load `.env` file.
2. Checks for `OPENAI_API_KEY`, `FINNHUB_API_KEY`, `QUIVERQUANT_API_KEY` via `os.getenv`.
3. Prints a warning list for any missing keys.
4. Returns `False` if any warnings; `True` if all present.

External libs: `python-dotenv`, `os`

`parse_args()` -> `argparse.Namespace`

Location: `controller.py:325`

Parameters: None

Returns: `argparse.Namespace` with attributes:

- `phase` (`str` | `None`): Run a single specific phase by ID.
- `from_phase` (`str` | `None`): Resume from this phase ID onward.
- `chef_only` (`bool`): Run only Chef GPT.
- `skip_scraping` (`bool`): Skip Phase 2.
- `list` (`bool`): List all phases and exit.
- `audit` (`bool`): Run verification audit only and exit.
- `skip_audit` (`bool`): Skip security/data quality checkpoints.

External libs: `argparse`

`list_phases()` -> `None`

Location: `controller.py:348`

Parameters: None

Returns: `None`

Behavior: Iterates over `PIPELINE`, printing each phase's id and name. If the phase's `output_check` path exists, prints a checkmark; otherwise blank.

`main()` -> `None`

Location: `controller.py:358`

Parameters: None (reads `sys.argv` via `parse_args`)

Returns: `None`

Behavior (full run flow):

1. Calls `parse_args()` and `list_phases()` if `--list`.
2. Prints startup banner.
3. Calls `check_env()` (non-blocking warning only).
4. Calls `load_state()`.
5. If `--audit`: imports and calls `run_all_agents(verbose=True)` from `agents.verification_runner`; exits with code 0 (PASS) or 1 (FAIL).
6. If `--chef-only`: finds Phase 11 step, runs it, exits.
7. If `--phase`: finds the matching step, runs it, exits.
8. Determines `steps_to_run`: full `PIPELINE` or sliced from `--from-phase` index.
9. Runs security audit checkpoint (unless `--skip-audit`).
10. Iterates `steps_to_run`:
 - If `--skip-scraping` and phase is 2, skips.
 - Calls `run_phase(step, state)`.
 - If success and `audit_checkpoint` is set (and not `--skip-audit`), calls `run_audit_checkpoint(checkpoint)`.
 - If phase failed and `required=True`, logs error, sets `failed_required=True`, breaks loop.
11. Prints final summary: either "completed with errors" or "Pipeline complete" with the path to `intelligence_report.json` and the Streamlit launch command.

Files written: `pipeline_state.json` (via `run_phase` → `save_state`)

chef_gpt.py — Phase 11: Intelligence Synthesis

File: `chef_gpt.py` **Phase:** 11 — Chef GPT Final Intelligence Layer **Role:** The final synthesis layer. Reads every prior phase's output JSON, assembles them into a single structured prompt, sends it to GPT-4o (or configured model), and writes the plain-English intelligence report. Implements the eight-question framework as the core analytical structure. Output is the final deliverable: `intelligence_report.json`.

Key constants:

Name	Value
<code>BASE_DIR</code>	<code>Path(__file__).parent</code>
<code>OUTPUT_PATH</code>	<code>BASE_DIR / "intelligence_report.json"</code>
<code>OPENAI_MODEL</code>	<code>os.getenv("THINKFREE_MODEL", "gpt-4o")</code>

LLM used: OpenAI `gpt-4o` (or override via `THINKFREE_MODEL` env var) **LLM temperature:** `0.3` **LLM response format:** `{"type": "json_object"}`

load_json(path: Path, default: Any = None) -> Any

Location: `chef_gpt.py:44`

Parameters:

- `path` (`Path`): Filesystem path to a JSON file.
- `default` (`Any`): Value returned if file is missing or malformed. Default: `None`.

Returns: Parsed JSON value, or `default`.

Behavior: Opens `path` in UTF-8 mode, calls `json.load`. On `FileNotFoundError` or `json.JSONDecodeError`, silently returns `default`.

External libs: `json`, `pathlib`

assemble_inputs() -> dict

Location: `chef_gpt.py:56`

Parameters: None

Returns: `dict` with keys mapping data source names to their loaded contents (or empty defaults).

Behavior: Calls `load_json` for each of the following files:

Key	File path	Default
<code>user_profile</code>	<code>user_profile.json</code>	<code>{}</code>
<code>economic_reasoning</code>	<code>news_output/economic_reasoning_summary.json</code>	<code>{}</code>
<code>sector_summaries</code>	<code>news_output/sector_summaries.json</code>	<code>{}</code>
<code>technical_signals</code>	<code>Module_2_Technical_Analysis/signal_output_phase3.json</code>	<code>[]</code>
<code>backtest_metrics</code>	<code>Module_2_Technical_Analysis/results_run/summary_metrics.json</code>	<code>{}</code>
<code>recession_data</code>	<code>recession_signals_output.json</code>	<code>{}</code>
<code>political_trades</code>	<code>GPT_Economy/Intelligence_layer (IN PROGRESS)/intelligence_output.json</code>	<code>{}</code>
<code>historical_parallel</code> s	<code>historical_parallel</code> s.json	<code>{}</code>
<code>opportunity_scores</code>	<code>opportunity_scores.json</code>	<code>{}</code>
<code>quantlib_metrics</code>	<code>quantlib_metrics.json</code>	<code>{}</code>

Returns the assembled dict directly. Missing files silently become empty defaults — the pipeline tolerates partial data.

build_prompt(inputs: dict) -> str

Location: `chef_gpt.py:106`

Parameters:

- `inputs` (`dict`): The dict returned by `assemble_inputs()`.

Returns: `str` — the complete GPT prompt (typically 2,000–5,000 characters).

Behavior (section by section):

Profile section (`chef_gpt.py:107–128`): Extracts `age`, `risk_tolerance`, `investment_goal`, `timeline`, `emotional_response` from `inputs["user_profile"]` with string defaults. Builds:

```
"The reader is {age} years old with a {risk} risk tolerance.
Their investment goal is {goal} over a {timeline} horizon.
When markets drop, they tend to {emotional}."
```

Economic summary section (`chef_gpt.py:131–137`): Reads `economic_reasoning.summary`, `bullish_sectors`, `bearish_sectors`. Builds:

```
"{econ_summary}\n\nBullish sectors: {...}\nBearish sectors: {...}"
```

Sector text section (`chef_gpt.py:139–145`): Iterates over up to 5 entries in `sector_summaries`. For each, takes the `sector_summary` field truncated at 400 characters.

Technical signals section (`chef_gpt.py:147–156`): Filters signals for `final_signal == "BUY"` (top 3) and `"SELL"` (top 3). Formats as:

```
"{ticker}: BUY signal (confidence: {confidence_score})"
"{ticker}: SELL signal (confidence: {confidence_score})"
```

Fallback: `"No strong buy or sell signals detected today."`

Backtest section (`chef_gpt.py:158–166`): If `backtest_metrics` has data:

```
"Historical backtest on current signals: CAGR {CAGR %}%, Sharpe ratio {Sharpe}, Max drawdown {Max Drawdown %}%"
```

Recession section (`chef_gpt.py:168–182`): Reads `recession_risk.score`, `recession_risk.label`, `recession_risk.plain_english_summary`, `real_world_impact.{mortgages,credit_cards,jobs,savings}`. Formats a multi-line block.

Political intelligence section (`chef_gpt.py:184–208`): If `political_trades` dict has `congressional_trade_links` or `housing_trades`:

- Iterates up to 5 `trade_links` items: `"{ticker}: {events[0][:200]}"`
- Iterates up to 3 `housing_trades`: `"{rep} – {tx} {ticker}"`
- Appends mandatory disclaimer.

Historical parallels section (`chef_gpt.py:210–244`): If `historical_parallel` is non-empty, iterates up to 3 parallels, each showing event type, period name/dates, narrative (400 chars), winners and losers with return percentages.

Opportunity scores section (`chef_gpt.py:246–260`): Reads `sector_opportunities` (top 5) and `ticker_opportunities` (BUY signals, top 3). Formats:

```
"Top scoring sectors: {sector} ({score}/10 {emoji}), ..."
"Top buy-signal tickers: {ticker} ({score}/10), ..."
"Recession discount applied: {recession_risk_used}/10"
```

QuantLib section (`chef_gpt.py:308–326`): If `quantlib_metrics.ticker_metrics` has data, takes top 5 and formats:

```
"– {ticker} ({signal}): P(+5% in 1mo)={prob:.0f}% Vol={vol:.0f}%/yr Daily VaR(95%)={var95:.2f}% Kelly={kelly:.1f}% of portfolio"
```

Output schema (`chef_gpt.py:263–305`): A JSON template (serialized as string) defining the exact structure GPT must return:

```
{
  "generated_at": "<ISO timestamp>",
  "profile_match": "<one sentence>",
  "headline": "<one compelling sentence>",
  "market_overview": {
    "what_happened": "<2–3 sentences, plain English>",
    "why_it_happened": "<2–3 sentences>",
    "who_benefits": "<list>",
    "who_is_hurt": "<list>"
  },
  "consumer_impact": {
    "summary": "<2–3 sentences>",
    "rent_and_housing": "<one sentence>",
    "groceries_and_gas": "<one sentence>"
  }
}
```

```

    "credit_cards_and_loans": "<one sentence>",
    "jobs_and_income": "<one sentence>"
  },
  "sector_outlook": {
    "bullish": ["<sector>"],
    "bearish": ["<sector>"],
    "why": "<one paragraph>"
  },
  "political_intelligence": {
    "summary": "<2-3 sentences>",
    "notable_events": ["<event 1>", "<event 2>"],
    "disclaimer": "This analysis identifies timing relationships..."
  },
  "market_signals": {
    "summary": "<plain English>",
    "top_opportunities": ["<ticker: plain English>"],
    "caution_flags": ["<ticker: plain English>"]
  },
  "historical_context": "<2-3 sentences>",
  "recession_risk": {
    "score": "<0-10>",
    "label": "<Low/Moderate/Elevated/High/Very High>",
    "plain_english": "<one paragraph>",
    "what_to_watch": "<one sentence>"
  },
  "what_it_means_for_you": "<2-3 sentences tailored to user>",
  "bottom_line": "<one sentence>",
  "disclaimer": "ThinkFree provides financial intelligence..."
}

```

GPT system prompt (`chef_gpt.py:428-436`):

"You are ThinkFree's AI financial analyst. You translate complex financial events into plain English for everyday people. You always write at a level that someone with no financial background can understand. You never use jargon without immediately explaining it. You are informative, objective, and always honest about uncertainty."

GPT user prompt instruction section (`chef_gpt.py:327-378`): Includes the eight-question framework explicitly:

For every major event, apply the 8-question framework:

1. What happened? (plain facts)
2. Why did it happen? (economic cause, explained simply)
3. Who benefits? (specific groups, companies, sectors)
4. Who is negatively affected? (specific groups who may be hurt)
5. How does this affect consumers? (rent, groceries, gas, credit cards, loans, jobs)
6. How does this affect businesses? (which types win or lose)
7. How does this affect markets? (sectors, stocks, bonds – plain English)
8. What happened historically? (use the historical parallels data above – cite actual periods and measured returns)

`run_chef_gpt()` -> dict

Location: `chef_gpt.py:387`

Parameters: None

Returns: dict — the parsed intelligence report.

Behavior (step by step):

1. Reads `OPENAI_API_KEY` from environment; raises `RuntimeError` if missing.
2. Calls `assemble_inputs()` to load all phase outputs.
3. Checks which data sources are non-empty and prints a summary list.
4. Calls `build_prompt(inputs)` to construct the full prompt.
5. Instantiates `OpenAI(api_key=api_key)` client.
6. Calls `client.chat.completions.create(model=OPENAI_MODEL, messages=[system_msg, user_msg], temperature=0.3, response_format={"type": "json_object"})`.
7. Extracts `response.choices[0].message.content`.
8. Parses JSON; on `json.JSONDecodeError` falls back to `{"raw_response": raw, "parse_error": "..."}.`
9. Stamps `generated_at` (UTC ISO), `data_sources_used`, and `user_profile` onto the report.
10. Writes to `intelligence_report.json` (UTF-8, indent=2).

- 11. Prints the headline and bottom_line fields to stdout.
- 12. Returns the report dict.

Files read: All 10 files listed in `assemble_inputs` . **Files written:** `intelligence_report.json` **External libs:** `openai` , `python-dotenv` , `json`

recession_signals.py — Phase 13: Recession Signals Engine

File: `recession_signals.py` **Phase:** 13 — Recession Indicator Engine **Role:** Downloads live macroeconomic data from Yahoo Finance (yield curve via `^TNX` / `^IRX` , VIX via `^VIX`) and FRED (unemployment rate `UNRATE` , GDP growth `A191RL1Q225SBEA`). Computes a graduated 0–10 recession risk score from these indicators. Generates plain-English explanations of each factor and translates the score into personal-finance impact guidance. Output is `recession_signals_output.json` , consumed by both `opportunity_score.py` (for the recession discount) and `chef_gpt.py` (for the briefing).

Key constants:

Name	Value
<code>BASE_DIR</code>	<code>Path(__file__).parent</code>
<code>OUTPUT_PATH</code>	<code>BASE_DIR / "recession_signals_output.json"</code>

`fetch_yield_curve()` -> dict

Location: `recession_signals.py:38`

Parameters: None

Returns: dict with keys `ten_year` (float|None), `three_month` (float|None), `spread` (float|None), `inverted` (bool).

Behavior:

- 1. Downloads `["^TNX", "^IRX"]` via `yf.download(period="3mo", interval="1d")` .
- 2. Extracts the most recent non-null Close value for each ticker.
- 3. `spread = ten_yr - three_mo` (10-year minus 3-month Treasury yield).
- 4. `inverted = spread < 0` .
- 5. Returns rounded values; on any exception returns all `None` with `inverted=False` .

External libs: `yfinance` , `pandas`

`fetch_vix()` -> dict

Location: `recession_signals.py:67`

Parameters: None

Returns: dict with keys `vix` (float|None), `stress_level` (str).

Behavior:

- 1. Downloads `"^VIX"` via `yf.download(period="5d", interval="1d")` .
- 2. Extracts most recent Close value.
- 3. Maps VIX level to stress label:

VIX range	Label
< 15	"Low – markets are calm"
15–19	"Normal – typical market conditions"
20–29	"Elevated – investors are nervous"
30–39	"High – significant market stress"
≥ 40	"Extreme – market panic conditions"

External libs: `yfinance` , `pandas`

```
_fetch_fred_csv(series_id: str) -> pd.Series | None
```

Location: `recession_signals.py:96`

Parameters:

- `series_id` (str): FRED series identifier (e.g. "UNRATE", "A191RL1Q225SBEA").

Returns: `pd.Series` of numeric values with date index, or `None` on failure.

Behavior:

1. Constructs URL: `https://fred.stlouisfed.org/graph/fredgraph.csv?id={series_id}`.
2. GETs with 10-second timeout.
3. Parses CSV with two columns: `["date", "value"]`.
4. Coerces `value` to numeric, drops NaN.
5. Returns the `value` Series.

External libs: `requests`, `pandas`, `io`

```
fetch_fred_indicators() -> dict
```

Location: `recession_signals.py:115`

Parameters: None

Returns: `dict` with keys:

- `available` (bool)
- `unemployment_rate` (float|None)
- `unemployment_rising` (bool)
- `gdp_growth` (float|None)
- `gdp_negative` (bool)

Behavior:

1. First attempts public FRED CSV endpoint via `_fetch_fred_csv`:
 - "UNRATE" → unemployment rate (monthly %)
 - "A191RL1Q225SBEA" → real GDP growth rate (quarterly %)
2. Computes `unemployment_rising = unemployment > unemp_prev` (last two observations).
3. Computes `gdp_negative = gdp_growth < 0`.
4. If public CSV fails, checks `FRED_API_KEY` env var; if present, uses `fredapi.Fred` library for the same series.
5. If both fail, returns `{"available": False, ...}`.

External libs: `requests`, `pandas`, `fredapi` (optional), `python-dotenv`

```
compute_recession_score(yield_curve: dict, vix_data: dict, fred: dict) -> dict
```

Location: `recession_signals.py:170`

Parameters:

- `yield_curve` (dict): Output of `fetch_yield_curve()`.
- `vix_data` (dict): Output of `fetch_vix()`.
- `fred` (dict): Output of `fetch_fred_indicators()`.

Returns: `dict` with keys `score` (float 0–10), `max_score` (10), `label` (str), `plain_english_summary` (str), `factors` (list[str]).

Scoring formula (reproduced from `recession_signals.py:170–318`):

The function accumulates `score` and `max_possible` as each indicator is evaluated. All partial scores use a `clamp(x, lo=0.0, hi=1.0)` helper.

Component 1 — Yield Curve (up to 4 points, `recession_signals.py:186–218`):

```
max_possible += 4
```

- **Inverted** (`spread < 0`): `curve_score = min(4.0, 4.0 × |spread| / 1.5)`
- **Not inverted:** `curve_risk = clamp((1.3 - spread) / 1.3)` — risk decays from 1.0 at `spread=0` to 0.0 at `spread≥1.3%`; `curve_score = 4.0 × curve_risk`

Plain-English thresholds for non-inverted curve:

- `spread < 0.5%`: "very flat"
- `spread < 1.0%`: "flattening"
- `spread >= 1.0%`: "normal"

Component 2 — VIX (up to 3 points, `recession_signals.py:221–234`):

```
max_possible += 3
```

```
vix_risk = clamp((vix - 14) / (40 - 14)) vix_contribution = 3.0 × vix_risk
```

Zero risk at $VIX \leq 14$; full 3 points at $VIX \geq 40$.

Component 3 — FRED indicators (up to 3 points, `recession_signals.py:237–287`):

```
max_possible += 3 (only if fred["available"])
```

GDP sub-component (up to 1.5 points):

- `gdp_negative`: +1.5
- Otherwise: `gdp_risk = clamp((2.5 - gdp) / 2.5)`, `contribution = 1.5 × gdp_risk`

Unemployment sub-component (up to 1.5 points):

- `unemp_rising` and `unemp > 5.0%`: +1.5
- `unemp_rising` and `unemp ≤ 5.0%`: +0.75
- Stable: `level_risk = clamp((unemp - 4.5) / 2.0)`, `contribution = 0.75 × level_risk`

Normalization (`recession_signals.py:289–294`):

```
normalized = round((score / max_possible) × 10, 1)
```

Label mapping (`recession_signals.py:296–310`):

Score	Label	Plain English
0–2	Low	"The economy shows no significant recession warning signs right now."
2.1–4	Moderate	"There are some warning signs but no imminent recession signal."
4.1–6	Elevated	"Multiple warning signs are flashing."
6.1–8	High	"Strong recession indicators present."
8.1–10	Very High	"Almost all recession warning signals are active."

```
explain_real_world_impact(score: float, yield_inverted: bool, vix: float | None) -> dict
```

Location: `recession_signals.py:325`

Parameters:

- `score` (float): The normalized recession risk score (0–10).
- `yield_inverted` (bool): Whether the yield curve is inverted.
- `vix` (float | None): Current VIX level.

Returns: dict with keys `mortgages`, `credit_cards`, `jobs`, `groceries_gas`, `savings` — each a plain-English string.

Behavior: Returns one of three pre-written impact dictionaries based on score thresholds:

- `score ≤ 3`: Low-risk messaging (stable, build emergency fund)
- `score ≤ 6`: Elevated-risk messaging (consider locking in fixed rate, pay down debt, 3-6 month savings)
- `score > 6`: High-risk messaging (recession-level warnings on credit, layoffs, cash reserves)

```
run_recession_signals() -> dict
```

Location: `recession_signals.py:387`

Parameters: None

Returns: dict — the full output structure written to disk.

Behavior:

1. Calls `fetch_yield_curve()`, `fetch_vix()`, `fetch_fred_indicators()` in sequence.
2. Calls `compute_recession_score(yield_curve, vix_data, fred)`.

3. Calls `explain_real_world_impact(score, inverted, vix)`.

4. Assembles output dict:

```
{
  "generated_at": "<UTC ISO>",
  "recession_risk": { score, max_score, label, plain_english_summary, factors },
  "raw_indicators": { yield_curve, vix, fred },
  "real_world_impact": { mortgages, credit_cards, jobs, groceries_gas, savings }
}
```

1. Writes to `recession_signals_output.json` (indent=2).

2. Prints score and label to stdout.

Files written: `recession_signals_output.json` **External libs:** `yfinance`, `pandas`, `numpy`, `requests`, `fredapi` (optional), `python-dotenv`

opportunity_score.py — Phase 8.9: Opportunity Scoring Engine

File: `opportunity_score.py` **Phase:** 8.9 — Opportunity Scoring (labeled Phase 8 in the roadmap) **Role:** Combines sector economic outlook, technical signals, and recession risk into a ranked 0–10 opportunity score for every sector and for individual tickers with signals. Applies user-profile multipliers. Output is consumed by `chef_gpt.py` and `portfolio_tracker.py`.

Key constants:

Name	Value
<code>BASE_DIR</code>	<code>Path(__file__).parent</code>
<code>OUTPUT_PATH</code>	<code>BASE_DIR / "opportunity_scores.json"</code>

`HIGH_GROWTH_SECTORS`: {"Information Technology", "Consumer Discretionary", "Communication Services"} `DEFENSIVE_SECTORS`: {"Health Care", "Utilities", "Consumer Staples"} `INCOME_SECTORS`: {"Real Estate", "Utilities", "Financials"}

`load_json(path: Path, default: Any = None) -> Any`

Location: `opportunity_score.py:33`

Identical pattern to `chef_gpt.load_json`. Opens path, parses JSON, returns `default` on failure.

`score_sector_outlook(sector: str, bullish_sectors: list, bearish_sectors: list) -> float`

Location: `opportunity_score.py:41`

Parameters:

- `sector` (`str`): Sector name (GICS standard).
- `bullish_sectors` (`list`): Bullish sector list from economic reasoning.
- `bearish_sectors` (`list`): Bearish sector list from economic reasoning.

Returns: `float` in `{0.0, 0.5, 1.0}`.

Formula:

```
if sector in bullish_sectors -> 1.0
if sector in bearish_sectors -> 0.0
else                          -> 0.5 (neutral)
```

`score_technical_signal(ticker: str, signals: list[dict]) -> float`

Location: `opportunity_score.py:50`

Parameters:

- `ticker` (`str`): Ticker symbol.
- `signals` (`list[dict]`): List of technical signal dicts from Phase 8.5.

Returns: `float` in `[0.0, 1.0]`.

Formula:

```
BUY: min(1.0, 0.5 + confidence × 0.1)
SELL: max(0.0, 0.5 - confidence × 0.1)
HOLD: 0.5
No signal found: 0.5
```

```
confidence = abs(sig.get("confidence_score", 0))
```

recession_discount(recession_score: float) -> float

Location: opportunity_score.py:64

Parameters:

- recession_score (float): Score from recession_signals_output.json (0–10).

Returns: float multiplier applied to opportunity scores.

Formula (step function):

Recession Score	Multiplier
≤ 2	1.00 (no discount)
> 2 and ≤ 4	0.90
> 4 and ≤ 6	0.75
> 6 and ≤ 8	0.60
> 8	0.45

profile_multiplier(sector: str, profile: dict) -> float

Location: opportunity_score.py:77

Parameters:

- sector (str): Sector name.
- profile (dict): User profile dict (keys: risk_tolerance, investment_goal).

Returns: float multiplier, capped at 1.5.

Formula:

```
multiplier = 1.0
if risk == "high" and sector in HIGH_GROWTH_SECTORS: multiplier = 1.2
elif risk == "low" and sector in DEFENSIVE_SECTORS: multiplier = 1.2
elif risk == "moderate": multiplier = 1.0 (unchanged)

if goal == "income" and sector in INCOME_SECTORS: multiplier *= 1.15

return min(1.5, multiplier)
```

build_sector_scores(bullish_sectors, bearish_sectors, signals, recession_risk_score, profile, sector_summaries) -> list[dict]

Location: opportunity_score.py:101

Parameters:

- bullish_sectors (list): From economic reasoning.
- bearish_sectors (list): From economic reasoning.
- signals (list[dict]): Technical signals.
- recession_risk_score (float): From recession signals.
- profile (dict): User profile.
- sector_summaries (dict): Sector summaries from Phase 5.

Returns: list[dict] sorted by opportunity_score descending.

Behavior:

1. Builds `all_sectors` as the union of bullish + bearish + sector_summaries keys. If empty, uses the 11 standard GICS sectors as defaults.
2. For each sector:

Full sector scoring formula (`opportunity_score.py:122–130`):

```
outlook = score_sector_outlook(sector, bullish_sectors, bearish_sectors) # 0/0.5/1
rec_adj = outlook × recession_discount(recession_risk_score)
pm       = profile_multiplier(sector, profile)
raw_score = rec_adj × pm
final    = min(10.0, raw_score × 10)
```

Direction labels:

- `final >= 7.0` : Bullish
- `final >= 5.0` : Neutral
- `final < 5.0` : Bearish

1. Each output record includes:

```
{
  "sector": "...",
  "opportunity_score": 0.0–10.0,
  "direction": "Bullish/Neutral/Bearish",
  "direction_emoji": "📈/📊/📉",
  "components": {
    "economic_outlook": ...,
    "recession_discount": ...,
    "profile_match": ...
  },
  "rationale": "..."
}
```

1. Sorts by `opportunity_score` descending.

`build_ticker_scores(signals, bullish_sectors, bearish_sectors, recession_risk_score, profile, sector_summaries) -> list[dict]`

Location: `opportunity_score.py:177`

Parameters: Same as `build_sector_scores` .

Returns: `list[dict]` sorted by `opportunity_score` descending, capped at top 25.

Ticker scoring formula (`opportunity_score.py:198–209`):

```
tech_score = score_technical_signal(ticker, signals) # 0.0–1.0
base       = tech_score × recession_discount(recession_risk_score)

if risk == "low" and final_sig == "BUY": base *= 0.85
if risk == "high" and final_sig == "SELL": base *= 0.85

final = min(10.0, base × 10)
```

Direction labels:

- `final >= 7.0` : Buy Signal
- `final >= 5.0` : Neutral / Hold
- `final < 5.0` : Sell Signal

Each record includes a `plain_english_signal` from `_signal_to_plain_english` .

`_signal_to_plain_english(ticker: str, signal: str, confidence: int, reasoning: str) -> str`

Location: `opportunity_score.py:235`

Parameters:

- `ticker` (`str`)
- `signal` (`str`): "BUY", "SELL", or "HOLD" .

- `confidence` (`int`): Confidence score (1–5).
- `reasoning` (`str`): Signal reasoning text.

Returns: `str` — plain-English explanation of the signal. Three pre-written templates, one per signal type. Includes the disclaimer "This is a signal to watch, not a guarantee."

`run_opportunity_scoring()` -> `dict`

Location: `opportunity_score.py:255`

Parameters: None

Returns: `dict` — the full output written to disk.

Behavior:

1. Loads: `economic_reasoning_summary.json`, `signal_output_phase3.json`, `recession_signals_output.json`, `user_profile.json`, `sector_summaries.json`.
2. Extracts `bullish_sectors`, `bearish_sectors` from economic reasoning; `recession_score` from recession risk dict (default 3.0 if not found).
3. Calls `build_sector_scores(...)` and `build_ticker_scores(...)`.
4. Assembles output:

```
{
  "generated_at": "...",
  "recession_risk_used": ...,
  "user_profile_id": "...",
  "sector_opportunities": [...],
  "ticker_opportunities": [...],
  "methodology": "..."
}
```

1. Writes to `opportunity_scores.json`.

Files read: `economic_reasoning_summary.json`, `signal_output_phase3.json`, `recession_signals_output.json`, `user_profile.json`, `sector_summaries.json` **Files written:** `opportunity_scores.json` **External libs:** `json`, `python-dotenv`, `pathlib`

quantlib_metrics.py — Phase 8.7: QuantLib Risk Metrics

File: `quantlib_metrics.py` **Phase:** 8.7 (a sub-phase within Phase 8 — Opportunity Quantification) **Role:** Computes rigorous risk-adjusted metrics for each ticker with a BUY or SELL signal. Uses Black-Scholes for probability of profit, parametric VaR at 95%, continuous Kelly criterion for position sizing, and annualized historical volatility. Falls back to pure numpy if QuantLib is unavailable. Output feeds into `chef_gpt.py`'s QuantLib section.

Key constants:

Name	Value	Meaning
<code>RISK_FREE_RATE</code>	<code>0.05</code>	5% annual risk-free rate
<code>LOOKBACK_DAYS</code>	<code>252</code>	1 trading year of history
<code>TARGET_HORIZON</code>	<code>21</code>	1 calendar month (~21 trading days)
<code>OUTPUT_PATH</code>	<code>BASE_DIR / "quantlib_metrics.json"</code>	

Optional imports:

- `QuantLib` (`_QL_AVAILABLE`): Used for Black-Scholes engine; falls back to numpy if unavailable.
- `yfinance` (`_YF_AVAILABLE`): Used for price history fetch.

`compute_annualized_vol(prices: list[float]) -> float`

Location: `quantlib_metrics.py:49`

Parameters:

- `prices` (`list[float]`): Ordered list of daily close prices.

Returns: `float` — annualized volatility as a decimal (e.g. 0.30 = 30%).

Formula:

```
log_returns = diff(log(prices))      # daily log returns
σ_daily      = std(log_returns, ddof=1)
σ_annual     = σ_daily × sqrt(252)
```

Default: `0.30` if fewer than 10 prices or fewer than 5 valid log returns.

External libs: `numpy`

`bs_prob_above_target(spot, target, vol, risk_free=0.05, time_years=21/252) -> float`

Location: `quantlib_metrics.py:63`

Parameters:

- `spot` (`float`): Current price.
- `target` (`float`): Target price (e.g. `spot × 1.05` for 5% upside).
- `vol` (`float`): Annualized volatility (decimal).
- `risk_free` (`float`): Risk-free rate. Default: `RISK_FREE_RATE` (0.05).
- `time_years` (`float`): Horizon in years. Default: `21/252 ≈ 0.0833`.

Returns: `float` in `[0, 1]` — the risk-neutral probability that `S_T > target`.

Formula (Black-Scholes N(d2)):

```
d1 = (ln(spot / target) + (risk_free + 0.5 × vol2) × T) / (vol × √T)
d2 = d1 - vol × √T
P(S_T > target) = N(d2)      # standard normal CDF
```

Implementation: If QuantLib is available, sets up a full `BlackScholesMertonProcess` with flat vol and rate term structures, prices a European call, then computes $N(d2)$ analytically via `scipy.special.ndtr`. Falls back to pure numpy/math `_norm_cdf(d2)`.

Edge case: Returns `0.5` if any input is ≤ 0 .

External libs: `QuantLib` (optional), `scipy.special.ndtr`, `math`

`_norm_cdf(x: float) -> float`

Location: `quantlib_metrics.py:125`

Parameters:

- `x` (`float`): Input value.

Returns: `float` — standard normal CDF at `x`.

Formula: $0.5 \times (1 + \text{erf}(x / \sqrt{2}))$

External libs: `math.erf`

`compute_var_95(prices: list[float]) -> float`

Location: `quantlib_metrics.py:131`

Parameters:

- `prices` (`list[float]`): Daily close prices.

Returns: `float` — daily parametric VaR at 95% confidence as a decimal fraction (e.g. $0.025 = 2.5\%$ of position).

Formula:

```
σ_annual = compute_annualized_vol(prices)
σ_daily  = σ_annual / √252
VaR_95   = 1.645 × σ_daily
```

The 1.645 factor is the 95th percentile of the standard normal distribution.

External libs: `math`

```
expected_return_lognormal(spot, vol, risk_free=0.05, horizon_years=21/252) -> float
```

Location: `quantlib_metrics.py:141`

Parameters:

- `spot` (`float`): Current price.
- `vol` (`float`): Annualized volatility (decimal).
- `risk_free` (`float`): Risk-free drift. Default: 0.05.
- `horizon_years` (`float`): Time horizon. Default: `21/252`.

Returns: `float` — expected price under lognormal model.

Formula (risk-neutral drift):

$$E[S_T] = \text{spot} \times \exp((\text{risk_free} - 0.5 \times \text{vol}^2) \times T)$$

Returns `spot` unchanged if inputs are invalid (≤ 0).

External libs: `math.exp`

```
kelly_fraction(prices: list[float], risk_free: float = 0.05) -> float
```

Location: `quantlib_metrics.py:155`

Parameters:

- `prices` (`list[float]`): Daily close prices.
- `risk_free` (`float`): Annual risk-free rate. Default: 0.05.

Returns: `float` — fraction of portfolio to allocate, in `[0.0, 0.25]` (0–25%).

Formula (continuous Kelly):

```
log_returns = diff(log(prices))
mu_annual   = mean(log_returns) * 252
sigma_annual = std(log_returns, ddof=1) * sqrt(252)
f*          = (mu_annual - risk_free) / sigma_annual**2
f_capped    = max(0.0, min(0.25, f*)) # quarter-Kelly cap for safety
```

Returns `0.0` if fewer than 10 prices or `sigma_annual <= 0`.

External libs: `numpy`, `math`

```
fetch_recent_prices(ticker: str, days: int = 252) -> list[float]
```

Location: `quantlib_metrics.py:176`

Parameters:

- `ticker` (`str`): Ticker symbol.
- `days` (`int`): Maximum number of trading days to return. Default: `LOOKBACK_DAYS` (252).

Returns: `list[float]` — most recent `days` close prices, or `[]` if yfinance unavailable or error.

Behavior: Calls `yf.download(ticker, period="1y", interval="1d", auto_adjust=True)`. Squeezes multi-level columns. Returns last `days` values from the Close column.

External libs: `yfinance`

```
compute_ticker_metrics(ticker: str, signal: dict) -> dict
```

Location: `quantlib_metrics.py:194`

Parameters:

- `ticker` (`str`): Ticker symbol.
- `signal` (`dict`): Signal dict from Phase 8.5 (must have `final_signal` key).

Returns: `dict` — complete metrics record for this ticker.

Behavior:

1. Calls `fetch_recent_prices(ticker)`.

2. If ≥ 10 prices:

- `spot = prices[-1]`
- `vol = compute_annualized_vol(prices)`
- `var_95 = compute_var_95(prices)`
- `target_bull = spot × 1.05` (5% upside target)
- `target_bear = spot × 0.95` (5% downside target)
- `prob_up = bs_prob_above_target(spot, target_bull, vol)`
- `prob_down = 1 - bs_prob_above_target(spot, target_bear, vol)`
- `exp_price = expected_return_lognormal(spot, vol)`
- `exp_return_pct = ((exp_price / spot) - 1) × 100`
- `kelly = kelly_fraction(prices)`

3. If < 10 prices: uses defaults (`vol=0.30`, `var_95=0.025`, etc.)

4. Returns:

```
{
  "ticker": "...",
  "current_price": 0.0,
  "annualized_volatility": 0.0,    // as %
  "var_95_daily": 0.0,            // as % of position
  "prob_5pct_upside_1mo": 0.0,    // as %
  "prob_5pct_downside_1mo": 0.0,  // as %
  "expected_return_1mo_pct": 0.0,
  "kelly_fraction": 0.0,          // as % of portfolio
  "technical_signal": "BUY/SELL/HOLD",
  "plain_english": "...",
  "disclaimer": "QuantLib metrics are theoretical estimates..."
}
```

`_plain_english_ql(ticker, signal, vol, prob_up, var_95, kelly) -> str`

Location: `quantlib_metrics.py:232`

Parameters: All floats/strings from `compute_ticker_metrics`.

Returns: `str` — plain-English summary of the ticker's risk metrics, one of three templates based on `signal` ("BUY", "SELL", or neutral).

BUY template includes: volatility label + %, theoretical 5%-gain probability, daily VaR, suggested Kelly position size. SELL template: volatility label + %, theoretical upside probability, daily VaR. Neutral: volatility, upside probability, no strong conviction statement.

`run_quantlib_metrics() -> dict`

Location: `quantlib_metrics.py:266`

Parameters: None

Returns: `dict` — the full output written to disk.

Behavior:

1. Prints QuantLib availability status.
2. Loads `signal_output_phase3.json`. On missing file, warns and uses empty list.
3. Filters to BUY and SELL signals only; sorts by `|confidence_score|` descending; takes top 15.
4. If no BUY/SELL signals: falls back to first 10 signals.
5. For each ticker, calls `compute_ticker_metrics(ticker, sig)` and appends result.
6. Assembles output:

```
{
  "generated_at": "...",
  "quantlib_version": "...",
  "risk_free_rate_used": 0.05,
  "horizon_trading_days": 21,
  "ticker_metrics": [...],
  "methodology": "..."
}
```

1. Writes to `quantlib_metrics.json`.

Files read: `Module_2_Technical_Analysis/signal_output_phase3.json` **Files written:** `quantlib_metrics.json` **External libs:** `QuantLib` (optional), `yfinance`, `numpy`, `scipy`, `math`

historical_correlation.py — Phase 7: Historical Correlation Engine

File: `historical_correlation.py` **Phase:** 7 — Historical Correlation Engine **Role:** Answers "What happened last time?" for current market events. Detects which historical event types are relevant to today's news by keyword scanning, then fetches sector ETF performance data from those historical periods via `yfinance`. Produces ranked winners/losers and a plain-English narrative for each event type. Output is consumed by `chef_gpt.py`'s `historical_context` field.

Key constants:

Name	Value
<code>BASE_DIR</code>	<code>Path(__file__).parent</code>
<code>OUTPUT_PATH</code>	<code>BASE_DIR / "historical_parallels.json"</code>

HISTORICAL_EVENTS (`historical_correlation.py:40`): A module-level list of 8 event type dicts, each containing:

- `event_type` (`str`): Internal identifier
- `label` (`str`): Human-readable name
- `trigger_keywords` (`list[str]`): Keywords that trigger detection
- `historical_periods` (`list[dict]`): List of dicts with `label`, `start`, `end`, `context`
- `plain_english` (`str`): Pre-written explanation of the event

Event types defined:

event_type	Label	# Historical Periods
<code>fed_rate_hike</code>	Federal Reserve Rate Hike Cycle	3 (2022, 2018, 2004–2006)
<code>fed_rate_cut</code>	Federal Reserve Rate Cut	3 (2020 COVID, 2008, 2019)
<code>inflation_spike</code>	Inflation Surge	2 (2021–2022, 1979–1982)
<code>tariff_announcement</code>	Tariff / Trade War	2 (2018–2019, 2002 Steel)
<code>recession</code>	Economic Recession	3 (2020 COVID, 2008–2009, 2001)
<code>ai_boom</code>	AI / Technology Boom	2 (2023–2024, 1995–2000)
<code>energy_shock</code>	Energy / Oil Price Shock	3 (2022 Ukraine, 2014, 1973 OPEC)
<code>defense_spending</code>	Defense / Military Spending Increase	2 (Post-9/11, Ukraine War)

SECTOR ETFs (`historical_correlation.py:151`): Maps 14 sector/asset labels to ETF tickers (XLK, XLV, XLF, XLE, XLY, XLP, XLI, XLB, XLRE, XLU, XLC, SPY, TLT, GLD).

detect_current_events(`economic_summary: str`, `enriched_articles: list[dict]`) -> `list[dict]`

Location: `historical_correlation.py:172`

Parameters:

- `economic_summary` (`str`): Text from `economic_reasoning_summary.json["summary"]`.
- `enriched_articles` (`list[dict]`): Up to 50 articles from `enriched_news_sentiment.json`.

Returns: `list[dict]` — filtered subset of **HISTORICAL_EVENTS** where at least one trigger keyword is found.

Behavior:

- Concatenates `economic_summary` + titles + summaries of first 50 articles, all lowercased.
- For each event in **HISTORICAL_EVENTS**, checks if `any(kw in combined_text for kw in event["trigger_keywords"])`.
- Returns matching events.

measure_period_performance(`ticker: str`, `start: str`, `end: str`) -> `dict | None`

Location: `historical_correlation.py:191`

Parameters:

- `ticker` (`str`): ETF or stock ticker.
- `start` (`str`): ISO date string "YYYY-MM-DD".
- `end` (`str`): ISO date string "YYYY-MM-DD".

Returns: `dict` or `None` if insufficient data.

Behavior:

1. Downloads price history via `yf.download(ticker, start=start, end=end, auto_adjust=True)`.
2. Returns `None` if empty or fewer than 5 trading days.
3. Computes:

```
start_price = close.iloc[0]
end_price   = close.iloc[-1]
peak_price  = close.max()
trough_price = close.min()
total_return = (end_price - start_price) / start_price * 100
max_drawdown = (trough_price - peak_price) / peak_price * 100
```

1. Returns:

```
{
  "ticker": "...",
  "start_price": ...,
  "end_price": ...,
  "total_return_pct": ...,
  "max_drawdown_pct": ...,
  "trading_days": ...
}
```

External libs: `yfinance`, `pandas`

`analyze_historical_period(period: dict) -> dict`

Location: `historical_correlation.py:222`

Parameters:

- `period` (`dict`): One historical period dict with `label`, `start`, `end`, `context`.

Returns: `dict` — the input period augmented with `sector_performance`, `winners`, `losers`.

Behavior:

1. Defines a sample of 8 ETFs: `{"Broad Market": "SPY", "Technology": "XLK", "Energy": "XLE", "Financials": "XLF", "Health Care": "XLV", "Consumer Staples": "XLP", "Bonds": "TLT", "Gold": "GLD"}`.
2. Calls `measure_period_performance` for each.
3. Ranks all sectors by `total_return_pct` descending.
4. Sets `winners` = top 3 as `"{label} ({ret:+.1f}%)"` strings.
5. Sets `losers` = bottom 3 in same format.

`generate_historical_narrative(event: dict, analyzed_periods: list[dict]) -> str`

Location: `historical_correlation.py:269`

Parameters:

- `event` (`dict`): An `HISTORICAL_EVENTS` entry.
- `analyzed_periods` (`list[dict]`): Output of `analyze_historical_period` for each period.

Returns: `str` — Markdown-formatted narrative.

Behavior: Builds a multi-paragraph string including:

- Event label as H2 header
- `event["plain_english"]` overview
- For each analyzed period: period label, context, SPY return direction/magnitude, best and worst performers

`run_historical_correlation() -> dict`

Location: `historical_correlation.py:301`

Parameters: None

Returns: `dict` — full output written to disk.

Behavior:

1. Loads `economic_reasoning_summary.json` and `enriched_news_sentiment.json` (both optional — warns if missing).
2. Calls `detect_current_events(economic_summary, enriched_articles)`.
3. If no events detected, defaults to `HISTORICAL_EVENTS[0]` (Fed Rate Hike context).
4. Iterates over detected events (up to 3):
 - For each event, iterates over its `historical_periods` (up to 2).
 - Calls `analyze_historical_period` for each.
 - Calls `generate_historical_narrative(event, analyzed_periods)`.
 - Appends result dict to `results`.
5. Assembles output:

```
{
  "generated_at": "...",
  "events_analyzed": ...,
  "historical_parallel": [...]
}
```

1. Writes to `historical_parallel.json`.

Files read: `news_output/economic_reasoning_summary.json`, `news_output/enriched_news_sentiment.json` **Files written:** `historical_parallel.json` **External libs:** `yfinance`, `pandas`, `numpy`, `python-dotenv`

`_extract_key_takeaway(event_type: str, periods: list[dict]) -> str`

Location: `historical_correlation.py:373`

Parameters:

- `event_type` (`str`): Event type identifier.
- `periods` (`list[dict]`): Analyzed historical periods.

Returns: `str` — a single-sentence takeaway about average market direction.

Behavior:

1. Extracts SPY `total_return_pct` from each period's `sector_performance`.
2. Computes simple average of available returns.
3. Maps to directional string:
 - `avg > 10%`: "markets generally rose an average of {avg:.1f}%"
 - `avg < -10%`: "markets generally fell an average of {avg:.1f}%"
 - Otherwise: "markets were mixed, averaging {avg:+.1f}%"
4. Appends disclaimer: "Past outcomes do not guarantee future results."

phase3_extraction.py — Phase 3: NLP Data Extraction

File: `phase3_extraction.py` **Phase:** 3 — Data Extraction & Structuring **Role:** Reads raw scraped articles and applies NLP and regex to extract structured financial intelligence: percentages, dollar values, dates, economic indicator mentions, comparison phrases, company names (via spaCy NER), and VADER sentiment scores. Outputs a sorted list of enriched article dicts. The output is the primary input to Phase 4 clustering.

Key paths:

Name	Value
<code>BASE_DIR</code>	<code>Path(__file__).parent</code>
<code>NEWS_DIR</code>	<code>BASE_DIR / "news_output"</code>

Name	Value
INPUT_PATH	NEWS_DIR / "articles_*.json" (latest)
OUTPUT_PATH	NEWS_DIR / "enriched_news_sentiment.json"

Optional imports (graceful degradation):

- `vaderSentiment.vaderSentiment.SentimentIntensityAnalyzer` → `VADER_AVAILABLE`
- `spacy` (model `en_core_web_sm`) → `SPACY_AVAILABLE`, `NLP`

Regex patterns (`phase3_extraction.py:61–69`):

Pattern	Expression	Matches
PERCENT_RE	<code>[+-]?\d+(?:\.\d+)?\s*%</code>	Signed/unsigned percentages
DOLLAR_RE	<code>`\\$\s\d{1,3}(?;\.\d{3})?(?:\.\d+)?(?:\s*(?:billion trillion B M T))?'`</code> (?:billion	million trillion B M T) ? Dollar amounts with optional scale
DATE_RE	Complex alternation	<code>MM/DD/YYYY</code> , month-name dates, <code>Q1 2024</code> , <code>FY2024</code>

`ECON_INDICATORS` (`phase3_extraction.py:71`): Dict of 10 named patterns (cpi, gdp, unemployment, fed_rate, inflation, interest_rate, recession, earnings, tariff, layoff) each mapped to a compiled case-insensitive regex.

`COMPARISON_PHRASES` (`phase3_extraction.py:84`): 16 exact phrases (e.g. "beat expectations", "missed estimates", "raised guidance", "record high").

`SENTIMENT_KEYWORDS` (`phase3_extraction.py:96`): Dict with "positive" (19 words) and "negative" (22 words) for keyword-based fallback sentiment.

`extract_percentages(text: str) -> list[str]`

Location: `phase3_extraction.py:113`

Returns: All regex matches of `PERCENT_RE` in `text`.

`extract_dollar_values(text: str) -> list[str]`

Location: `phase3_extraction.py:117`

Returns: All regex matches of `DOLLAR_RE` in `text`.

`extract_dates(text: str) -> list[str]`

Location: `phase3_extraction.py:121`

Returns: All regex matches of `DATE_RE` in `text`.

`extract_economic_indicators(text: str) -> list[str]`

Location: `phase3_extraction.py:125`

Returns: List of indicator names (keys from `ECON_INDICATORS`) whose regex patterns match in `text`.

`extract_comparison_phrases(text: str) -> list[str]`

Location: `phase3_extraction.py:133`

Returns: All case-insensitive matches of `COMPARISON_RE` (union of all 16 phrases) in `text`.

`extract_companies_spacy(text: str) -> list[str]`

Location: `phase3_extraction.py:137`

Parameters:

- `text` (`str`): Article text (processed only up to first 5,000 characters).

Returns: `list[str]` — up to 20 unique `ORG` entities extracted by spaCy NER. Returns `[]` if spaCy unavailable.

External libs: `spacy`

`score_sentiment_vader(text: str) -> dict`

Location: `phase3_extraction.py:145`

Parameters:

- `text` (`str`): Full article text.

Returns: `dict` with keys `compound` (float in [-1,1]), `pos`, `neg`, `neu` (floats in [0,1]).

Behavior: Instantiates `SentimentIntensityAnalyzer()` and calls `polarity_scores(text)`. Returns `{"compound": 0.0, "pos": 0.0, "neg": 0.0, "neu": 1.0}` if VADER unavailable.

External libs: `vaderSentiment`

`score_sentiment_keyword(text: str) -> dict`

Location: `phase3_extraction.py:152`

Parameters:

- `text` (`str`): Article text.

Returns: `dict` with keys `compound`, `pos`, `neg`, `neu`.

Formula (keyword fallback):

```
pos = count of positive keywords in text_lower
neg = count of negative keywords in text_lower
total = max(pos + neg, 1)
compound = (pos - neg) / total
pos_ratio = pos / total
neg_ratio = neg / total
neu = 1 - |compound|
```

`determine_sentiment_label(compound: float) -> str`

Location: `phase3_extraction.py:167`

Returns:

- `"positive"` if `compound >= 0.05`
- `"negative"` if `compound <= -0.05`
- `"neutral"` otherwise

`find_latest_articles_file() -> Path | None`

Location: `phase3_extraction.py:179`

Returns: Most recently modified `articles_*.json` file in `news_output/`, or `None`.

Behavior: Uses `Path.glob("articles_*.json")`, sorts by `stat().st_mtime` descending.

`load_articles(path: Path) -> list[dict]`

Location: `phase3_extraction.py:189`

Parameters:

- `path` (`Path`): Path to an articles JSON file.

Returns: `list[dict]` — normalized article dicts.

Behavior: Handles three input formats:

1. **Finnhub nested format:** `[{"symbol": ..., "finnhub": {"company_news": [...], ...}}]` — extracts `headline`, `summary`, `url`, `datetime`, `source`, `symbol` fields.
2. **Headline-keyed format:** `[{"headline": ..., ...}]` — copies record with `"title"` alias.
3. **Plain format:** `[{"title": ..., ...}]` — appended as-is.

`load_articles_with_fallback() -> list[dict]`

Location: `phase3_extraction.py:226`

Returns: `list[dict]` — articles loaded from the best available source.

Behavior:

1. Tries `find_latest_articles_file() -> load_articles()`.
2. If result is empty, falls back to `news_output/summaries.json`.
3. Returns `[]` if both fail.

`enrich_article(article: dict) -> dict`

Location: `phase3_extraction.py:245`

Parameters:

- `article (dict)`: Raw article dict with `title` and `summary` keys.

Returns: `dict` — original article augmented with `extracted`, `sentiment`, `data_richness_score`, `enriched_at`.

Behavior:

1. Concatenates `title + ". " + summary` as `full_text`.
2. Calls all extraction functions on `full_text`.
3. Runs VADER sentiment (or keyword fallback).
4. Computes sentiment label via `determine_sentiment_label`.
5. Computes data richness score:

Data richness formula (`phase3_extraction.py:267`):

```
richness_items = len(percentages) + len(dollar_vals) + len(dates) + len(indicators) * 2
richness_score = min(1.0, richness_items / 10.0)
```

1. Returns enriched dict with structure:

```
{
  "...original fields...",
  "extracted": {
    "percentages": [...],
    "dollar_values": [...],
    "dates": [...],
    "economic_indicators": [...],
    "comparison_phrases": [...],
    "companies_mentioned": [...]
  },
  "sentiment": {
    "compound": ...,
    "positive": ...,
    "negative": ...,
    "neutral": ...,
    "label": "positive/negative/neutral"
  },
  "data_richness_score": ...,
  "enriched_at": "..."
}
```

`run_extraction() -> list[dict]`

Location: `phase3_extraction.py:292`

Parameters: None

Returns: `list[dict]` — all enriched articles, sorted by `|sentiment.compound|` descending (most impactful first).

Behavior:

1. Calls `load_articles_with_fallback()`.
2. Iterates all articles, calling `enrich_article` for each with try/except per article.
3. Sorts by `abs(a["sentiment"]["compound"])` descending.
4. Creates `news_output/` directory if needed.

- 5. Writes to `enriched_news_sentiment.json` (UTF-8, indent=2).
- 6. Prints summary: total articles, positive/negative/neutral counts, NLP mode (spaCy/keyword).

Files read: `news_output/articles_*.json` or `news_output/summaries.json` **Files written:** `news_output/enriched_news_sentiment.json` **External libs:** `vaderSentiment` (optional), `spacy` (optional), `json`, `re`, `pathlib`

phase4_clustering.py — Phase 4: Article Scoring & Clustering

File: `phase4_clustering.py` **Phase:** 4 — Article Scoring & Clustering **Role:** Takes enriched articles from Phase 3, generates sentence embeddings, clusters similar articles (reducing information overload), assigns GICS sector labels, scores each article by impact, and selects top representative articles per cluster for downstream GPT summarization. Outputs `clustered_summaries.json`.

Key paths:

Name	Value
<code>BASE_DIR</code>	<code>Path(__file__).parent</code>
<code>NEWS_DIR</code>	<code>BASE_DIR / "news_output"</code>
<code>INPUT_PATH</code>	<code>NEWS_DIR / "enriched_news_sentiment.json"</code>
<code>OUTPUT_PATH</code>	<code>NEWS_DIR / "clustered_summaries.json"</code>

Optional imports:

- `sentence_transformers.SentenceTransformer` → `EMBEDDINGS_AVAILABLE`
- `sklearn.cluster.AgglomerativeClustering`, `sklearn.metrics.pairwise.cosine_similarity` → `SKLEARN_AVAILABLE`

GICS_SECTORS (`phase4_clustering.py:59`): Dict mapping 11 GICS sector names to keyword lists (3–20 keywords each) used for sector classification.

CREDIBILITY_SCORES (`phase4_clustering.py:109`): Dict mapping source names to credibility scores:

Source	Score
reuters, bloomberg	0.95
wsj, wall street journal	0.90
ft, financial times	0.90
cnbc	0.85
marketwatch	0.80
finnhub	0.75
yahoo	0.70
seekingalpha	0.65
default	0.60

HIGH_IMPACT_KEYWORDS (`phase4_clustering.py:124`): 28 high-impact financial keywords (earnings, fed, inflation, gdp, acquisition, merger, ipo, bankruptcy, tariff, layoff, etc.).

`assign_sector(text: str) -> str`

Location: `phase4_clustering.py:137`

Parameters:

- `text` (`str`): Combined article title + summary.

Returns: `str` — GICS sector name, or `"General Market"` if no match.

Behavior: For each GICS sector, counts keyword hits in `text_lower`. Returns the sector with the highest count; ties broken by dict order.

`get_credibility(source: str) -> float`

Location: `phase4_clustering.py:149`

Parameters:

- `source` (`str`): Article source name.

Returns: `float` — credibility score from `CREDIBILITY_SCORES` . Falls back to `0.60` ("default").

keyword_weight(text: str) -> float

Location: `phase4_clustering.py:157`

Parameters:

- `text` (`str`): Combined article text.

Returns: `float` in `[0.0, 1.0]` .

Formula: `min(1.0, keyword_hits / 5.0)` where `keyword_hits` is the count of `HIGH_IMPACT_KEYWORDS` found in `text_lower` .

profile_match_score(article: dict, profile: dict) -> float

Location: `phase4_clustering.py:163`

Parameters:

- `article` (`dict`): Enriched article dict.
- `profile` (`dict`): User profile dict.

Returns: `float` in `[0.0, 1.0]` .

Behavior:

- Base score: `0.5`
- If `risk_tolerance == "high"` and text contains growth/momentum/tech/ai/innovation terms: `+0.3`
- If `risk_tolerance == "high"` and text contains volatile/speculative/high-risk terms: `+0.2`
- If `risk_tolerance == "low"` and text contains dividend/stable/utility/bond/defensive terms: `+0.3`
- If `risk_tolerance == "low"` and text contains recession/safe haven/gold/treasury terms: `+0.2`
- If `risk_tolerance == "moderate"`: stays at `0.5`
- If `goal == "income"` and text contains dividend/yield/income/payout: `+0.2`
- If `goal == "growth"` and text contains growth/expansion/revenue growth/market share: `+0.2`
- Final: `min(1.0, score)`

compute_impact_score(article: dict, profile: dict) -> float

Location: `phase4_clustering.py:197`

Parameters:

- `article` (`dict`): Enriched article dict.
- `profile` (`dict`): User profile dict.

Returns: `float` in `[0.0, 1.0]` .

Formula (reproduced from `phase4_clustering.py:206–209`):

```
sentiment = |article.sentiment.compound|
credibility = get_credibility(article.source)
kw = keyword_weight(title + " " + summary)
pm = profile_match_score(article, profile)
richness = article.data_richness_score

impact = sentiment × credibility × (kw × 0.4 + pm × 0.3 + richness × 0.3)
return min(1.0, round(impact, 4))
```

The composite weight factor is `kw × 0.4 + pm × 0.3 + richness × 0.3` , ensuring that keyword relevance (40%), user-profile alignment (30%), and data richness (30%) all contribute to impact.

embed_articles(articles: list[dict]) -> np.ndarray | None

Location: `phase4_clustering.py:216`

Parameters:

- `articles` (`list[dict]`): All enriched articles.

Returns: `np.ndarray` of shape `(n_articles, embedding_dim)` as float32, or `None` if embeddings unavailable.

Behavior:

1. Constructs text strings: `(title + " " + summary)[:512]` for each article.
2. Loads `SentenceTransformer("all-MiniLM-L6-v2")`.
3. Encodes with `batch_size=64`, `show_progress_bar=False`.
4. Casts to `float32`.

External libs: `sentence-transformers`

`cluster_with_embeddings(articles: list[dict], embeddings: np.ndarray) -> dict[int, list[int]]`

Location: `phase4_clustering.py:230`

Parameters:

- `articles` (`list[dict]`): All enriched articles.
- `embeddings` (`np.ndarray`): Embedding matrix.

Returns: `dict[int, list[int]]` — mapping cluster label → list of article indices.

Behavior:

1. Computes `n_clusters = max(5, min(n // 10, 50))` — between 5 and 50.
2. Fits `AgglomerativeClustering(n_clusters=n_clusters, metric="cosine", linkage="average")`.
3. Maps labels to index lists.

External libs: `scikit-learn`

`cluster_by_sector(articles: list[dict]) -> dict[int, list[int]]`

Location: `phase4_clustering.py:248`

Parameters:

- `articles` (`list[dict]`): Articles with `gics_sector` already assigned.

Returns: `dict[int, list[int]]` — groups articles by their `gics_sector` field.

Behavior: Fallback when ML clustering is unavailable. Groups indices by sector string, converts to integer-keyed dict.

`deduplicate_within_cluster(cluster_articles: list[dict]) -> list[dict]`

Location: `phase4_clustering.py:259`

Parameters:

- `cluster_articles` (`list[dict]`): Articles in a single cluster.

Returns: `list[dict]` — deduplicated articles.

Behavior:

1. Normalizes each title: lowercase, removes non-alphanumeric characters.
2. Converts to word frozensets.
3. Compares new article against all previously seen titles using Jaccard-like overlap:

```
overlap = |words n seen_words| / max(|words|, |seen_words|)
is_dupe = overlap > 0.7
```

1. Skips duplicates; adds unique articles to result.

`re_sub_helper(pattern: str, repl: str, text: str) -> str`

Location: `phase4_clustering.py:283`

A thin wrapper around `re.sub`. Used internally for title normalization.

`run_clustering() -> dict`

Location: `phase4_clustering.py:291`

Parameters: None

Returns: `dict` — cluster-id → list of article dicts (written to disk).

Behavior:

1. Loads `enriched_news_sentiment.json`.
2. Loads `user_profile.json` if available.
3. For each article: assigns `gics_sector` via `assign_sector`, computes `impact_score` via `compute_impact_score`.
4. Chooses clustering method:
 - If both `EMBEDDINGS_AVAILABLE` and `SKLEARN_AVAILABLE`: calls `embed_articles` → `cluster_with_embeddings`. Method label: `"embedding+agglomerative"`.
 - Otherwise: calls `cluster_by_sector`. Method label: `"sector_fallback"`.
5. For each cluster:
 - Calls `deduplicate_within_cluster`.
 - Sorts by `impact_score` descending.
 - Assigns cluster-level sector as the most common `gics_sector` in the cluster (plurality vote).
 - Selects top 2 articles as representatives (for Phase 5 GPT summarization).
 - Tags all articles with `cluster_id` and `cluster_sector`.
6. Writes output as `{cluster_id: [article_dicts]}` to `clustered_summaries.json`.
7. Prints: total clusters, articles retained, clustering method.

Files read: `news_output/enriched_news_sentiment.json`, `user_profile.json` **Files written:** `news_output/clustered_summaries.json`

External libs: `sentence-transformers`, `scikit-learn`, `numpy`, `python-dotenv`

portfolio_tracker.py — Phase 10: Portfolio Tracker

File: `portfolio_tracker.py` **Phase:** 10 — Portfolio Tracker / Dashboard **Role:** A simulated paper-trading engine that tracks mock positions based on Phase 8.9 opportunity scores and Phase 8.5 signals. Opens new long positions when a ticker's opportunity score exceeds the minimum threshold, updates existing positions with current prices (sourced from QuantLib metrics to avoid extra API calls), closes positions that hit stop-loss or take-profit targets, and computes aggregate portfolio metrics. Educational simulation only — no live trading.

Key constants:

Name	Value	Meaning
<code>STARTING_CASH</code>	<code>100_000.0</code>	Initial simulated portfolio value
<code>MAX_POSITIONS</code>	<code>10</code>	Maximum concurrent open positions
<code>RISK_PCT</code>	<code>0.02</code>	2% of portfolio risked per trade
<code>MIN_SCORE</code>	<code>6.5</code>	Minimum opportunity score to open position
<code>STOP_LOSS_PCT</code>	<code>0.08</code>	8% stop loss below entry
<code>TAKE_PROFIT</code>	<code>0.15</code>	15% take profit above entry
<code>OUTPUT_PATH</code>	<code>BASE_DIR / "portfolio_snapshot.json"</code>	
<code>PORTFOLIO_LOG</code>	<code>BASE_DIR / "portfolio_log.json"</code>	(defined but not currently written)

`load_json(path: Path, default: Any = None) -> Any`

Same pattern as other modules. Returns `default` on `FileNotFoundError` or `JSONDecodeError`.

`load_existing_portfolio() -> dict`

Location: `portfolio_tracker.py:38`

Returns: `dict` — existing portfolio snapshot or fresh starting state.

Behavior: Calls `load_json(OUTPUT_PATH)`. If present, returns it. Otherwise returns:

```
{
  "cash": 100000.0,
```

```

"positions": {},
"closed_trades": [],
"inception_date": "<today>"
}

```

fetch_current_price(ticker: str, quantlib_metrics: list[dict]) -> float | None

Location: `portfolio_tracker.py:50`

Parameters:

- `ticker` (str): Ticker symbol.
- `quantlib_metrics` (list[dict]): List of QuantLib metric records (from `quantlib_metrics.json["ticker_metrics"]`).

Returns: float or None.

Behavior: Searches `quantlib_metrics` for a matching ticker (case-insensitive). Returns `current_price` as float if positive; else None. This avoids redundant yfinance calls since QuantLib already fetched prices.

compute_position_size(portfolio_value: float, price: float) -> int

Location: `portfolio_tracker.py:59`

Parameters:

- `portfolio_value` (float): Total current portfolio value (cash + open positions).
- `price` (float): Current stock price.

Returns: int — number of shares to buy.

Formula (risk-based position sizing, `portfolio_tracker.py:62–70`):

```

dollar_risk    = portfolio_value × RISK_PCT      (2% of portfolio)
stop_loss_price = price × (1 - STOP_LOSS_PCT)   (price × 0.92)
risk_per_share = price - stop_loss_price        (8% of price)
shares_by_risk = floor(dollar_risk / risk_per_share)

max_by_value   = floor(portfolio_value × 0.10 / price) (cap: 10% of portfolio per position)
shares         = min(shares_by_risk, max_by_value)

```

Returns 0 if `price <= 0` or `risk_per_share <= 0`.

open_position(portfolio, ticker, price, opportunity_score, signal_reasoning) -> bool

Location: `portfolio_tracker.py:74`

Parameters:

- `portfolio` (dict): Current portfolio state (mutated in-place).
- `ticker` (str): Ticker symbol.
- `price` (float): Current price.
- `opportunity_score` (float): Score from `opportunity_scores.json`.
- `signal_reasoning` (str): Plain-text reasoning (truncated to 200 chars).

Returns: bool — True if position was opened.

Behavior:

1. Returns False if ticker already in `positions` or `len(positions) >= MAX_POSITIONS`.
2. Computes `portfolio_value = cash + sum(shares × last_price)`.
3. Calls `compute_position_size(portfolio_value, price)`.
4. If `cost > cash`: recalculates as `floor(cash × 0.10 / price)` shares.
5. Deducts `cost = shares × price` from `portfolio["cash"]`.
6. Adds position record to `portfolio["positions"][ticker]`:

```

{
  "ticker": "...",
  "shares": N,
  "entry_price": ...,
  "last_price": ...,

```

```

"entry_date": "YYYY-MM-DD",
"cost_basis": ...,
"opportunity_score": ...,
"stop_loss": price × 0.92,
"take_profit_target": price × 1.15,
"unrealized_pnl": 0.0,
"unrealized_pnl_pct": 0.0,
"signal_reasoning": "...",
"status": "OPEN"
}

```

update_position(pos: dict, current_price: float) -> dict

Location: `portfolio_tracker.py:116`

Parameters:

- `pos` (dict): Open position record (mutated in-place).
- `current_price` (float): Latest price.

Returns: dict — updated position.

Formulas:

```

unrealized_pnl    = (current_price - entry_price) × shares
unrealized_pnl_pct = (current_price - entry_price) / entry_price × 100

if current_price <= stop_loss:      exit_signal = "STOP_LOSS"
elif current_price >= take_profit:  exit_signal = "TAKE_PROFIT"
else:                              exit_signal = None

```

close_position(portfolio: dict, ticker: str, current_price: float, reason: str) -> dict | None

Location: `portfolio_tracker.py:135`

Parameters:

- `portfolio` (dict): Portfolio state (mutated).
- `ticker` (str): Ticker to close.
- `current_price` (float): Exit price.
- `reason` (str): "STOP_LOSS" or "TAKE_PROFIT".

Returns: dict — closed trade record, or None if ticker not in positions.

Formulas:

```

proceeds    = shares × current_price
pnl         = proceeds - cost_basis
pnl_pct     = (current_price - entry_price) / entry_price × 100

```

Adds `proceeds` back to `portfolio["cash"]`. Appends closed trade record to `portfolio["closed_trades"]`.

compute_portfolio_metrics(portfolio: dict) -> dict

Location: `portfolio_tracker.py:162`

Parameters:

- `portfolio` (dict): Portfolio state dict.

Returns: dict with aggregate performance metrics.

Formulas:

```

open_value      = sum(shares × last_price) for all open positions
total_value     = cash + open_value

total_return    = total_value - STARTING_CASH
total_return_pct = total_return / STARTING_CASH × 100

win_rate_pct    = len(winning_trades) / len(all_closed_trades) × 100
gross_profit    = sum(pnl for winning trades)

```

```
gross_loss      = |sum(pnl for losing trades)|
profit_factor   = gross_profit / gross_loss  (∞ if no losses)

exposure_pct    = open_value / total_value × 100
```

Returns all values rounded to 2 decimal places.

run_portfolio_tracker() -> dict

Location: `portfolio_tracker.py:204`

Parameters: None

Returns: `dict` — full portfolio snapshot written to disk.

Behavior:

1. Loads: `signal_output_phase3.json`, `opportunity_scores.json`, `quantlib_metrics.json`, `user_profile.json`.
2. Builds `opp_lookup` (ticker → opportunity_score) and `sig_lookup` (ticker → signal dict).
3. Calls `load_existing_portfolio()`.
4. **Update cycle:** For each open position, fetches current price from QuantLib metrics; calls `update_position`; if `exit_signal` triggered, adds to close list.
5. **Close cycle:** Closes all positions in close list; prints P&L for each.
6. **Open cycle:** Filters signals for `final_signal == "BUY"`, sorts by opportunity score descending.
 - Adjusts `min_score` by risk profile:
 - `"low"`: 7.5 (conservative)
 - `"high"`: 5.5 (aggressive)
 - `"moderate"`: 6.5 (default)
 - Skips tickers below `min_score` or with no valid price.
 - Calls `open_position` for qualifying tickers.
7. Calls `compute_portfolio_metrics`.
8. Assembles snapshot dict (portfolio state + metrics + settings + disclaimer).
9. Writes to `portfolio_snapshot.json` (using `default=str` for date serialization).

Files read: `signal_output_phase3.json`, `opportunity_scores.json`, `quantlib_metrics.json`, `user_profile.json`, `portfolio_snapshot.json` (existing) **Files written:** `portfolio_snapshot.json` **External libs:** `json`, `datetime`, `pathlib`

reset_portfolio()

Location: `portfolio_tracker.py:316`

Behavior: Writes a fresh portfolio state with `STARTING_CASH = $100,000`, empty positions and trades, today as inception date. Invoked with `python portfolio_tracker.py --reset`.

congress_bills.py — Phase 6.5 Supplement: Congress Bills Intelligence

File: `congress_bills.py` **Phase:** 6.5 — Political Intelligence (supplement to the main `economic_engine_insider.py`) **Role:** Fetches recently enacted laws and pending bills from the Congress.gov API (119th Congress, 2025–2027). Correlates bill content with congressional stock trade disclosures by matching company and sector keywords against bill titles and action text, within a ±90-day time window. Adds politician party/state metadata via `politician_parties.json`. Outputs `congress_bills.json` for use in the political intelligence layer.

Key constants:

Name	Value
<code>BASE_DIR</code>	<code>Path(__file__).parent</code>
<code>OUTPUT_FILE</code>	<code>BASE_DIR / "congress_bills.json"</code>
<code>CONGRESS_API_KEY</code>	<code>os.getenv("CONGRESS_API_KEY", "")</code>
<code>CURRENT_CONGRESS</code>	<code>119</code>
<code>LOOKBACK_DAYS</code>	<code>180</code>
<code>BASE_URL</code>	<code>"https://api.congress.gov/v3"</code>

TICKER_KEYWORDS (`congress_bills.py:45`): Dict mapping 80+ ticker symbols to lists of keywords (company names, product names, policy topics) to search in bill text. Covers tech (AAPL, MSFT, NVDA, META, etc.), telecom (T, VZ), finance (JPM, BAC, GS), health care (JNJ, UNH, LLY), energy (XOM, CVX, NEE), industrials/defense (LMT, RTX, BA), and consumer sectors.

SECTOR_KEYWORDS (`congress_bills.py:142`): Dict mapping 10 sector names to keyword lists for sector-level bill matching (e.g. "Technology" → ["artificial intelligence", "AI", "semiconductor", "chip", ...]).

DISCLAIMER (`congress_bills.py:38`):

"This analysis identifies timing relationships between publicly available government disclosures and market events. It does not imply or allege wrongdoing of any kind."

_headers() → dict

Location: `congress_bills.py:156`

Returns: Empty dict. Congress.gov API uses `api_key` as a query parameter rather than an HTTP header.

_paginated_get(endpoint: str, params: dict, max_items: int = 250) → list[dict]

Location: `congress_bills.py:160`

Parameters:

- `endpoint` (str): API endpoint path (e.g. `"/law/119"`).
- `params` (dict): Additional query parameters.
- `max_items` (int): Maximum records to retrieve. Default: 250.

Returns: list[dict] — up to `max_items` records.

Behavior:

1. Iterates with `offset` pagination, fetching `limit = min(250, max_items)` records per request.
2. Appends `limit`, `offset`, `format="json"`, `api_key` to params.
3. Reads `data.get("bills", data.get("laws", []))` as the batch.
4. Stops when batch is empty, `offset >= total`, or `len(batch) < limit`.
5. Returns `items[:max_items]`.

External libs: `requests`

fetch_enacted_laws(since_days: int = 180) → list[dict]

Location: `congress_bills.py:189`

Parameters:

- `since_days` (int): Lookback window in days. Default: `LOOKBACK_DAYS` (180).

Returns: list[dict] — enacted laws with `latestAction.actionDate` within the lookback window.

Behavior:

1. Calls `_paginated_get(f"/law/{CURRENT_CONGRESS}", params={}, max_items=500)`.
2. Filters results: keeps laws where `latestAction.actionDate` is within `since_days` days of now. Includes laws with missing dates.
3. Prints count of matching laws.

API endpoint: GET `https://api.congress.gov/v3/law/119`

fetch_recent_bills(since_days: int = 180) → list[dict]

Location: `congress_bills.py:211`

Parameters:

- `since_days` (int): Lookback window in days.

Returns: list[dict] — recently updated pending bills.

Behavior: Calls `_paginated_get(f"/bill/{CURRENT_CONGRESS}", params={"sort": "updateDate+desc", "fromDateTime": since_iso}, max_items=500)`.

API endpoint: GET `https://api.congress.gov/v3/bill/119`

```
_text_matches(title: str, action_text: str, keywords: list[str]) -> bool
```

Location: `congress_bills.py:224`

Parameters:

- `title` (`str`): Bill title.
- `action_text` (`str`): Latest action text.
- `keywords` (`list[str]`): Keywords to search.

Returns: `bool` — `True` if any keyword appears in the combined lowercased string.

```
correlate_bills_to_tickers(all_bills: list[dict], trades: list[dict]) -> list[dict]
```

Location: `congress_bills.py:229`

Parameters:

- `all_bills` (`list[dict]`): Combined list of enacted laws + recent bills.
- `trades` (`list[dict]`): Congressional trade records from `intelligence_output.json["housing_trades"]`.

Returns: `list[dict]` — sorted list of correlation records (most recent first).

Behavior:

1. Loads `politician_parties.json` for name → `{party, party_abbr, state}` mapping.
2. Builds `ticker_trades` dict: ticker → list of trade records (with politician name, party, chamber, transaction, amount, date).
3. For each bill:
 - Parses `latestAction.actionDate`.
 - For each ticker in `TICKER_KEYWORDS` that has trades AND whose keywords match the bill text: adds to `matched_tickers`.
 - For each sector in `SECTOR_KEYWORDS` whose keywords match: adds to `matched_sectors`.
 - Skips bills with no matches.
 - Finds correlated trades within ± 90 days of `bill_dt`. Deduplicates by `politician|ticker|date`.
 - Builds correlation record:

```
{
  "bill_id": "...",
  "title": "...",
  "action_date": "...",
  "action_text": "...",
  "is_enacted": true/false,
  "matched_tickers": [...],
  "matched_sectors": [...],
  "correlated_trades": [
    {
      "politician": "...",
      "party": "...",
      "party_abbr": "...",
      "state": "...",
      "chamber": "...",
      "transaction": "...",
      "ticker": "...",
      "trade_date": "...",
      "amount": "...",
      "days_delta": N
    }
  ]
}
```

1. Sorts results by `action_date` descending.

Files read: `politician_parties.json`

```
main()
```

Location: `congress_bills.py:342`

Parameters: None (reads env vars)

Behavior:

1. Raises `RuntimeError` if `CONGRESS_API_KEY` is not set.
2. Calls `fetch_enacted_laws` and `fetch_recent_bills`.
3. Loads `intelligence_output.json` for trade data.
4. Calls `correlate_bills_to_tickers(all_bills, trades)`.
5. Assembles output:

```
{
  "disclaimer": "...",
  "generated_at": "...",
  "congress": 119,
  "lookback_days": 180,
  "enacted_laws": N,
  "recent_bills": N,
  "correlations": [...],
  "raw_laws": [
    { "title", "action_date", "action_text", "type", "number" }
  ]
}
```

1. Writes to `congress_bills.json`.
2. Prints top 5 correlations preview with `days_delta` direction ("before"/"after" bill).

Files read: `GPT_Economy/Intelligence_layer (IN PROGRESS)/intelligence_output.json`, `politician_parties.json` **Files written:** `congress_bills.json` **External libs:** `requests`, `python-dotenv`, `json`, `re`, `datetime`

politician_performance.py — Phase 6.5 Supplement: Politician Trading Performance

File: `politician_performance.py` **Phase:** 6.5 — Political Intelligence (supplement) **Role:** Pre-computes estimated P&L for every congressional stock trade using the midpoint of the disclosed dollar range and historical price data from yfinance (5-year history per ticker, one batch download per ticker to avoid rate limits). Aggregates by politician: total invested estimate, total estimated P&L, trade counts (buys/sells), and return percentage. Output is `politician_performance.json`, used by the political intelligence web UI.

Key paths:

Name	Value
<code>BASE_DIR</code>	<code>Path(__file__).parent</code>
<code>OUTPUT_FILE</code>	<code>BASE_DIR / "politician_performance.json"</code>

`_parse_range_midpoint(range_str: str) -> float`

Location: `politician_performance.py:34`

Parameters:

- `range_str` (`str`): Disclosed dollar range string from STOCK Act filing.

Returns: `float` — estimated midpoint in dollars.

Behavior (in order):

1. Tries `float(cleaned_str)` — handles plain numeric Amount fields.
2. Matches `"Over $X"` pattern: returns `X` as floor.
3. Matches `"$X - $Y"` pattern: returns `(X + Y) / 2`.
4. If only one number found: returns it.
5. Returns `0.0` if unparseable.

Examples:

```
"$1,001 - $15,000"      → 8000.5
"$50,001 - $100,000"    → 75000.5
"$1,000,001 - $5,000,000" → 3000000.5
"Over $5,000,000"       → 5000000.0
```

```
_build_ticker_history(tickers: list[str]) -> dict[str, pd.Series]
```

Location: `politician_performance.py:71`

Parameters:

- `tickers` (`list[str]`): All unique tickers across all trades.

Returns: `dict` mapping ticker → `pd.Series` of daily close prices (5-year history, date index).

Behavior:

1. For each ticker: calls `yf.download(ticker, period="5y", interval="1d", auto_adjust=True)`.
2. Squeezes multi-level column if needed.
3. Stores `close.dropna()` as the Series.
4. Rate limiting: 0.3-second sleep between calls; 1.0-second sleep every 10 tickers.
5. On error: stores empty `pd.Series`.

This single-batch-per-ticker approach avoids the per-trade-date API calls that would cause rate limiting.

External libs: `yfinance`, `pandas`, `time`

```
_price_on_date(series: pd.Series, date_str: str) -> float | None
```

Location: `politician_performance.py:104`

Parameters:

- `series` (`pd.Series`): Pre-fetched price history for a ticker.
- `date_str` (`str`): Target date string.

Returns: `float | None` — closing price on or nearest-before the target date.

Behavior:

1. Filters `series` to dates `<= pd.Timestamp(date_str)`.
2. Returns `float(available.iloc[-1])` — the most recent price on or before the trade date.
3. If no dates precede target (trade predates history), uses earliest available price.
4. Returns `None` on any exception.

External libs: `pandas`

```
main()
```

Location: `politician_performance.py:120`

Parameters: None

Behavior:

[LOAD_TRADES] (`politician_performance.py:122–143`):

1. Loads `intelligence_output.json["housing_trades"]`.
2. Loads `politician_parties.json` (name → party/state dict).
3. Filters to trades that have both `ticker` and `representative` fields.

[PRICE_FETCH] (`politician_performance.py:146–157`):

1. Collects unique tickers across all trades.
2. Calls `_build_ticker_history(unique_tickers)` — one download per ticker.

[COMPUTE] (`politician_performance.py:159–205`): For each trade:

```
midpoint      = _parse_range_midpoint(range_str)
price_then    = _price_on_date(series, date_str)
price_now     = series.iloc[-1]

raw_return    = (price_now - price_then) / price_then
if is_sale:   raw_return = -raw_return    # sold positions invert gains

pct_return    = raw_return * 100
est_pnl      = midpoint * raw_return
```

Produces a `trade_record` dict per trade with: politician, party, state, chamber, ticker, transaction, date, range_disclosed, midpoint_est, price_at_trade, price_current, pct_return, est_pnl.

[AGGREGATE] (`politician_performance.py:209–244`): Groups by politician using `defaultdict`. For each politician accumulates:

- `total_invested_est`: sum of `midpoint_est` for all trades
- `total_est_pnl`: sum of `est_pnl` for all trades
- `trade_count`, `buy_count`, `sell_count`

Computes per-politician estimated return:

```
pct = total_est_pnl / total_invested_est × 100
```

Sorts `summary_rows` by `Est. P&L ($)` descending.

[SAVE] (`politician_performance.py:265–276`): Writes:

```
{
  "disclaimer": "All figures are ESTIMATES based on midpoint of disclosed ranges...",
  "generated_at": "...",
  "politician_summary": [...sorted by P&L...],
  "trade_detail": [...all individual trades...]
}
```

to `politician_performance.json`.

Files read: `GPT_Economy/Intelligence_layer (IN PROGRESS)/intelligence_output.json`, `politician_parties.json` **Files written:** `politician_performance.json` **External libs:** `yfinance`, `pandas`, `json`, `re`, `datetime`, `collections.defaultdict`

Cross-Module Data Flow Summary

```
Phase 2 (Scraping)
├─ news_output/articles_*.json
│   └─ Phase 3 (phase3_extraction.py)
│       └─ news_output/enriched_news_sentiment.json
│           └─ Phase 4 (phase4_clustering.py)
│               └─ news_output/clustered_summaries.json
│                   └─ Phase 5 (GPT summarization – external)
│                       └─ news_output/sector_summaries.json

Phase 6 (Economic Reasoning – external)
├─ news_output/economic_reasoning_summary.json

Phase 6.5 (Political Intelligence – economic_engine_insider.py)
├─ intelligence_output.json
│   └─ congress_bills.py → congress_bills.json
│   └─ politician_performance.py → politician_performance.json

Phase 7 (historical_correlation.py)
├─ reads: economic_reasoning_summary.json + enriched_news_sentiment.json
└─ writes: historical_parallel.json

Phase 8.7 (quantlib_metrics.py)
├─ reads: signal_output_phase3.json
└─ writes: quantlib_metrics.json

Phase 8.9 (opportunity_score.py)
├─ reads: economic_reasoning_summary.json, signal_output_phase3.json,
│   └─ recession_signals_output.json, user_profile.json, sector_summaries.json
└─ writes: opportunity_scores.json

Phase 9 (Backtesting – external backtrader script)
├─ results_run/summary_metrics.json

Phase 10 (portfolio_tracker.py)
├─ reads: signal_output_phase3.json, opportunity_scores.json,
│   └─ quantlib_metrics.json, user_profile.json, portfolio_snapshot.json
└─ writes: portfolio_snapshot.json

Phase 13 (recession_signals.py)
├─ reads: FRED public CSV + yfinance (live data)
└─ writes: recession_signals_output.json
```

```
Phase 11 (chef_gpt.py) – FINAL SYNTHESIS
├─ reads: user_profile.json, economic_reasoning_summary.json,
│         sector_summaries.json, signal_output_phase3.json,
│         summary_metrics.json, recession_signals_output.json,
│         intelligence_output.json, historical_parallelsl.json,
│         opportunity_scores.json, quantlib_metrics.json
└─ writes: intelligence_report.json ← THE FINAL DELIVERABLE
```

```
Phase 12 (controller.py) – ORCHESTRATES ALL ABOVE
├─ reads/writes: pipeline_state.json
└─ invokes all scripts above via subprocess
```

Generated 2026-06-21. All line numbers reference the source files as they exist on that date. Formulas and prompts are reproduced verbatim from the code — nothing is invented.

Part 7B — Supporting Engines: Economic Reasoning, Clustering, Technical Analysis & Backtesting

This document is an exhaustive technical reference for every Python file that powers the non-scraping, non-UI pipeline stages of ThinkFree Finance. Each section covers: the file's role and phase, all external library dependencies, every function (signature, parameters, return value, behavior, files read/written), and verbatim reproduction of scoring formulas, indicator parameters, and GPT prompt templates found in the source code.

Table of Contents

1. [GPT_Economy/economic_engine.py](#) — Phase 6 FAISS Index Builder
2. [GPT_Economy/Reasoning_Report.py](#) — Phase 6 Economic Reasoning Engine
3. [Profile/article_clustering.py](#) — Phase 5 GPT Article Summarization & Validation
4. [Profile/GPT_article_clustering_summary.py](#) — Phase 4 Sector Summary Generator
5. [Module 2 Technical Analysis/ta_analysis.py](#) — Technical Analysis Engine
6. [Module 2 Technical Analysis/phase 2 q_lib.py](#) — Qlib Feature Export
7. [Module 2 Technical Analysis/phase 3 signal.py](#) — Signal Generation
8. [Module 2 Technical Analysis/pipeline_alpha.py](#) — Qlib LightGBM Pipeline
9. [Module 2 Technical Analysis/prepare_qlib_data.py](#) — Qlib Data Formatter
10. [Module 2 Technical Analysis/phase 4 backtrader.py](#) — Phase 9 Backtesting Engine
11. [validate_env.py](#) — Environment Validation Script

1. [GPT_Economy/economic_engine.py](#)

Role and Phase

Phase 6 — Economic Reasoning Engine (FAISS index build step). This is the *index construction* script. It walks a directory of economics PDF textbooks, extracts all page text, embeds each page using a local sentence-transformer model, and persists a FAISS flat L2 index alongside a JSON metadata file. The resulting index is later queried by [Reasoning_Report.py](#) to answer economic questions grounded in textbook knowledge.

macOS segfault constraint: This file calls `SentenceTransformer(EMBEDDING_MODEL)` at module level (line 19). On macOS with `torch==2.2.2`, this causes a segfault when the object is instantiated at import time. The known workaround (documented in [MEMORY.md](#)) is to avoid loading the model at runtime and instead use keyword-scan on FAISS metadata. The `economic_engine_insider.py` variant applies this workaround.

External Libraries

Library	Import	Purpose
<code>fitz</code> (PyMuPDF)	<code>import fitz</code>	PDF page text extraction
<code>faiss</code>	<code>import faiss</code>	Vector index construction and persistence
<code>numpy</code>	<code>import numpy as np</code>	Float32 array coercion
<code>tqdm</code>	<code>from tqdm import tqdm</code>	Progress bar over book files
<code>sentence_transformers</code>	<code>from sentence_transformers import SentenceTransformer</code>	Local embedding model
<code>os</code> , <code>json</code>	<code>stdlib</code>	Directory walking, JSON serialisation

Configuration Constants

```
ECON_BOOK_DIR = "/Users/allanaziz/Desktop/ThinkFree/ThinkFree-main/Economic_Books"
FAISS_STORE_DIR = <ECON_BOOK_DIR>/FAISS_Store
EMBEDDING_MODEL = "all-MiniLM-L6-v2" # 384-dimensional embeddings
INDEX_OUTPUT = <FAISS_STORE_DIR>/economic_knowledge_index.faiss
METADATA_OUTPUT = <FAISS_STORE_DIR>/economic_knowledge_metadata.json
```

The FAISS index is a `faiss.IndexFlatL2(384)` — exact brute-force L2 search, no quantization, 384 dimensions to match `all-MiniLM-L6-v2` output.

Functions

`extract_full_pages`

```
def extract_full_pages(pdf_path: str) -> list[str]
```

Parameters:

- `pdf_path` — absolute path to a `.pdf` file.

Returns: A list of stripped page-text strings, one element per page that contains non-whitespace text. Empty or whitespace-only pages are skipped. Pages that throw an extraction error are silently skipped; books that fail to open are silently skipped.

Behavior:

1. Opens the PDF with `fitz.open(pdf_path)`.
2. Iterates over every page index `page_number` from 0 to `len(doc) - 1`.
3. Calls `page.get_text("text")` in plain-text mode.
4. If the result is non-empty after `.strip()`, appends to the return list.
5. Per-page and per-book exceptions are caught and printed; the loop continues.

Files read: one PDF file per call. **Files written:** none.

`embed_texts`

```
def embed_texts(texts: List[str]) -> np.ndarray
```

Parameters:

- `texts` — a list of strings to embed (typically a batch of 10 pages).

Returns: A `np.ndarray` of shape `(len(texts), 384)` and dtype `float32`. On error, returns an empty array of shape `(0, 384)`.

Behavior:

1. Calls `model.encode(texts, convert_to_numpy=True)` where `model` is the module-level `SentenceTransformer("all-MiniLM-L6-v2")` instance (line 19).
2. Casts result to `float32` via `np.asarray(..., dtype="float32")`.
3. On any exception, prints the error and returns the empty fallback array.

Files read/written: none.

`main`

```
def main() -> None
```

Parameters: none.

Returns: none (side-effect: writes two files).

Behavior:

1. Creates a `faiss.IndexFlatL2(384)` index and an empty `metadata` list.
2. Walks `ECON_BOOK_DIR` recursively with `os.walk`, collecting all `.pdf` paths (case-insensitive suffix check).
3. For each PDF, derives a `title` from the filename stem (`os.path.splitext(os.path.basename(...))[0]`).
4. Calls `extract_full_pages` to get all page texts.
5. Processes pages in **batches of 10** (`range(0, len(pages), 10)`).
6. Calls `embed_texts(batch)` on each batch.
7. Validates: the result must be a 2-D `np.ndarray` with at least one row.
8. Adds valid embeddings to the FAISS index with `index.add(embeddings)`.
9. For each page in the batch, appends a metadata entry:

```
{"title": "<stem>", "page_index": <i+j>, "text": "<page_text>"}
```

- 1. Writes the FAISS index to `INDEX_OUTPUT` with `faiss.write_index`.
- 2. Writes all metadata to `METADATA_OUTPUT` as pretty-printed JSON.

Files read: all `.pdf` files under `ECON_BOOK_DIR`. **Files written:**

- `Economic_Books/FAISS_Store/economic_knowledge_index.faiss`
- `Economic_Books/FAISS_Store/economic_knowledge_metadata.json`

2. `GPT_Economy/Reasoning_Report.py`

Role and Phase

Phase 6 — Economic Reasoning Engine (query and synthesis step). Reads pre-computed sector summaries, loads the FAISS index built by `economic_engine.py`, selects the most relevant textbook modules using keyword classification, runs a LangChain `RetrievalQA` chain backed by GPT-4o, and writes a JSON report containing a plain-English economic analysis plus bullish/bearish sector lists.

External Libraries

Library	Import	Purpose
<code>faiss</code>	<code>import faiss</code>	Reading the persisted FAISS index
<code>numpy</code>	<code>import numpy as np</code>	Array handling
<code>python-dotenv</code>	<code>from dotenv import load_dotenv</code>	<code>.env</code> API key loading
<code>langchain_community</code>	<code>FAISS as LCFAISS</code> , <code>InMemoryDocstore</code>	LangChain vector store wrapper
<code>langchain</code>	<code>Document</code> , <code>RetrievalQA</code>	Document schema and QA chain
<code>langchain_openai</code>	<code>OpenAIEmbeddings</code> , <code>ChatOpenAI</code>	OpenAI embedding + chat model integration
<code>os</code> , <code>json</code> , <code>pathlib</code>	<code>stdlib</code>	File I/O

Path Constants

```
BASE_DIR = <repo_root>
SECTOR_SUMMARY_PATH = <BASE_DIR>/news_output/sector_summaries.json (read)
FAISS_INDEX_PATH = <BASE_DIR>/Economic_Books/FAISS_Store (read)
FAISS_INDEX_FILE = "economic_knowledge_index.faiss"
FAISS_METADATA_FILE = "economic_knowledge_metadata.json"
OUTPUT_PATH = <BASE_DIR>/news_output/economic_reasoning_summary.json (written)
```

Book Catalogue

`BOOK_METADATA` (lines 37-53) is a list of 15 dicts mapping book title stems to one of three modules:

- `"Economics Fundamentals (Macro & Micro)"` — Keynes, Friedman, Piketty, Dalio, Minsky, Kindleberger
- `"Behavioral, Fiscal, and Modern Policy"` — Kahneman, Shiller, Thaler, Krugman
- `"Investing and Market Behavior"` — Graham, Taleb, Malkiel, Lewis, Soros

Module Keyword Mapping

`MODULE_KEYWORDS` (lines 55-72) maps each module name to a keyword list used for automatic module selection:

```
"Economics Fundamentals (Macro & Micro)": [
    "inflation", "interest rate", "interest rates", "gdp", "monetary", "fiscal",
    "unemployment", "employment", "job market", "labor market", "supply", "demand",
    "aggregate", "productivity", "price level", "recession", "growth", "trade balance",
    "rate hike", "cpi", "federal reserve", "central bank",
]
"Behavioral, Fiscal, and Modern Policy": [
    "sentiment", "bias", "expectations", "behavioral", "consumer confidence",
    "fiscal deficit", "budget deficit", "gov spending", "stimulus", "fiscal policy",
    "irrational", "nudge", "modern monetary theory", "mmt", "confidence", "psychology",
]
"Investing and Market Behavior": [
    "stock", "stocks", "bond", "bonds", "portfolio", "volatility", "dividend",
    "etf", "yield", "valuation", "capital", "market", "hedge", "asset", "beta",
    "alpha", "mutual fund", "price to earnings", "pe ratio", "equity", "equities",
```

```
"securities", "market cap",
]
```

Functions

`_title_to_module_map`

```
def _title_to_module_map() -> dict[str, str]
```

Returns: A dict mapping lowercased, stripped book title stems to their module name, derived from `BOOK_METADATA`.

Behavior: Pure computation over the module-level constant list. No I/O.

`classify_modules`

```
def classify_modules(text: str) -> list[str]
```

Parameters:

- `text` — any string (typically the concatenation of all sector summaries).

Returns: A list of module name strings (0–3 elements) for which at least one keyword appears in `text.lower()`. The order mirrors `MODULE_KEYWORDS` iteration order.

Behavior: Simple substrng membership test; no regex or NLP. All three modules can be returned simultaneously.

`load_sector_summaries`

```
def load_sector_summaries() -> dict
```

Returns: The parsed JSON object from `SECTOR_SUMMARY_PATH`. Expected structure: `{sector_name: {"sector_summary": str, "articles_count": int}}`.

Files read: `news_output/sector_summaries.json`

`get_multi_module_retriever`

```
def get_multi_module_retriever(modules: list[str]) -> langchain VectorStoreRetriever
```

Parameters:

- `modules` — list of module name strings (output of `classify_modules`).

Returns: A LangChain retriever configured with `k=6` (return 6 documents per query).

Behavior:

1. Instantiates `OpenAIEmbeddings(model="text-embedding-ada-002")`.
2. Reads the FAISS index from disk with `faiss.read_index(str(faiss_file))`.
3. Reads metadata JSON and enriches each entry with its `module` field using `_title_to_module_map()`.
4. Wraps metadata entries as `langchain.schema.Document` objects (`page_content=m["text"]`, `metadata=m`).
5. Builds an `InMemoryDocstore` keyed by string integer index.
6. If `modules` is non-empty: filters `documents` to only those whose `metadata["module"]` is in `modules`. If filtering yields an empty set, falls back to all documents. Builds a new `LCFAISS` vector store from filtered docs using `LCFAISS.from_documents(filtered, embedding_model)`.
7. If `modules` is empty: constructs a full `LCFAISS(embedding_model, index, docstore, {})` directly from the pre-built FAISS index.
8. Returns `.as_retriever(search_kwargs={"k": 6})`.

Files read:

- `Economic_Books/FAISS_Store/economic_knowledge_index.faiss`
- `Economic_Books/FAISS_Store/economic_knowledge_metadata.json`

`run_reasoning_on_summaries`

```
def run_reasoning_on_summaries() -> str
```

Returns: The GPT-4o response string (the economic analysis report, up to ~800 words).

Behavior:

1. Calls `load_sector_summaries()` and concatenates all `sector_summary` values with double newlines: `"\n\n".join(v["sector_summary"] for v in summaries.values())`.
2. Calls `classify_modules(all_text)` to identify relevant knowledge modules.
3. Calls `get_multi_module_retriever(matched_modules)`.
4. Constructs `ChatOpenAI(model="gpt-4o", temperature=0.3)`.
5. Builds a `RetrievalQA.from_chain_type(llm=llm, retriever=retriever, return_source_documents=False)`.
6. Invokes the chain with the full prompt (see below).
7. Returns `result["result"]` if the response is a dict with a `"result"` key, otherwise `str(result)`.

GPT-4o Prompt Template (verbatim, lines 139-164):

```
You are an experienced financial advisor and economist trained in classical economic principles.
Based only on the provided economic textbook material, read the following sector summaries from different parts of the
market.

Create a summarized and unified explanation of what is happening in the economy right now.
Make it as detailed as possible, but less than 800 words, using the sector summaries provided.

Write clearly in plain English so the average person can understand. Focus on:
- What trends are emerging?
- If there is any political or war news, how it could affect the economy?
- What sectors are being affected, and how?
- Why are these things happening (the economic causes)?
- What are the consequences so far, and what might happen next?
- Which sectors may be positively or negatively affected by current events, and which are unaffected?
- What stocks would be good to buy right now, and which ones to avoid?

At the very end, list the bullish and bearish sectors in this exact format:

Bullish: [comma-separated sectors]
Bearish: [comma-separated sectors]

SECTOR DATA:
---
{all_text}
---
```

Model/settings: `gpt-4o`, temperature `0.3`, `return_source_documents=False`.

extract_bull_bear

```
def extract_bull_bear(summary_text: str) -> tuple[list[str], list[str]]
```

Parameters:

- `summary_text` — the full GPT output string.

Returns: A tuple (`bullish_sectors`, `bearish_sectors`) where each element is a list of title-cased sector name strings.

Behavior:

1. Takes the last 5 lines of `summary_text.strip().splitlines()`.
2. For each line, checks if it starts with `"bullish:"` (case-insensitive) or `"bearish:"` (case-insensitive).
3. Splits on `:` once, then splits the right side on `,`, strips whitespace, and applies `.title()` to each token.
4. Returns (`bullish`, `bearish`).

main

```
def main() -> None
```

Behavior:

- 1. Guards on `OPENAI_API_KEY` environment variable; raises `RuntimeError` if absent.
- 2. Calls `run_reasoning_on_summaries()` to get the report.
- 3. Calls `extract_bull_bear(report)`.
- 4. Creates `news_output/` if not present.
- 5. Writes JSON to `OUTPUT_PATH`:

```
{
  "summary": "<800-word report>",
  "bullish_sectors": ["Financials", ...],
  "bearish_sectors": ["Energy", ...]
}
```

Files written: `news_output/economic_reasoning_summary.json`

3. Profile/article_clustering.py

Role and Phase

Phase 5 — GPT Article Summarization & Validation. Despite the filename (`article_clustering.py`), this script is the GPT summarization and fact-retention validation layer. It reads clustered article data, calls GPT-4o to summarize each article in 3–5 data-rich sentences, validates that key numeric figures survived the summarization, assigns GICS sectors via keyword matching, and writes two output files: a full summarized article list and an updated `sector_summaries.json`.

Note: Phase 4 article clustering (KMeans/Agglomerative) is not present in this file. This script begins downstream of the clustering output and performs Phase 5 GPT summarization.

External Libraries

Library	Import	Purpose
<code>openai</code>	<code>from openai import OpenAI</code>	GPT-4o API calls
<code>python-dotenv</code>	<code>from dotenv import load_dotenv</code>	<code>.env</code> API key loading
<code>re</code>	<code>stdlib</code>	Regex patterns for numeric retention check
<code>json</code> , <code>os</code> , <code>pathlib</code>	<code>stdlib</code>	File I/O

Regex Patterns

```
PERCENT_PAT = re.compile(r"\b\d+(?:\.\d+)?%\b")
DOLLAR_PAT = re.compile(r"\$\d+(?:,\d{3})*(?:\.\d{2})?")
DATE_PAT = re.compile(
    r"\b(?:\d{1,2}[/-])?\d{1,2}[/-]\d{2,4}\b"
    r"|\b(?:Jan|Feb|Mar|Apr|May|Jun|Jul|Aug|Sep|Oct|Nov|Dec) [a-z]*\s+\d{1,2},?\s+\d{2,4}\b"
)
```

These three patterns are used in `validate_retention` to count numeric facts before and after summarization.

GICS Sector Definitions

```
GICS_SECTORS = [
    "Communication Services", "Consumer Discretionary", "Consumer Staples",
    "Energy", "Financials", "Health Care", "Industrials",
    "Information Technology", "Materials", "Real Estate", "Utilities",
]
```

Sector Keyword Map (used in `assign_sector`)

```
{
  "Information Technology": ["tech", "software", "semiconductor", "ai", "cloud", "chip"],
  "Health Care":           ["health", "pharma", "drug", "biotech", "hospital", "fda", "medical"],
  "Financials":            ["bank", "finance", "interest rate", "fed", "mortgage", "insurance"],
  "Energy":                ["oil", "gas", "energy", "opec", "fuel", "pipeline", "renewable"],
  "Consumer Discretionary": ["retail", "auto", "consumer", "amazon", "tesla", "housing"],
  "Consumer Staples":      ["food", "grocery", "staples", "walmart", "costco"],
}
```

```

"Industrials":      ["industrial", "manufacturing", "defense", "aerospace", "logistics"],
"Materials":       ["material", "mining", "steel", "copper", "aluminum", "commodity"],
"Real Estate":     ["real estate", "reit", "property", "housing market", "rent"],
"Utilities":       ["utility", "electric", "water", "power grid"],
"Communication Services": ["media", "telecom", "streaming", "social media", "google", "meta"],
}

```

Falls back to **"General"** if no keyword matches.

Functions

assign_sector

```
def assign_sector(text: str) -> str
```

Parameters:

- **text** — raw article text (title + summary concatenated).

Returns: A GICS sector name string, or **"General"** if no keyword matches.

Behavior: Lowercases **text**, iterates the sector keyword map in definition order. Returns the first matching sector. Does not handle multiple-sector assignment.

summarize_article

```
def summarize_article(client: OpenAI, model: str, article: dict) -> str
```

Parameters:

- **client** — an initialized **OpenAI** client instance.
- **model** — the model name string (hardcoded as **"gpt-4o"** at call site in **main**).
- **article** — a dict with keys **"title"** (or **"headline"**), **"summary"** (or **"content"**).

Returns: A stripped GPT response string (3–5 sentences). On API failure, returns up to the first 500 characters of the original **summary**.

GPT Prompt Template (verbatim, lines 70-76):

```

Summarize the following financial news article.
Include all quantitative data (percentages, dollar amounts, dates).
Keep it concise but data-rich (3-5 sentences):

---
{full_text}
---

```

Model/settings: **model** parameter (caller passes **"gpt-4o"**), **temperature=0.3**.

validate_retention

```
def validate_retention(original: str, summary: str) -> bool
```

Parameters:

- **original** — the source text (title + summary joined by space).
- **summary** — the GPT-produced summary.

Returns: **True** if the summary retains at least half the numeric values found in the original, or if the original contains no numeric values. **False** otherwise.

Retention formula:

```

orig_vals = PERCENT_PAT.findall(original)
            + DOLLAR_PAT.findall(original)
            + DATE_PAT.findall(original)

summ_vals = PERCENT_PAT.findall(summary)
            + DOLLAR_PAT.findall(summary)
            + DATE_PAT.findall(summary)

```

```
passes = len(summ_vals) >= len(orig_vals) // 2
```

If `orig_vals` is empty, returns `True` unconditionally. Note: this uses integer floor division (`// 2`), so 1 value retained out of 2 original passes (`0 >= 1` → fails; but `1 >= 1` passes).

main

```
def main() -> None
```

Behavior:

1. Reads `OPENAI_API_KEY` from environment; raises `RuntimeError` if absent.
2. Reads `clustered_summaries.json` — raises `FileNotFoundError` if missing.
3. Normalizes the raw JSON to a flat list of article dicts (handles both `list` and `dict` input shapes).
4. Initializes `OpenAI(api_key=api_key)` and sets `model = "gpt-4o"`.
5. For each article:
 - Calls `assign_sector(title + " " + summary)` if no `sector` key is already present.
 - Calls `summarize_article(client, model, article)`.
 - Calls `validate_retention(full_text, gpt_summary)`. If it fails, prints a warning and falls back to `summary[:500]` (or the GPT result if non-empty).
 - Builds `enriched` dict by spreading all original article fields plus `"title"`, `"sector"`, `"gpt_summary"`.
 - Adds to `summarized` list and to `sector_buckets[sector]`.
6. Writes `clustered_articles_with_summaries.json`:

```
{"clusters": [{"cluster_id": 0, "top_articles": [<enriched article dicts>]]}
```

1. Reads existing `sector_summaries.json` if present (to avoid overwriting sectors already summarized).
2. For each new sector bucket: joins all `gpt_summary` strings, truncates to 2000 characters, and writes to `sector_summaries.json` only if the sector key is not already present.

Files read:

- `news_output/clustered_summaries.json`
- `news_output/sector_summaries.json` (if exists, for merge)

Files written:

- `news_output/clustered_articles_with_summaries.json`
- `news_output/sector_summaries.json` (merged update)

4. Profile/GPT_article_clustering_summary.py

Role and Phase

Phase 4 (sector-level GPT summarization) — runs after Phase 4 clustering. Takes the `clustered_summaries.json` (output of article clustering), groups articles by GICS sector (`gics_sector` field), and runs a two-pass GPT summarization pipeline: first chunking articles into intermediate 4-sentence summaries, then combining intermediates into a single 12+ sentence sector overview with numeric retention validation. Writes `sector_summaries.json`.

External Libraries

Library	Import	Purpose
<code>openai</code>	<code>from openai import OpenAI</code>	Chat completion API
<code>python-dotenv</code>	<code>from dotenv import load_dotenv</code>	<code>.env</code> loading
<code>re</code>	<code>stdlib</code>	Numeric retention regex
<code>itertools</code>	<code>from itertools import islice</code>	Chunking helper
<code>os, json</code>	<code>stdlib</code>	File I/O

Configuration Constants

```
BASE_DIR    = <repo_root>
NEWS_DIR    = <BASE_DIR>/news_output
INPUT_FILE  = <NEWS_DIR>/clustered_summaries.json    (read)
OUTPUT_FILE = <NEWS_DIR>/sector_summaries.json      (written)

API_KEY     = os.getenv("OPENAI_API_KEY")
MODEL       = os.getenv("MODEL", "gpt-3.5-turbo")   # default: gpt-3.5-turbo
TEMPERATURE = float(os.getenv("TEMPERATURE", 0.3))
CHUNK_SIZE  = int(os.getenv("CHUNK_SIZE", 50))      # max articles per chunk
```

Regex Patterns (Retention Check)

```
percent_re = re.compile(r"\b\d+(?:\.\d+)?%\b")
dollar_re  = re.compile(r"\$\d{1,3}(?:,\d{3})*(?:\.\d+)?")
```

GPT Prompt Templates (verbatim, lines 32-47)

Phase 1 — Chunk summarization:

System:

You are a professional financial analyst. Summarize the following article summaries into exactly 4 sentences, including every percentage and dollar figure.

User:

Chunk of {n} article summaries:
{entries}
Please summarize into 4 sentences, preserving all numeric details.

Phase 2 — Final sector overview:

System:

You are a professional financial analyst. Combine the intermediate summaries into a comprehensive sector overview. Your response must be at least 12 sentences long and include all percentages and dollar figures.

User:

Combine {n} intermediate summaries for the {sector} sector into a cohesive overview of at least 12 sentences, preserving all numeric data:
{chunks}

Functions

chunked

```
def chunked(seq, size) -> Generator[list, None, None]
```

Parameters:

- `seq` — any iterable.
- `size` — chunk size (int).

Returns: Generator yielding lists of up to `size` elements.

Behavior: Uses `itertools.islice` to consume the iterator in fixed-size windows. Each yielded list is `[first] + list(islice(it, size-1))`.

Module-Level Pipeline (not wrapped in a `main` function)

The summarization pipeline runs at module import time (no `if __name__ == "__main__"` guard beyond the execution context). The sequence is:

1. **Load input:** Opens `INPUT_FILE`. Normalizes to a dict `clusters` keyed by cluster label. If the raw JSON is a list, wraps it as `{'0': raw}`.
2. **Group by sector:** Iterates all articles; extracts `gics_sector` (default `'Unknown'`) and builds `sector_articles: dict[str, list[str]]`. Each article becomes the string `"- {summary} (URL: {url})"`.
3. **Per-sector two-pass summarization:**
 - **Pass 1 (chunking):** Splits the article list into chunks of `CHUNK_SIZE` (default 50). For each chunk, calls the OpenAI chat API (MODEL, TEMPERATURE, chunk prompt). Collects responses as `intermediate: list[str]`.
 - **Pass 2 (final overview):** Joins intermediates as `"- {s}"` lines and calls the final-overview prompt. Collects `final_text`.
 - **Retention check:** Counts `orig_vals = percent_re.findall(chunks_joined) + dollar_re.findall(chunks_joined)` and `found_vals` in `final_text`. Prints a warning if `len(found_vals) < len(orig_vals) * 0.8` (i.e., less than 80% of numeric tokens retained).
 - Stores `output[sector] = {"sector_summary": final_text.strip(), "articles_count": len(lines)}`.
4. **Save:** Writes `OUTPUT_FILE` as pretty-printed JSON.

Retention threshold formula:

```
warn if len(found_vals) < len(orig_vals) * 0.8
```

(80% numeric retention threshold; only a warning, not a regeneration trigger.)

Files read: `news_output/clustered_summaries.json` **Files written:** `news_output/sector_summaries.json`

5. [Module_2_Technical_Analysis/ta_analysis.py](#)

Role and Phase

Phase 5/6 bridge — Technical Analysis Engine. Downloads 1-year daily OHLCV data for all tickers in bullish and bearish sectors (from the economic reasoning output), computes four technical indicators (MACD, RSI, SMA50, SMA200) using either `talib` (preferred) or the `ta` library fallback, derives three binary signal flags, and saves a unified CSV for downstream use by `phase_3_signal.py`, `prepare_qlib_data.py`, and `phase_4_backtrader.py`.

External Libraries

Library	Import	Purpose
<code>yfinance</code>	<code>import yfinance as yf</code>	OHLCV data download
<code>pandas</code>	<code>import pandas as pd</code>	DataFrame operations
<code>numpy</code>	<code>import numpy as np</code>	Array operations
<code>talib</code>	<code>import talib as _talib</code>	TA-Lib C-extension indicators (optional)
<code>ta</code>	<code>import ta as _ta</code>	Pure-Python TA fallback
<code>tqdm</code>	<code>from tqdm import tqdm</code>	Progress bar
<code>concurrent.futures</code>	<code>ThreadPoolExecutor, as_completed</code>	Parallel ticker downloads (5 workers)
<code>json, os, warnings, pathlib</code>	<code>stdlib</code>	I/O and suppression of FutureWarnings

Configuration Constants

```
LOOKBACK_DAYS = 365
BASE_DIR      = <repo_root>
OUTPUT_DIR    = Module_2_Technical_Analysis/
RAW_DATA_DIR  = <BASE_DIR>/raw_data/
MAPPING_PATH  = <BASE_DIR>/Profile/flat-ui_data-Sun Jun 15 2025.csv
REASONING_PATH = <BASE_DIR>/news_output/economic_reasoning_summary.json
OUTPUT_CSV    = Module_2_Technical_Analysis/ta_analysis_detailed.csv
```

Indicator Parameters

Indicator	Period / Setting
MACD (fast)	12

Indicator	Period / Setting
MACD (slow)	26
MACD (signal)	9
RSI	14
SMA (short)	50
SMA (long)	200

Signal Derivation Rules

```

rsi_signal = -1 if RSI > 70 (overbought)
            = 1 if RSI < 30 (oversold)
            = 0 otherwise
macd_signal = 1 if MACD line > MACD signal line (bullish)
            = -1 if MACD line <= MACD signal line (bearish)
trend_signal = 1 if SMA50 > SMA200 (golden cross / uptrend)
            = -1 if SMA50 < SMA200 (death cross / downtrend)

```

Functions

load_sector_mapping

```
def load_sector_mapping() -> pd.DataFrame
```

Returns: A DataFrame with at minimum columns `ticker` (str) and `sector` (str). Ticker symbols have `.` replaced with `-` for yfinance compatibility (e.g., `BRK.B` → `BRK-B`).

Files read: `Profile/flat-ui_data-Sun Jun 15 2025.csv` (S&P 500 constituent list with GICS sectors).

load_bullish_bearish

```
def load_bullish_bearish() -> tuple[list[str], list[str]]
```

Returns: A tuple `(bullish_sectors, bearish_sectors)`. Returns `([], [])` on any error or if `REASONING_PATH` does not exist.

Files read: `news_output/economic_reasoning_summary.json` (output of `Reasoning_Report.py`).

save_raw_csv

```
def save_raw_csv(ticker: str, df_raw: pd.DataFrame) -> None
```

Parameters:

- `ticker` — ticker symbol string.
- `df_raw` — raw OHLCV DataFrame from yfinance.

Behavior: Creates `RAW_DATA_DIR` if needed, then saves `df_raw` to `<RAW_DATA_DIR>/<ticker>.csv` with index.

Files written: `raw_data/<ticker>.csv`

compute_indicators

```
def compute_indicators(df_close: pd.Series) -> pd.DataFrame
```

Parameters:

- `df_close` — a pandas Series of closing prices, indexed by date.

Returns: A DataFrame indexed identically to `df_close`, containing columns:

- `macd` — MACD line value (float)
- `macd_signal_line` — MACD signal line value (float)
- `rsi` — RSI value (float, 0–100)

- `sma_50` — 50-day simple moving average (float)
- `sma_200` — 200-day simple moving average (float)
- `rsi_signal` — integer signal flag (-1, 0, 1)
- `macd_signal` — integer signal flag (1 or -1)
- `trend_signal` — integer signal flag (1 or -1)

Behavior (TA-Lib path, `_TALIB=True`):

```
close = np.ascontiguousarray(s.values)
macd_line, macd_signal_line, _ = _talib.MACD(close)      # default: 12/26/9
rsi_arr = _talib.RSI(close)                             # default: 14
sma_50_arr = _talib.SMA(close, timeperiod=50)
sma_200_arr = _talib.SMA(close, timeperiod=200)
```

Behavior (ta fallback, `_TALIB=False`):

```
macd_obj = _ta.trend.MACD(s)                            # default: fast=12, slow=26, signal=9
macd = macd_obj.macd().values
macd_sig = macd_obj.macd_signal().values
rsi = _ta.momentum.RSIIndicator(s).rsi().values         # default window: 14
sma_50 = _ta.trend.SMAIndicator(s, window=50).sma_indicator().values
sma_200 = _ta.trend.SMAIndicator(s, window=200).sma_indicator().values
```

Signal rules applied via `numpy.where` (vectorized, lines 92-94):

```
rsi_signal = np.where(rsi > 70, -1, np.where(rsi < 30, 1, 0))
macd_signal = np.where(macd > macd_signal_line, 1, -1)
trend_signal = np.where(sma_50 > sma_200, 1, -1)
```

run_ta_analysis

```
def run_ta_analysis(ticker: str) -> pd.DataFrame | None
```

Parameters:

- `ticker` — a ticker symbol string.

Returns: A DataFrame of the last `LOOKBACK_DAYS` (365) rows of indicators with a prepended `ticker` column, or `None` on any error.

Behavior:

1. Downloads 1 year of daily OHLCV with `yf.download(ticker, period="1y", interval="1d", progress=False)`.
2. If the result has a `pd.MultiIndex` column (multiple tickers in one download), extracts the `"Close"` slice with `df.xs("Close", axis=1, level=0)`.
3. Drops NaN close prices.
4. Calls `compute_indicators(df_close)`.
5. Takes `df_ind.tail(LOOKBACK_DAYS)`.
6. Inserts `ticker` as the first column.
7. Saves raw OHLCV to `raw_data/<ticker>.csv` via `save_raw_csv`.
8. Returns the trimmed indicator DataFrame. Returns `None` on any exception.

main

```
def main() -> None
```

Behavior:

1. Loads the sector mapping CSV and the bullish/bearish sector lists.
2. If either sector list is non-empty, filters the S&P 500 constituent list to tickers in those sectors only. Otherwise processes all S&P 500 tickers.
3. Runs `run_ta_analysis` concurrently using `ThreadPoolExecutor(max_workers=5)`.
4. Concatenates all non-None results, drops all-NaN rows, resets the index, and renames `"Date"` → `"date"` for downstream compatibility.
5. Writes the combined DataFrame to `OUTPUT_CSV`.

Files read:

- `Profile/flat-ui_data-Sun Jun 15 2025.csv`
- `news_output/economic_reasoning_summary.json` (if exists)

Files written:

- `Module_2_Technical_Analysis/ta_analysis_detailed.csv`
- `raw_data/<ticker>.csv` for each successfully downloaded ticker

6. `Module_2_Technical_Analysis/phase_2_q_lib.py`

Role and Phase

Qlib feature export — bridge between legacy `ta_analysis_results.json` and Qlib's CSV format. This is an early-stage utility script (not integrated with the newer `ta_analysis_detailed.csv` flow). It reads a now-superseded `ta_analysis_results.json`, converts signal strings to integers, generates a single-day synthetic datetime row per ticker, and writes a Qlib-compatible CSV.

Note: This script uses hardcoded absolute paths and processes only the latest day's data (`dates[-1:]`). It is a proof-of-concept bridge, predating the `prepare_qlib_data.py` + `pipeline_alpha.py` flow.

External Libraries

Library	Import	Purpose
<code>pandas</code>	<code>import pandas as pd</code>	DataFrame operations
<code>json</code>	<code>stdlib</code>	Reading TA results
<code>os</code>	<code>stdlib</code>	Directory creation
<code>datetime</code> , <code>timedelta</code>	<code>stdlib</code>	Synthetic date generation

Module-Level Pipeline (no functions defined)

1. **Load:** Reads `/Users/allanaziz/Desktop/ThinkFree/ThinkFree-main/news_output/ta_analysis_results.json` (hardcoded path).
2. **Combine:** Concatenates `ta_data["bullish_analysis"]` and `ta_data["bearish_analysis"]` .
3. **Date range:** Generates 365 dates going back from today; uses only the last date (`dates[-1:]`).
4. **Row construction:** For each stock in `combined` , creates one row:

```
{
  "datetime": date.strftime("%Y-%m-%d"),
  "instrument": ticker.upper(),
  "rsi": stock["rsi"],
  "macd": stock["macd"],
  "sma_50": stock["sma_50"],
  "sma_200": stock["sma_200"],
  "close": stock["close"],
  "rsi_signal": 1 if "oversold" else -1 if "overbought" else 0,
  "macd_signal": 1 if "bullish" else -1,
  "trend_signal": 1 if "bullish" else -1,
  "label": None,
}
```

1. **Save:** Writes `qlib_data/qlib_feature_data.csv` (no index).

Files read: `news_output/ta_analysis_results.json` **Files written:** `qlib_data/qlib_feature_data.csv`

7. `Module_2_Technical_Analysis/phase_3_signal.py`

Role and Phase

Phase 3b — Multi-indicator signal generation and voting. Reads `ta_analysis_detailed.csv` , takes the most recent row per ticker (representing the current signal state), applies a weighted voting logic across RSI, MACD, SMA golden/death cross, OBV, ADX, and CCI, and outputs a `signal_output_phase3.json` with per-ticker BUY/SELL/HOLD verdicts and confidence scores.

External Libraries

Library	Import	Purpose
<code>pandas</code>	<code>import pandas as pd</code>	CSV reading and DataFrame operations
<code>json</code> , <code>os</code> , <code>pathlib</code>	<code>stdlib</code>	File I/O

Configuration Constants

```
LOOKBACK_DAYS = 5          # Not used in current logic (groupby tail(1) used instead)
TA_CSV_PATH   = Module_2_Technical_Analysis/ta_analysis_detailed.csv
OUTPUT_PATH   = Module_2_Technical_Analysis/signal_output_phase3.json
```

Signal Voting Algorithm

The `confidence` score is an integer that increments/decrements per indicator:

Indicator	Condition	Effect
RSI	< 30 (oversold)	confidence += 1, added to support
RSI	> 70 (overbought)	confidence -= 1, added to support
RSI	30–70	added to contradiction
MACD vs Signal	MACD > signal line	confidence += 1 ("bullish crossover")
MACD vs Signal	MACD < signal line	confidence -= 1 ("bearish crossover")
MACD vs Signal	equal	added to contradiction
SMA50 vs SMA200	SMA50 > SMA200	confidence += 1 ("golden cross")
SMA50 vs SMA200	SMA50 < SMA200	confidence -= 1 ("death cross")
OBV	> 0	confidence += 1
OBV	<= 0	added to contradiction
ADX	< 20	added to contradiction ("weak trend")
ADX	>= 20	added to support
CCI	abs(CCI) < 100	added to contradiction ("neutral zone")
CCI	abs(CCI) >= 100	added to support

Final signal rule:

```
if confidence >= 2: signal = "BUY"
elif confidence <= -2: signal = "SELL"
else:
    signal = "HOLD"
```

Note: OBV, ADX, and CCI are handled if present in the CSV but are not computed by `ta_analysis.py`. They would be `NaN` in the standard pipeline, so their contributions default to contradiction or are skipped.

Functions

`analyze_row`

```
def analyze_row(row: pd.Series) -> dict
```

Parameters:

- `row` — a single row from the TA CSV as a pandas Series, expected to contain: `rsi`, `macd`, `macd_signal_line`, `sma_50`, `sma_200`, `obv`, `adx`, `cci`, `date`, `ticker`.

Returns: A dict:

```
{
  "date": "YYYY-MM-DD",
  "ticker": "AAPL",
```

```

"final_signal": "BUY" | "SELL" | "HOLD",
"confidence_score": -3..+3,
"supporting_indicators": ["RSI (oversold at 28.3)", "MACD bullish crossover"],
"contradicting_indicators": ["CCI in neutral zone"],
"reasoning": "Indicators suggest a bullish setup. Supporting: ... Conflicting: ..."
}

```

Behavior: Implements the voting table documented above. NaN values are handled via `pd.isna()` checks before each indicator block. The `macd_signal_line` column name is used (not `macd_signal`) to avoid confusion with the binary flag.

main

```
def main() -> None
```

Behavior:

1. Checks `TA_CSV_PATH` existence; prints error and returns if absent.
2. Reads CSV; if no `"date"` column, renames the first column to `"date"`.
3. Sorts by `["ticker", "date"]`; takes the most recent row per ticker with `groupby("ticker").tail(1)`.
4. Applies `analyze_row` to each row via list comprehension.
5. Writes the resulting list to `OUTPUT_PATH`.
6. Prints summary counts: BUY / SELL / HOLD totals.

Files read: `Module_2_Technical_Analysis/ta_analysis_detailed.csv` **Files written:** `Module_2_Technical_Analysis/signal_output_phase3.json`

8. Module_2_Technical_Analysis/pipeline_alpha.py

Role and Phase

Qlib LightGBM Alpha Signal Pipeline. Loads the Qlib-formatted data pickle produced by `prepare_qlib_data.py`, constructs a `DataHandlerLP` with `Fillna` and `DropnaLabel` processors, automatically splits dates into train/test segments (train to second-last date, test on last two dates), trains a LightGBM gradient-boosting model for 1000 rounds, generates next-day return predictions, and saves them to `predictions.csv`.

External Libraries

Library	Import	Purpose
<code>qlib</code>	<code>import qlib</code>	Quantitative research library initialization
<code>qlib.config</code>	<code>REG_CN</code>	Region configuration (China region used as structural template)
<code>qlib.data.dataset.loader</code>	<code>StaticDataLoader</code>	In-memory data loader
<code>qlib.data.dataset.handler</code>	<code>DataHandlerLP</code>	Feature processing pipeline
<code>qlib.data.dataset</code>	<code>DatasetH</code>	Segmented dataset
<code>qlib.contrib.model.gbdt</code>	<code>LGBModel</code>	LightGBM model
<code>pandas</code>	<code>import pandas as pd</code>	DataFrame handling
<code>os</code>	<code>stdlib</code>	Path construction

Configuration

```

provider_uri = "/Users/allanaziz/Desktop/ThinkFree/ThinkFree-main/qlib_data"
data_path    = <provider_uri>/formatted/data.pkl

```

Qlib is initialized with `REG_CN` (China region) for structural compatibility, not because the data is Chinese.

Functions

to_iso

```
def to_iso(val) -> str
```

Parameters:

- `val` — a datetime-like value.

Returns: ISO date string ("YYYY-MM-DD") if `val` has a `.date()` method, otherwise `str(val)`.

Behavior: Used to convert min/max datetime index values to handler date bounds.

Module-Level Pipeline (not wrapped in a function)

The entire pipeline executes at module level:

1. **Qlib init:** `qlib.init(provider_uri=provider_uri, region=REG_CN)`
2. **Load data:** `pd.read_pickle(data_path)` — expects a DataFrame with a `(datetime, instrument)` MultiIndex.
3. **DateTime dtype fix:** If the `datetime` index level is not already `datetime64`, converts it.
4. **MultiIndex column construction:** If the DataFrame columns are not already a MultiIndex with `feature` / `label` levels:
 - Wraps all existing columns under `feature` level.
 - Creates a next-day return label: `close.pct_change().shift(-1)` wrapped under the `label` level.
 - Concatenates feature and label DataFrames.
5. **DataHandlerLP construction:**
 - `infer_processors`: `[{"class": "Fillna", "kwargs": {"fields_group": "feature"}}]`
 - `learn_processors`: `[{"class": "DropnaLabel", "kwargs": {}}]`
 - `process_type`: `"append"`
6. **Auto-segment dates:**

```
train = (dates[0], dates[-2]) # all dates except last
test  = (dates[-2], dates[-1]) # second-to-last and last date
```

Fallback: if fewer than 3 unique dates, both segments use `dates[0]`.

1. **DatasetH creation:** `DatasetH(handler=handler, segments=segments)`
2. **LightGBM training:** `model = LGBModel(); model.fit(dataset)` — 1000 boosting rounds, L2 loss.
3. **Prediction:** `preds = model.predict(dataset)` — shape `(n_test_rows,)`, values are predicted next-day relative returns.
4. **Save:** `preds.to_csv(<provider_uri>/predictions.csv)`

Files read: `qlib_data/formatted/data.pkl` **Files written:** `qlib_data/predictions.csv`

9. Module_2_Technical_Analysis/prepare_qlib_data.py**Role and Phase**

Qlib data formatting — prerequisite for `pipeline_alpha.py`. Converts the TA analysis CSV (output of `ta_analysis.py`) into Qlib's required pickle format: a DataFrame with a `(datetime, instrument)` MultiIndex, sorted ascending.

External Libraries

Library	Import	Purpose
<code>pandas</code>	<code>import pandas as pd</code>	CSV reading, DataFrame transformation, pickle serialization
<code>os</code>	<code>stdlib</code>	Directory creation

Configuration Constants

```
CSV_PATH      = "/Users/allanaziz/Desktop/ThinkFree/ThinkFree-main/qlib_data/formatted/ta_analysis_detailed.csv"
QLIB_OUTPUT_DIR = "/Users/allanaziz/Desktop/ThinkFree/ThinkFree-main/qlib_data/formatted"
OUTPUT_FILE    = <QLIB_OUTPUT_DIR>/data.pkl
```

Note: The hardcoded CSV path points to a copy of `ta_analysis_detailed.csv` placed in `qlib_data/formatted/`, not the original output location (`Module_2_Technical_Analysis/ta_analysis_detailed.csv`). Manual copying is required between these runs.

Functions

convert_to_qlib_format

```
def convert_to_qlib_format() -> None
```

Parameters: none.

Returns: none (side-effect: writes a `.pkl` file).

Behavior:

1. Reads `CSV_PATH` with `pd.read_csv`.
2. Drops entirely empty rows with `df.dropna(how="all")`.
3. Ensures a date column exists (`"date"`, `"datetime"`, or `"Date"`). Raises `ValueError` if none found.
4. Renames `"ticker"` → `"instrument"` if present.
5. Renames the date column to `"datetime"`.
6. Converts `"datetime"` to `datetime64` with `pd.to_datetime`.
7. Sets the MultiIndex: `df.set_index(["datetime", "instrument"]).sort_index()`.
8. Creates `QLIB_OUTPUT_DIR` if needed.
9. Serializes with `df.to_pickle(OUTPUT_FILE)`.

Files read: `qlib_data/formatted/ta_analysis_detailed.csv` **Files written:** `qlib_data/formatted/data.pkl`

10. Module_2_Technical_Analysis/phase_4_backtrader.py

Role and Phase

Phase 9 — Backtesting Engine. The core simulation layer. Reads the TA signal CSV, downloads full OHLCV history for all tickers in the signal universe, recomputes RSI/MACD/SMA/ATR indicators for validation, feeds data into a Backtrader `Cerebro` instance, runs either the improved long-only strategy (`ImprovedLongOnlyStrategy`) or the legacy long/short strategy (`MultiTickerSignalStrategy`), computes a full suite of performance metrics (CAGR, Sharpe, Sortino, Calmar, max drawdown, win rate, profit factor, expectancy, time in market, average bars held), and writes equity curve, trade log, and summary metrics JSON.

The engine supports configuration via JSON file (`results_run/backtest_config.json`) that overrides CLI arguments, enabling reproducible runs without command-line juggling.

External Libraries

Library	Import	Purpose
<code>backtrader</code>	<code>import backtrader as bt</code>	Strategy execution engine
<code>yfinance</code>	<code>import yfinance as yf</code>	OHLCV history download
<code>pandas</code>	<code>import pandas as pd</code>	DataFrames and time series
<code>numpy</code>	<code>import numpy as np</code>	Numerical operations
<code>argparse</code> , <code>json</code> , <code>os</code> , <code>math</code> , <code>pathlib</code>	<code>stdlib</code>	Configuration, I/O, math
<code>dataclasses</code>	<code>@dataclass</code>	Typed configuration and metrics containers

Global Configuration Defaults

```

DEFAULT_INITIAL_CASH: float = 100_000.0
DEFAULT_MAX_POSITIONS: int = 10
DEFAULT_COMMISSION: float = 0.0005      # 5 basis points per side
DEFAULT_SLIPPAGE_PCT: float = 0.0005     # 5 basis points
DEFAULT_RISK_FREE: float = 0.0
MIN_BARS_WARMUP: int = 220               # bars required before SMA200 is valid

# Indicator periods
RSI_PERIOD: int = 14
MACD_FAST: int = 12
MACD_SLOW: int = 26
MACD_SIGNAL: int = 9
SMA_SHORT: int = 50

```

```
SMA_LONG: int      = 200
ATR_PERIOD: int    = 14

MAX_PORTFOLIO_DRAWDOWN: float = 0.35      # 35% portfolio-level hard stop
```

Column Alias Map

```
COLUMN_ALIASES = {
    'MACD': 'macd',
    'MACD_Signal': 'macd_signal',
    'macd_signal_line': 'macd_signal',
    'macd_sig': 'macd_signal_flag',
    'macd_signal': 'macd_signal_flag',
    'RSI': 'rsi',
    'SMA50': 'sma_50',
    'SMA200': 'sma_200',
    'rsi_sig': 'rsi_signal',
    'trend_sig': 'trend_signal',
    'Ticker': 'ticker',
    'SYMBOL': 'ticker',
    'symbol': 'ticker',
    'DATE': 'Date',
    'date': 'Date',
    'timestamp': 'Date',
}
```

Applied by `normalize_columns` to handle CSV column naming variants from different upstream sources.

Functions

ensure_dir

```
def ensure_dir(path: str) -> None
```

Creates `path` and all parents with `os.makedirs(path, exist_ok=True)`.

log

```
def log(msg: str) -> None
```

Prints `msg` with `flush=True` for real-time output in long-running backtest runs.

to_float_scalar

```
def to_float_scalar(x: Any, default: float = 0.0) -> float
```

Purpose: Safe coercion of Pandas/NumPy scalars and Series objects to Python `float`. Handles: `pd.Series` (takes first element), `np.generic` / `np.number` (via `.item()`), and anything else via `float(x)`. Returns `default` on any exception.

to_int_scalar

```
def to_int_scalar(x: Any, default: int = 0) -> int
```

Same as `to_float_scalar` but coerces to `int`. Used for signal values (-1, 0, 1) and position sizes.

normalize_columns

```
def normalize_columns(df: pd.DataFrame) -> pd.DataFrame
```

Parameters: A DataFrame with potentially variant column names.

Returns: The same DataFrame with columns renamed per `COLUMN_ALIASES`. All column names are coerced to `str`. Unrecognized columns are left unchanged.

load_signals

```
def load_signals(signals_csv: str) -> Dict[str, pd.DataFrame]
```

Parameters:

- **signals_csv** — absolute path to the signal CSV (e.g., **ta_analysis_detailed.csv**).

Returns: A dict mapping ticker symbols (str) to DatetimeIndex-indexed DataFrames of their signals. Each DataFrame has at minimum columns: **rsi_signal**, **macd_signal_flag**, **trend_signal** (defaulting to 0 if absent in source CSV).

Behavior:

1. Reads CSV; applies **normalize_columns**.
2. Validates presence of **"Date"** and **"ticker"** columns; raises **ValueError** if absent.
3. Converts **"Date"** to **datetime64**.
4. Ensures **rsi_signal**, **macd_signal_flag**, **trend_signal** columns exist (fills with 0 if missing).
5. Sorts by **["ticker", "Date"]**.
6. Groups by ticker; for each group, sets **"Date"** as index.
7. Returns **{ticker_str: group_df}**.

Files read: the signals CSV path argument.

ta_validate

```
def ta_validate(price_df: pd.DataFrame) -> pd.DataFrame
```

Parameters:

- **price_df** — a DataFrame with OHLCV columns (**Open**, **High**, **Low**, **Close**, **Volume**) indexed by date.

Returns: The same DataFrame with appended recalculation columns:

Column	Formula				
rsi_recalc	Wilder RSI(14) via EWM: $100 - 100 / (1 + RS)$, where $RS = \text{gain_ewm} / \text{loss_ewm}$, $\alpha=1/14$				
macd_recalc	$\text{EMA}(\text{close}, 12) - \text{EMA}(\text{close}, 26)$				
macd_signal_recalc	$\text{EMA}(\text{macd_recalc}, 9)$				
sma50_recalc	$\text{close.rolling}(50).mean()$				
sma200_recalc	$\text{close.rolling}(200).mean()$				
atr_recalc	Wilder ATR(14): $\text{EWM}(\text{TR}, \text{span}=14)$ where $\text{TR} = \max(\text{H-L},$	H-		L-	)`
		prevC	,	prevC	
rsi_signal_recalc	1 if rsi_recalc < 30, -1 if > 70, else 0				
macd_signal_flag_recalc	1 if macd_recalc > macd_signal_recalc , -1 if <, else 0				
trend_signal_recalc	1 if sma50 > sma200 , -1 if <, else 0				

RSI formula (Wilder EWM, lines 228-233):

```
delta = close.diff()
up = delta.clip(lower=0.0)
down = -delta.clip(upper=0.0)
gain = up.ewm(alpha=1/RSI_PERIOD, adjust=False).mean()
loss = down.ewm(alpha=1/RSI_PERIOD, adjust=False).mean()
rs = gain / loss.replace(0, np.nan)
rsi = 100 - (100 / (1 + rs))
```

ATR formula (lines 248-255):

```
tr = max(|High - Low|, |High - prev_Close|, |Low - prev_Close|)
atr = tr.ewm(span=ATR_PERIOD, adjust=False).mean()
```

Behavior notes: Handles MultiIndex columns defensively by flattening to level-0. Validates required OHLCV columns; raises `ValueError` if any are missing.

Class: `PandasDataAdj`

```
class PandasDataAdj(bt.feeds.PandasData)
```

A Backtrader data feed subclass that maps standard OHLCV column names to Backtrader line names:

```
params = (
    ('datetime', None),    # from DataFrame index
    ('open', 'Open'),
    ('high', 'High'),
    ('low', 'Low'),
    ('close', 'Close'),
    ('volume', 'Volume'),
    ('openinterest', -1), # not used
)
```

Dataclass: `StrategyConfig`

```
@dataclass
class StrategyConfig:
    max_positions: int = 10
    hard_dd_stop: float = 0.20 # 20% portfolio drawdown halt
    risk_free: float = 0.0
    min_align: int = 2 # 2 of 3 signals must agree
    use_recalc: bool = True # use recomputed indicators
    min_price: float = 5.0 # skip stocks below $5
    min_volume: int = 200_000 # skip stocks below 200k avg volume
    risk_pct: float = 0.015 # 1.5% of portfolio equity risked per trade
    atr_stop_mult: float = 2.0 # initial stop = entry - 2×ATR
    atr_trail_mult: float = 2.5 # trailing stop = peak - 2.5×ATR
    long_only: bool = True # no short selling
    trend_filter: bool = True # only enter above SMA200
```

Class: `ImprovedLongOnlyStrategy`

```
class ImprovedLongOnlyStrategy(bt.Strategy)
```

The primary production strategy. Long-only, ATR-risk-sized, with trailing stop and trend filter.

params:

- `signals_map` — `Dict[str, pd.DataFrame]` keyed by ticker.
- `config` — a `StrategyConfig` instance.
- `final_date` — the last date of the signal CSV (triggers forced liquidation).

State tracking:

- `equity_curve` — list of `(date, portfolio_value)` tuples.
- `days_in_market` — count of calendar days with at least one open position.
- `max_equity_seen` — running maximum portfolio value (for drawdown calculation).
- `hard_stop_triggered` — boolean; prevents re-entry after drawdown halt.
- `open_info` — dict of open position metadata keyed by ticker.
- `trades_log` — list of closed trade dicts.
- `_peak_since_entry` — dict of per-ticker highest close since entry (trailing stop reference).

`__init__`

Initializes all state. Sets `max_equity_seen` from `broker.getvalue()` at construction time.

`_today_signals`

```
def _today_signals(self, tkr: str, dt: pd.Timestamp) -> Tuple[int, int, int]
```

Returns: (`rsi_signal`, `macd_signal_flag`, `trend_signal`) for ticker `tkr` on date `dt`. Returns `(0, 0, 0)` if the ticker or date is not found. Prefers `*_recalc` columns when `use_recalc=True`.

`_get_atr`

```
def _get_atr(self, tkr: str, dt: pd.Timestamp) -> float
```

Returns: ATR value for `tkr` on `dt` from the `atr_recalc` column of `signals_map[tkr]`. Returns `0.0` if unavailable, non-finite, or zero.

`_get_sma200`

```
def _get_sma200(self, tkr: str, dt: pd.Timestamp) -> float
```

Returns: SMA200 value. Checks `sma200_recalc` first, then `sma_200`. Returns `0.0` if unavailable.

`_long_vote`

```
def _long_vote(self, sigs: Tuple[int, int, int]) -> bool
```

Returns: `True` if at least `min_align` of the three signals equal `+1` (bullish).

```
bullish = sum(s == 1 for s in sigs)
return bullish >= cfg.min_align
```

`_exit_vote`

```
def _exit_vote(self, sigs: Tuple[int, int, int]) -> bool
```

Returns: `True` if at least `min_align` of the three signals equal `-1` (bearish) — triggers position exit.

`_open_positions_count`

```
def _open_positions_count(self) -> int
```

Returns: Count of data feeds with non-zero position size.

`_risk_sized_shares`

```
def _risk_sized_shares(self, data: bt.feeds.PandasData, tkr: str, dt: pd.Timestamp) -> int
```

Returns: Number of shares to buy, sized so that a `2×ATR` adverse move equals `risk_pct` of portfolio.

ATR-based sizing formula:

```
risk_dollars = equity × risk_pct          # e.g., $100,000 × 0.015 = $1,500
stop_distance = ATR × atr_stop_mult      # e.g., ATR × 2.0
shares = int(risk_dollars / stop_distance)

# Fallback if ATR is zero:
stop_distance = price × 0.05              # 5% of price

# Hard cap: no single position > 25% of portfolio
max_shares = int((equity × 0.25) / price)
shares = max(0, min(shares, max_shares))
```

`next`

Called by Backtrader once per bar. Core execution logic:

1. Appends `(current_date, portfolio_value)` to `equity_curve`.
2. Increments `days_in_market` if any position is open.
3. Updates `max_equity_seen`; computes rolling drawdown:

```
dd = 1 - (pv / max_equity_seen)
```

1. If `dd >= hard_dd_stop (20%)` and `hard_stop_triggered` is False: closes all positions and sets `hard_stop_triggered = True`.
2. For each data feed:
 - Skips if no new bar since last check.
 - Liquidates on `final_date`.
 - Skips all processing if `hard_stop_triggered`.
 - Applies `min_price` and `min_volume` filters (closes existing position if ticker falls below filter).
 - **If long position exists:**
 - Updates trailing stop peak: `_peak_since_entry[ticker] = max(prev_peak, close_px)`.
 - ATR trailing stop check: if `close_px < peak - ATR × atr_trail_mult (2.5)`, closes position.
 - Signal exit: if `_exit_vote` is True, closes position.
 - **If no position:**
 - Checks `_open_positions_count() < max_positions`.
 - Checks `_long_vote` returns True (at least 2 of 3 signals bullish).
 - Trend filter: requires `close_px > SMA200`.
 - Sizes via `_risk_sized_shares`; calls `self.buy(data=data, size=size)`.

notify_order

```
def notify_order(self, order: bt.Order) -> None
```

No-op override (order confirmations are not logged individually).

notify_trade

```
def notify_trade(self, trade: bt.Trade) -> None
```

On trade open: Stores `{entry_date, entry_price, size, side: "Long"}` in `open_info[ticker]`.

On trade close: Computes and appends to `trades_log`:

```
{
    "ticker":      str,
    "side":        "Long",
    "entry_date":  "YYYY-MM-DD",
    "exit_date":   "YYYY-MM-DD",
    "entry_price": float (6 decimals),
    "exit_price":  float (6 decimals) | null,
    "pnl":         float (2 decimals),
    "bars_held":   int,
    "pnl_pct":     float (2 decimals) | null,
}
```

Exit price formula: `exit_price = entry_price + (pnl / size)`.

PnL% formula: `pnl_pct = (pnl / (entry_price × abs(size))) × 100`.

Class: MultiTickerSignalStrategy

```
class MultiTickerSignalStrategy(bt.Strategy)
```

Legacy long/short strategy — kept for backwards compatibility. Used automatically when `long_only=False`.

Key differences from `ImprovedLongOnlyStrategy`:

- Supports short selling: `sell(data=data, size=size)` when `vote == -1`.
- Uses equal-weight position sizing (portfolio value divided by `max_positions`).
- No ATR trailing stop; exits only on signal vote reversal.
- No trend filter (no SMA200 entry guard).
- `_aligned_vote` requires zero opposing signals (stricter): `pos >= min_align AND neg == 0` for long; `neg >= min_align AND pos == 0` for short.

`_aligned_vote`

```
def _aligned_vote(self, sigs: Tuple[int, int, int]) -> int
```

Returns: `+1` for long vote (bullish majority, zero bearish), `-1` for short vote (bearish majority, zero bullish), `0` for hold. Stricter than `ImprovedLongOnlyStrategy._long_vote`.

`_target_shares`

```
def _target_shares(self, data: bt.feeds.PandasData) -> int
```

Equal-weight sizing formula:

```
alloc_value = portfolio_equity / max_positions
shares      = int(alloc_value // close_price)
```

Dataclass: `Metrics`

```
@dataclass
class Metrics:
    sharpe: float
    sortino: float
    calmar: float
    max_drawdown_pct: float
    max_dd_duration_days: int
    time_in_market_pct: float
    cagr: float
    win_rate: float = 0.0
    profit_factor: float = 0.0
    expectancy: float = 0.0
    avgBarsHeld: float = 0.0
    total_trades: int = 0
```

`compute_extended_trade_metrics`

```
def compute_extended_trade_metrics(trades_df: pd.DataFrame) -> dict
```

Parameters: A DataFrame with at least a `"pnl"` column and optionally `"bars_held"`.

Returns: A dict with keys `win_rate`, `profit_factor`, `expectancy`, `avgBarsHeld`, `total_trades`.

Formulas:

```
win_rate      = count(pnl > 0) / total_trades      (then × 100 for %)
gross_profit  = sum(pnl > 0)
gross_loss    = abs(sum(pnl < 0))
profit_factor = gross_profit / gross_loss          (inf if gross_loss == 0)
avg_win       = mean(pnl > 0)
avg_loss      = abs(mean(pnl < 0))
expectancy    = (win_rate × avg_win) - ((1 - win_rate) × avg_loss)
avgBarsHeld   = mean(bars_held)
```

`compute_metrics`

```
def compute_metrics(
    equity_df: pd.DataFrame,
    days_in_market: int,
```

```
trades_df: Optional[pd.DataFrame] = None
) -> Metrics
```

Parameters:

- `equity_df` — DataFrame with `"date"` and `"equity"` columns.
- `days_in_market` — int count of days with an open position.
- `trades_df` — optional trade log DataFrame.

Returns: A populated `Metrics` dataclass.

All formulas:

```
# Daily returns
ret = equity.pct_change().fillna(0)

# Sharpe (annualized, daily returns, no risk-free adjustment by default)
sharpe = (mean(ret) / std(ret, ddof=0)) * sqrt(252)

# Sortino (uses only negative return days in denominator)
neg = ret[ret < 0]
ds = std(neg, ddof=0)
sortino = (mean(ret) / ds) * sqrt(252)          (inf if ds == 0)

# CAGR
years = (end_date - start_date).days / 365.0
cagr = (end_equity / start_equity) ^ (1/years) - 1

# Max Drawdown
roll_max = equity.cummax()
dd_series = equity / roll_max - 1.0
max_dd_pct = abs(dd_series.min())              (as percentage)

# Max Drawdown Duration (consecutive days below rolling max)
max_dur = longest run of dd_series < 0

# Calmar
calmar = cagr / max_dd_pct                     (inf if max_dd_pct == 0)

# Time in Market
time_in_market_pct = (days_in_market / len(equity_df)) * 100
```

All metrics are rounded:

- `sharpe`, `sortino`, `calmar`: 4 decimal places
- `max_drawdown_pct`, `cagr`, `time_in_market_pct`: 2 decimal places
- Trade metrics: 1–2 decimal places

adjust_ohlc_with_adjclose

```
def adjust_ohlc_with_adjclose(df: pd.DataFrame) -> pd.DataFrame
```

Applies split and dividend adjustment to OHLC prices using the `Adj Close` column from yfinance:

```
factor = Adj Close / Close
Open, High, Low, Close *= factor
```

Drops the `Adj Close` column after adjustment. Returns `df` unchanged if `"Adj Close"` is not present.

fetch_history

```
def fetch_history(ticker: str, start: pd.Timestamp, end: pd.Timestamp) -> pd.DataFrame
```

Parameters:

- `ticker` — ticker symbol.
- `start`, `end` — date range.

Returns: Adjusted OHLCV DataFrame with a `datetime64` index, sorted ascending. Returns an empty DataFrame if yfinance returns nothing.

Behavior: Calls `yf.download(..., auto_adjust=False)` to get raw splits/dividends data, then applies `adjust_ohlc_with_adjclose` manually. Selects only `["Open", "High", "Low", "Close", "Volume"]`.

build_universe

```
def build_universe(signals_map: Dict[str, pd.DataFrame]) -> List[str]
```

Returns: Sorted list of ticker strings from the `signals_map` keys (all tickers in the signal CSV).

run_backtest

```
def run_backtest(
    signals_csv: str,
    out_dir: str,
    initial_cash: float = 100_000.0,
    commission: float = 0.0005,
    slippage_pct: float = 0.0005,
    max_positions: int = 10,
    risk_free: float = 0.0,
    dd_stop: float = 0.20,
    min_align: int = 2,
    use_recalc: bool = True,
    verbose_mismatch: bool = True,
    min_price: float = 5.0,
    min_volume: int = 200_000,
    long_only: bool = True,
    trend_filter: bool = True,
    risk_pct: float = 0.015,
) -> Tuple[pd.DataFrame, pd.DataFrame, Metrics]
```

Returns: (`equity_df`, `trades_df`, `metrics`) where `equity_df` has columns `["date", "equity"]`, `trades_df` has all trade log fields, and `metrics` is a `Metrics` dataclass.

Behavior:

1. Creates `out_dir`.
2. Loads and normalizes signals with `load_signals(signals_csv)`.
3. Determines global date range from all signal DataFrames.
4. Sets `hist_start = start_dt - max(365, SMA_LONG+20) days` to ensure SMA200 warm-up.
5. For each ticker: downloads OHLCV, warns if fewer than `MIN_BARS_WARMUP (220)` warm-up bars, runs `ta_validate`, trims to `[start_dt:]`, merges recalculated signals back into `signals_map[tkr]` at matching dates.
6. Optionally logs signal mismatches between CSV-provided and recalculated values if `verbose_mismatch=True`.
7. Configures Backtrader `Cerebro`: sets cash, commission, percent slippage.
8. Adds one `PandasDataAdj` feed per ticker.
9. Selects `ImprovedLongOnlyStrategy` (if `long_only=True`) or `MultiTickerSignalStrategy`.
10. Runs `cerebro.run(maxcpus=1)`.
11. Trims equity curve to `[start_dt:]`, computes metrics.
12. Saves:
 - `<out_dir>/portfolio_equity_curve.csv`
 - `<out_dir>/trade_log.csv`
 - `<out_dir>/summary_metrics.json`

Files read: signals CSV, yfinance download (network). **Files written:**

- `results_run/portfolio_equity_curve.csv`
- `results_run/trade_log.csv`
- `results_run/summary_metrics.json`

load_json_config

```
def load_json_config(config_path: str) -> Dict[str, Any]
```

Returns: Parsed JSON dict from `config_path`, or `{}` if file does not exist or is malformed.

Default config path: `Module_2_Technical_Analysis/results_run/backtest_config.json`

parse_args

```
def parse_args(argv: Optional[List[str]] = None) -> argparse.Namespace
```

Defines all CLI arguments with defaults matching the strategy defaults. Boolean arguments accept `"1"`, `"true"`, `"yes"` (case-insensitive). Key arguments:

Argument	Type	Default	Description
<code>--signals_csv</code>	str	None	Required path to TA CSV
<code>--out_dir</code>	str	<code>results_run_<timestamp></code>	Output directory
<code>--initial_cash</code>	float	100,000	Starting capital
<code>--commission</code>	float	0.0005	Brokerage commission fraction
<code>--slippage_pct</code>	float	0.0005	Slippage fraction
<code>--max_positions</code>	int	10	Maximum concurrent positions
<code>--dd_stop</code>	float	0.35	Portfolio drawdown halt level
<code>--min_align</code>	int	2	Min signals to agree for entry/exit
<code>--use_recalc</code>	bool	True	Use recomputed indicators
<code>--min_price</code>	float	5.0	Skip below this price
<code>--min_volume</code>	int	200,000	Skip below this volume
<code>--long_only</code>	bool	True	Disable short selling
<code>--trend_filter</code>	bool	True	Require Close > SMA200 for entry
<code>--risk_pct</code>	float	0.015	Portfolio risk fraction per trade

overlay_args_with_json

```
def overlay_args_with_json(args: argparse.Namespace, json_cfg: Dict[str, Any]) -> argparse.Namespace
```

Behavior: For each key in `json_cfg` that matches an attribute of `args`, calls `setattr(args, k, v)`. JSON config values override CLI defaults; this allows a stable JSON config to act as persistent settings without repeated CLI flags.

copy_config_to_outdir

```
def copy_config_to_outdir(config_source: str, out_dir: str) -> None
```

Copies the JSON config file to `<out_dir>/config_used.json` for reproducibility tracking. Silently skips if the source does not exist.

main (backtrader)

```
def main(argv: Optional[List[str]] = None) -> None
```

Execution sequence:

1. `parse_args(argv)` — CLI defaults.
2. `load_json_config(DEFAULT_JSON_CONFIG)` — JSON override.
3. `overlay_args_with_json` — merge.
4. Validate `signals_csv` is set; exit with usage instructions if not.
5. `ensure_dir(out_dir)`.

- 6. `copy_config_to_outdir`.
- 7. Log all run parameters.
- 8. `run_backtest(...)`.
- 9. Print full performance summary table to stdout.

Files written: All files from `run_backtest` plus `config_used.json`.

11. `validate_env.py`

Role and Phase

Environment / pre-flight validation script. Not part of the data pipeline. Run before any pipeline execution to verify that all required API keys are set, all critical Python packages are importable, all key pipeline script files exist, and all required output directories exist. Optionally tests live OpenAI API connectivity with `--test-api`.

External Libraries

Library	Import	Purpose
<code>python-dotenv</code>	<code>from dotenv import load_dotenv</code>	<code>.env</code> file loading
<code>openai</code>	<code>from openai import OpenAI</code>	API connectivity test
<code>importlib.metadata</code>	<code>stdlib</code>	Checks <code>sentence-transformers</code> version without importing it (avoids torch segfault)
<code>argparse</code> , <code>os</code> , <code>sys</code> , <code>pathlib</code>	<code>stdlib</code>	CLI argument parsing, env access, exit codes

Segfault mitigation: `sentence-transformers` import is deliberately checked via `importlib.metadata.version("sentence-transformers")` rather than `__import__("sentence-transformers")` (line 110-115). This avoids loading `torch` at import time, which causes a macOS segfault on `torch==2.2.2` when `SentenceTransformer()` is instantiated.

Functions

`load_env`

```
def load_env() -> None
```

Calls `load_dotenv(BASE_DIR / ".env")` using python-dotenv. If python-dotenv is not installed, prints a warning and continues (OS environment variables will still be read).

`check`

```
def check(label: str, passed: bool, detail: str = "") -> bool
```

Parameters:

- `label` — the check name to display.
- `passed` — True/False result.
- `detail` — optional description string.

Returns: `passed` (allows chaining in `if not check(...)`).

Behavior: Prints "  `{label}` — `{detail}`" or "  `{label}` — `{detail}`".

`section`

```
def section(title: str) -> None
```

Prints a formatted section header with a 55-character dashed border. Used to visually separate check categories.

validate_api_keys

```
def validate_api_keys() -> list[str]
```

Returns: A list of required API key names that are missing or contain placeholder values.

Keys checked:

Key	Required?	Purpose
OPENAI_API_KEY	Yes	GPT summarization, Chef GPT (Phases 5, 11)
FINNHUB_API_KEY	Yes	News scraping (Phase 2)
SEC_API_KEY	No	SEC Edgar filings
QUANDL_API_KEY	No	Macro data (Phase 6)
QUIVERQUANT_API_KEY	Yes	Political intelligence (Phase 6.5)
FRED_API_KEY	No	FRED macro data (Phase 13)

A key is considered "set" if: non-empty, does not equal `"your_{key_lower}_here"`, and does not contain `"your_"`.

validate_imports

```
def validate_imports() -> list[str]
```

Returns: A list of `"pip install <package>"` strings for required packages that fail to import.

Packages checked:

Module	Package	Required?	Purpose
dotenv	python-dotenv	Yes	API key management
openai	openai	Yes	GPT summarization
streamlit	streamlit	Yes	Dashboard
pandas	pandas	Yes	Data handling
numpy	numpy	Yes	Numerical operations
yfinance	yfinance	Yes	Market data
feedparser	feedparser	Yes	RSS news scraping
requests	requests	Yes	HTTP requests
vaderSentiment	vaderSentiment	No	Sentiment analysis
sklearn	scikit-learn	No	Clustering algorithms
faiss	faiss-cpu	No	Vector search
backtrader	backtrader	No	Backtesting engine
fredapi	fredapi	No	FRED macro data
plotly	plotly	No	Charts
sentence-transformers	(metadata only)	No	Article embeddings — checked via <code>importlib.metadata</code> , not <code>__import__</code>

validate_files

```
def validate_files() -> list[str]
```

Returns: A list of relative path strings for required files that are missing.

Files checked:

File	Required?	Purpose
phase3_extraction.py	Yes	Phase 3 — Data extraction

File	Required?	Purpose
<code>phase4_clustering.py</code>	Yes	Phase 4 — Article clustering
<code>recession_signals.py</code>	Yes	Phase 13 — Recession signals
<code>historical_correlation.py</code>	Yes	Phase 7 — Historical correlation
<code>opportunity_score.py</code>	Yes	Phase 8.9 — Opportunity scoring
<code>chef_gpt.py</code>	Yes	Phase 11 — Chef GPT synthesis
<code>controller.py</code>	Yes	Phase 12 — Pipeline controller
<code>dashboard/app.py</code>	Yes	Streamlit dashboard
<code>agents/security_audit.py</code>	No	Security audit agent
<code>agents/data_quality.py</code>	No	Data quality agent
<code>agents/verification_runner.py</code>	No	Verification runner
<code>.env</code>	Yes	API keys config
<code>requirements.txt</code>	Yes	Package manifest

`validate_directories`

```
def validate_directories() -> None
```

Behavior: Checks (and creates if missing) three directories:

- `news_output/`
- `Module_2_Technical_Analysis/results_run/`
- `agents/`

All three are reported as passing regardless of whether they pre-existed.

`test_openai_api`

```
def test_openai_api() -> bool
```

Returns: `True` if the OpenAI API responds successfully; `False` if the key is absent or any exception occurs.

Behavior: Sends a minimal chat completion request:

```
client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[{"role": "user", "content": "Say OK"}],
    max_tokens=5,
)
```

Checks `bool(response.choices[0].message.content)`. Only invoked when `--test-api` is passed.

`main (validate_env)`

```
def main() -> None
```

Behavior:

1. Parses `--test-api` flag.
2. Calls `load_env()`.
3. Runs all four validation sections in order: API keys, imports, files, directories.
4. If `--test-api`: calls `test_openai_api()`.
5. Exits with `sys.exit(0)` if all required checks passed, `sys.exit(1)` otherwise.
6. Prints actionable remediation instructions for each failure category.

Appendix: Data Flow Summary

```
PDF Books
↓
economic_engine.py      → economic_knowledge_index.faiss
                        economic_knowledge_metadata.json
↓
clustered_summaries.json (Phase 4 clustering output – separate script)
↓
article_clustering.py (Phase 5 GPT summarization)
                        → clustered_articles_with_summaries.json
                        sector_summaries.json (partial)
↓
GPT_article_clustering_summary.py (Phase 4 sector overview)
                        → sector_summaries.json (full, 12-sentence overviews)
↓
Reasoning_Report.py (Phase 6 economic reasoning)
                        → economic_reasoning_summary.json
                        (bullish_sectors, bearish_sectors)
↓
ta_analysis.py (TA engine)
                        → ta_analysis_detailed.csv
                        raw_data/<ticker>.csv
↓
phase_3_signal.py (signal generation)
                        → signal_output_phase3.json
↓
prepare_qlib_data.py    → qlib_data/formatted/data.pkl
↓
pipeline_alpha.py (LightGBM) → qlib_data/predictions.csv
↓
phase_4_backtrader.py (backtesting)
                        → results_run/portfolio_equity_curve.csv
                        results_run/trade_log.csv
                        results_run/summary_metrics.json
```

The Money Trail Detective Engine

The Money Trail Detective Engine upgrades the old timing-only "suspicion score" into a chain-of-evidence **Case File**. Instead of asking only "how many trades happened before this law passed," it scores each law as a **CASE STRENGTH** built from six evidence links, and leans **multiplicative on the two critical links** so a case cannot score high on trade volume alone.

The 6-link model

Each link returns a strength in **0..1**.

1. **The Law.** What economic action the bill takes (funds, authorizes, awards contracts, changes a rule or tariff). Where does money flow. Source: bill title and latest-action text (Congress.gov).
1. **The Beneficiary (materiality). CRITICAL.** The specific companies whose revenue depends on the law, not a keyword sector tag. Source: a curated materiality map for high-impact areas (defense, energy, health, tech); falls back to keyword-matched tickers (clearly labeled lower-confidence).
1. **The Access. CRITICAL.** Did a trader sponsor or cosponsor the bill, or sit on the committee of jurisdiction. Sources: Congress.gov sponsors/cosponsors, and committee rosters from the unitedstates/congress-legislators project.
1. **The Trade.** Did access holders trade the beneficiary companies ****before the committee/markup milestone**** (re-anchored timing), not just before final passage. Sources: Congress.gov action timeline, plus the correlated trades.
1. **The Payoff.** Abnormal return, not raw profit. Each pre-milestone trade's market-adjusted return (QuiverQuant **excess_return**, winsorized) is compared to the member's own baseline (median), and the abnormal portion is scored.
1. **The Pattern.** Cross-law repetition per member and clustering of multiple members trading the same beneficiary before the same milestone.

Scoring formula and weights

```
CASE STRENGTH = round(100 * gate * evidence)
```

```
gate      = (0.15 + 0.85 * Beneficiary) * (0.15 + 0.85 * Access)  # critical, multiplicative
evidence  = 0.20*Law + 0.35*Trade + 0.25*Payoff + 0.20*Pattern    # supporting, additive (sums to 1)
```

The **gate** collapses toward ~0.02 when either critical link is absent, so the score is capped in the single digits regardless of how many trades exist. Access weights: sponsor **1.0**, cosponsor **0.8**, committee-of-jurisdiction **0.7**, none **0.1**. Tier bands (UI): Strong case **>=70**, Notable **45-69**, Emerging **25-44**, Thin **<25**.

Data sources per link

Link	Source	Endpoint / file
Law	Congress.gov	bill title + latest-action text (already in congress_bills.json)
Beneficiary	curated map + keyword fallback	MATERIALITY in build_money_trail.py
Access	Congress.gov + congress-legislators	/bill/{c}/{type}/{n}/committees , .../cosponsors , bill sponsors ; committee-membership-current.json
Trade	Congress.gov timeline	/bill/.../actions parsed into milestones, re-anchored to markup/committee
Payoff	QuiverQuant	/bulk/congresstrading excess_return , winsorized, vs per-member median baseline
Pattern	derived	computed across the full case set

Keys (**CONGRESS_API_KEY**, **QUIVERQUANT_API_KEY**) are read from **.env**, never logged or written to output.

File and function map

- `webapp/build_money_trail.py` -- the engine.
 - `class MoneyTrailDetectiveEngine` -- `link_law`, `link_beneficiary`, `link_access`, `link_trade`, `link_payoff`, `link_pattern`, `score_case`.
- `fetch_bill_evidence(bill_id)` -- Phase 1+2: actions timeline, committees, sponsors, cosponsors.
- `load_committee_rosters()` / `committee_member_names()` -- Phase 2 access.
- `load_excess_index()` -- Phase 3: excess-return index + per-member baseline.
- `MATERIALITY` -- Phase 4 curated beneficiary map.
- `infer_beneficiaries_via_gpt()` -- Phase 4 gpt-4o scaffold (NOT enabled).
- `verdict_sentence()` -- the plain-language verdict.
- Output: `webapp/js/money_trail_data.js` (`window.MONEY_TRAIL`), exported to `private_data/MONEY_TRAIL.json` (core bundle), refreshed by the scheduler `money_trail` job (daily).
- `webapp/js/app.js` -- front end.
 - `mtCaseFor`, `lawScore`, `caseTier` -- leaderboard scoring from the engine.
 - `lawLeaderboard` / `renderMarkets` -- the entry-point leaderboard.
 - `caseFile(billId)` -- the Case File modal.
 - `caseTimeline(c)` -- the SVG milestone-plus-trades timeline.
 - `caseEvidenceCards(c)` -- the six evidence-board cards.

The Case File UI

The leaderboard ("The Money Trail Detective" screen) is the entry point: every tracked law ranked by case strength, with a tier pill and quick stats (beneficiaries, access level, abnormal return). Clicking a row opens the Case File:

- A plain-language **verdict** at the top, plus an honest note on the weakest or missing link so it reads as evidence, not accusation.
- A centerpiece **timeline**: one track for the bill with milestone markers; trades

plotted below as points, colored by buy/sell and sized by disclosed amount, so pre-markup clustering is visually obvious. The anchor milestone is highlighted.

- The **chain of evidence** as six cards (The Law, Who It Pays, Who Knew, Who Traded, The Payoff, The Pattern), each with its own strength bar and a short "why this matters."

UI copy uses no emojis and no em dashes, and color comes only from the semantic theme tokens (`--info`, `--warning`, `--danger`, `--success`); pink is reachable only through Customize Display.

Phase 4: gpt-4o materiality (enabled, titles + summaries path)

`infer_beneficiaries_via_gpt(title, summary)` reads a bill's title and summary (not full text) and proposes the specific publicly-traded companies whose revenue most directly depends on it, labeled **model-inferred**. It runs only for laws the curated map does not already cover, and results are cached in `money_trail_gpt_cache.json` so re-runs do not re-spend. Enable with the `--gpt` flag (`build_money_trail.py --gpt`); the scheduler job runs the cached path.

Actual first run: 40 of 80 laws needed inference (the rest were curated), for about **\$0.15** on gpt-4o. Materiality basis now splits roughly curated 40, model-inferred 16, keyword fallback 24. The fuller-bill-text path remains an option at higher cost (~\$2 to \$8) and is not enabled.

Known limitations (evidenced vs inferred)

- **Beneficiary** is evidenced only for the curated areas (defense, energy, health, tech). Outside those, it is keyword-inferred and the case is labeled and capped accordingly. The gpt-4o path (above) would close this gap.
- **Access** is evidenced for sponsors, cosponsors, and current committee rosters. Historical committee membership (for older trades) and leadership roles are not yet wired.
- **Trade timing** uses the milestones Congress.gov exposes in the action log;

true non-public markup dates can lag the public action text.

- **Payoff** uses QuiverQuant `excess_return` (market-adjusted) winsorized to

+/-100 points against a per-member median baseline. It is an estimate, not a full event study with a fitted factor model.

- **Name matching** between QuiverQuant trader names and Congress.gov rosters is

fuzzy; a missed match understates access (conservative, not inflating).

Everything the engine outputs is a timing relationship between public disclosures, legislative milestones, and market-adjusted returns. It does not imply or allege wrongdoing of any kind.

Part 8 — Python Verification Agent Layer

Source tree: `agents/` (relative to repo root `ThinkFree-main/`) **Files covered:** `__init__.py`, `data_quality.py`, `security_audit.py`, `strategy_audit.py`, `verification_runner.py`

Summary Table

Agent	Module	Triggers After	Blocks On	Output Written
DataQualityAgent	<code>agents/data_quality.py</code>	Phase 2 (scraping) and Phase 3 (extraction)	Any BLOCKER - severity finding	<code>pipeline_audit.json</code> (via runner)
SecurityAuditAgent	<code>agents/security_audit.py</code>	Every pipeline execution	Any CRITICAL or HIGH finding	<code>pipeline_audit.json</code> (also writes directly)
StrategyAuditAgent	<code>agents/strategy_audit.py</code>	Phase 9 (backtesting)	Any BLOCKER - severity finding	<code>pipeline_audit.json</code> (via runner)
VerificationRunner	<code>agents/verification_runner.py</code>	Invoked by <code>controller.py</code> or directly	Any agent returning FAIL or ERROR	<code>pipeline_audit.json</code> (combined)

Package Root — `agents/__init__.py`

File: `agents/__init__.py` (line 1–2)

The package init is a single-line comment:

```
# ThinkFree Finance – Verification Agent Layer
```

No imports, no `__all__`, no public API defined at the package level. All agents are imported directly by `verification_runner.py` using their module paths.

DataQualityAgent — `agents/data_quality.py`

Overview

Pipeline checkpoint: After Phase 2 (raw scraping) and Phase 3 (NLP extraction). Also validates Phase 4 (clustering), Phase 6 (economic reasoning), and Phase 13 (recession signals).

What it checks:

Phase	Check Category	Severity Possible
Phase 2	Missing title, empty title, missing URL, no content body, stale/unparseable timestamps	BLOCKER, WARN
Phase 3	Null sentiment, missing compound score, zero data richness, no extracted entities, sort order	BLOCKER, WARN, INFO
Phase 4	Empty clustering output, too few clusters, missing GICS sector, missing impact score	BLOCKER, WARN, INFO
Phase 6	Missing economic reasoning file, missing required keys, thin summary	INFO, WARN
Phase 13	Missing recession output, missing risk score, score out of 0–10 range	INFO, WARN

Pass/fail verdict logic: FAIL if any finding has `severity == "BLOCKER"`. No blockers → PASS. Warnings do not block.

Global threshold constant:

```
PASS_THRESHOLD = 0.80 # agents/data_quality.py:22
```

This constant is defined but its value (80%) is referenced conceptually in per-field checks — the actual blocking thresholds are implemented inline per field (e.g., `> 0.2` for missing title, `> 0.3` for missing URL, `> 0.5` for null sentiment).

Files read:

File Path	Purpose
news_output/articles_*.json (most recent by sort)	Phase 2 raw articles
news_output/enriched_news_sentiment.json	Phase 3 enriched/extracted articles
news_output/clustered_summaries.json	Phase 4 clustered output
news_output/economic_reasoning_summary.json	Phase 6 economic reasoning
recession_signals_output.json (repo root)	Phase 13 recession signals

Files written: None directly. Results are returned as a `dict` and written by `verification_runner.py` into `pipeline_audit.json`.

`class QualityFinding`

File: `agents/data_quality.py`, lines 25–40

```
class QualityFinding:
    def __init__(self, severity: str, phase: str, check: str, message: str, count: int = 1)
```

Parameters:

Parameter	Type	Description
severity	str	One of "BLOCKER", "WARN", "INFO"
phase	str	Pipeline phase label, e.g. "phase2", "phase3", "phase6"
check	str	Short machine-readable check ID, e.g. "missing_title", "null_sentiment"
message	str	Human-readable description of the finding
count	int	Number of affected records (default 1 ; 0 for structural/aggregate findings)

Instance attributes: `self.severity`, `self.phase`, `self.check`, `self.message`, `self.count`

`QualityFinding.to_dict()`

```
def to_dict(self) -> dict
```

Returns a flat dictionary with keys `severity`, `phase`, `check`, `message`, `count`. Used when serializing findings into the `pipeline_audit.json` output.

`check_phase2_output(articles)`

File: `agents/data_quality.py`, lines 47–132

```
def check_phase2_output(articles: list[dict]) -> list[QualityFinding]
```

Purpose: Validates the raw scraped article list produced by `phase2_data_scraping.py`.

Parameters: `articles` — list of raw article dicts.

Returns: List of `QualityFinding` objects.

Step-by-step behavior:

1. **Empty guard** (line 52–56): If `articles` is empty, immediately returns a single `BLOCKER` finding with check `"empty_output"`. No further checks run.
1. **Per-article field checks** (lines 63–73): Iterates all articles and counts:
 - `missing_title`: `art.get("title")` is falsy.
 - `empty_title`: title exists but `len(title) < 10`.
 - `missing_url`: neither `art.get("url")` nor `art.get("link")` is truthy.
 - `no_summary`: none of `summary`, `content`, or `description` fields present.
1. **Missing title threshold** (lines 77–82):
 - If `missing_title / total > 0.2` → `BLOCKER`
 - Otherwise → `WARN`

1. **Empty title** (lines 84–88): Any `empty_title > 0` → `WARN ("thin_title")`.
1. **Missing URL threshold** (lines 90–95):
 - If `missing_url / total > 0.3` → `BLOCKER`
 - Otherwise → `WARN`
1. **No content** (lines 97–101): Any `no_summary > 0` → `WARN ("no_content")`.
1. **Freshness check** (lines 103–131): Iterates articles, tries to parse `published`, `publishedAt`, or `datetime` field as ISO 8601 (with `Z` → `+00:00` substitution). Counts articles published within the last 48 hours.
 - If `fresh_count == 0` and articles exist → `WARN ("stale_data")`
 - If `fresh_count < total * 0.5` → `WARN ("mostly_stale")`

`check_phase3_output(articles)`

File: `agents/data_quality.py`, lines 139–214

```
def check_phase3_output(articles: list[dict]) -> list[QualityFinding]
```

Purpose: Validates enriched articles from `phase3_extraction.py`, which adds sentiment, entity extraction, and data richness scores.

Parameters: `articles` — list of enriched article dicts.

Returns: List of `QualityFinding` objects.

Step-by-step behavior:

1. **Empty guard** (lines 143–148): Empty list → single `BLOCKER ("empty_output")`, return immediately.
1. **Per-article checks** (lines 153–172): Iterates all articles counting:
 - `null_sentiment`: `art.get("sentiment")` is `None`, or it is a dict but `compound` key is absent or `None`.
 - `missing_compound`: `sentiment` is not a dict or lacks `"compound"`.
 - `zero_richness`: `art.get("data_richness_score")` is `None` or `0`.
 - `missing_entities`: `art.get("entities", {})` is empty or all entity lists are empty.
1. **Null sentiment threshold** (lines 174–179):
 - If `null_sentiment / total > 0.5` → `BLOCKER`
 - Otherwise → `WARN`
1. **Missing compound** (lines 181–184): Any → `WARN ("missing_compound")`.
1. **Low richness threshold** (lines 186–191):
 - If `zero_richness > total * 0.5` → `WARN ("low_richness")`.
1. **Low entity extraction threshold** (lines 193–199):
 - If `missing_entities > total * 0.7` → `INFO ("low_entity_extraction")`. Severity intentionally `INFO` because spaCy may not be installed.
1. **Sort order check** (lines 201–212): If `len(articles) >= 3`, samples the first 10 articles, extracts `abs(compound)` for each, and checks whether the list is already sorted descending. If not → `INFO ("not_sorted")`.

`check_phase4_output(data)`

File: `agents/data_quality.py`, lines 221–268

```
def check_phase4_output(data: dict) -> list[QualityFinding]
```

Purpose: Validates the clustered output dict from `phase4_clustering.py`.

Parameters: `data` — dict mapping cluster IDs to lists of article dicts.

Returns: List of `QualityFinding` objects.

Step-by-step behavior:

1. **Empty guard** (lines 226–231): Empty dict → `BLOCKER ("empty_output")`, return immediately.
1. **Cluster count check** (lines 233–238):
 - `total_clusters = len(data)`. If `< 2` → `WARN ("too_few_clusters")`. Expected 5+ clusters.

- `total_articles = sum(len(v) for v in data.values() if isinstance(v, list))`.

1. **Per-article checks within clusters** (lines 240–250): Iterates each cluster's article list counting:

- `no_sector: art.get("gics_sector")` is falsy.
- `no_impact: art.get("impact_score")` is `None`.

1. **Missing sector threshold** (lines 252–256):

- If `no_sector > total_articles * 0.3` → `WARN ("missing_sector")`.

1. **Missing impact threshold** (lines 258–262):

- If `no_impact > total_articles * 0.3` → `WARN ("missing_impact")`.

1. **OK summary** (lines 263–267): Always appends an `INFO ("ok")` finding with cluster and article counts.

`check_economic_reasoning(data)`

File: `agents/data_quality.py`, lines 275–301

```
def check_economic_reasoning(data: dict) -> list[QualityFinding]
```

Purpose: Validates Phase 6 economic reasoning output from `GPT_Economy/Reasoning_Report.py`.

Parameters: `data` — dict loaded from `economic_reasoning_summary.json`.

Step-by-step behavior:

1. **Not-generated guard** (lines 279–283): Empty dict → `INFO ("not_generated")`, return immediately. Soft failure — Phase 6 is optional.
1. **Required keys check** (lines 285–292): Checks for `"summary"`, `"bullish_sectors"`, `"bearish_sectors"`. Missing any → `WARN` with key-specific check name.
1. **Thin summary check** (lines 294–299): If `summary` is a string with `len < 50` → `WARN ("thin_summary")`.

`check_recession_output(data)`

File: `agents/data_quality.py`, lines 308–331

```
def check_recession_output(data: dict) -> list[QualityFinding]
```

Purpose: Validates Phase 13 recession signals output.

Parameters: `data` — dict loaded from `recession_signals_output.json`.

Step-by-step behavior:

1. **Not-generated guard** (lines 312–316): Empty dict → `INFO ("not_generated")`, return immediately.
1. **Score presence check** (lines 318–321): `data.get("recession_risk", {}).get("score")`. If `None` → `WARN ("missing_score")`.
1. **Score range check** (lines 323–328): If score exists but not in `[0, 10]` → `WARN ("score_out_of_range")`.

`load_json_safe(path)`

File: `agents/data_quality.py`, lines 338–343

```
def load_json_safe(path: Path) -> Any
```

Purpose: Safely loads a JSON file. Returns `None` on `FileNotFoundError` or `json.JSONDecodeError` without raising. Used by `run_data_quality_check` for every file it reads.

`run_data_quality_check(verbose)`

File: `agents/data_quality.py`, lines 346–445

```
def run_data_quality_check(verbose: bool = True) -> dict
```

Purpose: Main entry point for the DataQualityAgent. Loads all pipeline output files and runs all check functions.

Parameters: `verbose` — if `True`, prints progress and verdict to stdout.

Returns: A `dict` with the following structure:

```
{
  "audit_timestamp": "<ISO 8601>Z",
  "verdict": "PASS" | "FAIL",
  "summary": {
    "blockers": <int>,
    "warnings": <int>,
    "info": <int>
  },
  "findings": [ <QualityFinding.to_dict()>, ... ],
  "message": "<human-readable verdict string>"
}
```

Step-by-step execution:

1. Glob `news_output/articles_*.json`, sort descending (most recent first), load the newest file. Handles both `list` format (standard) and Finnhub nested dict format (flattens `company_news` arrays). Calls `check_phase2_output`.
1. Load `news_output/enriched_news_sentiment.json`. If a list, calls `check_phase3_output`. Otherwise appends `INFO` (`"not_found"`).
1. Load `news_output/clustered_summaries.json`. If a dict, calls `check_phase4_output`. Otherwise appends `INFO` (`"not_found"`).
1. Load `news_output/economic_reasoning_summary.json`. Calls `check_economic_reasoning(econ or {})`.
1. Load `recession_signals_output.json`. Calls `check_recession_output(recession or {})`.
1. Partitions all findings by severity. **Verdict is FAIL if `len(blockers) > 0`, else PASS.**
1. Prints summary to stdout if `verbose=True` (max 5 blocking/warning findings shown).

Does not write any file directly. Returns the result dict.

SecurityAuditAgent — `agents/security_audit.py`

Overview

Pipeline checkpoint: Runs on every pipeline execution (no phase dependency).

What it checks:

Check	Function	Severity
Hardcoded secrets and dangerous code patterns in all <code>.py</code> files	<code>scan_python_files_for_secrets</code>	CRITICAL, HIGH, MEDIUM
API keys / secret patterns in JSON output files	<code>scan_json_outputs_for_secrets</code>	CRITICAL, HIGH
<code>.env</code> file existence and <code>.gitignore</code> coverage	<code>check_env_file</code>	HIGH, CRITICAL
Bandit static analysis (subprocess)	<code>run_bandit</code>	CRITICAL/HIGH/MEDIUM (from bandit)
CVE scan via safety (subprocess)	<code>run_safety_check</code>	HIGH
Pipeline output structure validation	<code>validate_pipeline_output_structure</code>	INFO, MEDIUM, HIGH

Pass/fail verdict logic: FAIL if any finding has `severity == "CRITICAL"` OR `severity == "HIGH"`. Medium, Low, and Info do not block.

```
passed = len(critical) == 0 and len(high) == 0 # agents/security_audit.py:365
```

CI gate behavior: When run as `__main__`, exits with `sys.exit(0 if _result.get("passed") else 1)` (line 403), allowing CI pipelines and pre-commit hooks to block on failure.

Note: There is a subtle bug at line 403: `_result.get("passed")` should be `_result.get("verdict") == "PASS"` since the result dict contains key `"verdict"`, not `"passed"`. As written, `get("passed")` always returns `None` (falsy), so `__main__` always exits with code 1. `verification_runner.py` correctly reads `result.get("verdict")` and is unaffected.

Files read:

- All `*.py` files recursively under repo root (excluding `venv/`, `venv_qlib/`, `__pycache__`, `.git/`, `site-packages/`)
- First 20 `*.json` files under `news_output/` and `GPT_Economy/`
- `.env` (for existence, gitignore coverage, and rotated-key detection)
- `.gitignore`
- `user_profile.json` (structure validation)

- `news_output/economic_reasoning_summary.json` (structure validation)

Files written:

- `pipeline_audit.json` (repo root) — written directly by `run_security_audit` at line 384–385, even when called standalone (unlike other agents which only write via the runner).

Secret Detection Patterns

File: `agents/security_audit.py`, lines 34–42

```
SECRET_PATTERNS = [
    (re.compile(r'sk-proj-[A-Za-z0-9-]{40,}'), "OpenAI API key"),
    (re.compile(r'sk-[A-Za-z0-9]{48}'), "OpenAI API key (legacy)"),
    (re.compile(r'"api_key"\s*:\s*" [A-Za-z0-9-]{20,}"'), "Hardcoded API key in JSON"),
    (re.compile(r'password\s*=\s*["\']{8,}["\']', re.I), "Hardcoded password"),
    (re.compile(r'secret\s*=\s*["\']{8,}["\']', re.I), "Hardcoded secret"),
    (re.compile(r'Bearer [A-Za-z0-9-]{20,}'), "Hardcoded Bearer token"),
    (re.compile(r'[A-Za-z0-9]{40}'), None), # Generic long token – flagged only in assignment context
]
```

The first 4 patterns from `SECRET_PATTERNS` are used in `scan_json_outputs_for_secrets` (line 165: `SECRET_PATTERNS[:4]`). All labeled patterns are checked; the generic 40-char token pattern (index 6, `label=None`) is present in the list but the JSON scanner skips it via `if label and ...`.

Dangerous Code Patterns:

File: `agents/security_audit.py`, lines 44–51

```
DANGEROUS_PATTERNS = [
    (re.compile(r'\beval\s*\(\'', "Use of eval() – potential code injection"),
    (re.compile(r'\bexec\s*\(\'', "Use of exec() – potential code injection"),
    (re.compile(r'subprocess.*shell\s*=\s*True\)', "subprocess with shell=True – command injection risk"),
    (re.compile(r'pickle.loads\b\)', "pickle.loads – unsafe deserialization"),
    (re.compile(r'yaml\.load\s*\([^\)]*\)(?!.*Loader\)', "yaml.load without Loader – unsafe deserialization"),
    (re.compile(r'os\.system\s*\(\'', "os.system() – prefer subprocess"),
]
```

All six dangerous patterns are checked on every line of every `.py` file. Any match → **HIGH** severity finding with category `"dangerous_code"`.

Dynamic secret fragments from `.env`:

File: `agents/security_audit.py`, lines 53–69

```
def _secret_fragments():
    frags = ["sk-proj-", "sk-ant-", "ghp-", "github_pat-"]
    try:
        for line in (Path(__file__).resolve().parent.parent / ".env").read_text(...).splitlines():
            if "=" in line and not line.strip().startswith("#"):
                v = line.split("=", 1)[1].strip()
                if len(v) >= 16:
                    frags.append(v[:14]) # first 14 chars of each real key value
    except Exception:
        pass
    return frags

KNOWN_BAD_STRINGS = _secret_fragments()
```

`KNOWN_BAD_STRINGS` is built at import time. It starts with four generic provider prefixes:

- `"sk-proj-"` (OpenAI project keys)
- `"sk-ant-"` (Anthropic API keys)
- `"ghp-"` (GitHub Personal Access Tokens)
- `"github_pat-"` (GitHub fine-grained PATs)

Then reads the live `.env` file and appends the first 14 characters of every value that is 16+ characters long. This means real API key material from `.env` is scanned for directly, without ever storing the full key in the committed source file.

class Finding

File: `agents/security_audit.py`, lines 75–91

```
class Finding:
    def __init__(self, severity: str, category: str, file: str, line: int, message: str)
```

Parameters:

Parameter	Type	Description
<code>severity</code>	<code>str</code>	One of <code>"CRITICAL"</code> , <code>"HIGH"</code> , <code>"MEDIUM"</code> , <code>"LOW"</code> , <code>"INFO"</code>
<code>category</code>	<code>str</code>	Machine-readable category: <code>"exposed_secret"</code> , <code>"dangerous_code"</code> , <code>"env_assignment"</code> , <code>"secret_in_output"</code> , <code>"missing_env"</code> , <code>"env_not_gitignored"</code> , <code>"rotated_key_in_env"</code> , <code>"bandit_<test_id>"</code> , <code>"cve"</code> , <code>"missing_output"</code> , <code>"malformed_output"</code> , <code>"invalid_json"</code> , <code>"tool_missing"</code> , <code>"tool_error"</code>
<code>file</code>	<code>str</code>	Relative file path (relative to repo root)
<code>line</code>	<code>int</code>	Line number (<code>0</code> for file-level findings)
<code>message</code>	<code>str</code>	Human-readable description

Finding.to_dict()

```
def to_dict(self) -> dict
```

Returns `{"severity", "category", "file", "line", "message"}`. Used for JSON serialization.

scan_python_files_for_secrets(root)

File: `agents/security_audit.py`, lines 93–138

```
def scan_python_files_for_secrets(root: Path) -> list[Finding]
```

Purpose: Scans all `.py` files in the project for hardcoded secrets and dangerous code patterns.

Parameters: `root` — the repo root `Path`.

Step-by-step behavior:

- `root.rglob("*.py")` — finds all Python files recursively.
- Skips files where any path component is in `{"venv", "venv_qlib", "__pycache__", ".git", "site-packages"}`.
- Reads each file with `errors="replace"` (handles binary content gracefully).
- Iterates line by line:
 - Known bad strings:** For each string in `KNOWN_BAD_STRINGS`, checks `if bad_str in line`. Match → `CRITICAL` finding, category `"exposed_secret"`, message shows first 8 chars followed by `...`.
 - Dangerous patterns:** For each of 6 patterns in `DANGEROUS_PATTERNS`, checks `pattern.search(line)`. Match → `HIGH` finding, category `"dangerous_code"`.
 - Direct env assignment:** Checks `re.search(r'os\.environ\[\"(OPENAI|API|SECRET|KEY|TOKEN|PASSWORD)\', line, re.I)`. If matched and line does not contain `"os.getenv"` or `"load_dotenv"` → `MEDIUM` finding, category `"env_assignment"`.

scan_json_outputs_for_secrets(root)

File: `agents/security_audit.py`, lines 141–174

```
def scan_json_outputs_for_secrets(root: Path) -> list[Finding]
```

Purpose: Checks pipeline output JSON files for accidentally embedded API keys.

Step-by-step behavior:

- Scans `news_output/` and `GPT_Economy/` directories (if they exist) for all `*.json` files.
- Samples the **first 20 files only** (line 150: `output_files[:20]`).
- For each file, reads full text content:
 - Checks each `KNOWN_BAD_STRINGS` fragment. Match → `CRITICAL` finding, breaks (one finding per file).

- Checks `SECRET_PATTERNS[:4]` (the four most specific patterns). Labeled match → **HIGH** finding, breaks.

`check_env_file(root)`

File: `agents/security_audit.py`, lines 177–211

```
def check_env_file(root: Path) -> list[Finding]
```

Purpose: Verifies that the `.env` file exists, is gitignored, and does not still contain previously-rotated keys.

Step-by-step behavior:

1. If `root / ".env"` does not exist → **HIGH** finding (`"missing_env"`), return immediately.
2. Reads `.gitignore`. If `".env"` not a substring of its content → **HIGH** finding (`"env_not_gitignored"`).
3. Reads `.env` content. For each string in `KNOWN_BAD_STRINGS`, checks if present → **CRITICAL** finding (`"rotated_key_in_env"`), prompting immediate key rotation.

`run_bandit(root)`

File: `agents/security_audit.py`, lines 214–244

```
def run_bandit(root: Path) -> list[Finding]
```

Purpose: Runs the `bandit` static analysis tool as a subprocess and parses its JSON output.

Subprocess call:

```
python -m bandit -r <root> --exclude <root>/venv,<root>/venv_qlib -f json -q
```

Step-by-step behavior:

1. Runs bandit with 60-second timeout.
2. If stdout is non-empty, parses as JSON. Processes up to the first 10 `"results"` entries.
3. Each bandit result maps to a `Finding` with:
 - `severity`: `issue.get("issue_severity", "MEDIUM").upper()`
 - `category`: `"bandit-" + issue.get("test_id", "unknown").lower()`
 - `file`: bandit's `filename` with root prefix stripped
 - `line`: bandit's `line_number`
 - `message`: `"[<test_id>] <issue_text>"`
4. On `FileNotFoundError` (bandit not installed) → **INFO** finding (`"tool_missing"`).
5. On any other exception → **INFO** finding (`"tool_error"`).

`run_safety_check()`

File: `agents/security_audit.py`, lines 247–276

```
def run_safety_check() -> list[Finding]
```

Purpose: Runs `safety check` to identify dependencies with known CVEs.

Subprocess call:

```
python -m safety check --json
```

Step-by-step behavior:

1. Runs safety with 30-second timeout.
2. Parses stdout as JSON. Handles both list format (legacy safety) and dict with `"vulnerabilities"` key.
3. Processes up to 5 vulnerabilities. Each maps to a `Finding` with:
 - `severity`: `"HIGH"`
 - `category`: `"cve"`
 - `file`: `"requirements.txt"`

- `message: "Vulnerable package: <name>. <advisory[:100]>"`
4. On `FileNotFoundError` → `INFO ("tool_missing")`.
 5. On exception → `INFO ("tool_error")`.

`validate_pipeline_output_structure(root)`

File: `agents/security_audit.py`, lines 279–322

```
def validate_pipeline_output_structure(root: Path) -> list[Finding]
```

Purpose: Checks that key pipeline output files exist and contain their required top-level keys.

Checks defined (lines 283–294):

File	Required Keys	Label
<code>user_profile.json</code>	<code>["age", "risk_tolerance", "risk_score"]</code>	<code>"User Profile"</code>
<code>news_output/economic_reasoning_summary.json</code>	<code>["summary", "bullish_sectors", "bearish_sectors"]</code>	<code>"Economic Reasoning"</code>

Step-by-step behavior per check:

1. If file does not exist → `INFO` finding (`"missing_output"`). Continue to next check.
2. Load and parse JSON. If `json.JSONDecodeError` → `HIGH` finding (`"invalid_json"`).
3. For each required key missing from the parsed dict → `MEDIUM` finding (`"malformed_output"`).

`run_security_audit(verbose)`

File: `agents/security_audit.py`, lines 329–397

```
def run_security_audit(verbose: bool = True) -> dict
```

Purpose: Main entry point for SecurityAuditAgent. Runs all six check functions and writes `pipeline_audit.json`.

Execution order:

1. `scan_python_files_for_secrets(BASE_DIR)`
2. `scan_json_outputs_for_secrets(BASE_DIR)`
3. `check_env_file(BASE_DIR)`
4. `run_bandit(BASE_DIR)`
5. `run_safety_check()`
6. `validate_pipeline_output_structure(BASE_DIR)`

Verdict logic (line 365):

```
passed = len(critical) == 0 and len(high) == 0
```

Return dict structure:

```
{
  "audit_timestamp": "<ISO 8601>Z",
  "verdict": "PASS | FAIL",
  "summary": {
    "critical": <int>,
    "high": <int>,
    "medium": <int>,
    "info": <int>,
    "total": <int>
  },
  "findings": [ <Finding.to_dict()>, ... ],
  "message": "<human-readable verdict>"
}
```

Writes directly to `pipeline_audit.json` (line 384–385). This is the only agent that writes to disk independently of the runner.

Verbose output (when `verbose=True`): Prints progress, verdict label, and the first 5 critical/high findings with `file:line - message` format.

StrategyAuditAgent — `agents/strategy_audit.py`

Overview

Pipeline checkpoint: After Phase 9 (backtesting). Reads output from `Module_2_Technical_Analysis/results_run/`.

What it checks:

Check ID	Metric	Threshold	Severity
<code>negative_sharpe</code>	Sharpe ratio	<code>< 0</code>	BLOCKER
<code>low_sharpe</code>	Sharpe ratio	<code>< 0.5</code>	WARN
<code>sharpe_ok</code>	Sharpe ratio	<code>>= 0.5</code>	INFO
<code>excessive_drawdown</code>	Max Drawdown %	<code>> 35</code>	BLOCKER
<code>high_drawdown</code>	Max Drawdown %	<code>> 20</code>	WARN
<code>negative_cagr</code>	CAGR %	<code>< 0</code>	BLOCKER
<code>suspicious_cagr</code>	CAGR %	<code>> 100</code>	WARN
<code>no_trades</code>	Trade count	<code>== 0</code>	BLOCKER
<code>too_few_trades</code>	Trade count	<code>< 5</code>	WARN
<code>overtrading</code>	Trade count	<code>> 500</code>	WARN
<code>low_market_exposure</code>	Time in Market %	<code>< 5</code> (and trades > 0)	WARN
<code>possible_lookahead</code>	CAGR > 50 AND trades < 20	Heuristic combined	WARN

Pass/fail verdict logic: FAIL if any finding has `severity == "BLOCKER"`. Warnings do not block.

Files read:

File Path	Purpose
<code>Module_2_Technical_Analysis/results_run/summary_metrics.json</code>	Numeric strategy performance metrics
<code>Module_2_Technical_Analysis/results_run/trade_log.csv</code>	Trade log (line count used to determine trade count)

Files written: None directly. Results returned as dict for the runner to include in `pipeline_audit.json`.

`class StrategyFinding`

File: `agents/strategy_audit.py`, lines 32–47

```
class StrategyFinding:
    def __init__(self, severity: str, check: str, message: str, value: Any = None, plain_english: str = "")
```

Parameters:

Parameter	Type	Description
<code>severity</code>	<code>str</code>	<code>"BLOCKER"</code> , <code>"WARN"</code> , or <code>"INFO"</code>
<code>check</code>	<code>str</code>	Machine-readable check ID (e.g., <code>"negative_sharpe"</code> , <code>"excessive_drawdown"</code>)
<code>message</code>	<code>str</code>	Technical description for developers
<code>value</code>	<code>Any</code>	The raw metric value that triggered the finding (e.g., <code>-0.43</code> for Sharpe)
<code>plain_english</code>	<code>str</code>	Plain-language explanation for the dashboard, written for non-technical users

`StrategyFinding.to_dict()`

```
def to_dict(self) -> dict
```

Returns `{"severity", "check", "message", "value", "plain_english"}`.

`load_json(path)`

File: `agents/strategy_audit.py`, lines 50–55

```
def load_json(path: Path) -> dict
```

Returns empty dict `{}` on `FileNotFoundError` or `json.JSONDecodeError`. Safe loader; does not raise.

`count_trades(trade_log_path)`

File: `agents/strategy_audit.py`, lines 58–64

```
def count_trades(trade_log_path: Path) -> int
```

Opens the CSV trade log and counts `len(lines) - 1` (subtracts the header row). Returns `-1` if the file is not found (indicating Phase 9 has not been run).

`run_strategy_audit(verbose)`

File: `agents/strategy_audit.py`, lines 67–202

```
def run_strategy_audit(verbose: bool = True) -> dict
```

Purpose: Main entry point for StrategyAuditAgent.

Step-by-step behavior:

1. Loads `summary_metrics.json` via `load_json`. If empty → single `INFO` finding (`"missing_backtest"`) and returns immediately via `_build_result`.
1. Extracts metric values with safe float conversion and `None` handling:

```
sharpe      = float(metrics.get("Sharpe", 0) or 0)
sortino     = float(metrics.get("Sortino", 0) or 0) if metrics.get("Sortino") not in (None, float("inf")) else None
max_drawdown = float(metrics.get("Max Drawdown %", 0) or 0)
cagr        = float(metrics.get("CAGR %", 0) or 0)
time_in_market = float(metrics.get("Time in Market %", 0) or 0)
trade_count = count_trades(TRADE_LOG_PATH)
```

1. Check 1 — Sharpe Ratio (lines 95–115):

- `< 0` → BLOCKER `"negative_sharpe"` with plain-English: "A negative Sharpe means the strategy is losing more money than it would if you just held cash."
- `< 0.5` → WARN `"low_sharpe"`, target is Sharpe > 1.0.
- `>= 0.5` → INFO `"sharpe_ok"`.

1. Check 2 — Max Drawdown (lines 117–133):

- `> 35` → BLOCKER `"excessive_drawdown"` with plain-English explaining retail investor behavior.
- `> 20` → WARN `"high_drawdown"` recommending trailing stops.

1. Check 3 — CAGR (lines 135–153):

- `< 0` → BLOCKER `"negative_cagr"` explaining signals predicting wrong direction or excessive costs.
- `> 100` → WARN `"suspicious_cagr"` flagging lookahead bias as the probable cause.

1. Check 4 — Trade Count (lines 155–179):

- `== 0` → BLOCKER `"no_trades"` with specific remediation: "Lower min_align from 3 to 2, or check that signal columns are mapped correctly."
- `< 5` → WARN `"too_few_trades"` (statistically meaningless sample).
- `> 500` → WARN `"overtrading"` (commission/slippage risk).

1. Check 5 — Time in Market (lines 181–189):

- `< 5% AND trade_count != 0` → WARN `"low_market_exposure"` suggesting signal alignment requirements are too strict.

1. Check 6 — Lookahead Bias Heuristic (lines 191–201):

- `cagr > 50 AND trade_count < 20` → WARN `"possible_lookahead"` explaining that entry fills should be on the next bar (t+1), not the signal bar (t).

1. Calls `_build_result(findings, verbose)` to package and return.

`_build_result(findings, verbose)`

File: `agents/strategy_audit.py`, lines 205–247

```
def _build_result(findings: list[StrategyFinding], verbose: bool) -> dict
```

Purpose: Packages findings into the return dict, generates plain-English summary, and prints verbose output.

Verdict logic: FAIL if any BLOCKER finding present.

Plain-English summary generation:

- If blockers: `"Strategy audit found {n} critical issue(s) that must be fixed: {first 2 blocker messages joined by '; '}"`
- If warnings only: `"Strategy passed basic checks but has {n} warning(s) to review."`
- If clean: `"Strategy audit passed. No critical issues detected."`

Return dict structure:

```
{
  "audit_timestamp": "<ISO 8601>Z",
  "verdict": "PASS" | "FAIL",
  "summary": {
    "blockers": <int>,
    "warnings": <int>,
    "info": <int>
  },
  "plain_english_summary": "<string>",
  "findings": [ <StrategyFinding.to_dict()>, ... ],
  "recommendations": [ "<string>", ... ]
}
```

Verbose output: Prints verdict label, first 5 blocker+warning findings, and first 3 recommendations.

`_build_recommendations(findings)`

File: `agents/strategy_audit.py`, lines 250–273

```
def _build_recommendations(findings: list[StrategyFinding]) -> list[str]
```

Purpose: Generates a prioritized, actionable recommendation list based on which check IDs fired.

Rule table:

Triggered Checks	Recommendations Added
<code>"no_trades"</code> or <code>"too_few_trades"</code>	1) Reduce <code>min_align</code> from 3 to 2. 2) Verify <code>COLUMN_ALIASES</code> maps <code>'macd_signal'</code> to <code>'macd_signal_flag'</code> .
<code>"negative_cagr"</code> or <code>"negative_sharpe"</code>	1) Switch to LONG-ONLY strategy. 2) Add ATR trailing stops (2x ATR below peak). 3) Add trend filter: only enter long when <code>Close > SMA200</code> .
<code>"excessive_drawdown"</code> or <code>"high_drawdown"</code>	1) Per-trade risk sizing: risk 1–2% equity per trade. 2) Reduce max drawdown stop from 35% to 15%.
<code>"possible_lookahead"</code>	Verify entry fills on next bar (t+1, not t). Reference <code>cerebro.resampled_data</code> .
None of the above	<code>"Continue monitoring strategy performance as market conditions change."</code>

VerificationRunner — `agents/verification_runner.py`

Overview

`verification_runner.py` is the orchestration entry point that invokes all three agents, collects their results, computes an overall verdict, and writes the unified `pipeline_audit.json`. It is designed to be called by `controller.py` (Phase 12) at defined pipeline checkpoints, or run directly from the command line.

CLI interface:

```
python agents/verification_runner.py [--phase {security,data,strategy,all}] [--quiet]
```

Default: `--phase all`.

Exit codes:

- `0` — overall verdict is `PASS`
- `1` — overall verdict is `FAIL`, `ERROR`, or `UNKNOWN`

Files written:

- `pipeline_audit.json` (repo root) — combined output from all agents.

Constants and Imports

File: `agents/verification_runner.py`, lines 23–30

```
BASE_DIR = Path(__file__).parent.parent
AUDIT_OUTPUT = BASE_DIR / "pipeline_audit.json"
sys.path.insert(0, str(BASE_DIR))

from agents.security_audit import run_security_audit
from agents.data_quality import run_data_quality_check
from agents.strategy_audit import run_strategy_audit
```

The runner inserts `BASE_DIR` into `sys.path` so agent imports resolve regardless of working directory.

`run_all_agents(verbose)`

File: `agents/verification_runner.py`, lines 33–121

```
def run_all_agents(verbose: bool = True) -> dict
```

Purpose: Runs all three agents in sequence, catches exceptions, and produces a combined audit result.

Execution order and crash isolation:

Each agent is wrapped in a `try/except Exception`:

```
[1/3] SecurityAuditAgent → results["security"]
[2/3] DataQualityAgent   → results["data_quality"]
[3/3] StrategyAuditAgent → results["strategy_audit"]
```

If an agent raises an uncaught exception, a synthetic error result is inserted:

```
{
  "verdict": "ERROR",
  "message": "<AgentName> crashed: <exception>",
  "findings": []
}
```

Overall verdict logic (lines 91–94):

```
all_verdicts = [r.get("verdict", "UNKNOWN") for r in results.values()]
any_fail = any(v in ("FAIL", "ERROR") for v in all_verdicts)
overall = "FAIL" if any_fail else "PASS"
```

`ERROR` and `FAIL` both count as pipeline failures. `UNKNOWN` is not in the fail-set, so an unrecognized verdict defaults to `PASS` (edge case for future agents).

Combined output written to `pipeline_audit.json` :

```
{
  "pipeline_audit_timestamp": "<ISO 8601>Z",
  "overall_verdict": "PASS" | "FAIL",
  "agents_run": ["security", "data_quality", "strategy_audit"],
  "agent_verdicts": {
    "security": "PASS" | "FAIL" | "ERROR",
    "data_quality": "PASS" | "FAIL" | "ERROR",
    "strategy_audit": "PASS" | "FAIL" | "ERROR"
  },
  "agent_results": {
    "security": { <full SecurityAuditAgent result> },
    "data_quality": { <full DataQualityAgent result> },
    "strategy_audit": { <full StrategyAuditAgent result> }
  },
  "message": "<human-readable overall verdict>"
}
```

Verbose output: Prints banner, per-agent verdict with icons, and path to the audit file.

`run_single_agent(agent_name, verbose)`

File: `agents/verification_runner.py` , lines 124–134

```
def run_single_agent(agent_name: str, verbose: bool = True) -> dict
```

Purpose: Dispatches to a single agent by name string. Accepted names: `"security"` , `"data"` , `"strategy"` .

On unrecognized name: prints error and `sys.exit(1)` .

***Note:** When a single agent is run via `run_single_agent` , results are **not** written to `pipeline_audit.json` by the runner (only `SecurityAuditAgent` writes directly). `DataQualityAgent` and `StrategyAuditAgent` results are only returned in-memory.*

`main()`

File: `agents/verification_runner.py` , lines 137–157

```
def main()
```

Purpose: CLI entry point. Parses `--phase` and `--quiet` arguments.

Argument parsing:

Argument	Choices	Default	Effect
<code>--phase</code>	<code>security</code> , <code>data</code> , <code>strategy</code> , <code>all</code>	<code>all</code>	Which agent(s) to run
<code>--quiet</code>	flag	off	Suppresses verbose output

Exit behavior (lines 155–157):

```
verdict = result.get("overall_verdict", result.get("verdict", "UNKNOWN"))
sys.exit(0 if verdict == "PASS" else 1)
```

Reads `"overall_verdict"` for the combined run, falls back to `"verdict"` for single-agent runs. Exits `1` on anything except `"PASS"` .

Cross-Agent Architecture Notes

Data Flow



```
controller.py (Phase 12)
├─ calls verification_runner.py
│   └─ [1] SecurityAuditAgent — reads .py files, .env, .gitignore, JSON outputs
│       └─ writes pipeline_audit.json (directly)
│       └─ [2] DataQualityAgent — reads news_output/*.json, recession_signals_output.json
│           └─ returns dict only
│       └─ [3] StrategyAuditAgent — reads Module_2_Technical_Analysis/results_run/
│           └─ returns dict only
└─ writes pipeline_audit.json (combined, overwrites SecurityAuditAgent's direct write)
```

Severity Vocabulary by Agent

Agent	Severity Levels	Block Level
DataQualityAgent	BLOCKER, WARN, INFO	BLOCKER
SecurityAuditAgent	CRITICAL, HIGH, MEDIUM, LOW, INFO	CRITICAL or HIGH
StrategyAuditAgent	BLOCKER, WARN, INFO	BLOCKER

pipeline_audit.json Schema (Combined)

The file at repo root is overwritten on every full runner invocation. The schema nests the full result dict from each agent under "agent_results":

```
pipeline_audit.json
├─ pipeline_audit_timestamp (ISO 8601 UTC string)
├─ overall_verdict ("PASS" | "FAIL")
├─ agents_run (list of agent keys)
├─ agent_verdicts (flat key → verdict summary)
├─ agent_results
│   └─ security (full SecurityAuditAgent result)
│   └─ data_quality (full DataQualityAgent result)
│   └─ strategy_audit (full StrategyAuditAgent result)
└─ message (human-readable overall verdict)
```

Pipeline Error Reporting

The CLAUDE.md architecture document specifies a pipeline_errors.json file for failed-verdict logging, but the current implementation consolidates all findings directly into pipeline_audit.json. No separate pipeline_errors.json is written by any agent in the current codebase.

Planned Agents Not Yet Implemented

Per CLAUDE.md, the following agents are planned but have no corresponding files in agents/ :

Planned Agent	Planned Trigger	Status
FinancialFactAgent	After Phase 5 (GPT summarization)	Not implemented
EconomicReasoningAgent	After Phase 6 (reasoning engine)	Not implemented
RecessionSignalAgent	After Phase 13 (recession engine)	Not implemented

These would live in agents/verification/ per the roadmap, though the three current agents live directly in agents/ .

Part 9 — The Cybersecurity Sub-Agent and Its Training

ThinkFree ships with a development-time **AI sub-agent**: an attacker-perspective security reviewer defined as a Claude Code sub-agent. This part documents the agent's definition, its operating rules, and — in full — the knowledge base it was **trained** on (extracted from the founder's curated security-research folder). "Training" here means embedding distilled, verified knowledge into the agent's prompt and reference files; a Claude Code sub-agent learns through its system prompt plus the files it loads, not through fine-tuning.

9.1 Agent definition

File: `.claude/agents/cybersecurity-agent.md`

The file is a Markdown document with YAML front-matter (the agent's metadata) and a system-prompt body. Front-matter fields:

Field	Value	Meaning
<code>name</code>	<code>cybersecurity-agent</code>	The invocable agent name
<code>description</code>	attacker-perspective reviewer; triggers on "check security", "audit before launch", etc.	When the harness auto-selects it
<code>tools</code>	<code>Read, Grep, Glob, Bash, WebFetch</code>	Read + search + run scanners + fetch references
<code>model</code>	<code>inherit</code>	Uses the session model (so reviews get the strongest model)

The agent runs in a read/search/execute posture — it audits and reports, and can drive scanners — but is not given Edit/Write by default, so it cannot silently change code.

9.2 Operating rules (baked into the prompt)

- 1. **Never break functionality.** Any proposed fix must preserve behavior; if it changes behavior, that is called out explicitly. This mirrors the project-wide constraint in Part 2.4.
- 1. **Authorized scope only.** It audits ThinkFree's own repository and its own running instance. It never uses the offensive tooling against third parties.
- 1. **Verify before you alarm.** A finding is real only with an exact `file:line` and a concrete attack path; speculation is labeled as such.
- 1. **Defense in depth.** Client-side checks are UX, never security; controls that matter are enforced server-side; the browser is assumed hostile.
- 1. **No secrets in output.** It never prints a real key, token, password, or hash.

9.3 Stack translation (why generic advice was not enough)

The source material assumes a Supabase/Postgres "Row-Level Security" stack. ThinkFree is **FastAPI + stdlib SQLite + bcrypt + PyJWT**. The agent was given an explicit map so it audits the real architecture: the "enable RLS" lesson becomes **"every `/api/*` endpoint must scope results to the authenticated `current_user` and must never trust an id, email, or tier sent by the client"** — the FastAPI/ SQLite way to prevent IDOR / broken access control. The agent's prompt contains a full description of ThinkFree's real components (the static gate, the JWT cookie, the gated `private_data/` endpoints, the tier model, the market-hours pause).

9.4 The training corpus (source folder)

The knowledge was extracted from `ThinkFree/Agents/Cybersecurity Sub Agent/`, which contains three written guides and a set of GitHub repositories:

Written guides (read in full):

- *How to Prevent Vibe Coded Apps From Being Hacked: Security Checklist and Prompts* (Giga AI / Namanyay Goel) — the three highest-leverage controls: row-level security / access control, rate limiting, and keeping API keys out of code.
- *I'm a Bug Hunter. Here is how I prevent my Vibe-Coded apps from getting

hacked.* (BehiSec) — the attacker mindset ("security is finding the exceptions developers forgot"), eight infrastructure mistakes, and a pre-launch checklist (no secrets in front end, all routes verify auth server-side, users can only access their own data — test with two accounts, dependencies current, `.env` gitignored, DB not internet-exposed, debug off in prod).

- *How to ACTUALLY make your (vibe coded) apps secure (from an actual hacker)*

(cryptoviksant) — CAPTCHA/bot gating, HTTPS everywhere, sanitize every input on both sides, update dependencies (including transitive), WAF, logging/monitoring, and the closing advice: "Read the OWASP Top 10 and dig deeper."

Repositories (inspected directly):

Repo	What it is	How it trained the agent
vibe-guard	28-rule static scanner (TypeScript)	Source of the rule checklist (9.5)
vibe-pen-tester / vibe-coding-penetration-tester	LLM multi-agent pen-testers (Python)	Source of the active-test playbook (9.6)
PayloadsAllTheThings	Payloads per vuln class	Source of the payload cheatsheet (9.7)
SecLists	Wordlists (passwords, discovery, fuzzing)	Source of the wordlist path index (9.7)
public-pentesting-reports	Real reports (Cure53, Doyensec, Bishop Fox, Trail of Bits, NCC)	Reference for finding write-up style
PentestGPT	USENIX Security 2024 autonomous pen-test agent	Methodology reference
awesome-pentest	Curated tool index	Reference index
supabase-js	Supabase client	Excluded — not ThinkFree's stack

9.5 Knowledge file 1 — `vibe-guard-rules.md` (the 28-rule checklist)

Extracted from vibe-guard's `src/rules/*.ts`, this file distills all 28 static rules into a grep-driven checklist. Each rule carries a severity, a plain-English description of the dangerous code shape, 2–4 concrete signatures to grep for, and the fix. The 28 rules:

Web/application (20): broken-access-control, missing-authentication, sql-injection, xss-detection, csrf-protection, exposed-secrets, hardcoded-sensitive-data, unvalidated-input, directory-traversal, insecure-file-upload, insecure-deserialization, insecure-session-management, insecure-random-generation, open-cors, insecure-http, missing-security-headers, insecure-configuration, insecure-error-handling, insecure-logging, insecure-dependencies.

Infrastructure (3): dockerfile-security, container-registry-security, kubernetes-security.

AI-specific (5): prompt-injection-detection, ai-agent-access-control, ai-data-leakage-prevention, ai-generated-code-validation, mcp-server-security. These matter because ThinkFree runs GPT / Chef GPT, so prompt injection and AI-data-leakage are live risks, not hypotheticals.

A notable direct hit: the **insecure-random-generation** rule flags non-crypto randomness used as a security boundary — which is exactly what the static-mode gate's `cyrb53` hash is. The agent therefore knows to flag any reliance on `cyrb53` for real security (it is acceptable only as soft UX).

9.6 Knowledge file 2 — `pentest-playbook.md` (active-test methodology)

Extracted from the two LLM pen-testers, this file documents *how* to actively test each vulnerability class and, crucially, *how to validate a finding before reporting it* (the swarm explicitly drops unvalidated findings).

- **Architecture:** discovery → a specialist agent per vuln class → validation → report.

- **Vuln classes documented:** XSS (browser-instrumented `window.__xss_triggered` confirmation plus reflection checks across four contexts), SQL injection (error/auth-bypass/UNION/time-based with the real SQL error-string set), CSRF (token + SameSite inspection, PoC generation as evidence), Authentication (weak policy, insecure cookies, session-id-in-URL, enumeration, missing logout, default creds, client-side bypass), IDOR (id-parameter probing with access-denied suppression logic), and SSRF (URL/param discovery with internal-IP confirmation).

- **Validation discipline:** a two-stage gate (LLM expert review, then a deterministic re-check) so plausible-but-wrong findings do not survive.

- **SSRF defenses worth copying:** block targets that resolve to private/loopback IPs, and only trust `X-Forwarded-For` behind a verified proxy.

9.7 Knowledge file 3 — `payloads-cheatsheet.md` (payloads + wordlists)

Extracted from PayloadsAllTheThings and SecLists, with an authorization-only notice at the top. It provides the canonical, compact payload set per class — XSS, SQLi, SSRF, IDOR, command injection, directory traversal, CSRF, insecure file upload, and JWT attacks (none-algorithm, RS256→HS256 confusion, null signature, weak-secret brute force, `kid / jku` injection) — and an index of the most useful specific SecLists wordlist paths (common passwords, default creds, web-content discovery, usernames, vuln-specific fuzzing lists). The JWT section is directly relevant because ThinkFree's sessions are JWTs.

9.8 How the agent uses its training

The agent's prompt instructs it to **read all three knowledge files at the start of every audit**, then work the checklist top-down in priority order: broken access control → secret leakage → rate limiting → injection → transport/headers → infrastructure/deploy → dependencies. Its output is a structured report: a PASS/FAIL verdict, findings (each with severity, OWASP category, `file:line`, the attack, the minimal non-breaking fix, and a confidence level), manual tests to run (e.g. the two-account IDOR test), and pre-launch reminders (key rotation, production flags). It can also drive the existing Python static scanner `agents/security_audit.py` (Part 8).

9.9 Relationship to the runtime verification agents

This sub-agent is a **development-time reviewer** (it reasons over the code and helps an engineer). It complements — but is distinct from — the **runtime** Python verification agents in `agents/` (Part 8), which execute inside the data pipeline and gate it with pass/fail verdicts. Together they implement the blueprint's "two LLMs catch more than one" principle: independent review plus automated static gating.

9.10 Distribution note

The agent and its knowledge base live under `.claude/`, which is gitignored, so they are local to the development machine and were not pushed to the public repository. Making them shareable would require either relocating them or adding a scoped gitignore exception; this is a deliberate, reversible choice.

Part 10 — Library & Dependency Inventory

This part lists every third-party library the platform depends on, what it is used for, and any version pin and the reason for it. Two dependency sets exist: the **pipeline** set (`requirements.txt` , installed into the legacy `tf_env`) and the **backend** set (`server/requirements.txt` , installed into a clean `server/.venv`). The two are deliberately kept separate so the lean web backend does not drag in the heavy ML/quant stack.

10.1 Backend dependencies (`server/requirements.txt`)

Installed into a clean virtualenv (never reuse `tf_env`).

Library	Pin	Purpose
fastapi	0.111.0	The web framework: routing, dependency injection, request validation
uvicorn[standard]	0.30.1	ASGI server that runs the FastAPI app
bcrypt	4.1.3	Password hashing (slow by design; resists brute force)
PyJWT	2.8.0	Signed session tokens stored in the httpOnly cookie
python-dotenv	1.0.1	Loads secrets from <code>server/.env</code> (never committed)

The backend otherwise relies on the Python **standard library** only: `sqlite3` (user store), `hmac` (constant-time beta-key compare), `hashlib` , `datetime` , `json` , `re` , `os` , `pathlib` , `zoneinfo` (market-hours), and `secrets` .

10.2 Pipeline dependencies (`requirements.txt`)

Grouped by role. Several pins are mandatory for macOS / Python 3.10 compatibility (see 10.4).

AI / LLM & embeddings

Library	Pin	Purpose
openai	>=1.0.0	GPT calls for summarization, reasoning, and the Chef GPT briefing
langchain	1.3.9	LLM orchestration framework
langchain-community	>=0.3	Community integrations (vector stores, loaders)
langchain-openai	>=0.2	OpenAI provider for LangChain
tiktoken	(unpinned)	Token counting for prompt budgeting
sentence-transformers	2.7.0	Sentence embeddings for clustering / semantic search
transformers	4.41.2	Underlying model library for embeddings
torch	2.2.2	Tensor backend for transformers
faiss-cpu	(unpinned)	Vector index for the economic-reasoning knowledge base

Quant / technical analysis / backtesting

Library	Pin	Purpose
backtrader	(unpinned)	Backtesting engine (Phase 9)
QuantLib	(unpinned)	Quantitative finance math (Phase 8 metrics)
ta	(unpinned)	Technical-analysis indicators
scikit-learn	(unpinned)	Clustering (KMeans/Agglomerative), ML utilities
scipy	(unpinned)	Scientific computing used by the quant/stat code
numpy	<2	Numerical arrays (pinned, see 10.4)
pandas	>=2.0.0	Dataframes throughout the pipeline

Market & economic data

Library	Pin	Purpose
yfinance	(unpinned)	Yahoo Finance market data
fredapi	(unpinned)	FRED macroeconomic series (yield curve, etc.)
requests	(unpinned)	HTTP client for external APIs
feedparser	(unpinned)	RSS news ingestion
beautifulsoup4	(unpinned)	HTML parsing/scraping

NLP / sentiment

Library	Pin	Purpose
vaderSentiment	(unpinned)	Lexicon sentiment scoring
(nltk)	(imported)	NLP utilities used in extraction

Delivery / visualization (legacy prototype)

Library	Pin	Purpose
streamlit	>=1.32.0	The legacy <code>dashboard/app.py</code> prototype UI
plotly	>=5.0.0	Charts in the Streamlit prototype

Validation, security & utilities

Library	Pin	Purpose
pydantic	>=2.0.0	Schema validation for inter-module data contracts
validators	(unpinned)	Input sanitization for URLs, emails, tickers
ratelimit	(unpinned)	Decorator-based rate limiting on external API calls
python-dotenv	>=1.0.0	Loads pipeline secrets from <code>.env</code>
tqdm	(unpinned)	Progress bars for long builds

Other third-party libraries seen in imports

Library	Where	Purpose
qlib	<code>Module_2_Technical_Analysis/</code>	Quant research data layer for TA/backtesting
fitz (PyMuPDF)	book/PDF ingestion	Reading economics PDFs into the knowledge base
ijson	bulk SEC ingestion	Streaming JSON parse of large filings
dulwich	<code>push_to_github.py</code>	Pure-Python git operations

10.3 Standard-library usage (notable)

Across the pipeline the stdlib does much of the heavy lifting: `json` (58 files), `pathlib` (50), `os` (28), `urllib` (25), `datetime` (24), `typing` (19), `re` (18), `time` (14), plus `sys`, `collections`, `math`, `argparse`, `concurrent.futures`, `threading`, `subprocess`, `gzip`, `zipfile`, `csv`, `io`, `difflib`, `functools`, `itertools`, `dataclasses`, `logging`, `ssl`, `hashlib`, `hmac`, `sqlite3`. `from __future__ import annotations` appears in 51 files (forward-reference typing).

10.4 Version-pin rationale

The blueprint records hard pins for macOS / Python 3.10 stability:

- `numpy < 2` — required for compatibility with the torch 2.2.x ABI.
- `sentence-transformers == 2.7.0`, `transformers == 4.41.2`,

`torch == 2.2.2` — this exact triple is the known-good combination on macOS Python 3.10. A documented constraint:

`SentenceTransformer()` can **segfault** on macOS with torch 2.2.2 if the model is loaded at runtime; the economic engine therefore avoids

loading the model live and instead keyword-scans FAISS metadata. Engineers reproducing the environment must honor these pins.

10.5 Frontend dependencies

None. The `webapp/` front end has **zero external or CDN dependencies**. It is pure vanilla JavaScript, HTML, and CSS, all served locally. `index.html` loads only three local scripts up front (`boot.js` , `auth_seed.js` , `auth.js`); every other module is injected locally by `boot.js` after authentication. The logout animation and all icons are hand-authored inline SVG — no icon library, no framework (no React/Vue/jQuery), and no analytics or third-party trackers.

This is a deliberate security and performance choice: there is no supply-chain surface in the browser bundle, nothing phones home, and the only code the browser runs is code in this repository.

10.6 Security tooling (referenced by CI and the audit agent)

The CI security workflow and the static audit agent use: **bandit** (static analysis), **safety** / **pip-audit** (dependency CVE scans), and **detect-secrets** (pre-commit secret blocking). These are invoked as scanners (see Parts 8 and 11) rather than imported as libraries.

Part 11 — Security Model & Deployment

11.1 The security model in one line

Anything sent to the browser is public. So the hosted app sends only the UI shell and keeps keys, Python, data, and identity on the server, released only to a logged-in user.

11.2 What the model achieves

Goal	Status	How
No API keys exposed	Achieved	Keys live in server env vars (<code>server/.env</code> / host dashboard), read only by Python; never sent to the browser
Python code unreadable	Achieved	Python runs server-side and is never served; <code>.py</code> stays out of <code>webapp/</code>
Data files not downloadable	Achieved	Datasets moved to <code>private_data/</code> (not web-served), returned only via <code>/api/data/*</code> behind login; the server hard-404s any <code>*_data.js</code> path
Tiers / auth not bypassable	Achieved	Login, sessions, and tier cadence are enforced server-side; editing the browser cannot change them
"Animated JavaScript" unreadable	Not possible	The browser must download JS to run it; it is minified, but UI/animation code is always readable. It contains no secrets or bulk data, so this is acceptable

The honest caveat is stated plainly to stakeholders: front-end JavaScript is always readable by the browser. Only keys, Python, and bulk data can truly be hidden — and those are.

11.3 Authentication & session security

- **Passwords** are hashed with **bcrypt** (slow by design). Plaintext is never stored.
- **Sessions** are **PyJWT** tokens placed in a cookie that is **httpOnly** (JavaScript, and therefore an XSS payload, cannot read it), **Secure** in production, and **SameSite=lax**.
 - **JWT revocation:** the user row carries a `token_valid_after` epoch; a token whose `iat` predates it is rejected. Logout bumps this value to "now," which immediately invalidates any older or stolen cookie.
 - **Brute-force throttling (in-memory):** login is limited per IP+email (5 / 15 min) and per IP (a 20 / 15 min spray cap); signup is limited per IP (10 / hour) to protect the closed-beta invite key. Failures clear on success.
 - **Beta-key check** uses `hmac.compare_digest` (constant time) so the invite key cannot be guessed by timing.

11.4 Signup integrity controls

- **Age gating (liability):** a required consent checkbox ("I confirm I am 18 or older, or I am at least 13 and have explicit permission from a parent or guardian"), plus a hard floor of 13 enforced **both** client-side (the quiz rejects ages below 13) and server-side (the profile endpoint re-checks). Neither can be bypassed by editing the browser.
- **Email validity:** a stricter format regex (real TLD required) on both client and server, plus a client-side **DNS-over-HTTPS domain-existence check** (MX, falling back to A record) that rejects unregistered nonsense domains while accepting any real **custom** domain. It fails **open** if DNS is unreachable so legitimate users are never locked out. The documented limitation: this catches nonexistent domains, not typos at real domains; true mailbox verification would require sending a confirmation code (a future enhancement).

11.5 Transport & HTTP hardening (server middleware)

- **Security headers** on every response: Content-Security-Policy, HSTS (Strict-Transport-Security), X-Frame-Options / frame-ancestors, X-Content-Type-Options.
- **HTTPS redirect** in production via `X-Forwarded-Proto` (defaults to https)

avoid redirect loops; exempts `/healthz`).

- **CORS** wired from `config.ALLOWED_ORIGINS` (inert under a single-origin

deployment, ready if the front end and API are ever split).

- **Per-tier rate limiting + market-hours pause** on the live-data endpoint

(`_last_live` tracking; zoneinfo-based market check), so lower tiers stop generating API calls after the close while premium tiers keep refreshing.

- The server **refuses to boot in production** if `TF_SECRET_KEY` is still the placeholder value.

11.6 Secret hygiene

- Secrets live only in gitignored `.env` (root, for the pipeline) and

`server/.env` (for the backend), or in the host's dashboard env vars.

- Gitignored and confirmed never-tracked: `.env`, `server/.env`,

`webapp/js/auth_seed.js` (holds password hashes), `private_data/*`, `server/users.db`, and `.claude/` (except the security agent, which is deliberately re-included for sharing — see Part 9.10).

- Before every push, sensitive paths are verified gitignored *and* absent from

the tracked tree, and a key-pattern scan is run over everything committable.

11.7 CI security gate

`.github/workflows/security.yml` runs on push/PR and executes **detect-secrets**, **bandit**, **safety**, and the project's own `agents/security_audit.py`. The audit `sys.exit(1)`s on FAIL, so a failing security check blocks the pipeline.

11.8 Local run

```
# 1. Clean virtualenv (do NOT reuse tf_env)
python3 -m venv server/.venv
server/.venv/bin/pip install -r server/requirements.txt

# 2. Secrets
cp server/.env.example server/.env
python3 -c "import secrets; print(secrets.token_urlsafe(48))" # -> TF_SECRET_KEY
# edit server/.env: set TF_SECRET_KEY, TF_BETA_KEY, keep TF_PRODUCTION=0 locally

# 3. Build the gated datasets from the webapp/js/*_data.js bundles
python3 scripts/extract_data_to_json.py # writes private_data/*.json + manifest.json

# 4. Run
server/.venv/bin/uvicorn main:app --app-dir server --reload --port 8000
# open http://localhost:8000
```

Gating sanity checks:

```
curl -i http://localhost:8000/api/data/bundle # expect 401 (not logged in)
curl -i http://localhost:8000/js/data.js      # expect 404 (blocked)
curl -i http://localhost:8000/api/auth/me    # expect 401
```

11.9 Production hosting (recommended path)

1. Push to a **private** GitHub repo; confirm `server/.env`, `private_data/*`, and

`users.db` are gitignored.

1. Deploy to Render / Railway / Fly.io:

- Start: `uvicorn main:app --app-dir server --host 0.0.0.0 --port $PORT`
- Build: `pip install -r server/requirements.txt`
- Set env vars in the dashboard (not a committed file): `TF_SECRET_KEY`,

`TF_BETA_KEY`, `TF_PRODUCTION=1`, `TF_ALLOWED_ORIGINS=https://your-domain`, plus the data-pipeline keys.

- Run `scripts/extract_data_to_json.py` at build time or mount `private_data/`

as a persistent volume.

1. Put **Cloudflare** in front: automatic HTTPS/TLS, HTTP/2, WAF (managed

ruleset), Bot Fight Mode, "Always Use HTTPS," and rate limiting on `/api/*`. This is the DDoS/bot/TLS layer that makes the backend "safe for beta."

11.10 Pre-launch security checklist

- [] `TF_SECRET_KEY` is a fresh 48+ char random value (app refuses to start in prod with the placeholder).
- [] `TF_PRODUCTION=1` in prod (cookies `Secure`, HSTS sent).
- [] **Rotate every data-pipeline API key** (LegiScan, Open States, Finnhub, OpenAI, etc.) before launch. Any key ever committed, shared, or pasted into a config is considered burned.
- [] No key string appears anywhere under `webapp/`: `grep -rEi "api[_]?key|secret|token" webapp/`.
- [] `private_data/`, `server/.env`, `users.db` are not in git: `git status --ignored`.
- [] The old `webapp/js/*_data.js` are removed from the deployed shell once the front end fetches from `/api/data/*` (the server blocks them regardless).
- [] Cloudflare WAF + rate limiting on `/api/auth/*`.
- [] Backups of `users.db` (or move to managed Postgres before scaling past beta).

11.11 Known not-yet-done (by design)

- Password reset and email verification (needs an email provider; arguably

redundant for an invite-key beta).

- The TLS certificate itself (the host's responsibility; set `TF_PRODUCTION=1`).
- End-to-end run of the extractor + uvicorn in this sandbox (no network, broken

legacy venv); the backend Python all compiles cleanly and the JS parses, but the founder must run the full login → data → render flow on a real machine.

Part 12 — Dataset & Data-Source Catalog

This document maps every data bundle consumed by the ThinkFree Finance front-end to its `window.NAME` global, the file it lives in, the build script that produces it, the external government or commercial data source it draws from, whether it is **core** (sent immediately after login), **lazy** (fetched on first State Legislature view), or **live** (refreshed on each user's tier cadence), and which UI panels it powers.

Political-intelligence framing rule (non-negotiable). Every dataset that touches congressional trading, lobbying activity, government contracts, campaign finance, or legislative action describes timing relationships between publicly available government disclosures and market events. It does not imply or allege wrongdoing of any kind. This framing is enforced in the disclaimer fields embedded in each affected data file and is repeated in every UI card that renders the data.

Master Catalog Table

Global name	Webapp file	Build script	External source	Classification	Powers in UI
TF_DATA	js/data.js	build_data.py	Congress.gov RSS, QuiverQuant, pipeline JSONs	core	Politician trades, bills, news feed, correlation engine, dashboard header
IW_DATA	js/influence_data.js	build_influence.py	LittleSis (littlesis.org) bulk dump	core	InfluenceWeb graph — board members, executives, donors, owners
RELATIONSHIPS	js/relationships_data.js	build_relationships.py	U.S. Senate Lobbying Disclosure Act (LDA) API	core	InfluenceWeb relationship edges — lobbying firms, lobbyists, issue areas, bills
NP_DATA	js/nonprofit_data.js	build_nonprofits.py	ProPublica Nonprofit Explorer (IRS Form 990)	core	InfluenceWeb industry-group nodes — revenue, expenses, assets
FEC_DATA	js/fec_data.js	build_fec.py	OpenFEC / Federal Election Commission API	core	Politician hover cards — receipts, PAC vs. individual money, top contributors
SEC_DATA	js/sec_data.js	build_sec.py	sec-api.io (SEC EDGAR)	core	Company detail panel — board members, directors, latest annual filing
SECBULK_DATA	js/secbulk_data.js	build_secbulk.py	SEC EDGAR bulk zips (companyfacts + submissions)	core	Company financials — revenue, assets, net income, SIC, filing counts
USA_DATA	js/usaspending_data.js	build_usaspending.py	USASpending.gov federal awards API (2020-2025)	core	Company detail + InfluenceWeb — total federal contracts, top awarding agencies
STATES_DATA	js/states_data.js	build_states.py + build_bea.py + build_bls.py + build_census.py	FRED (St. Louis Fed), BEA RPP 2023, BLS CPI, Census ACS	core	US Map choropleth, State Accountability panel — Prosperity/Pressure scores

Global name	Webapp file	Build script	External source	Classification	Powers in UI
SP500	js/sp500_data.js	build_sp500.py	github.com/datasets/s-and-p-500-companies (GICS, public domain)	core	Sector membership lists in InfluenceWeb; ticker universe for price lookups
QUANT_DATA	js/quant_data.js	build_quant.py	yfinance history + QuantLib (Black-Scholes/VaR/Kelly) + ta indicators	core	Predictive Market Signals panel — RSI, MACD, SMA, VaR, Kelly, Sharpe
MEMBER_BILLS	js/member_bills.js	build_member_bills.py	Congress.gov API	core	Politician hover cards — sponsored + cosponsored bill counts
PRICES_DATA	js/prices_data.js	build_prices.py	Finnhub (finnhub.io)	live	Every ticker popup — price, % change, market cap, industry; Quant signals
NEWS_INTEL	js/news_intel.js	build_news_intel.py	ThinkFree pipeline + GPT-4o-mini (Chef GPT)	live	News Intelligence panel — per-sector and per-ticker AI summaries
LEGISCAN_DATA	js/legiscan_data.js	build_legiscan.py	LegiScan Public API	lazy	State Legislature Watch — bills, sponsors, vote breakdowns, economic topics
OPENSTATES_DATA	js/openstates_data.js	build_openstates.py	Open States / Plural API v3	lazy	State Legislature Watch — legislator photos, email, party, chamber
US_MAP_PATHS / US_MAP_VIEWBOX	js/usmap_paths.js	(static — not generated)	Wikimedia Blank US Map SVG (public domain)	static logic asset	US Map SVG renderer — 50-state outlines

Classification definitions (from `scripts/extract_data_to_json.py` and `server/data.py`): - **core** — sent in a single `GET /api/data/bundle` response immediately after login; populates the globals required by every view. - **lazy** — fetched only on first visit to the State Legislature view via `GET /api/data/state`; kept off the initial page load because the two files together are ~3.5 MB. - **live** — fetched via `GET /api/data/live`; cadence is enforced per user tier (lower tiers pause during market-closed hours); contains prices and AI-generated news intelligence. - **static logic asset** — not a data bundle; always served directly by the static file host (not blocked by the server middleware); contains map geometry, not financial data.

In **static mode** (plain `file://` open without the FastAPI backend) the globals are set by injecting `<script>` tags for each file in the order defined in `boot.js`; `legiscan_data.js` and `openstates_data.js` are still lazy-injected on first State Legislature view. In **server mode** the same `_data.js` files on disk are blocked by the server middleware (`_BLOCKED_SUFFIXES` in `server/main.py`); the data arrives only through the authenticated `/api/data/*` routes.

Dataset Groups

Core — Primary Intelligence Bundle

TF_DATA — ThinkFree Master Dataset

- **File:** `webapp/js/data.js` (~1.41 MB)
- **Global:** `window.TF_DATA`
- **Build script:** `webapp/build_data.py`
- **Source:** Aggregates from the full ThinkFree Python pipeline — `politician_performance.json`, `enriched_news_sentiment.json`, `signal_output_phase3.json`, `economic_reasoning_summary.json`, `news_output/sector_summaries.json`, `articles.json`,

`correlation/top_bills.json` , `bill_correlations/*.json`

- **Top-level keys:** `generated_at` , `user` , `disclaimer` , `news` , `recent_trades` , `politicians` , `bills` , `correlation` , `sectors`
- **Powers:** Every dashboard panel. The `recent_trades` list drives the congressional trading accountability layer; `politicians` populates hover cards and FEC lookups; `news` feeds the News Feed view; `bills` drives legislative activity; `correlation` powers the sector-bill correlation engine.
- **Political-intelligence note:** The `disclaimer` field embedded in `TF_DATA` states: *"This analysis identifies timing relationships between publicly available government disclosures and market events. It does not imply or allege wrongdoing of any kind."* This disclaimer is referenced by `app.js` and reproduced in every politician-trade card.

IW_DATA — InfluenceWeb Relationship Graph

- **File:** `webapp/js/influence_data.js` (~247 KB)
- **Global:** `window.IW_DATA`
- **Build script:** `webapp/build_influence.py`
- **Source:** LittleSis (littlesis.org) bulk entity + relationship dump (two gzip files: `entities.json.gz` ~73 MB, `relationships.json.gz` ~105 MB). Build script streams both files and extracts only companies matched to ThinkFree tickers. LittleSis relationship category IDs used: `CAT_POSITION=1` (board/exec), `CAT_DONATION=5` , `CAT_OWNERSHIP=10` , `CAT_LOBBYING=7` .
- **Top-level keys:** `source` , `companies` (keyed by ticker), each entry containing `name` , `entity_id` , `blurb` , `website` , `board` , `executives` , `donors` , `relationships`
- **Powers:** The InfluenceWeb interactive graph — the centrepiece accountability visual showing board interlocks, executive mobility, and donor networks between corporations, politicians, and industry groups.

RELATIONSHIPS — Federal Lobbying Disclosure Graph

- **File:** `webapp/js/relationships_data.js` (~283 KB)
- **Global:** `window.RELATIONSHIPS`
- **Build script:** `webapp/build_relationships.py`
- **Source:** U.S. Senate Lobbying Disclosure Act (LDA) API (`lda.senate.gov/api/v1/filings/`). Anonymous access, rate-limited to ~15 requests/minute. Year 2024 filings. No API key required.
- **Top-level keys:** `source` , `year` , `disclaimer` , `byTicker` — each ticker entry contains `client` , `filings` , `spend_fmt` , `firms` (lobbying firms), `lobbyists` , `issues` , `bills`
- **Powers:** InfluenceWeb relationship edges — lobbying spend, which firms were hired, which issue areas and specific bills were lobbied on. Cross-referenced with `IW_DATA` for graph rendering.
- **Political-intelligence note:** Embedded `disclaimer` field states: *"Public federal lobbying disclosures. Describes relationships only; does not imply wrongdoing."*

NP_DATA — Industry Group Financials (IRS Form 990)

- **File:** `webapp/js/nonprofit_data.js` (~8 KB)
- **Global:** `window.NP_DATA`
- **Build script:** `webapp/build_nonprofits.py`
- **Source:** ProPublica Nonprofit Explorer API (`projects.propublica.org/nonprofits/api/v2`). No authentication required. Data originates from IRS Form 990 public filings.
- **Top-level keys:** `source` , `byName` — each entry: `ein` , `name` , `city` , `state` , `ntee` , `ntee_label` , `revenue` , `revenue_fmt` , `expenses` , `expenses_fmt` , `assets` , `assets_fmt` , `year` , `url`
- **Organisations covered:** ~24 major lobbying nonprofits and trade associations (e.g., PhRMA, American Petroleum Institute, US Chamber of Commerce, Business Roundtable, Semiconductor Industry Association).
- **Powers:** InfluenceWeb industry-group nodes — shows real financial scale (revenue, assets) of the advocacy organizations that appear in the lobbying graph.

FEC_DATA — Campaign Finance

- **File:** `webapp/js/fec_data.js` (~28 KB)
- **Global:** `window.FEC_DATA`
- **Build script:** `webapp/build_fec.py`

- **Source:** OpenFEC API (api.open.fec.gov/v1/). API key read from `.env` as `FEC_API_KEY` , never written to output. Data covers politicians tracked in ThinkFree's `politician_performance.json` .
- **Top-level keys:** `source` , `byName` — each politician entry: `candidate_id` , `fec_name` , `party` , `office` , `url` , `receipts` , `receipts_fmt` , `from_individuals` , `from_pacs` , `disbursements` , `cash_on_hand` , `top_contributors` .
- **Powers:** Politician detail panel and hover cards — total fundraising, PAC vs. individual money split, top-contributor employer breakdown. Part of the accountability score pipeline (`scores.js`).
- **Political-intelligence note:** This data describes publicly reported campaign-finance disclosures. It does not imply or allege wrongdoing of any kind.

SEC_DATA — Board & Director Data (sec-api.io)

- **File:** `webapp/js/sec_data.js` (~165 KB)
- **Global:** `window.SEC_DATA`
- **Build script:** `webapp/build_sec.py`
- **Source:** sec-api.io (third-party SEC EDGAR aggregator). API key read from `.env` as `SECAPI_IO_KEY` . Pulls current Directors & Board Members and the latest annual DEF 14A proxy for each InfluenceWeb company. Resume-safe: already-fetched companies are preserved across runs.
- **Top-level keys:** `source` , `byTicker` — each entry: `entityName` , `filedAt` , `board` (array of `{name, position, age, since, committees, independent}`)
- **Powers:** Company detail panel — board composition, director independence, committee memberships, and director age/tenure. Cross-referenced against `IW_DATA` for the InfluenceWeb graph node detail view.

SECBULK_DATA — SEC EDGAR Bulk Financials

- **File:** `webapp/js/secbulk_data.js` (~61 KB)
- **Global:** `window.SECBULK_DATA`
- **Build script:** `webapp/build_secbulk.py`
- **Source:** SEC EDGAR official bulk datasets — `companyfacts.zip` and `submissions.zip` (downloaded manually to `~/Downloads` ; multi-GB files, CIK-keyed). Reads only the per-company JSON slices for tracked tickers. No network at runtime; no rate limits.
- **Top-level keys:** `source` , `byTicker` — each entry: `cik` , `name` , `sic` , `exchange` , `ein` , `filings_total` , `last_filing` , `form_counts` , `revenue` , `revenue_fmt` , `fy` , `assets` , `assets_fmt` , `net_income` , `net_income_fmt` , `liabilities` .
- **Powers:** Company financials panel — real XBRL balance-sheet figures (revenue, assets, net income), SIC classification, total filing count, filing type breakdown. Also used by `app.js` to resolve company names from tickers (`stockName()`).

USA_DATA — Federal Government Contracts

- **File:** `webapp/js/usaspending_data.js` (~217 KB)
- **Global:** `window.USA_DATA`
- **Build script:** `webapp/build_usaspending.py`
- **Source:** USASpending.gov API (api.usaspending.gov). No API key required. Covers federal contract awards types A–D, fiscal years 2020–2025, for every S&P 500 company and all other InfluenceWeb-tracked tickers.
- **Top-level keys:** `source` , `byTicker` — each entry: `recipient` , `total_contracts` , `total_contracts_fmt` , `top_agencies` (array of `{agency, amount, amount_fmt}`), `url` .
- **Powers:** Company detail panel, InfluenceWeb contractor view — total dollar value of federal contracts, top awarding agencies (DoD, DHS, HHS, etc.). A key element of the government-industry relationship layer.
- **Political-intelligence note:** This data is drawn from mandatory public federal award disclosures. It identifies which companies received taxpayer-funded contracts and from which agencies. It does not imply or allege wrongdoing of any kind.

STATES_DATA — State Economic Indicators

- **File:** `webapp/js/states_data.js` (~25 KB)
- **Global:** `window.STATES_DATA`
- **Build scripts (pipeline of four):**
 1. `webapp/build_states.py` — FRED series (`{ST}UR` , `{ST}STHPI` , `MEH0INUS{ST}A672N` , `{ST}PCPI`). API key: `FRED_API_KEY` .
 2. `webapp/build_bea.py` — BEA Regional Price Parities 2023. API key: `BEA_API_KEY` .

3. `webapp/build_bls.py` — BLS regional CPI year-over-year inflation by Census region.
 4. `webapp/build_census.py` — Census ACS 1-year: median home value, median rent, median household income, poverty rate, population.
- **External sources:** Federal Reserve Bank of St. Louis (FRED), Bureau of Economic Analysis (BEA), Bureau of Labor Statistics (BLS), U.S. Census Bureau ACS.
 - **Top-level keys:** `source`, `byState` — each state entry: `name`, `unemployment`, `house_price_index`, `house_price_growth`, `per_capita_income`, `income_growth`, `median_household_income`, `prosperity_score`, `pressure_score`, `median_home_value`, `median_rent`, `poverty_count`, `population`, `poverty_rate`, `price_to_income`, `affordability`, `rent_burden_pct`, `cost_of_living`, `real_purchasing_power`
 - **Powers:** US Map choropleth (colour-coded by Prosperity or Pressure score), State Accountability panel, and the "how this affects your state" contextualisation layer. All 50 states + DC covered.

SP500 — S&P 500 Sector Membership

- **File:** `webapp/js/sp500_data.js` (~56 KB)
- **Global:** `window.SP500`
- **Build script:** `webapp/build_sp500.py`
- **Source:** `github.com/datasets/s-and-p-500-companies` (public domain CSV: ticker, company, GICS sector, GICS sub-industry). No API key required.
- **Top-level keys:** `source`, `byTicker` — each entry: `name`, `sector` (ThinkFree mapped sector), `gics` (original GICS sector), `sub` (GICS sub-industry); also `bySector` summary counts.
- **Sector mapping:** GICS is remapped to ThinkFree's internal sectors (e.g., `Industrials` → `Defense` if aerospace/defense sub-industry, `Health Care` → `Pharmaceuticals` if pharma/biotech, `Financials` → `Financial Services`, etc.).
- **Powers:** InfluenceWeb sector lists ("View All" ticker membership per sector); also the ticker universe seed for `build_prices.py`.

QUANT_DATA — Predictive Market Signals

- **File:** `webapp/js/quant_data.js` (~7 KB)
- **Global:** `window.QUANT_DATA`
- **Build script:** `webapp/build_quant.py`
- **Source:** Live price history via `yfinance`; quantitative math from `quantlib_metrics.py` (Black-Scholes $N(d^2)$ probability, annualized volatility, 95% VaR, lognormal expected return, Kelly criterion); technical indicators from the `ta` library (RSI, MACD, SMA-50/200 crossovers). No API keys.
- **Top-level keys:** `source`, `byTicker` — each entry: `price`, `volatility`, `var95`, `prob_up`, `prob_down`, `exp_return_1mo`, `kelly`, `sharpe`, `rsi`, `rsi_signal`, `macd`, `macd_signal_line`, `macd_cross`, `sma50`, `sma200`, `trend`, `ta_score`, `ta_signal`, `ql_signal`
- **Candidate tickers:** Derived from the `TF_DATA` news ticker mentions and recent congressional trades — ~22 most-active tickers with available price data.
- **Powers:** Predictive Market Signals panel in `predictions.js` — RSI, MACD, trend signals, QuantLib probability scores, Kelly fractions, and Sharpe ratios shown per ticker.

MEMBER_BILLS — Congressional Legislative Activity

- **File:** `webapp/js/member_bills.js` (~4.5 KB)
- **Global:** `window.MEMBER_BILLS`
- **Build script:** `webapp/build_member_bills.py`
- **Source:** Congress.gov API (`api.congress.gov/v3/member/`). API key: `CONGRESS_API_KEY`, read from `.env`, never logged. Covers every politician in `TF_DATA.politicians` that has a bioguide ID.
- **Top-level keys:** `source`, `byBioguide` — each entry: `sponsored` (count), `cosponsored` (count), `total`
- **Powers:** Politician hover cards — corrects the misleading "0–1 bills" that appeared when only trade-correlated bills were counted; now shows full real legislative activity.

Live — Tier-Gated Refresh Bundle

PRICES_DATA — Live Market Prices

- **File:** `webapp/js/prices_data.js` (~186 KB)

- **Global:** `window.PRICES_DATA`
- **Build script:** `webapp/build_prices.py`
- **Source:** Finnhub (`finnhub.io`). API key: `FINNHUB_API_KEY` , read from `.env` , never written to output or logs. Free tier: 60 calls/minute; build sleeps 1.1 s between requests. Covers the full S&P 500 plus all tickers referenced in SEC bulk, InfluenceWeb, USASpending, congressional trades, and bill tickers.
- **Top-level keys:** `source` , `byTicker` — each entry: `price` , `change` , `change_pct` , `market_cap` , `market_cap_fmt` , `name` , `industry`
- **Powers:** Every company popup and ticker card across the app — real-time price, percentage change, market cap, and industry classification. Also seeds the `build_quant.py` candidate ticker selection.

NEWS_INTEL — AI News Intelligence (Chef GPT)

- **File:** `webapp/js/news_intel.js` (~12 KB)
- **Global:** `window.NEWS_INTEL`
- **Build script:** `webapp/build_news_intel.py`
- **Source:** ThinkFree's own pipeline outputs (local, no API): `signal_output_phase3.json` (technical signals), `quant_data.js` (QuantLib metrics), `news_output/sector_summaries.json` , `economic_reasoning_summary.json` . Final plain-English translation: one batched GPT-4o-mini call per sector (~8–10 API calls total). Model configurable via `THINKFREE_NEWS_MODEL` env var.
- **Top-level keys:** `source` , `model` , `api_calls` , `bySector` (sector → `summary` , per-ticker `what_it_means`), `byTicker`
- **Powers:** News Intelligence panel — plain-English sector and per-ticker summaries combining technical signals, QuantLib scores, and economic reasoning, translated by Chef GPT into jargon-free language for the everyday investor.

Lazy — State Legislature Bundle (loaded on first State view)

LEGISCAN_DATA — State Legislative Activity

- **File:** `webapp/js/legiscan_data.js` (~2.03 MB — largest dataset)
- **Global:** `window.LEGISCAN_DATA`
- **Build script:** `webapp/build_legiscan.py`
- **Source:** LegiScan Public API (`api.legiscan.com`). API key: `LEGISCAN_API_KEY` , read from `.env` . Free-tier budget: 30,000 queries/month; typical build uses ~400 calls. Covers all 50 states + DC.
- **Top-level keys:** `source` , `disclaimer` , `byState` — each state: `state` , `name` , `session` (`id` , `name` , `year_start/end` , `special`), `bill_total` , `status_breakdown` , `relevant_count` , `bills` (array of up to 40 economically-relevant bills with sponsors and roll-call vote summaries for top 6)
- **Economic topic tags:** taxes, housing, wages, health, energy, education, environment, criminal justice, agriculture, infrastructure, trade (applied at build time to classify bills for the "how this affects your life" translation layer).
- **Powers:** State Legislature Watch panel — full state-by-state bill activity, economic relevance scoring, sponsor attribution, vote breakdowns.
- **Political-intelligence note:** Embedded `disclaimer` states: *"This data describes publicly available state-legislative activity. It does not imply or allege wrongdoing of any kind."*

OPENSTATES_DATA — State Legislator Profiles

- **File:** `webapp/js/openstates_data.js` (~1.43 MB)
- **Global:** `window.OPENSTATES_DATA`
- **Build script:** `webapp/build_openstates.py`
- **Source:** Open States / Plural API v3 (`v3.openstates.org/people`). API key: `OPENSTATES_API_KEY` , read from `.env` . Rate-limited to 1 req/sec on the free tier (~500/day). Build degrades gracefully — a state that fails is skipped, not fatal. Covers all 50 states + DC.
- **Top-level keys:** `source` , `byState` — each state: `name` , `legislators` (array of {`name` , `party` , `chamber` , `district` , `image` , `email` , `openstates_url`})
- **Powers:** State Legislature Watch — legislator photo, email address, party badge, chamber, and district alongside the LegiScan bill data. Cross-referenced by name at render time in `app.js` .

Static Logic Asset

US_MAP_PATHS / US_MAP_VIEWBOX — SVG Map Geometry

- **File:** `webapp/js/usmap_paths.js` (~31 KB)
- **Globals:** `window.US_MAP_VIEWBOX` (string), `window.US_MAP_PATHS` (object keyed by 2-letter state code)
- **Build script:** Not generated — static file, checked into the repo.
- **Source:** Wikimedia-derived Blank US Map SVG (public domain).
- **Top-level keys:** `US_MAP_VIEWBOX` (SVG viewBox string), `US_MAP_PATHS` (state code → SVG path `d` string)
- **Classification:** Loaded as a normal logic script in `boot.js` `LOGIC_SCRIPTS`; not blocked by the server middleware; contains no financial data or personal information.
- **Powers:** The US Map choropleth renderer in `app.js` — draws the clickable 50-state outline and fills each state with the Prosperity/Pressure score colour.

Server-Mode Endpoint Map

When the FastAPI backend (`server/main.py` + `server/data.py`) is active, the data flows through three authenticated endpoints rather than static `<script>` tags:

Endpoint	Datasets delivered	Trigger
<code>GET /api/data/bundle</code>	All core datasets as a single JSON object keyed by global name	Immediately after login
<code>GET /api/data/state</code>	<code>LEGISCAN_DATA</code> + <code>OPENSTATES_DATA</code> (lazy)	First visit to State Legislature view
<code>GET /api/data/live</code>	<code>PRICES_DATA</code> + <code>NEWS_INTEL</code> (live)	Per-tier cadence; paused when market closed for lower tiers

The manifest that drives this routing is written by `scripts/extract_data_to_json.py` to `private_data/manifest.json` and read at runtime by `server/data.py` → `_manifest()`. The `private_data/` directory is gitignored entirely (only `.gitignore` is tracked) — the datasets it holds are never committed to the repository or served statically.

Build Script Reference

Build script	Output file	Requires <code>.env</code> key	Run frequency
<code>webapp/build_data.py</code>	<code>js/data.js</code>	(none; reads local pipeline JSONs)	After each full pipeline run
<code>webapp/build_influence.py</code>	<code>js/influence_data.js</code>	(none; reads LittleSis bulk gz from <code>~/Downloads</code>)	When LittleSis dump is refreshed
<code>webapp/build_relationships.py</code>	<code>js/relationships_data.js</code>	(none; Senate LDA is anonymous)	Quarterly / on demand
<code>webapp/build_nonprofits.py</code>	<code>js/nonprofit_data.js</code>	(none; ProPublica API is open)	Annually
<code>webapp/build_fec.py</code>	<code>js/fec_data.js</code>	<code>FEC_API_KEY</code>	Election-cycle refresh
<code>webapp/build_sec.py</code>	<code>js/sec_data.js</code>	<code>SECAPI_IO_KEY</code>	Quarterly / resume-safe
<code>webapp/build_secbulk.py</code>	<code>js/secbulk_data.js</code>	(none; reads SEC bulk zips from <code>~/Downloads</code>)	When SEC publishes updated bulk zips
<code>webapp/build_usaspending.py</code>	<code>js/usaspending_data.js</code>	(none; USASpending API is open)	Monthly
<code>webapp/build_states.py</code>	<code>js/states_data.js</code> (base)	<code>FRED_API_KEY</code>	Monthly
<code>webapp/build_bea.py</code>	<code>js/states_data.js</code> (enriches)	<code>BEA_API_KEY</code>	Annually (BEA RPP lags ~12 mo)
<code>webapp/build_bls.py</code>	<code>js/states_data.js</code> (enriches)	(none)	Monthly
<code>webapp/build_census.py</code>	<code>js/states_data.js</code> (enriches)	(none — Census ACS open)	Annually
<code>webapp/build_sp500.py</code>	<code>js/sp500_data.js</code>	(none; GitHub CSV)	When index composition changes
<code>webapp/build_quant.py</code>	<code>js/quant_data.js</code>	(none; yfinance + local QuantLib)	Daily
<code>webapp/build_member_bills.py</code>	<code>js/member_bills.js</code>	<code>CONGRESS_API_KEY</code>	Weekly
<code>webapp/build_prices.py</code>	<code>js/prices_data.js</code>	<code>FINNHUB_API_KEY</code>	Daily / live-tier

Build script	Output file	Requires <code>.env</code> key	Run frequency
<code>webapp/build_news_intel.py</code>	<code>js/news_intel.js</code>	<code>OPENAI_API_KEY</code> (or <code>THINKFREE_NEWS_MODEL</code> env)	After each news pipeline run
<code>webapp/build_legiscan.py</code>	<code>js/legiscan_data.js</code>	<code>LEGISCAN_API_KEY</code>	Weekly (30k/month budget)
<code>webapp/build_openstates.py</code>	<code>js/openstates_data.js</code>	<code>OPENSTATES_API_KEY</code>	Monthly (500/day free tier)
<code>webapp/minify_data.py</code>	<i>(minifies all <code>_data.js</code> in place)</i>	<i>(none)</i>	Before production deploy
<code>scripts/extract_data_to_json.py</code>	<code>private_data/*.json</code> + <code>private_data/manifest.json</code>	<i>(none)</i>	Before launching the FastAPI server

Part 12B — Current State of the Live Application

This part documents the platform **as it runs in production today** at `thinkfree.onrender.com`. It captures the subsystems and changes that shipped after the earlier parts were first written, so the reference reflects the live product. It records capabilities and design, not live point figures.

12B.1 Deployment on Render

The application is deployed on **Render** from the `main` branch of the GitHub repository, described by `render.yaml` (a Blueprint). One **web service** runs the FastAPI backend, which serves the static front-end and the gated data API. A **persistent disk** (mounted at `/var/data`) holds the SQLite user store so accounts and sessions survive redeployments. Secrets are provided as dashboard environment variables (`sync: false`), never committed. The build reinstalls backend dependencies, rebuilds `private_data/` from the committed data snapshots, and compiles the marketing landing bundle.

A separate **daily cron service** runs the free and low-cost data builders and commits the refreshed `*_data.js` snapshots back to `main`; that push triggers the web service to redeploy with fresh data. Git is the durable hand-off, because a cron job and the web service cannot share a Render disk.

12B.2 Tiered refresh and pricing

`server/config.py` holds the single source of truth for the tier catalogue. Tiers differ only in how often a user may pull fresh live data; every tier reads the same public records and shows the same transparent math. The public tiers are:

Tier	Refresh cadence	Price
Free	Daily briefing	Free
Pro, Hourly	Every hour	5 per month
Pro, 30-minute	Every 30 minutes, incl. after hours	15 per month
Pro, 5-minute	Every 5 minutes, incl. after hours	25 per month

An internal **beta** tier grants 5-minute access to closed-beta testers and is never sold. The refresh cadence is **enforced server-side** in the data API, so it cannot be sped up by editing the browser. Tier keys are stable across pricing changes because they are persisted on user rows and inside signed tokens.

12B.3 The Money Trail engine

The Money Trail engine scores each law on a transparent, six-link **chain of evidence**: the law, who it pays, who knew, who traded, the payoff, and the pattern. Every link contributes a visible, weighted component to the score, so a reader can see exactly how a suspicion score was reached rather than trusting a black box. The output obeys the political-intelligence framing rule: it describes **timing relationships** between public disclosures and market events and never alleges wrongdoing. A cached, no-cost path builds the scores without a paid language-model step.

12B.4 Congressional trading, QuiverQuant, and corrected statistics

Congressional-trade data is sourced from **QuiverQuant** and STOCK Act disclosures, joined to government-contract files already in the repository. The politician scoreboard uses the **corrected methodology** in Part 2B: members are ranked on **excess return versus SPY over the identical holding window**, sales are framed as **avoided or foregone** exposure rather than summed with buy profit-and-loss, and dollar totals are shown as **ranges** to reflect the brackets in the disclosures. This replaces the earlier raw-dollar, buy-and-sell-combined presentation.

12B.5 Scheduler and autonomous refresh

Two mechanisms keep the served data fresh without a manual redeploy:

- An **in-process background refresher** (a FastAPI daemon thread) re-pulls the economy series on a timer and writes them to the persistent disk, so the served data updates continuously between deploys.
- The **daily cron** (Part 12B.1) refreshes the broader snapshot set and commits them back to `main`, triggering a redeploy.

No paid language-model steps run in the scheduled path; the cron runs only the free and cheap builders.

12B.6 OFR Hedge Fund Monitor

A **Hedge Fund Monitor** screen surfaces hedge-fund and leverage indicators from the U.S. Office of Financial Research (OFR) Hedge Fund Monitor, a free, no-key public data source. It gives the everyday reader a plain-English read on where systemic leverage is building.

12B.7 Data Centers tracker

A **Data Centers** screen maps and lists U.S. data-center activity by county, combining public sources (Census, EIA, curated locations, and OpenStreetMap and FCC coverage) into a browseable, paginated view, framed around what the buildout means for local power demand, jobs, and costs.

12B.8 Economy and cost-of-living tracker

An **Economy** screen tracks the prices people actually pay, groceries, rent, gas, mortgages, car loans, jobs, and more, against a fixed baseline, using a large set of FRED indicators. It translates macro moves into what they mean for a household budget in plain English, which is the platform's core positioning.

12B.9 Presidential feed

A **Presidential** feed tracks executive-branch activity relevant to markets and the economy, presented in the same translate-to-plain-English style as the rest of the product.

12B.10 Security hardening

The backend enforces a real security posture for cloud hosting:

- **Authentication** via bcrypt password hashing and signed PyJWT session tokens in

an httpOnly cookie, with server-side session records on the persistent disk.

- **Closed-beta invite codes:** single-use codes gate account creation and are

consumed on redemption, replacing an earlier shared beta key.

- **Gated data:** the datasets are served only through authenticated `/api/data/*`

endpoints; the raw bulk-data files are blocked from direct download, and the app returns unauthorized without a session.

- **Hardening headers on every response:** a Content-Security-Policy, `nosniff`,

`X-Frame-Options: DENY`, a referrer policy, a permissions policy, and, in production, HSTS with an HTTPS redirect.

- **Rate limiting:** a coarse global per-IP throttle under precise per-endpoint

limits on the auth routes.

- **Self-hosted fonts:** the app fonts are self-hosted and the CSP declares

`font-src 'self'`, removing an external font dependency.

- Dependency versions are pinned and patched against known advisories, and a

standing attacker-perspective review runs before changes ship.

12B.11 Public landing and app theme

The public **marketing landing** at `/` is a standalone React and Vite bundle, fully isolated from the application: it imports no application code, data, or auth, and it is served as static assets. It has its **own scoped Content-Security-Policy** that applies only to `/` and the landing assets; every application and API route keeps the original, unchanged application CSP. The landing routes its login and sign-in actions to the application, which owns the real authentication flow.

The application itself received a **theme-level visual upgrade** delivered entirely through its central design tokens (color, type scale, spacing, radii, shadows, motion) plus polish of shared components. It is centralized and reversible, applies to both light and dark themes and to mobile and desktop, and preserves all functionality and information architecture. The user-selectable accent presets, including the pink and purple options, remain available; the default brand accent stays a single restrained cyan.

The influence-graph nodes render clean local ticker monograms rather than remote logos, which removed a remote image dependency and eliminated broken-image placeholders.

Part 13 — Appendices

Appendix A — Project file index (non-vendored)

Backend (`server/`)

File	Lines	Role
<code>main.py</code>	110	App, middleware, static serving, health, data-file blocking
<code>auth.py</code>	288	bcrypt + JWT auth, signup/login/logout/profile/a11y, rate limiting
<code>data.py</code>	111	Gated <code>/api/data/*</code> (bundle/state/live), per-tier cadence, market pause
<code>db.py</code>	107	stdlib SQLite user store, JWT revocation column
<code>config.py</code>	63	Env loading, tiers, secret-placeholder guard

Runtime verification agents (`agents/`)

File	Lines	Role
<code>data_quality.py</code>	449	Data-integrity gate after scraping/extraction
<code>security_audit.py</code>	403	Secret/CVE/static-analysis gate (CI-blocking)
<code>strategy_audit.py</code>	277	Backtest sanity gate (lookahead, Sharpe, drawdown)
<code>verification_runner.py</code>	161	Orchestrates the agents at checkpoints

Core intelligence pipeline (repo root)

File	Lines	Phase
<code>controller.py</code>	451	12 — orchestration / human-in-the-loop
<code>chef_gpt.py</code>	469	11 — final intelligence briefing
<code>recession_signals.py</code>	429	13 — recession risk score
<code>congress_bills.py</code>	409	6.5 — political intelligence
<code>phase4_clustering.py</code>	383	4 — clustering
<code>portfolio_tracker.py</code>	334	10 — mock portfolio
<code>phase3_extraction.py</code>	333	3 — extraction
<code>historical_correlation.py</code>	400	7 — historical analogs
<code>quantlib_metrics.py</code>	326	8 — quant metrics
<code>opportunity_score.py</code>	314	8 — opportunity scoring
<code>politician_performance.py</code>	290	6.5 — politician tracking
<code>validate_env.py</code>	249	env/config validation

Supporting engines

File	Lines	Area
<code>Module_2_Technical_Analysis/phase_4_backtrader.py</code>	1149	Backtesting (Phase 9)
<code>GPT_Economy/Reasoning_Report.py</code>	200	Economic reasoning report
<code>Profile/GPT_article_clustering_summary.py</code>	187	GPT cluster summaries
<code>Module_2_Technical_Analysis/ta_analysis.py</code>	165	Technical indicators
<code>Module_2_Technical_Analysis/pipeline_alpha.py</code>	158	Alpha pipeline
<code>Module_2_Technical_Analysis/phase_3_signal.py</code>	142	Signal generation
<code>Profile/article_clustering.py</code>	118	Embedding clustering

File	Lines	Area
GPT_Economy/economic_engine.py	101	FAISS economic engine

Offline data build scripts (webapp/)

build_data.py (824), build_legiscan.py (280), build_secbulk.py (164), build_influence.py (174), build_states.py (155), build_fec.py (153), build_usaspending.py (148), build_prices.py (145), build_relationships.py (146), build_nonprofits.py (139), build_openstates.py (133), build_census.py (131), build_quant.py (121), build_sec.py (119), build_bea.py (103), build_bls.py (98), fetch_politician_photos.py (83), build_sp500.py (81), build_member_bills.py (74), minify_data.py (60), scripts/extract_data_to_json.py (78).

Frontend (webapp/js/)

File	Lines	Role
app.js	1847	Router, page rendering, modals, search, refresh
influenceweb.js	1201	Interactive relationship graph
auth.js	594	Dual-mode gate, quiz, tiers, signup integrity
congress.js	582	Political-intelligence views
genimpact.js	337	Generational-impact scoring
predictions.js	323	Predictive market signals
scoreinfo.js	302	Progressive score explanations
scores.js	211	Score display
boot.js	148	Mode detection + script injection
glossary.js	95	Jargon annotation
*_data.js	(generated)	17 datasets (Part 12)

Other

Path	Role
dashboard/app.py (3624)	Legacy Streamlit prototype (kept, untouched)
iphone-app/	SwiftUI client reusing web data (not build-verified)
.claude/agents/cybersecurity-agent.md + cybersecurity-kb/	The AI sub-agent + its training (Part 9)
CLAUDE.md	The canonical blueprint
DEPLOY.md	Hosting + security guide

Appendix B — Environment variables

Variable	Used by	Purpose
TF_SECRET_KEY	backend	JWT signing secret (48+ random chars)
TF_BETA_KEY	backend	Closed-beta invite key
TF_PRODUCTION	backend	1 in prod (Secure cookies, HSTS, HTTPS redirect)
TF_ALLOWED_ORIGINS	backend	CORS allow-list
TF_SESSION_TTL_SECONDS	backend	Session lifetime
LEGISCAN_API_KEY	build_legiscan.py	State legislation
OPENSTATES_API_KEY	build_openstates.py	State legislators
FINNHUB_API_KEY	build_prices.py	Market prices
FEC_API_KEY	build_fec.py	Campaign finance
CENSUS_API_KEY	build_census.py	Census data
BEA_API_KEY	build_bea.py	Economic accounts

Variable	Used by	Purpose
BLS_API_KEY	build_bls.py	Labor statistics
OPENAI_API_KEY	pipeline / news intel	GPT calls
FRED_API_KEY	recession signals	Macro series

(See Part 4 for the authoritative per-script mapping.)

Appendix C — Glossary

- **Blueprint** / `CLAUDE.md` — the committed project charter that governs mission, framing, roadmap, and constraints.
- **Chef GPT** — the final intelligence layer (Phase 11) that merges all upstream outputs into one plain-English briefing.
- **cyrb53** — a fast non-cryptographic hash used only for the soft static-mode gate; never a real security boundary.
- **Dual-mode** — the front end runs identically as a static site or behind the FastAPI backend, decided at runtime by `boot.js`.
- **Eight-question framework** — the eight questions every major analysis must answer (Part 1).
- **IDOR** — Insecure Direct Object Reference; accessing another user's data by changing an identifier. Prevented by server-side per-user scoping.
- **Intelligence report** — the platform's output. ThinkFree never emits a trade order.
- **Political-intelligence framing rule** — all such output describes timing relationships between public disclosures and market events and never alleges wrongdoing.
- **RLS (Row-Level Security)** — the database-level per-user data isolation concept from the source material; in ThinkFree's FastAPI/SQLite stack it is implemented as server-side authorization on every endpoint.
- **Tier** — a subscription level controlling data-refresh cadence (e.g. beta = 15-minute refresh), enforced server-side.
- **Verification agent** — a runtime Python gate that returns pass/fail at a pipeline checkpoint (Part 8), distinct from the development-time AI sub-agent (Part 9).

Appendix D — Document provenance

This document was compiled from a direct reading of the source tree (every file listed in Appendix A), the canonical blueprint (`CLAUDE.md`), the deployment guide (`DEPLOY.md`), and the security knowledge base in `.claude/agents/cybersecurity-kb/`. The code-reference parts (4, 5, 6A, 6B, 7A, 7B, 8, 12) were produced by reading the actual source and reproducing real signatures, formulas, and patterns; where a subsystem could not be executed end-to-end in the build environment, that is stated rather than asserted as verified.